

TRẦN QUỐC LONG – TẠ VIỆT CƯỜNG

GIÁO TRÌNH
HỌC MÁY THỐNG KÊ

NHÀ XUẤT BẢN ĐẠI HỌC QUỐC GIA HÀ NỘI

Mục lục

Mục lục	i
Danh sách hình vẽ	xi
Danh sách bảng	xv
Lời nói đầu	xix
Chương 1 Giới thiệu	1
1.1 Học máy là gì ?	1
1.2 Ứng dụng của Học máy	4
1.3 Các tình huống áp dụng Học máy	8
1.4 Các phương pháp và bài toán trong Học máy . . .	11
1.4.1 Học có giám sát	11
1.4.2 Học không giám sát	13
1.4.3 Học tăng cường	14
1.4.4 Các bài toán lai	15
1.4.5 Các kỹ thuật thường dùng trong Học máy .	16
1.5 Quá trình phát triển và triển khai mô hình học máy	18
1.6 Tổng kết chương	20
1.7 Bài tập	20

Tài liệu tham khảo 23**Chương 2 Bài toán phân lớp 25**

2.1	Định nghĩa	26
2.2	Tổng quan về dữ liệu	28
2.3	Cách tiếp cận Học máy thống kê	28
2.4	Ước lượng xác suất lỗi	30
2.5	Hàm phân lớp tối ưu Bayes	34
2.6	Mô hình hồi quy Logistic	36
2.6.1	Nguyên lý Ước lượng hợp lý cực đại	37
2.6.2	Thuật toán huấn luyện	39
2.7	Thuật toán K láng giềng gần nhất	42
2.8	Thuật toán Cây quyết định	44
2.8.1	Thuật toán ID3	47
2.8.2	Thuật toán C4.5	48
2.8.3	Thuộc tính liên tục	49
2.9	Tình huống áp dụng: Phân lớp hoa IRIS	50
2.9.1	Giới thiệu dữ liệu đầu vào	50
2.9.2	Các thư viện sử dụng	50
2.9.3	Các bước triển khai chính	51
2.10	Tổng kết chương	52
	Bài tập	52

Tài liệu tham khảo 55**Chương 3 Mạng nơ-ron nhiều lớp 57**

3.1	Mô hình	58
3.1.1	Mô hình nơ-ron đơn vị	58
3.1.2	Lớp nơ-ron	59
3.1.3	Mạng nơ-ron nhiều lớp lan truyền tới	61
3.2	Huấn luyện mô hình mạng nơ-ron nhiều lớp	64

3.3	Các thuật toán tối ưu	67
3.3.1	Tính hội tụ của phương pháp xuống đồi bằng đạo hàm	68
3.3.2	Thuật toán xuống đồi bằng đạo hàm ngẫu nhiên	72
3.3.3	Thuật toán SGD với quán tính	72
3.3.4	Thuật toán ước lượng quán tính thay đổi - Adam	74
3.4	Các vấn đề liên quan đến tính hội tụ	75
3.5	Tình huống áp dụng: Nhận dạng chữ số MNIST . .	77
3.5.1	Giới thiệu thư viện Pytorch	77
3.5.2	Xây dựng mô hình mạng nơ-ron nhiều lớp .	77
3.5.3	Các bước triển khai chính	78
3.6	Tổng kết chương	79
	Bài tập	79
	Tài liệu tham khảo	83
	Chương 4 Máy véc-tơ hỗ trợ	85
4.1	Mô hình nơ-ron nhân tạo Perceptron	86
4.1.1	Pha huấn luyện	87
4.1.2	Tính dừng của thuật toán huấn luyện	88
4.2	Máy véc-tơ hỗ trợ lề cứng	90
4.3	Máy véc-tơ hỗ trợ lề mềm	95
4.4	Phương pháp nhân hóa	101
4.5	Tình huống áp dụng: Phân lớp chữ số MNIST bằng SVM	105
4.6	Tổng kết chương	106
	Bài tập	107
	Tài liệu tham khảo	111

Chương 5	Mạng nơ-ron tích chập	113
5.1	Mô hình hoá dữ liệu dạng ảnh	114
5.2	Phép toán tích chập	117
5.2.1	Lớp tích chập	118
5.2.2	Lan truyền ngược qua lớp tích chập	122
5.3	Các lớp phổ biến được sử dụng trong mạng tích chập	123
5.3.1	Lớp gộp	123
5.3.2	Lớp liên kết đầy đủ	125
5.3.3	Lớp triệt tiêu ngẫu nhiên	125
5.3.4	Lớp chuẩn hóa loạt	126
5.4	Kiến trúc mạng CNN xương sống	127
5.4.1	Mạng VGG	127
5.4.2	Mạng Inception	128
5.4.3	Mạng ResNet	130
5.4.4	Mạng MobileNet	132
5.5	Bài toán nhận dạng hình ảnh	134
5.5.1	Bài toán nhận dạng ảnh	135
5.5.2	Bài toán nhận dạng vật thể trong ảnh	136
5.5.3	Bài toán phân vùng ngữ nghĩa ảnh	137
5.6	Một số chiến lược huấn luyện	137
5.6.1	Học chuyển đổi	138
5.6.2	Điều chỉnh hệ số học	139
5.6.3	Các chiến lược huấn luyện nâng cao khác	139
5.7	Tình huống áp dụng: Phân loại ảnh CIFAR10	140
5.8	Tổng kết chương	141
	Bài tập	143
	Tài liệu tham khảo	147
Chương 6	Mạng nơ-ron cho dữ liệu dạng chuỗi	149
6.1	Mô hình hoá dữ liệu dạng chuỗi	150

6.2	Mạng nơ-ron hồi quy	153
6.2.1	Mạng nơ-ron hồi quy sâu	155
6.2.2	Mạng nơ-ron hồi quy hai chiều	156
6.2.3	Kiến trúc Mã hoá – Giải mã	156
6.2.4	Những vấn đề trong mạng RNN	157
6.3	Bộ nhớ ngắn hạn dài	158
6.4	Mạng nơ-ron biến đổi Transformer	161
6.5	Một số bài toán nhận thức kiểu chuỗi	165
6.5.1	Bài toán dịch máy thống kê	165
6.5.2	Bài toán phát hiện chủ đề	166
6.6	Tình huống áp dụng: học phép toán cộng	167
6.6.1	Mô tả bài toán	167
6.6.2	Các bước triển khai	168
6.7	Tổng kết chương	169
	Bài tập	170
Tài liệu tham khảo		171
Chương 7 Mô hình hồi quy		173
7.1	Bài toán hồi quy	174
7.1.1	Hàm hồi quy tối ưu	175
7.1.2	Sự tráo đổi giữa độ lệch và phương sai	176
7.2	Mô hình hồi quy tuyến tính	178
7.2.1	Huấn luyện mô hình	181
7.3	Các phương pháp đánh giá mô hình	183
7.3.1	Đánh giá đầu ra mô hình hồi quy	184
7.3.2	Đánh giá chất lượng trọng số	186
7.4	Mạng nơ-ron hồi quy	187
7.4.1	Hồi quy với hệ cơ sở phi tuyến	187
7.4.2	Mạng nơ-ron	188
7.5	Hàm lỗi	190

7.6	Hồi quy bằng K láng giềng gần nhất	191
7.7	Mô hình Cây quyết định trong hồi quy	193
7.8	Tình huống áp dụng: Dự đoán giá nhà	194
7.9	Tổng kết chương	195
	Bài tập	196
Tài liệu tham khảo		199
Chương 8 Học tăng cường		201
8.1	Giới thiệu về quá trình quyết định Markov	202
8.1.1	Mô hình toán học của học tăng cường	203
8.1.2	Một số loại môi trường cơ bản của học tăng cường	207
8.2	Thuật toán tối ưu quy hoạch động	209
8.2.1	Giá trị tối ưu	209
8.2.2	Giá trị trạng thái - hành động	210
8.2.3	Thuật toán lặp giá trị	211
8.2.4	Thuật toán lặp chính sách	216
8.3	Học tăng cường và phương pháp học hàm Q	219
8.3.1	Học mô hình quá trình quyết định Markov	220
8.3.2	Học hàm Q thông qua kinh nghiệm	221
8.4	Các thuật toán học xấp xỉ	226
8.5	Áp dụng học sâu cho học tăng cường	228
8.5.1	Mạng học sâu cho hàm giá trị Q	228
8.5.2	Phương pháp Xuống đồi bằng đạo hàm của chính sách	232
8.5.3	Phương pháp tối ưu chính sách	235
8.6	Tình huống áp dụng: Sử dụng mạng DQN để điều khiển thăng bằng	236
8.6.1	Mô tả bài toán	236
8.6.2	Các bước triển khai	237

8.7	Tổng kết chương	238
	Bài tập	239
Tài liệu tham khảo		241
Chương 9 Học kết hợp		243
9.1	Giới thiệu	244
9.2	Phương pháp Bagging	245
9.2.1	Tổng quan phương pháp	245
9.2.2	Mô hình Rừng ngẫu nhiên	248
9.3	Phương pháp Boosting	251
9.4	Tăng cường bằng đạo hàm	256
9.4.1	Tăng cường bằng đạo hàm bậc 1	258
9.4.2	Tăng cường bằng đạo hàm bậc 2	259
9.5	Tình huống áp dụng: XGBoost	261
9.6	Tổng kết chương	263
	Bài tập	263
Tài liệu tham khảo		267
Chương 10 Bài toán trích xuất đặc trưng		269
10.1	Giới thiệu	270
10.2	Phương pháp phân tích thành phần chính	271
10.2.1	Thành phần chính đầu tiên	272
10.2.2	Các thành phần chính tiếp theo	275
10.2.3	Giảm chiều dữ liệu	277
10.3	Bộ tự mã hóa sâu	278
10.3.1	Giới thiệu	278
10.3.2	Huấn luyện bộ tự mã hóa sâu	279
10.3.3	Bộ tự mã hoá khử nhiễu	280
10.3.4	Bộ tự mã hoá biến phân	281

10.4	Trích chọn đặc trưng trong bài toán học có giám sát	283
10.4.1	Phương pháp lọc	284
10.4.2	Phương pháp gói	286
10.4.3	Phương pháp nhúng	287
10.5	Tình huống áp dụng: Phân tích thành phần chính trên dữ liệu CIFAR-10	288
10.6	Tổng kết chương	289
	Bài tập	289
	Tài liệu tham khảo	293
	Chương 11 Mô hình sinh	295
11.1	Bài toán sinh dữ liệu	296
11.2	Ước lượng mật độ bằng mô hình tham số	297
11.2.1	Ước lượng hợp lý cực đại	298
11.2.2	Ước lượng hậu nghiệm cực đại	301
11.3	Thuật toán EM - Cực đại hoá kì vọng	305
11.3.1	Cận dưới bằng chứng	307
11.3.2	Thuật toán cực đại hoá kì vọng	308
11.3.3	Thuật toán EM trên phân bố xác suất họ mũ	310
11.4	Ước lượng mật độ phi tham số	314
11.5	Biểu diễn phân bố bằng mạng nơ-ron	317
11.5.1	Biểu diễn mật độ bằng hàm năng lượng . . .	317
11.5.2	Biểu diễn phân bố bằng hàm sinh	320
11.6	Tình huống áp dụng: Generative Adversarial Networks (GAN)	325
11.7	Tổng kết chương	328
	Bài tập	329
	Tài liệu tham khảo	331
	Chương 12 Mô hình đồ thị xác suất	333

12.1	Giới thiệu	334
12.2	Mô hình Bayes ngây thơ	335
12.2.1	Pha huấn luyện	336
12.2.2	Pha suy luận	337
12.2.3	Lưu ý khi ước lượng xác suất	339
12.3	Mạng Bayes	340
12.3.1	D-chia cắt	341
12.3.2	Quan hệ độc lập xác suất có điều kiện	344
12.4	Trường ngẫu nhiên Markov	345
12.4.1	Đồ thị nhân tử	346
12.5	Suy luận chính xác trên đồ thị dạng cây	347
12.5.1	Tính xác suất biên của một đỉnh	348
12.5.2	Tính xác suất cực đại	351
12.6	Mô hình Markov ẩn HMM	352
12.6.1	Bài toán suy luận xác suất	355
12.6.2	Bài toán suy luận chuỗi trạng thái	356
12.6.3	Bài toán huấn luyện	356
12.7	Trường ngẫu nhiên có điều kiện	359
12.7.1	Pha huấn luyện	360
12.7.2	Pha suy luận	361
12.8	Tình huống áp dụng: Bài toán gán nhãn từ loại	364
12.9	Tổng kết chương	366
	Bài tập	366
Tài liệu tham khảo		369
Chương 13 Triển khai mô hình học máy		371
13.1	Vòng đời phát triển mô hình học máy	372
13.2	Thu thập và tiền xử lý dữ liệu	373
13.2.1	Thu thập dữ liệu	374
13.2.2	Tiền xử lý dữ liệu	376

13.2.3	Trích chọn đặc trưng	377
13.3	Triển khai mô hình học máy	380
13.3.1	Triển khai mô hình	380
13.3.2	Chuẩn bị mô hình trước khi triển khai . . .	381
13.3.3	Đóng gói mô hình	382
13.3.4	Các phương pháp triển khai mô hình	383
13.3.5	Đánh giá và giám sát mô hình sau triển khai	384
13.4	Các nền tảng hỗ trợ triển khai mô hình	385
13.4.1	Công cụ đóng gói và triển khai	385
13.4.2	Công cụ quản lý và giám sát mô hình	385
13.4.3	Công cụ tối ưu hóa và tăng tốc suy luận . .	386
13.4.4	Công cụ CI/CD cho học máy	386
13.5	Tình huống áp dụng: Triển khai mô hình qua giao thức lập trình ứng dụng REST	387
13.6	Tổng kết chương	389
	Bài tập	389
	Tài liệu tham khảo	391
	Chỉ mục	393

Danh sách hình vẽ

1.1	Tỉ lệ băng thông email là thư rác.	2
1.2	Bộ dữ liệu chữ số viết tay MNIST.	3
1.3	Giải quyết vấn đề bằng cách tiếp cận Học máy.	4
1.4	Quá trình phát triển, triển khai mô hình Học máy. . .	19
2.1	Hàm lỗi Logistic $-\log \sigma(s)$ với $s = yf(\mathbf{x})$	39
2.2	Minh hoạ kết quả cập nhật của trọng số theo hướng của đạo hàm.	40
2.3	Thuật toán KNN với $k = 3$ và $k = 5$	43
2.4	Cây quyết định phân lớp.	45
2.5	Cây quyết định phân chia tập đầu vào cho bài toán phân lớp.	45
3.1	Mô hình nơ-ron.	58
3.2	Một lớp nơ-ron có p đầu ra.	60
3.3	Mạng nơ-ron nhiều lớp lan truyền tới.	62
4.1	Tập dữ liệu khả tách tuyến tính.	87
4.2	Mô hình Máy véc-tơ hỗ trợ SVM.	92

4.3	Hàm lỗi bản lề (hinge loss).	97
5.1	Minh họa về ảnh cấp sáng được lưu trữ dưới dạng ma trận hai chiều.	115
5.2	Ví dụ về ảnh cấp xám 28×28 được lấy từ tập MNIST với nhãn cần phải học là tập các chữ số từ 0 đến 9. . .	115
5.3	Minh họa về ba kênh màu của ảnh RGB.	116
5.4	Ví dụ về dữ liệu dùng cho bài toán phát hiện vật thể trong ảnh.	117
5.5	Cấu trúc chung của mạng CNN.	118
5.6	Ví dụ phép tích chập trên ảnh cấp xám.	118
5.7	Ví dụ phép tích chập trên ảnh ba chiều.	121
5.8	Ví dụ kết quả của lớp gộp theo giá trị lớn nhất trong vùng tiếp nhận.	124
5.9	Các mạng VGG-16 và VGG-19.	128
5.10	Cấu trúc mô-đun inception.	130
5.11	Kết nối phần dư.	131
5.12	Mạng ResNet-50.	132
5.13	Tích chập trên từng kênh.	133
5.14	Kiến trúc mạng MobileNet-v1.	134
6.1	Ví dụ về mẫu tín hiệu âm thanh	152
6.2	Tính toán của RNN trên chuỗi dữ liệu.	154
6.3	Biểu diễn rút gọn của mạng RNN.	155
6.4	Mạng nơ-ron RNN hai chiều.	156
6.5	Kiến trúc Mã hoá – Giải mã.	157
6.6	Một bước tính toán trong mạng LSTM.	159

6.7	Cấu trúc tổng quát của mô hình Transformer	162
7.1	Mô hình hồi quy tuyến tính.	178
7.2	Minh hoạ lỗi của mô hình hồi quy tuyến tính	179
7.3	Hệ số R cho các trường hợp khác nhau.	185
7.4	Cây quyết định hồi quy tập đầu vào $\mathcal{X}$	193
8.1	Mô hình đơn giản minh hoạ về tác tử thông minh học cách tương tác với môi trường	203
8.2	Mô hình tính toán giá trị tối ưu $V^*(s)$ sử dụng công thức đệ quy	212
8.3	Mô hình tính toán giá trị V_{t+1} sử dụng giá trị V_t	213
8.4	Mô hình tính toán giá trị $V^\pi(s)$ với chính sách π	217
8.5	Mô hình tương tác giữa tác tử và môi trường trong học tăng cường	219
8.6	Cấu trúc cơ bản của DQN	229
9.1	Học kết hợp	245
9.2	Phương pháp Bagging	247
9.3	Phương pháp AdaBoost	252
9.4	Phương pháp Tăng cường bằng đạo hàm	258
10.1	Quá trình trích xuất đặc trưng và khôi phục lại dữ liệu thô	271
10.2	Minh hoạ phương pháp phân tích thành phần chính PCA trên dữ liệu 2 chiều	273
10.3	Minh hoạ quá trình mã hoá và giải mã trong bước huấn luyện của bộ tự mã hoá sâu	280

10.4	Minh hoạ quá trình mã hoá và giải mã trong bước huấn luyện của bộ tự mã hoá biến phân	282
10.5	Lược đồ phương pháp gói	286
10.6	Tập dữ liệu trong $\mathbb{R}^2$	290
11.1	Minh hoạ phương pháp ước lượng mật độ tham số . . .	297
11.2	Phân phối Nhị phân với $\theta = 0.5$, $N = 20$ và $n = 10$. .	304
11.3	Minh hoạ mô hình GAN	322
12.1	Mô hình Bayes ngây thơ.	336
12.2	Ví dụ minh hoạ về mạng Bayes mô tả quan hệ giữa các thuộc tính.	341
12.3	Các tình huống D-chia cắt.	341
12.4	Một đồ thị nhân tử có 3 nhân tử.	346
12.5	Cây con trong đồ thị nhân tử.	348
12.6	Mạng Bayes cho mô hình HMM.	354
13.1	Quá trình phát triển, triển khai mô hình Học máy. . .	372

Danh sách bảng

1.1	Ví dụ về bộ dữ liệu trong bài toán phân loại thư rác theo tiêu đề	12
2.1	Dữ liệu cho bài toán phân lớp học hàm XOR	27
2.2	Dữ liệu phân lớp cho cây quyết định	53
3.1	Bảng các hàm kích hoạt thông dụng	59
3.2	Cấu trúc mạng nơ-ron nhiều lớp cho bài toán MNIST .	78
3.3	Dữ liệu phân lớp	80
4.1	Dữ liệu huấn luyện	107
5.1	Cấu trúc mô hình học sâu tích chập cho bài toán nhận dạng ảnh màu CIFAR10	142
6.1	Ví dụ về dữ liệu dạng văn bản được thu thập trên mạng internet về các thông tin bất động sản.	151
6.2	Ví dụ về các chủ đề và dữ liệu chuỗi cần phân loại chủ đề	166
6.3	Ví dụ về bộ dữ liệu D	168
7.1	So sánh ba mô hình hồi quy tuyến tính, hồi quy Logistic và perceptron	180

7.2	Dữ liệu huấn luyện	197
11.1	Các lớp trong kiến trúc của Generator trong mô hình GAN	327
11.2	Các lớp trong kiến trúc của Discriminator trong mô hình GAN	327
12.1	Giá trị của các nhân tử trong đồ thị nhân tử dạng cây	367

Danh sách thuật toán

2.1	Thuật toán huấn luyện mô hình hồi quy Logistic . .	41
2.2	Thuật toán huấn luyện trực tuyến mô hình hồi quy Logistic	41
2.3	Thuật toán KNN	43
2.4	Khung thuật toán huấn luyện Cây quyết định . . .	46
2.5	Dò ngưỡng θ cho thuộc tính liên tục A^k	49
3.1	Thuật toán lan truyền ngược	65
3.2	Thuật toán huấn luyện mạng nơ-ron MLP	67
3.3	Thuật toán SGD huấn luyện mạng nơ-ron MLP . .	73
4.1	Thuật toán huấn luyện Perceptron	88
4.2	Thuật toán huấn luyện mô hình SVM từ bài toán gốc	100
6.1	Lan truyền tới trong mạng RNN	154
6.2	Lan truyền tới trong mạng LSTM	160
6.3	Tính toán của lớp Chú ý (Attention)	164
7.1	Thuật toán KNN hồi quy	192
7.2	Thuật toán huấn luyện cây hồi quy CART	194
8.1	Thuật toán lập giá trị	214
8.2	Thuật toán lập chính sách	218
8.3	Thuật toán SARSA dựa trên chính sách tham lam ϵ	225
8.4	Thuật toán SARSA với hàm xấp xỉ tuyến tính Q .	228

8.5	Thuật toán huấn luyện DQN (Huấn luyện mạng DQN với chính sách tham lam ϵ)	231
8.6	Thuật toán tối ưu chính sách (Policy Gradient) . .	235
9.1	Phương pháp Bootstrap	246
9.2	Phương pháp Bagging	247
9.3	Huấn luyện mô hình Rừng ngẫu nhiên	249
9.4	Dự đoán với Rừng ngẫu nhiên	249
9.5	Phương pháp AdaBoost	254
9.6	Dự đoán với AdaBoost	254
9.7	Phương pháp SAMME cho phân lớp C lớp	257
9.8	Dự đoán với SAMME (AdaBoost đa lớp)	257
9.9	Phương pháp Tăng cường bằng đạo hàm bậc 1 (Gradient Boosting)	259
9.10	Dự đoán với Tăng cường bằng đạo hàm bậc 1 . . .	260
9.11	Tăng cường bằng đạo hàm bậc 2	261
10.1	Thuật toán Phân tích thành phần chính (PCA) . .	276
11.1	Thuật toán Cực đại hoá Kỳ vọng (EM Algorithm) .	309
11.2	Thuật toán K-means	314
11.3	Ước lượng mật độ KDE (Kernel Density Estimation)	316
11.4	Huấn luyện mô hình năng lượng (EBM Training) .	320
11.5	Thuật toán huấn luyện mô hình GAN	323
11.6	Thuật toán huấn luyện mô hình WGAN	326
12.1	Thuật toán huấn luyện mô hình Bayes ngây thơ . .	338
12.2	Thuật toán suy luận bằng mô hình Bayes ngây thơ	339
12.3	Thuật toán tính $p(x)$ cho mọi thuộc tính x trên cây	350
12.4	Thuật toán Baum-Welch huấn luyện mô hình HMM	358
12.5	Thuật toán huấn luyện mô hình CRF cho đồ thị nhân tử dạng cây	362

Lời nói đầu

Trong thời đại dữ liệu ngày nay, việc đưa ra quyết định không còn dựa vào trực giác hay kinh nghiệm cá nhân mà được hỗ trợ mạnh mẽ bởi các phân tích chính xác và sâu sắc từ dữ liệu. Học máy thống kê, với khả năng kết hợp giữa toán học và sức mạnh tính toán, đóng vai trò là nền tảng trong việc chuyển đổi dữ liệu thô thành tri thức có giá trị. Đây không chỉ là một công cụ, mà còn là cầu nối giúp con người hiểu rõ hơn về các xu hướng, mối quan hệ và dự đoán chính xác những thay đổi trong tương lai.

Với ứng dụng rộng rãi trong các lĩnh vực từ y tế, kinh doanh, khoa học môi trường đến trí tuệ nhân tạo, học máy thống kê đã trở thành trái tim của các hệ thống thông minh hiện đại. Nó không chỉ giúp tối ưu hóa hiệu suất mà còn mở ra cơ hội giải quyết những vấn đề phức tạp tưởng chừng bất khả thi với thuật toán và lập trình thông thường. Học máy thống kê, vì thế, không chỉ là lựa chọn mà đã trở thành một yếu tố cốt lõi định hình tương lai của xã hội và công nghệ.

Theo báo cáo từ McKinsey Global Institute, trong thập kỷ qua, dữ liệu đã trở thành một tài sản quý giá của mọi tổ chức và doanh nghiệp, với tốc độ tăng trưởng vượt bậc. Tuy nhiên, chỉ khoảng 20% dữ liệu hiện nay được khai thác một cách hiệu quả. Điều này

đã dẫn đến một nhu cầu ngày càng lớn về các chuyên gia có khả năng phân tích và đưa ra quyết định dựa trên dữ liệu. Trong đó, học máy thống kê nổi bật như một công cụ mạnh mẽ giúp giải quyết các bài toán phức tạp trong nhiều lĩnh vực, từ tài chính, y tế, giáo dục, đến thương mại và sản xuất.

Sinh viên ngày nay cần nhận thức rằng để trở thành nhân lực cạnh tranh trong thị trường lao động, họ phải nắm vững kiến thức và kỹ năng về học máy thống kê. Đây không chỉ là một công cụ giúp tăng khả năng phân tích dữ liệu, mà còn là nền tảng để phát triển các ứng dụng thông minh và đưa ra những giải pháp sáng tạo trong công việc. Hơn nữa, những kiến thức và kỹ năng này còn mang lại lợi ích lớn trong việc giải quyết các vấn đề thực tiễn trong cuộc sống hàng ngày.

Nhận thấy tầm quan trọng của học máy thống kê, nhiều trường đại học trên thế giới đã đưa môn học này trở thành một phần không thể thiếu trong chương trình đào tạo các ngành liên quan đến trí tuệ nhân tạo và khoa học dữ liệu. Việc đào tạo bài bản không chỉ tập trung vào lý thuyết mà còn nhấn mạnh vào các phương pháp thực hành hiện đại. Giáo trình này ra đời nhằm đáp ứng nhu cầu cấp bách trong việc cung cấp kiến thức nền tảng và kỹ năng ứng dụng học máy thống kê, giúp sinh viên hoàn thiện chuẩn đầu ra và đáp ứng tốt các yêu cầu công việc sau này.

Học máy thống kê là một lĩnh vực rộng lớn với nhiều phương pháp tiếp cận và quan điểm khác nhau. Trong giáo trình này, chúng tôi tập trung giới thiệu các khái niệm cơ bản và cái nhìn tổng thể nhất về học máy thống kê, giúp người học xây dựng nền tảng kiến thức vững chắc. Ngoài những nguyên lý chung và những kỹ năng cơ bản cần thiết để ứng dụng học máy thống kê trong thực tế, giáo trình còn giới thiệu một số chi tiết kỹ thuật cơ bản của các mô hình học máy thống kê chính, từ đó giúp người học hiểu rõ hơn về tầm quan trọng và khả năng ứng dụng của học máy thống kê.

Trong thực tiễn, mỗi lĩnh vực ứng dụng sẽ có các đặc thù riêng, yêu cầu những phương pháp và kỹ thuật phù hợp để giải quyết các vấn đề khác nhau. Giáo trình này không tham vọng bao quát toàn bộ những ứng dụng đó, mà nhằm cung cấp một nền tảng kiến thức tổng quát, từ đó người học có thể tiếp tục nghiên cứu và trang bị thêm các kỹ năng chuyên sâu hơn để đáp ứng nhu cầu thực tiễn trong việc khai thác và phân tích dữ liệu.

Giáo trình được xây dựng dựa trên kinh nghiệm giảng dạy và thực tiễn triển khai các ứng dụng học máy thống kê của chúng tôi trong nhiều năm qua. Quá trình biên soạn giáo trình đã tham khảo các tài liệu uy tín và phổ biến trên thế giới về học máy thống kê, cũng như cập nhật từ những nghiên cứu gần đây nhằm mang lại cho người học một cái nhìn toàn diện và hiện đại về lĩnh vực này. Chúng tôi đã kết hợp giữa lý thuyết nền tảng và các kỹ thuật thực hành, từ đó tạo nên một tài liệu không chỉ phục vụ cho việc học tập, mà còn hỗ trợ người học áp dụng hiệu quả vào thực tế. Các nội dung chính trong giáo trình bao gồm:

- **Chương 1. Giới thiệu:** Chương này giới thiệu tổng quan về học máy thống kê, vai trò của nó trong phân tích dữ liệu và các ứng dụng thực tiễn. Nội dung bao gồm định nghĩa, phân loại học máy và các thách thức chính trong lĩnh vực này.
- **Chương 2. Bài toán phân lớp:** Chương này tập trung vào các phương pháp phân loại, các mô hình cơ bản như Cây quyết định, K-láng giềng, Hồi quy Logistic. Người học sẽ tìm hiểu cách lựa chọn và đánh giá các mô hình phân loại.
- **Chương 3. Mô hình mạng nơ-ron nhiều lớp:** Chương này giải thích kiến trúc và cách hoạt động của mạng nơ-ron nhiều lớp. Nội dung bao gồm các khái niệm cơ bản về lan truyền xuôi, lan truyền ngược và tối ưu hóa hàm lỗi.

- **Chương 4. Mô hình máy véc-tơ hỗ trợ:** Chương này giới thiệu Máy véc-tơ hỗ trợ, một kỹ thuật mạnh mẽ trong phân lớp và hồi quy. Các khái niệm về tối ưu hoá độ rộng của lề, phương pháp nhân hoá và ứng dụng của máy véc-tơ hỗ trợ được giải thích rõ ràng.
- **Chương 5. Mô hình mạng nơ-ron tích chập:** Chương này trình bày kiến trúc mạng nơ-ron tích chập, một mô hình học sâu phổ biến cho xử lý dữ liệu hình ảnh. Các khái niệm như tích chập, gộp, các mô hình mạng xương sống và các ứng dụng trong nhận dạng hình ảnh sẽ được giới thiệu.
- **Chương 6. Mô hình mạng nơ-ron cho dữ liệu dạng chuỗi:** Chương này đề cập đến các mô hình học sâu cho dữ liệu tuần tự, bao gồm mạng Nơ-ron hồi quy, mạng Bộ nhớ ngắn hạn dài và Máy biến đổi. Ứng dụng chính của các mô hình này trong xử lý ngôn ngữ tự nhiên và dự đoán chuỗi thời gian cũng được thảo luận.
- **Chương 7. Mô hình hồi quy:** Chương này tập trung vào các kỹ thuật hồi quy, từ hồi quy tuyến tính đến hồi quy phi tuyến. Người học sẽ tìm hiểu cách xây dựng và đánh giá mô hình hồi quy trong các bài toán dự đoán.
- **Chương 8. Học tăng cường:** Chương này giới thiệu học tăng cường, một nhánh của học máy dựa trên khái niệm tác tử thông minh thu thập kinh nghiệm từ môi trường và tối ưu phần thưởng nhận được. Các thuật toán phổ biến như học sâu cho hàm Q và phương pháp đi theo đạo hàm chính sách cũng được mô tả chi tiết.
- **Chương 9. Học kết hợp:** Chương này trình bày các phương pháp học tổ hợp như Bagging, Boosting và Boosting bằng đạo hàm. Nội dung nhấn mạnh cách kết hợp nhiều mô hình để nâng cao hiệu quả dự đoán.

- **Chương 10. Bài toán trích xuất đặc trưng:** Chương này tập trung vào kỹ thuật xây dựng và chọn lựa đặc trưng, một bước quan trọng trong quy trình học máy. Các phương pháp trích xuất, chọn lọc, giảm chiều dữ liệu và bộ tự mã hoá biến phân cũng được giới thiệu.
- **Chương 11. Mô hình sinh:** Chương này trình bày các mô hình sinh dựa trên thuật toán cực đại hoá kì vọng và mạng sinh đối nghịch. Người học sẽ tìm hiểu cách các mô hình này tạo ra dữ liệu mới từ phân phối ẩn.
- **Chương 12. Mô hình xác suất đồ thị:** Chương này tập trung vào các mô hình đồ thị xác suất như mạng Bayesian và trường ngẫu nhiên Markov.
- **Chương 13. Triển khai mô hình học máy:** Chương này cung cấp kiến thức về triển khai các mô hình học máy trong môi trường sản xuất. Nội dung bao gồm tối ưu hóa mô hình, giám sát hiệu suất và bảo trì hệ thống.

Cuốn giáo trình này có thể được sử dụng bởi sinh viên, học viên cao học và người đã đi làm. Để hiểu tổng quan về Học máy, người học có thể đọc các Chương 1, Chương 2 và Chương 13. Để hiểu rõ hơn về các mô hình học máy cơ bản, người học có thể đọc các chương 3, 4, 5, 6, 7, 8, 9, 10. Để hiểu về các mô hình học máy nâng cao, người học có thể đọc các chương 11, 12. Ngoài ra, cuốn sách cũng có thể được sử dụng để tham khảo về một nội dung cụ thể trong lĩnh vực học máy thống kê.

Để hoàn thành giáo trình này, chúng tôi xin chân thành cảm ơn các ý kiến đóng góp, sự giúp đỡ động viên của đồng nghiệp tại Bộ môn Khoa học máy tính, Khoa Công nghệ thông tin và tại Viện Trí tuệ nhân tạo, Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội.

Giáo trình chắc chắn không thể tránh khỏi những thiếu sót và hạn chế. Chúng tôi rất mong nhận được sự góp ý và phản hồi từ bạn đọc để giáo trình ngày càng hoàn thiện hơn. Chúng tôi xin trân trọng cảm ơn.

Các tác giả.

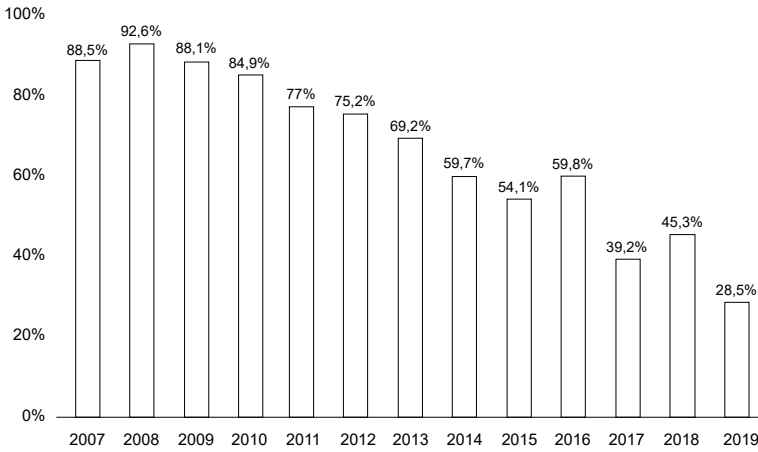
Chương 1

Giới thiệu

Trong chương này, chúng ta sẽ tiếp cận khái niệm Học máy từ những góc nhìn cơ bản nhất, bao gồm: học máy là gì, phạm vi ứng dụng và các tình huống áp dụng thực tế. Chương này làm bước đệm để người học nắm bắt và kết nối được với toàn bộ nội dung của cuốn sách, từ bài toán phân lớp (Chương 2) đến các mô hình nơ-ron phức tạp (Chương 3, 4, 5) và các hướng nâng cao như mô hình sinh (Chương 11) hay mô hình đồ thị xác suất (Chương 12).

1.1 Học máy là gì ?

Học máy là một ngành khoa học về các thuật toán và các chương trình máy tính tự động cải tiến thông qua kinh nghiệm. Trong đó, các thuật toán học máy được thiết kế để phân tích dữ liệu, nhận biết mẫu và học từ kinh nghiệm mà không cần phải lập trình cụ thể. Học máy là một phần của trí tuệ nhân tạo và liên quan với việc phát triển các mô hình dự đoán và quyết định. Một số ví dụ của học máy bao gồm:



Hình 1.1: Tỷ lệ băng thông email là thư rác.

Ví dụ 1.1 (Phân loại thư rác). Các chương trình quản lý thư điện tử hiện nay không thể thiếu mô-đun phân loại và lọc thư rác do số lượng thư rác quá lớn. Hình 1.1 cho thấy tỷ lệ băng thông dành cho thư rác giảm qua nhiều năm liên tục, một phần là do các mô-đun lọc thư rác ngày càng hiệu quả. Để xây dựng mô-đun lọc thư rác, thuật toán lọc thư quan sát và học từ một tập dữ liệu lớn gồm hàng triệu thư điện tử được gán nhãn là thư thường hoặc thư rác. Thuật toán tự điều chỉnh để tìm cách mô tả được mối quan hệ giữa nội dung thư với nhãn của nó. Sau khi học xong, thuật toán được sử dụng để phân loại, lọc các thư điện tử mới không có trong tập dữ liệu nói trên. Điểm đặc biệt là độ chính xác khi phân loại thư ngày càng tăng nếu tập dữ liệu ban đầu lớn hơn.

Ví dụ 1.2 (Phân loại chữ số viết tay). Thư từ, bưu phẩm trong hệ thống bưu điện của Mỹ được phân loại theo mã số bưu điện của từng vùng. Khi gửi bưu phẩm, người gửi ghi mã số bưu điện vào các ô được định sẵn (Hình 1.2). Khi nhận bưu phẩm, các bưu trạm sử dụng hệ thống đọc chữ số viết tay tự động để phân luồng, phân phối bưu phẩm do số lượng bưu phẩm hàng ngày của nước Mỹ rất

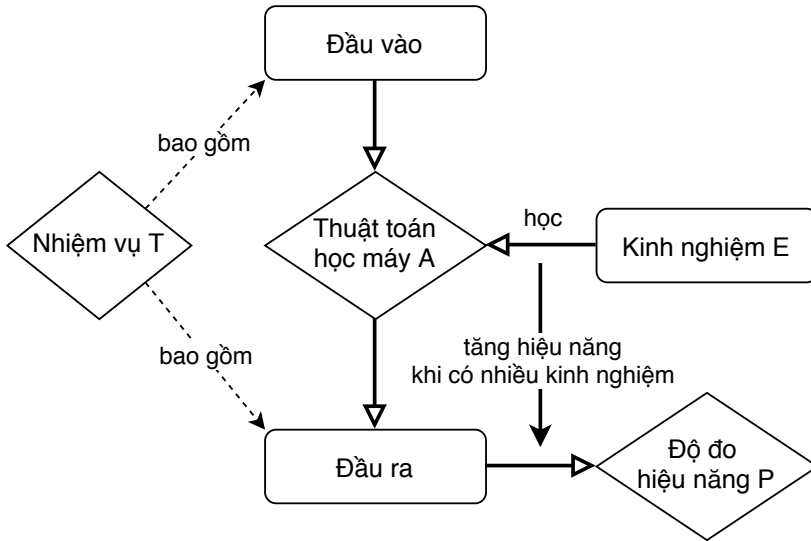


Hình 1.2: Bộ dữ liệu chữ số viết tay MNIST.

lớn (6,2 tỉ bưu phẩm vào năm 2019). Bộ dữ liệu chữ số viết tay MNIST được thu thập và công bố bởi Viện Tiêu chuẩn và Công nghệ của Mỹ bao gồm 70.000 ảnh chữ số viết tay kích thước 28x28, được gán nhãn từ 0 đến 9 là bộ dữ liệu “Hello World” dành cho ngành Học máy. Nhiều mô hình Học máy khi phát minh ra được đánh giá bởi bộ dữ liệu MNIST.

Qua các ví dụ trên, ta thấy để giải quyết một nhiệm vụ hoặc một vấn đề bằng cách tiếp cận Học máy, trước tiên, cần định nghĩa nhiệm vụ T bằng cách chỉ rõ đầu vào, đầu ra cần có của thuật toán; sau đó, chọn một độ đo hiệu năng P cho thuật toán cần tìm, độ đo này nói chung không phụ thuộc vào thuật toán cụ thể mà chỉ phụ thuộc vào đầu ra của thuật toán cùng đầu ra đích mong muốn; cuối cùng, xây dựng bộ dữ liệu hoặc kinh nghiệm E từ những hoạt động lịch sử của nhiệm vụ đó để thuật toán học (Hình 1.3).

Định nghĩa 1.3 (Thuật toán Học máy - Tom Mitchell). Thuật toán $\mathcal{A}$ được coi là học từ kinh nghiệm E đối với lớp bài toán (hoặc lớp nhiệm vụ) T và độ đo hiệu năng P nếu hiệu năng đo bằng P của $\mathcal{A}$ đối với nhiệm vụ trong T tăng với kinh nghiệm E .



Hình 1.3: Giải quyết vấn đề bằng cách tiếp cận Học máy.

Thuật toán $\mathcal{A}$ khi huấn luyện trên kinh nghiệm E cho ta mô hình học máy $\mathcal{A}(E)$ đã được huấn luyện. Mục tiêu của ngành Học máy có thể chia ra làm ba hướng chính: nghiên cứu cách tiếp cận học máy cho các lớp nhiệm vụ T ; phát triển các thuật toán học máy $\mathcal{A}$; và đánh giá chất lượng, hiệu năng của mô hình học máy $\mathcal{A}(E)$ đã được huấn luyện.

1.2 Ứng dụng của Học máy

Trong những năm gần đây và trong tương lai gần, Học máy đã, đang và sẽ có ứng dụng trong nhiều ngành nghề, đi vào mọi mặt đời sống của con người. Học máy hiện là phân ngành mũi nhọn của ngành học về Trí tuệ nhân tạo. Liên tục có hàng loạt nghiên cứu, công nghệ mới về Học máy được áp dụng ở quy mô công nghiệp. Trong phần này, chúng ta cùng xem xét một số ứng dụng nổi bật của Học máy đã làm thay đổi tư duy, công việc và lối sống của con người.

- *Y tế và chăm sóc sức khỏe.* Học máy đã sản sinh nhiều kỹ thuật và công cụ trong chẩn đoán và tiên lượng tình hình sức khỏe, bệnh tật trong lĩnh vực y tế. Dự đoán diễn biến của bệnh - dịch bệnh, trích xuất tri thức y khoa từ kết quả nghiên cứu lâm sàng, hỗ trợ điều trị tâm lý và quản lý bệnh nhân nói chung là những lĩnh vực y tế đã cho thấy ứng dụng của Học máy. Ngoài ra, Học máy còn phân tích hồ sơ bệnh án, xử lý mất mát thông tin, giám sát các thông số từ cảm biến tại phòng chăm sóc đặc biệt để cảnh báo.
- *Giao thông.* Học máy đang được sử dụng hàng ngày khi chúng ta sử dụng các ứng dụng bản đồ như Google Maps để di chuyển. Học máy xác định và dự đoán tình hình giao thông giúp người dùng tránh các cung đường đông đúc và đến đích đúng giờ. Học máy sử dụng thông tin vị trí và vận tốc của hàng triệu người dùng đang bật định vị Hệ thống định vị toàn cầu - GPS để hình thành bản đồ giao thông và phân tích, dự đoán các vị trí có thể tắc nghẽn. Ngoài ra, các hãng xe chia sẻ như Uber, Grab sử dụng thông tin vị trí xe và vị trí người đặt xe làm đầu vào cho mô hình Học máy dùng để xếp lịch, lập kế hoạch cung đường, thời điểm đến đích cũng như giá từng cuộc xe.
- *Mạng xã hội* Hầu hết chúng ta đều có một hoặc nhiều tài khoản mạng xã hội. Mạng xã hội đem lại niềm vui và có thể gây nghiện. Mạng xã hội giúp mọi người gần nhau hơn trong đời sống số, chia sẻ với nhau về công nghệ, những kỹ năng thú vị, tin tức nóng nổi và hơn hết giữ liên lạc với những người chúng ta muốn. Học máy đóng vai trò cốt lõi trong việc phát triển các nền tảng mạng xã hội. Tính năng *Giới thiệu bạn bè* sử dụng Học máy để dự đoán hai người có thân quen với nhau trong cuộc sống thật hay không thông qua các mối liên hệ giữa hai người và những người quen chung, các sở thích, các nhóm và nơi làm việc chung. Tính năng *Nhận dạng khuôn mặt* sử dụng mô hình Học máy

để đánh dấu các khuôn mặt xuất hiện trong ảnh, nhận dạng người xuất hiện trong các bức ảnh và tự động thông báo cho họ về sự xuất hiện của họ trên mạng xã hội. Đối với người dùng, quá trình này nhìn qua tưởng chừng rất trơn tru và thân thiện, nhưng toàn bộ quá trình kỹ thuật và hệ thống phục vụ phía sau của tính năng này cực kì phức tạp.

- *Trợ lý ảo thông minh.* Chỉ với số tiền không quá lớn, hiện nay chúng ta đã có thể sử dụng một ứng dụng trợ lý ảo như Siri, Google Assistants, Alexa và rất nhiều thiết bị khác. Điện thoại thông minh như Pixel, iPhone và loa thông minh như Echo và Google Home cũng không quá đắt đỏ. Samsung cũng đang đưa trợ lý ảo Bixby lên tivi của hãng. Nhiệm vụ của trợ lý ảo là hỗ trợ “ông/bà chủ” (chúng ta) trong mọi công việc hàng ngày và chúng ta chỉ cần ra lệnh. Trợ lý ảo hỗ trợ đặt lịch hẹn, báo thức, cập nhật tin tức. Để làm được các công việc đó, trợ lý ảo phải hiểu ngôn ngữ tự nhiên ở cả dạng văn bản và dạng âm thanh. Chúng phải trả lời các câu hỏi như "Giá đặt một bát phở bò ở gần đây thế nào?", "Thời tiết ngày mai ở Thành phố Hồ Chí Minh ra sao?". Một loạt các kỹ thuật và công nghệ Học máy đã được áp dụng trong trợ lý ảo để xử lý những yêu cầu như vậy của “ông/bà chủ”.
- *Tối ưu kết quả tìm kiếm.* Các máy tìm kiếm như Google, Bing đều dùng Học máy để cải thiện kết quả tìm kiếm. Các thuật toán Học máy cá nhân hoá quan sát các sở thích của từng cá nhân để sắp xếp lại kết quả tìm kiếm trước khi hiển thị cho người dùng. Khi kết quả tìm kiếm phù hợp và có ích, người dùng sẽ nán lại lâu hơn trên một trang web. Toàn bộ thông tin này được lưu lại và học để lần sau kết quả tìm kiếm sẽ càng sát với ý định của người dùng. Ngược lại, nếu người dùng không ấn vào các đường dẫn thứ 1, thứ 2 mà chọn đường dẫn thứ 4, thứ 5 trong danh sách, ngay lập tức, máy tìm kiếm hiểu là mô hình dự đoán thứ

tự đường dẫn của nó cần phải cập nhật lại để lần sau phục vụ tốt hơn.

- *Giám sát - an ninh.* Đối với một người hoặc một nhóm nhân viên an ninh, việc theo dõi màn hình của hàng trăm camera liên tục truyền tải hình ảnh về trung tâm an ninh là việc làm cực kì mệt mỏi và buồn tẻ. Tương tự như vậy, với tổ chức và doanh nghiệp hoạt động dựa trên uy tín, việc giám sát thông tin trên mạng xã hội có ý nghĩa sống còn nhưng khả năng giám sát hàng trăm trang web, hàng triệu người dùng, bài viết bằng tay và mắt người cực kì hạn chế.

Các thuật toán Học máy ngày nay đã có khả năng thực hiện các nhiệm vụ giám sát camera, mạng xã hội để phát hiện ra các hành vi không mong muốn và tự động cảnh báo, giảm công sức giám sát bằng người và giảm thiệt hại cho cộng đồng, tổ chức.

Trong hệ thống thanh toán, ngân hàng, các thuật toán Học máy đã giúp cảnh báo các giao dịch giả mạo, làm không gian số trong sạch và tạo niềm tin cho con người giao dịch trên đó. Các hệ thống thanh toán ngày nay có một loạt các công cụ giúp giám sát giao dịch, phân biệt giữa giao dịch hợp lệ và không hợp lệ.

- *Chăm sóc khách hàng.* Bạn đã từng thấy cửa sổ chat bật lên mỗi khi bạn ghé thăm website của một cửa hàng? Khả năng cao đó là một tác tử hội thoại tự động được lập trình bởi các kỹ thuật Học máy. Những tác tử hội thoại này đóng vai trò nhân viên chăm sóc khách hàng, có nhiệm vụ giúp đỡ, trả lời các câu hỏi của khách.

Bằng khả năng phân tích ngôn ngữ và khả năng truy xuất vào cơ sở dữ liệu của cửa hàng, tác tử hội thoại có thể trả lời khá chính xác yêu cầu của khách, tăng đáng kể xác suất mua hàng và sự hài lòng của khách hàng.

- *Gợi ý sản phẩm.* Thương mại điện tử đang thống lĩnh thị trường bán lẻ trong những năm gần đây do tính tiện lợi cũng như sự đa dạng của hàng hoá, đặc biệt là có nhiều khuyến mại hấp dẫn về giá cả và giao hàng tận nhà. Dễ thấy trên các trang web thương mại điện tử, mỗi khi chúng ta xem một sản phẩm, lập tức ứng dụng web hiển thị các mặt hàng tương tự, các gói sản phẩm đi kèm, cũng như gợi ý sản phẩm mỗi khi chúng ta quay lại trang web đó.

Các thuật toán Học máy gợi ý sản phẩm đăng sau các website như vậy đã phân tích hành vi duyệt web, hành vi mua bán trong lịch sử, sở thích, giỏ hàng đã mua của hàng triệu người dùng để dự đoán hành vi tiếp theo và cá nhân hoá dự đoán này đến từng người dùng.

1.3 Các tình huống áp dụng Học máy

Học máy là công cụ rất mạnh để giải quyết nhiều vấn đề như các mục trước đã đề cập. Tuy nhiên, như mọi công cụ mà con người đã tạo ra, Học máy chỉ nên áp dụng ở những ngữ cảnh, tình huống nhất định chứ không nên áp dụng Học máy một cách mù quáng cho tất cả các vấn đề. Trong mục này, chúng ta xem xét một số tình huống thực tế mà áp dụng cách tiếp cận Học máy được khuyến nghị:

- *Nhiệm vụ quá phức tạp để lập trình tương minh:* Khi lập trình bình thường, chúng ta hay sử dụng các lệnh rẽ nhánh (if - else) nhằm mô tả các luật dùng để ra quyết định. Khi số lượng luật bùng nổ, có quá nhiều trường hợp cần xét vượt quá khả năng lập trình, lúc đó ta nên nghĩ đến việc áp dụng Học máy. Mô hình Học máy sẽ tự động tìm ra quy luật của việc ra quyết định hoặc dự đoán. Như ví dụ về phân loại thư rác ở mục trước, rất khó để ta có thể liệt kê tất cả các luật xác định thư rác do từ vựng của mỗi ngôn ngữ có thể lên đến hàng trăm nghìn từ. Tình huống

này cũng đúng khi phải phân tích các mẫu dữ liệu có hàng nghìn hoặc hàng triệu thông tin như các thuật toán đánh giá sở thích người dùng để hiển thị quảng cáo trên website. Các thông tin đơn lẻ này không những nhiều mà còn có thể có mối tương quan với nhau mà không một chương trình nào có thể mô tả bằng các lệnh rẽ nhánh trực tiếp. Khi đó sử dụng một mô hình Học máy là lựa chọn hợp lý.

- *Nhiệm vụ thay đổi thường xuyên*: Một số bài toán thường xuyên thay đổi theo thời gian khiến cho chương trình phải cập nhật liên tục. Ví dụ như khi phân tích một trang web để tìm ra giá cả của sản phẩm. Nếu lập trình tường minh, chúng ta sẽ tìm các thẻ HTML chứa giá của sản phẩm. Tuy nhiên, cách tiếp cận này gặp phải vấn đề khi trang web cập nhật hoặc được thiết kế lại, thẻ HTML ban đầu có thể không còn đúng. Khi đó một thuật toán Học máy đã được huấn luyện bằng hàng triệu trang web sản phẩm, từ hàng nghìn website có thể sẽ tự động đoán được giá sản phẩm trên trang web mới.
- *Nhiệm vụ là vấn đề nhận thức*: Ngày nay, khi làm việc với văn bản, ảnh, âm thanh hoặc phim, rất khó để không dùng Học máy. Các mô hình Học máy hiện đại gần đây như mạng nơ-ron sâu đã có khả năng thực hiện các nhiệm vụ nhận thức như: phân loại hình ảnh, phát hiện vị trí vật thể, phân vùng ngữ nghĩa trên ảnh, phân tích cấu trúc văn bản, phát hiện thực thể trong văn bản, trả lời hội thoại, dịch văn bản, nhận dạng âm thanh và tổng hợp âm thanh. Các mô hình Học máy đã thực hiện các nhiệm vụ nhận thức trên ở mức độ xấp xỉ khả năng của con người, trong đó đã vượt qua con người ở một số nhiệm vụ trong các đánh giá khách quan. Do đó, khi phải thực hiện một nhiệm vụ nhận thức trên các loại dữ liệu trên, hãy thử cách tiếp cận bằng Học máy.
- *Nhiệm vụ chưa được nghiên cứu kỹ lưỡng*: Với các hiện tượng

chưa được khoa học giải thích kỹ nhưng lại có dữ liệu, sử dụng mô hình Học máy có thể tìm ra các quy luật, các mối tương quan trong dữ liệu, hỗ trợ ra quyết định. Hiện có khá nhiều nghiên cứu sử dụng Học máy để khai thác y văn, các công trình khoa học trong y tế nhằm tìm ra các quy luật trong điều trị lâm sàng. Các bác sĩ khó có thể đọc được hàng nghìn hoặc hàng trăm nghìn bài báo về các phương pháp điều trị, các đánh giá trên người bệnh và tổng hợp lại. Đây chính là sức mạnh của Học máy, tìm kiếm quy luật trong dữ liệu lớn. Một ví dụ khác là việc bảo đảm an ninh mạng. Mỗi ngày, số lượng thông tin lưu trữ của mạng rất lớn, bao gồm thông tin của từng thành phần trong mạng và thông tin giữa các thành phần với nhau. Con người khó có thể phân tích chi tiết tất cả các thông tin như vậy để phát hiện ra các cuộc tấn công vào hệ thống. Đặc biệt, tin tặc thường xuyên nghĩ ra những cách tấn công mới, chưa từng có. Trong trường hợp này, các mô hình Học máy mô tả được trạng thái bình thường của hệ thống sẽ rất nhạy trong việc phát hiện ra những tình huống bất thường và cảnh báo được nguy cơ tấn công.

- *Nhiệm vụ có mục tiêu đơn giản*: Học máy ngày nay đặc biệt tốt trong trường hợp đầu ra cần đạt của nhiệm vụ là các câu trả lời Có hoặc Không hoặc một con số. Học máy chưa tốt trong các tình huống phải đưa ra quyết định mà không có đầy đủ thông tin cũng như phải lập kế hoạch trong dài hạn.

Khi quyết định lựa chọn cách tiếp cận Học máy để giải quyết một bài toán thực tế, chúng ta cũng cần tính đến các chi phí khi sử dụng Học máy, bao gồm: chi phí thu thập, chuẩn bị và làm sạch dữ liệu; chi phí huấn luyện mô hình; và chi phí triển khai, sử dụng, giám sát và bảo trì mô hình. Trong các nhóm chi phí trên, việc thu thập, chuẩn bị và làm sạch dữ liệu liên quan đến nhân lực

gán nhãn và xử lý dữ liệu. Việc huấn luyện và triển khai sử dụng mô hình yêu cầu nhân lực được đào tạo chuyên sâu về Học máy và một số máy móc đặc biệt có thể khá đắt đỏ. Việc giám sát và bảo trì mô hình cũng đòi hỏi nguồn lực để liên tục duy trì sự hoạt động của mô hình. Do đó, ngay cả khi có thể áp dụng Học máy, chúng ta cũng cần tính đến *hiệu quả chi phí* của việc vận hành một hệ thống như vậy.

1.4 Các phương pháp và bài toán trong Học máy

Các bài toán cơ bản trong Học máy được phân thành ba loại chính: Học có giám sát, Học không giám sát và Học tăng cường. Mỗi loại bài toán này đều có những đặc điểm riêng và yêu cầu các phương pháp, mô hình khác nhau. Cách chia này dựa trên thông tin mà mô hình Học máy nhận được từ dữ liệu. Chúng ta sẽ tìm hiểu kỹ hơn về mỗi loại bài toán này trong các mục sau.

1.4.1 Học có giám sát

Đây là bài toán Học máy yêu cầu xây dựng mô hình mô tả mối quan hệ giữa đầu vào và đầu ra dựa trên dữ liệu là một tập hợp các cặp đầu vào - đầu ra. Đầu ra đóng vai trò hướng dẫn, giám sát của người thầy đối với mô hình hoặc thuật toán Học máy. Bài toán Học có giám sát thường được phát biểu dưới dạng:

Định nghĩa 1.4 (Học có giám sát).

Cho tập dữ liệu $\mathcal{D} = \{(x_i, y_i)\}, i = 1, 2, \dots, n$, trong đó $x_i \in \mathcal{X}$ là tập đầu vào, $y_i \in \mathcal{Y}$ là tập đầu ra, n là kích thước của dữ liệu $\mathcal{D}$, hãy tìm một hàm $f : \mathcal{X} \rightarrow \mathcal{Y}$ xấp xỉ mối quan hệ giữa đầu vào và đầu ra được chỉ ra trong tập dữ liệu $\mathcal{D}$, đánh giá bởi độ đo hiệu suất $P(f)$.

Ví dụ 1.5 (Phân loại thư rác theo tiêu đề). Khi $\mathcal{X}$ là tập các tiêu đề thư điện tử, $\mathcal{Y} = \{\text{thư rác}, \text{thư thường}\}$, ta có bài toán Học phân

Bảng 1.1: Ví dụ về bộ dữ liệu trong bài toán phân loại thư rác theo tiêu đề

Chỉ số	Tiêu đề	Phân loại
1	Cuộc họp vào thứ 6 tuần này	Thư thường
2	Giảm giá 50% cho tất cả sản phẩm	Thư rác
3	Thông báo về việc tăng lương	Thư thường

lớp thư rác - thư thường dùng trong các chương trình quản lý thư điện tử như Outlook, Thunderbird, Gmail. Bảng 1.1 là một ví dụ về bộ dữ liệu trong bài toán phân loại thư rác theo tiêu đề có số lượng mẫu $n = 3$. Cụ thể với mẫu thứ nhất, đầu vào x_1 là tiêu đề thư điện tử “Cuộc họp vào thứ 6 tuần này” và đầu ra y_1 là nhãn “thư thường”.

Ví dụ 1.6 (Dự đoán giá). Khi $\mathcal{X}$ là giá cổ phiếu trong lịch sử tính trong khoảng thời gian nhất định như trong 1 ngày hoặc 1 tuần. Khi đó, giá trị mục tiêu $\mathcal{Y} = \mathbb{R}^+$ sẽ là giá cổ phiếu tại thời điểm sắp tới. Với thiết lập này, ta có bài toán Học dự đoán giá cổ phiếu tương lai.

Khi $\mathcal{Y}$ là tập hợp rời rạc, ta nói bài toán Học máy là *bài toán phân lớp*. Khi $\mathcal{Y}$ là tập số thực $\mathcal{R}$, ta nói bài toán học máy là *bài toán hồi quy*.

Hiệu suất $P(f)$ trong bài toán phân lớp (Chương 2) thường là số lỗi trung bình trên tập dữ liệu $\mathcal{D}$, còn được gọi là *hàm lỗi 0-1* và được định nghĩa như công thức (1.1).

$$P(f) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(f(x_i) \neq y_i). \quad (1.1)$$

Trong định nghĩa trên, $\mathbb{I}(\cdot)$ là *hàm chỉ báo* được định nghĩa như sau:

$$\mathbb{I}(p) = \begin{cases} 1 & \text{nếu } p \text{ đúng,} \\ 0 & \text{nếu } p \text{ sai.} \end{cases}$$

Ví dụ, nếu ta tính được giá trị lỗi $P(f) = 0,1$, ta nói hàm f chính xác 90% trên tập dữ liệu $\mathcal{D}$.

Hiệu suất $P(f)$ trong bài toán hồi quy (Chương 7) được tính bằng *trung bình bình phương sai số* MSE (mean squared error) trên tập dữ liệu $\mathcal{D}$ và được định nghĩa như công thức (1.2).

$$P(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2. \quad (1.2)$$

1.4.2 Học không giám sát

Đây là bài toán Học máy yêu cầu xây dựng mô hình mô tả hoặc trích xuất quy luật của dữ liệu. Đặc điểm của các bài toán học không giám sát là không có dữ liệu về đầu ra mong muốn (tập $\mathcal{Y}$). Thuật toán, mô hình học không giám sát cần hiểu dữ liệu mà không cần thông tin về đầu ra. Có ba loại bài toán chính trong Học không giám sát, bao gồm:

- *Phân cụm* (Chương 11): Tìm kiếm các nhóm, các cụm “gần nhau” trong dữ liệu;
- *Ước lượng mật độ* (Chương 11): Ước lượng phân bố xác suất của dữ liệu.
- *Trích xuất đặc trưng* (Chương 10): Tìm kiếm các đặc trưng quan trọng, các biểu diễn ẩn của dữ liệu.

Trong thiết lập không cần dữ liệu mục tiêu để học, việc triển khai các thuật toán Học không giám sát thường dựa trên các giả định về cấu trúc của dữ liệu hoặc các quy luật tiềm ẩn trong dữ liệu. Vì thế với mỗi bài toán, chúng ta lại có những độ đo riêng để đánh giá hiệu suất của mô hình Học không giám sát. Ví dụ, trong bài toán phân cụm, chúng ta có thể sử dụng độ đo về khoảng cách trung bình tương đối của các cụm để đánh giá chất lượng phân

cụm. Trong bài toán ước lượng mật độ, chúng ta có thể sử dụng độ đo độ thích hợp để ước lượng phân bố xác suất của dữ liệu. Ngoài ra, trong bài toán trích xuất đặc trưng chúng ta cũng có thể sử dụng các độ đo sai khác với dữ liệu đầu vào. Các chương 10 và 11 sẽ đề cập chi tiết hơn về các bài toán Học không giám sát cũng như các độ đo liên quan.

1.4.3 Học tăng cường

Học tăng cường là lĩnh vực Học máy học cách điều khiển các tác tử trong một môi trường sử dụng phản hồi hoặc điểm thưởng của môi trường đối với hành động của tác tử. Học tăng cường (Chương 8) được áp dụng khi không có một tập dữ liệu huấn luyện cố định sẵn mà hiệu suất được mô tả dưới dạng mục tiêu như điểm thưởng hoặc trạng thái thắng - thua. Mục tiêu chính của bài toán Học tăng cường là tìm cách điều khiển tác tử để nhận được điểm thưởng lớn nhất dựa trên thông tin và kinh nghiệm nhận được khi tác tử tương tác với môi trường. Có thể nói Học tăng cường rất gần với Học có giám sát, tuy nhiên các phản hồi trong Học tăng cường có độ trễ hoặc bị nhiễu. Ví dụ khi ta chơi cờ vua, và thực hiện 1 nước đi nào đó, thì kết quả của nước đi này là thắng hoặc thua có thể xảy ra trong nhiều bước đi tiếp theo.

Bên cạnh đó khái niệm “kinh nghiệm” trong Học tăng cường cũng tách biệt so với dữ liệu huấn luyện trong Học có giám sát và Học không giám sát. Kinh nghiệm của tác tử được cập nhật liên tục qua mỗi bước tương tác với môi trường và có tính phụ thuộc thời gian. Các hành động của tác tử dựa trên mô hình học máy tại một thời điểm xác định đều có ảnh hưởng đến tập kinh nghiệm có thể nhận được trong tương lai. Điều này tạo ra một vòng lặp phức tạp giữa hành động – kinh nghiệm – mô hình Học máy. Các thuật toán Học tăng cường phải xác định được mối quan hệ nhân quả giữa hành động và kinh nghiệm để tối ưu hóa hiệu suất của tác tử.

1.4.4 Các bài toán lai

1.4.4.1 Học bán giám sát

Đây là bài toán Học máy khi chúng ta có cả dữ liệu có nhãn và dữ liệu chưa gán nhãn. Mục tiêu của các thuật toán học bán giám sát là tận dụng tất cả dữ liệu đang có, kể cả khi chưa được gán nhãn. Các tiếp cận này đặc biệt hữu dụng bởi lượng dữ liệu chưa được gán nhãn lớn hơn nhiều so với lượng dữ liệu đã được gán nhãn. Điều này đúng đối với dữ liệu ảnh, dữ liệu văn bản và dữ liệu âm thanh trong trường hợp việc gán nhãn dữ liệu rất đắt đỏ. Về mặt cách tiếp cận, Học bán giám sát có thể sử dụng các kỹ thuật Học không giám sát để hình thành quy luật của dữ liệu, sau đó kết hợp với việc sử dụng Học có giám sát để gán nhãn dữ liệu dựa trên quy luật đã tìm được. Hiện nay cách tiếp cận này rất phổ biến trong các mô hình học sâu, đặc biệt là trong các mô hình ngôn ngữ lớn.

1.4.4.2 Học tự giám sát

Đây là cách tiếp cận trong Học không giám sát nhưng sử dụng phương pháp Học có giám sát. Kỹ thuật thường dùng ở đây là sử dụng chính dữ liệu đầu vào x để chế tạo ra đầu ra $y = f(x)$ (trong trường hợp đơn giản nhất, có thể $y = x$). Sau đó sử dụng Học có giám sát trên tập dữ liệu gồm các cặp (x, y) đã được tạo ra. Như vậy, mô hình học tự giám sát sẽ học được một biểu diễn mới của x có tính bất biến với một số biến đổi được thiết kế bởi biến đổi f .

Ví dụ 1.7 (Tô màu ảnh đen trắng). Giả sử ta có một tập ảnh màu $I_1, \dots, I_n$. Ta thiết kế dữ liệu như sau $\mathcal{D} = \{(x_i = \text{Gray}(I_i), y_i = I_i)\}, i = 1, \dots, n$. Trong đó $\text{Gray}(\cdot)$ là hàm biến đổi ảnh màu thành ảnh đen trắng. Nếu áp dụng một thuật toán Học có giám sát lên tập dữ liệu $\mathcal{D}$, ta sẽ thu được một mô hình có khả năng nhận vào một ảnh đen trắng và tô màu ảnh đó.

1.4.5 Các kỹ thuật thường dùng trong Học máy

1.4.5.1 Học đa nhiệm vụ

Đây là kỹ thuật Học máy có giám sát xây dựng mô hình Học máy giải quyết nhiều nhiệm vụ cùng một lúc. Nghĩa là, với cùng một đầu vào, mô hình tính toán ra nhiều đầu ra và được đánh giá hiệu suất trên tất cả các đầu ra này. Nói một cách hình thức thì đầu ra $y = (y_1, y_2, \dots, y_K) \in \mathcal{Y} = \mathcal{Y}_1 \times \mathcal{Y}_2 \times \dots \times \mathcal{Y}_K$ với $\mathcal{Y}_k, k = 1, \dots, K$ là tập đầu ra của nhiệm vụ thứ k . Người ta hy vọng rằng, khi các nhiệm vụ có mối liên hệ nào đó thì cách giải quyết chúng cũng phải có điểm chung. Do vậy, việc huấn luyện mô hình giải quyết đồng thời nhiều nhiệm vụ sẽ cho kết quả tốt hơn khi huấn luyện mô hình với từng nhiệm vụ đơn lẻ. Một tình huống rất hay xảy ra khi giải quyết các nhiệm vụ là một nhiệm vụ có rất nhiều dữ liệu đã được gán nhãn còn các nhiệm vụ khác lại có rất ít dữ liệu được gán nhãn. Khi đó, Học đa nhiệm vụ sẽ sử dụng được nhiều dữ liệu gán nhãn hơn, giúp cải thiện hiệu suất các nhiệm vụ có ít dữ liệu. Vấn đề khó khăn chính của việc Học đa nhiệm vụ so với cách Học có giám sát thông thường là làm sao để các nhiệm vụ không can thiệp lẫn nhau, tức là không làm giảm hiệu suất của các nhiệm vụ khác.

1.4.5.2 Học chủ động

Đây là kỹ thuật Học máy cho phép mô hình đưa ra yêu cầu gán nhãn hoặc làm rõ thông tin về những dữ liệu mà mô hình còn cảm thấy mơ hồ, khó đưa ra dự đoán. Với kỹ thuật này, mô hình được huấn luyện với ít mẫu dữ liệu cần được gán nhãn hơn. Mô hình chỉ đưa ra yêu cầu gán nhãn khi thấy cần thiết, do đó, giảm chi phí và thời gian gán nhãn mà vẫn huấn luyện được mô hình có hiệu suất cao. Trong giáo trình này chúng tôi không đi sâu vào kỹ thuật này, tuy nhiên, người học có thể tìm hiểu thêm trong tài liệu ở phần Tài liệu tham khảo ở cuối chương.

1.4.5.3 Học trực tuyến

Đây là kỹ thuật huấn luyện, cập nhật tham số của mô hình học máy ngay khi có dữ liệu mới. Hình thức học trực tuyến rất có ích khi dữ liệu đến theo luồng hoặc phân bố của dữ liệu thay đổi theo thời gian. Do đó, mô hình cũng phải thay đổi thường xuyên để đảm bảo hiệu suất khi có thay đổi trong dữ liệu. Để đánh giá thuật toán học trực tuyến, ta thường dùng đại lượng “độ hối tiếc” (regret) là hiệu số giữa hiệu suất thực tế khi học trực tuyến và hiệu suất khi biết trước toàn bộ dữ liệu. Mục tiêu của việc huấn luyện là giảm độ hối tiếc nhỏ nhất có thể. Ví dụ, đối với bài toán phân lớp

$$R = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(f_i(x_i) \neq y_i) - \min_f \frac{1}{n} \sum_{i=1}^n \mathbb{I}(f(x_i) \neq y_i). \quad (1.3)$$

Trong đó $\min_f \frac{1}{n} \sum_{i=1}^n \mathbb{I}(f(x_i) \neq y_i)$ là số lỗi trung bình tốt nhất có thể nếu ta biết trước toàn bộ tập dữ liệu còn $\frac{1}{n} \sum_{i=1}^n \mathbb{I}(f_i(x_i) \neq y_i)$ là số lỗi trung bình thực tế trong quá trình cập nhật mô hình trực tuyến $f_1 \rightarrow f_2 \rightarrow \dots \rightarrow f_n$.

1.4.5.4 Học chuyển đổi

Học chuyển đổi (Mục 5.6) là kỹ thuật huấn luyện mô hình trên một nhiệm vụ, sau đó mô hình được huấn luyện tiếp cho một nhiệm vụ đích khác. Cách huấn luyện này cho phép xây dựng mô hình cho nhiệm vụ đích với ít dữ liệu có sẵn nhưng tận dụng tối đa các nguồn dữ liệu của nhiệm vụ khác có liên quan đến nhiệm vụ đích. Học chuyển đổi khác Học đa nhiệm vụ ở chỗ mô hình cho từng nhiệm vụ được huấn luyện tuần tự. Ngoài ra, khi chuyển đổi từ nhiệm vụ này sang nhiệm vụ khác, mô hình có thể được thay đổi đôi chút để phù hợp với nhiệm vụ mới nhưng vẫn giữ phần lớn các cấu trúc mô hình của nhiệm vụ cũ. Học chuyển đổi là kỹ thuật được áp dụng phổ biến, đặc biệt cho các mô hình Học sâu.

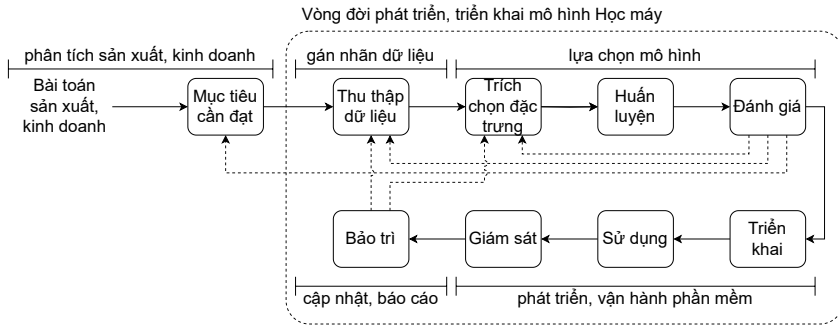
1.4.5.5 Học kết hợp

Học kết hợp (Chương 9) là kỹ thuật kết hợp nhiều mô hình để đưa ra quyết định có hiệu suất cao hơn quyết định của từng mô hình đơn lẻ. Học kết hợp bao gồm hai bước: (i) xây dựng các mô hình cơ sở, (ii) kết hợp quyết định của các mô hình cơ sở. Học kết hợp là kỹ thuật Học máy rất hiệu quả, giúp tăng hiệu suất và giảm biến động khi ra quyết định. Khi áp dụng một thuật toán Học máy vào thực tế, người ta thường tính đến cách tiếp cận Học kết hợp để tăng hiệu suất của thuật toán Học máy đó.

1.5 Quá trình phát triển và triển khai mô hình học máy

Một vòng đời phát triển và triển khai mô hình học máy được mô tả trong Hình 1.4, bao gồm các bước chính:

1. *Thu thập và chuẩn bị*: Dữ liệu được thu thập, lưu trữ và làm sạch.
2. *Trích chọn đặc trưng dữ liệu*: Tìm kiếm, trích xuất các đặc tính quan trọng từ dữ liệu thu thập được.
3. *Huấn luyện mô hình*: Dữ liệu được chia thành hai phần: một phần dùng để huấn luyện mô hình, một phần dùng để đánh giá hiệu năng của mô hình. Mô hình được huấn luyện trên dữ liệu huấn luyện, sau đó được đánh giá hiệu năng trên dữ liệu đánh giá.
4. *Đánh giá mô hình*: Mô hình được đánh giá hiệu năng dựa trên các tiêu chí như độ chính xác, độ hài lòng, thời gian chạy, độ phức tạp, độ tin cậy, độ ổn định.
5. *Triển khai mô hình*: Mô hình được triển khai vào môi trường thực tế, sẵn sàng phục vụ người dùng. Trong quá trình triển



Hình 1.4: Quá trình phát triển, triển khai mô hình Học máy.

khai, cần chú ý đến các vấn đề như hiệu năng, bảo mật và tính ổn định của mô hình.

6. *Sử dụng mô hình*: Mô hình được sử dụng để giải quyết các vấn đề thực tế, đưa ra dự đoán, phân loại, gợi ý, tối ưu hóa, phân cụm, phân tích dữ liệu.
7. *Giám sát mô hình*: Mô hình cần được giám sát thường xuyên theo các tiêu chí đề ra ban đầu của mô hình. Các vấn đề xảy ra trong quá trình sử dụng mô hình cần được phát hiện và xử lý kịp thời.
8. *Bảo trì mô hình*: Mô hình cần được bảo trì, cập nhật để đảm bảo giữ được hiệu suất. Đặc biệt, khi dữ liệu thay đổi, mô hình cần được cập nhật để phù hợp với dữ liệu mới.

Tám bước trong vòng đời phát triển - triển khai mô hình Học máy nêu trên được vận dụng một cách linh hoạt. Nếu có vấn đề xảy ra cần thay đổi ở các bước trước, chúng ta có thể quay lại các bước trước đó để chỉnh sửa, hoàn thiện (xem Hình 1.4). Tại mỗi bước, thường các lập trình viên sử dụng các công cụ hoặc nền tảng có sẵn nhưng đôi khi họ phải tự xây dựng công cụ mới do yêu cầu nhiệm vụ hoặc yêu cầu về dữ liệu đặt ra.

1.6 Tổng kết chương

Tổng hợp lại, Chương 1 đã trình bày bức tranh khái quát của Học máy, nhấn mạnh tầm quan trọng của dữ liệu và các bài toán học máy phổ biến. Những ví dụ cụ thể cho thấy Học máy đóng vai trò then chốt trong nhiều ứng dụng đời sống. Các chương tiếp theo sẽ lần lượt đi sâu vào từng bài toán cụ thể, đưa ra các phương pháp và thuật toán chi tiết.

1.7 Bài tập

1. Hãy định nghĩa học máy và so sánh sự khác nhau giữa học máy và các thuật toán truyền thống?
2. Hãy mô tả một ví dụ thực tế trong đó học máy có thể cải tiến hiệu năng qua kinh nghiệm.
3. Vì sao dữ liệu lớn lại quan trọng đối với học máy? Liên hệ điều này với ví dụ về phân loại thư rác?
4. Hãy liệt kê và giải thích một số ứng dụng của học máy trong đời sống hàng ngày.
5. Hãy chọn một ứng dụng học máy mà bạn cảm thấy thú vị nhất và mô tả chi tiết cách học máy được sử dụng trong ứng dụng đó.
6. Học máy có thể được ứng dụng trong lĩnh vực chăm sóc sức khỏe để giúp dự đoán và quản lý bệnh tật như thế nào?
7. Trong tình huống nào thì áp dụng học máy là có lợi nhất? Đưa ra một ví dụ minh họa cho mỗi tình huống.
8. Tại sao học máy lại phù hợp với các nhiệm vụ phức tạp và nó khác gì so với lập trình truyền thống trong việc xử lý các tình huống này?

9. Làm thế nào học máy có thể hỗ trợ việc phân tích các trang web liên tục thay đổi như trong ví dụ tìm kiếm giá sản phẩm?
10. Hãy so sánh hai phương pháp học có giám sát và học không giám sát. Đưa ra ví dụ thực tế cho mỗi phương pháp.
11. Giải thích bài toán phân loại thư rác và bài toán dự đoán giá cổ phiếu trong ngữ cảnh học có giám sát.
12. Trong các kỹ thuật học tăng cường, đâu là thách thức chính khi thiết kế một mô hình có phản hồi chậm từ môi trường? Bạn có ý tưởng nào để giải quyết thách thức này?
13. Hãy mô tả các bước chính trong vòng đời phát triển một mô hình học máy.
14. Tại sao việc thu thập và chuẩn bị dữ liệu là một bước quan trọng trong vòng đời của mô hình học máy?
15. Hãy giải thích vai trò của bước trích chọn đặc trưng và tại sao bước này giúp cải thiện hiệu năng của mô hình.

Tài liệu tham khảo

- [1] Jordan, Michael I. and Mitchell, Tom M., *Machine learning: Trends, perspectives, and prospects*, Science, vol. 349, no. 6245, pp. 255–260, 2015.
- [2] Bishop, Christopher M., *Pattern recognition and machine learning*, Springer, 2006.
- [3] Domingos, Pedro, *A few useful things to know about machine learning*, Communications of the ACM, vol. 55, no. 10, pp. 78–87, 2012.
- [4] Sutton, Richard S. and Barto, Andrew G., *Reinforcement learning: An introduction*, MIT Press, 2018.
- [5] Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron, *Deep learning*, MIT Press, 2016.
- [6] Amershi, Saleema, Begel, Andrew, Bird, Christian, DeLine, Robert, Gall, Harald, Kamar, Ece, Nushi, Besmira, et al., *Software engineering for machine learning: A case study*, Proceedings of the 41st International Conference on Software Engineering, pp. 291–300, 2019.

- [7] LeCun, Yann, Bengio, Yoshua, and Hinton, Geoffrey, *Deep learning*, Nature, vol. 521, no. 7553, pp. 436–444, 2015.
- [8] Mitchell, Tom M., *Machine learning*, McGraw Hill, 1997.

Chương 2

Bài toán phân lớp

Chương 2 tập trung vào một trong những bài toán quan trọng nhất của Học máy: Bài toán phân lớp. Con người có một khả năng đặc biệt là khả năng phân biệt các đối tượng khác nhau và hình thành các khái niệm. Về mặt toán học, một khái niệm là một tập hợp các thực thể cùng loại. Ví dụ, khái niệm quả cam là tập hợp tất cả các quả cam, khái niệm ô tô là tập hợp mọi chiếc ô tô mà con người đã từng và sẽ làm ra. Như vậy, khả năng hình thành một khái niệm trong ý thức của con người là *khả năng phân loại* đối tượng thành hai lớp: các đối tượng thuộc về khái niệm đó và các đối tượng khác, không thuộc về khái niệm đó. Trong Học máy, ta gọi đó là bài toán phân lớp. Ta sẽ tìm hiểu dữ liệu huấn luyện, cách đánh giá tính sẵn sàng và mức độ đại diện của dữ liệu, cũng như các mô hình phân lớp thống kê cơ bản như Hồi quy Logistic, K láng giềng gần nhất hay Cây quyết định. Chương này có mối liên hệ mật thiết với Chương 7 về các mô hình hồi quy và Chương 4 về mô hình máy vector hỗ trợ vì cả ba đều hướng tới nhiệm vụ dự đoán dựa trên dữ liệu có nhãn.

2.1 Định nghĩa

Tổng quát hơn, ta có thể phân lớp các thực thể (hoặc đối tượng) thành nhiều lớp. Phát biểu một bài toán phân lớp như sau

Định nghĩa 2.1 (Bài toán phân lớp). Cho $\mathcal{X}$ là tập đầu vào, $\mathcal{Y} = \{1, 2, \dots, C\}$ là tập các phân lớp của $\mathcal{X}$ và tập dữ liệu huấn luyện $D = \{(x_i, y_i)\}, i = 1, 2, \dots, n$ với $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$ sao cho $y_i = f(x_i)$ với $f : \mathcal{X} \rightarrow \mathcal{Y}$ là một hàm phân lớp ẩn chưa biết (hàm mục tiêu). Hãy tìm một hàm phân lớp $h : \mathcal{X} \rightarrow \mathcal{Y}$ sao cho $h(x)$ gần giống với $f(x)$ với mọi $x \in \mathcal{X}$.

Tập huấn luyện D còn gọi là tập dữ liệu dùng để xác định các tham số của mô hình học máy. Trong tập dữ liệu D , mỗi giá trị (x_i, y_i) được gọi là mẫu huấn luyện. Trong đó, $x_i \in \mathcal{X}$ được gọi là đặc trưng và $y_i \in \mathcal{Y}$ được gọi là nhãn. Chúng ta có hai định nghĩa cơ bản về không gian đặc trưng và không gian nhãn như sau:

Định nghĩa 2.2 (Không gian đặc trưng). Không gian đặc trưng là tập hợp tất cả các véc-tơ biểu diễn đầu vào của bài toán phân lớp. Không gian đặc trưng thường là không gian số thực $\mathbb{R}^d$ với d chiều.

Định nghĩa 2.3 (Không gian nhãn). Không gian nhãn là tập tất cả các nhãn của bài toán phân lớp. Không gian nhãn thường là tập rời rạc $\mathcal{Y}$ với C lớp.

Ví dụ 2.4 (Một số bài toán phân lớp). Để làm quen với khái niệm phân lớp, ta sẽ cùng tìm hiểu một số thiết lập bài toán phân lớp đơn giản.

1. Bài toán phân lớp hàm logic XOR: trong bài toán phân lớp hàm logic XOR, nhiệm vụ đặt ra là học ra được mô hình dự đoán kết quả của phép toán logic XOR dựa trên giá trị đầu vào. Dữ liệu được thể hiện ở bảng 2.1. Tập dữ liệu D có tổng cộng bốn mẫu,

Bảng 2.1: Dữ liệu cho bài toán phân lớp học hàm XOR

	x^1	x^2	y
x_1	0	0	0
x_2	0	1	1
x_3	1	0	1
x_4	1	1	0

mỗi mẫu có hai đặc trưng tương ứng. Các đặc trưng của mẫu huấn luyện được kí hiệu là x^1 và x^2 , nhận giá trị 0 hoặc 1. Do tính ràng buộc về giá trị logic nên các đặc trưng này được coi là rời rạc. Kết quả phân lớp thành tập $y = 0$ và $y = 1$ nên số lớp $C = 2$. Trong ví dụ cơ bản này, chúng ta thấy được số lượng mẫu cần thiết để phân lớp chỉ cần bốn mẫu.

2. Bài toán phân lớp hoa IRIS: trong bài toán phân lớp hoa IRIS, dữ liệu đầu vào là tập các đặc trưng của cánh hoa và đài hoa. Từ đó, các giá trị này được sử dụng để phân lớp vào các lớp con của loài hoa IRIS. Tập đặc trưng được sử dụng là: chiều dài và chiều rộng của cánh hoa và đài hoa. Các đặc trưng này nhận giá trị thực, nên không gian đặc trưng là $\mathbb{R}^4$. Số lớp con là ba lớp nên $C = 3$.
3. Bài toán phân lớp MNIST: trong bài toán phân lớp MNIST, mỗi đối tượng là một ảnh chữ số đã được rời rạc hóa thành các điểm ảnh. Chiều dài và chiều rộng của một ảnh MNIST là 28 nên số đặc trưng tổng cộng có $28 \times 28 = 784$. Các chữ số nhận giá trị từ 0 đến 9 nên tổng cộng số lớp phải phân lớp là $C = 10$.

Trong định nghĩa trên, C gọi là số lớp của bài toán phân lớp. Khi C bằng 2, ta nói đây là bài toán *phân lớp nhị phân*. Trên thực tế việc phân lớp với C bằng 2 có ý nghĩa quan trọng về mặt lý thuyết. Vì nếu có thể phân lớp đúng được với hai lớp bất kì thì

chúng ta có thể tổng quát hóa để xây dựng bộ phân lớp đúng với $C' > 2$ lớp.

2.2 Tổng quan về dữ liệu

Tập dữ liệu huấn luyện $D = \{(x_i, y_i)\}, i = 1, 2, \dots, n$ là đối tượng nghiên cứu chính của học máy. Trong học máy thống kê, dữ liệu D thường được coi như xuất phát từ một phân bố ngẫu nhiên $\mathcal{P}$. Trước khi áp dụng học máy vào các bài toán thực tế, bước tạo ra tập dữ liệu D được gọi là bước thu thập dữ liệu hoặc bước lấy mẫu. Quá trình lấy mẫu để có được tập D từ $\mathcal{P}$ bao gồm: thu thập, xử lý và lưu trữ dữ liệu có ảnh hưởng tiên quyết đến khả năng thành công của một dự án học máy. Thông thường, đây là quá trình tốn nhiều thời gian và được tạo thành từ nhiều bước nhỏ hơn.

Hiện nay, dưới thời đại công nghệ dữ liệu lớn, lượng dữ liệu được thu thập rất nhiều và người ta đã tạo ra khá nhiều bộ dữ liệu chuẩn. Đây là một trong những điều kiện cần thiết để các mô hình có thể được phát triển và đánh giá, so sánh với nhau. Tuy nhiên, để dữ liệu có thể được sử dụng hiệu quả trong các bài toán học máy, dữ liệu được thu thập cần tuân thủ một số tính chất nhất định, bao gồm: tính sẵn sàng, tính đại diện và tính tin cậy của dữ liệu, tính tương quan trong dữ liệu và số lượng mẫu cần thiết của tập dữ liệu. Chúng ta sẽ đề cập chi tiết hơn đến các tính chất này trong Chương 13 Triển khai mô hình học máy.

2.3 Cách tiếp cận Học máy thống kê

Định nghĩa 2.1 đã cho thấy nhiệm vụ cần phải giải quyết của bài toán phân lớp nhưng chưa làm rõ làm thế nào để đánh giá độ “tốt” của mô hình phân lớp h một cách định lượng. Bên cạnh đó, việc tìm được một mô hình phân lớp h tốt phụ thuộc nhiều vào tập dữ

liệu được sử dụng. Để giải quyết vấn đề này, chúng ta có thể tiếp cận bài toán phân lớp qua lăng kính của ngành xác suất thống kê.

Định nghĩa 2.5 (Bài toán phân lớp thống kê). Cho $\mathcal{X}$ là tập đầu vào, $\mathcal{Y} = \{1, 2, \dots, C\}$ là tập các phân lớp của $\mathcal{X}$ và tập dữ liệu huấn luyện $D = \{(x_i, y_i)\}, i = 1, 2, \dots, n$ với $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$ được lấy mẫu theo *phân bố dữ liệu* $\mathcal{P}(x, y)$, hãy tìm một mô hình phân lớp $h : \mathcal{X} \rightarrow \mathcal{Y}$ phân lớp tốt trên phân bố $\mathcal{P}$. Nghĩa là xác suất lỗi phân lớp $\mathbb{P}_{(X,Y) \sim \mathcal{P}}\{h(X) \neq Y\}$ nhỏ.

Trong đó sự kiện phân lớp sai $\{h(X) \neq Y\}$ là một sự kiện ngẫu nhiên trong phân bố ngẫu nhiên $\mathcal{P}$ của các biến ngẫu nhiên X, Y .

Điểm khác của Định nghĩa 2.5 so với Định nghĩa 2.1 là các mẫu dữ liệu (x, y) được lấy mẫu theo một phân bố xác suất $\mathcal{P}$ nhất định. Định nghĩa 2.5 cho phép mô tả sự không chắc chắn của nhãn y trong bài toán phân lớp, khác với Định nghĩa 2.1 nhãn y được xác định hoàn toàn bởi hàm mục tiêu $f(x)$.

Mọi tính toán định lượng của bài toán phân lớp được tính toán trên phân bố $\mathbb{P}$. Ta gọi xác suất của sự kiện phân lớp sai là *xác suất lỗi*, tỉ lệ lỗi hay rủi ro kì vọng của mô hình phân lớp h . Xác suất này được định nghĩa theo công thức sau:

$$\text{err}_{\mathcal{P}}(h) = \mathbb{P}_{(X,Y) \sim \mathcal{P}}\{h(X) \neq Y\} = \mathbb{E}_{\mathcal{P}}\mathbb{I}[h(X) \neq Y], \quad (2.1)$$

với $\mathbb{I}$ là hàm chỉ thị (indicator function) của sự kiện phân lớp sai

$$\mathbb{I}[h(X) \neq Y] = \begin{cases} 1, & \text{nếu } h(X) \neq Y, \\ 0, & \text{nếu } h(X) = Y. \end{cases}$$

Việc giải bài toán phân lớp thống kê là tìm một mô hình phân lớp h^* sao cho

$$h^* = \arg \min_h \text{err}_{\mathcal{P}}(h). \quad (2.2)$$

Trên thực tế, nói chung ta không biết chính xác phân bố $\mathcal{P}$ của dữ liệu mà chỉ biết tập dữ liệu D được giả sử lấy mẫu từ phân bố này. Do đó, đại lượng $\text{err}_{\mathcal{P}}(h)$ không thể tính được mà phải ước lượng thông qua tập dữ liệu D . Tức là ta xấp xỉ $\text{err}_{\mathcal{P}}(h)$ bằng đại lượng $\widehat{\text{err}}_D(h)$ được tính trên tập dữ liệu D :

$$\widehat{\text{err}}_D(h) = \frac{1}{n} \underbrace{\sum_{i=1}^n \mathbb{I}[h(x_i) \neq y_i]}_{\text{Đếm số lỗi trên tập } D}. \quad (2.3)$$

Đại lượng $\widehat{\text{err}}_D(h)$ ở công thức (2.3) được gọi là *tỉ lệ lỗi* (error rate) hoặc *rủi ro thực nghiệm* (empirical risk) của h trên tập dữ liệu D .

2.4 Ước lượng xác suất lỗi

Xấp xỉ $\text{err}_{\mathcal{P}}(h)$ bằng $\widehat{\text{err}}_D(h)$ tốt đến mức nào? Định lý sau cho chúng ta biết rằng nếu tập dữ liệu D càng lớn thì xấp xỉ này càng chính xác.

Định lý 2.6 (Xấp xỉ rủi ro kì vọng bằng rủi ro thực nghiệm). *Xét một hàm phân lớp h cố định, nếu các mẫu dữ liệu (x_i, y_i) được lấy mẫu độc lập từ phân bố $\mathcal{P}$ thì với mọi giá trị $\epsilon > 0$,*

$$\mathbb{P}_{D \sim \mathcal{P}^n} [|\widehat{\text{err}}_D(h) - \text{err}_{\mathcal{P}}(h)| < \epsilon] \geq 1 - 2e^{-2n\epsilon^2}. \quad (2.4)$$

Hay nói cách khác, với mọi giá trị $\epsilon, \delta > 0$, với xác suất không thấp hơn $1 - \delta$ trên tập dữ liệu D có $n \geq \frac{1}{2\epsilon^2} \ln\left(\frac{2}{\delta}\right)$ mẫu dữ liệu, ta có

$$|\widehat{\text{err}}_D(h) - \text{err}_{\mathcal{P}}(h)| < \epsilon.$$

Chứng minh: Áp dụng định lý Hoeffding với $X_i = \mathbb{I}[h(x_i) \neq y_i]$ và $\mu = \text{err}_{\mathcal{P}}(h) = \mathbb{E}_{\mathcal{P}}[X_i]$, ta có $0 \leq X_i \leq 1$ nên

$$\begin{aligned} \mathbb{P}_{D \sim \mathcal{P}^n} [|\widehat{\text{err}}_D(h) - \text{err}_{\mathcal{P}}(h)| \geq \epsilon] &= \mathbb{P} \left[\left| \frac{1}{n} \sum_{i=1}^n X_i - \mu \right| \geq \epsilon \right] \\ &\leq 2e^{-2n\epsilon^2} \leq \delta. \end{aligned}$$

Giải bất đẳng thức trên ta có

$$n \geq \frac{1}{2\epsilon^2} \ln \left(\frac{2}{\delta} \right)$$

□

Định lý 2.6 cho thấy, khi số mẫu dữ liệu n càng lớn thì xác suất để lỗi thực nghiệm gần bằng lỗi kì vọng ngày càng lớn. Ví dụ, nếu $\epsilon = 1\%$, $n = 100000$, tỉ lệ lỗi trung bình trên 100.000 mẫu dữ liệu là 10% thì tỉ lệ lỗi thực tế nằm trong khoảng $[10\% \pm 1\%]$ với xác suất không thấp hơn $1 - 2e^{-20} \approx 1 - 4.10^{-9}$, tức là gần bằng 1. Đây cũng là một trong các lý do tại sao Học máy lại cần nhiều dữ liệu để huấn luyện mô hình như vậy. Ước lượng sai số của Định lý 2.6 là cận dưới đúng với mọi phân bố $\mathcal{P}$ nên ước lượng này thường được xem là ước lượng không chặt.

Như vậy, khi có nhiều dữ liệu, việc cực tiểu hóa rủi ro thực nghiệm $\widehat{\text{err}}_D(h)$ có thể dẫn đến việc cực tiểu hóa rủi ro kì vọng $\text{err}_{\mathcal{P}}(h)$. Ta gọi cách tiếp cận này là nguyên tắc *Tối thiểu hóa rủi ro thực nghiệm* (Empirical Risk Minimization - ERM). Trong cách tiếp cận này, ta sẽ chọn một hàm phân lớp h_D trong một tập các hàm phân lớp $\mathcal{H}$ sao cho

$$h_D = \arg \min_{h \in \mathcal{H}} \widehat{\text{err}}_D(h). \quad (2.5)$$

Trong trường hợp đơn giản nhất là chọn h trong một tập hữu hạn các hàm phân lớp $\mathcal{H}$ có chứa h^* , nguyên tắc Tối thiểu hóa rủi ro thực nghiệm có thể xấp xỉ được h^* với xác suất cao (PAC - Probably Approximately Correct). Định lý này được phát biểu như sau:

Định lý 2.7 (Học xấp xỉ với xác suất cao (PAC Learning)). *Xét lớp hàm phân lớp $\mathcal{H}$ hữu hạn và một hàm phân lớp $h^* \in \mathcal{H}$, nếu*

- $n \geq \frac{2}{\epsilon^2} \ln \left(\frac{2|\mathcal{H}|}{\delta} \right)$ mẫu dữ liệu $(x_i, y_i), i = 1, 2, \dots, n$ được lấy mẫu độc lập từ phân bố $\mathcal{P}$ và
- $\text{err}_{\mathcal{P}}(h^*) = 0$,

thì với mọi giá trị $\epsilon > 0$ và $\delta > 0$, với xác suất không thấp hơn $1 - \delta$ trên tập dữ liệu D , ta có:

$$\text{err}_{\mathcal{P}}(h_D) \leq \epsilon. \quad (2.6)$$

Chứng minh: Sử dụng Định lý 2.6, với mỗi $h \in \mathcal{H}$, ta có

$$\mathbb{P}_{D \sim \mathcal{P}^n} [|\widehat{\text{err}}_D(h) - \text{err}_{\mathcal{P}}(h)| > \epsilon/2] \leq 2e^{-2n\epsilon^2/4} = 2e^{-n\epsilon^2/2}.$$

Do đó để xác suất tồn tại ít nhất một $h \in \mathcal{H}$ như trên là

$$\mathbb{P}_{D \sim \mathcal{P}^n} [\exists h \in \mathcal{H} : |\widehat{\text{err}}_D(h) - \text{err}_{\mathcal{P}}(h)| > \epsilon/2] \leq 2|\mathcal{H}|e^{-n\epsilon^2/2} \leq \delta.$$

Vậy ta có xác suất để cùng lúc mọi $h \in \mathcal{H}$ đều có $|\widehat{\text{err}}_D(h) - \text{err}_{\mathcal{P}}(h)| \leq \epsilon/2$ ít nhất là $1 - \delta$.

Ta có với xác suất không thấp hơn $1 - \delta$,

$$\begin{aligned} \text{err}_{\mathcal{P}}(h_D) &\leq \widehat{\text{err}}_D(h_D) + \epsilon/2 \\ [\text{do ERM}] &\leq \widehat{\text{err}}_D(h^*) + \epsilon/2 \\ &\leq \text{err}_{\mathcal{P}}(h^*) + \epsilon/2 + \epsilon/2 \\ &\leq \text{err}_{\mathcal{P}}(h^*) + \epsilon \\ [\text{do } \text{err}_{\mathcal{P}}(h^*) = 0] &= \epsilon, \end{aligned}$$

Giải $2|\mathcal{H}|e^{-n\epsilon^2/2} \leq \delta$ ta có

$$n \geq \frac{2}{\epsilon^2} \ln \left(\frac{2|\mathcal{H}|}{\delta} \right).$$

□

Định lý 2.7 cho thấy rằng nếu số mẫu dữ liệu n đủ lớn, ta có thể sử dụng nguyên tắc Tối thiểu hóa rủi ro thực nghiệm để tìm ra một hàm phân lớp h_D có xác suất lỗi $\text{err}_{\mathcal{P}}(h_D)$ nhỏ hơn một giá trị ϵ bất kỳ với xác suất cao $1 - \delta$. Định lý này cũng cho thấy rằng số mẫu dữ liệu n cần thiết tỉ lệ thuận với kích thước $\log |\mathcal{H}|$. Hay nói cách khác, độ lớn của lớp hàm phân lớp $\mathcal{H}$ phần nào quyết định độ khó của bài toán phân lớp.

Một giả thiết của Định lý 2.7 là lớp hàm phân lớp $\mathcal{H}$ là hữu hạn. Giả thiết này có thật sự không thực tế? Trong thực hành, khi ta lập trình các mô hình học máy, ta thường sử dụng các hàm phân lớp có thể được biểu diễn bằng một số lượng hữu hạn các tham số là số thực (64 bit). Giả sử một mô hình cần N bit nhị phân để biểu diễn thì số lượng mô hình khác nhau mà ta có thể tạo ra là $|\mathcal{H}| = 2^N$. Do đó, số lượng mô hình khác nhau mà ta có thể tạo ra là hữu hạn. Khi đó sử dụng Định lý 2.7 là hợp lý và số lượng mẫu dữ liệu cần thiết là

$$n \geq \frac{2}{\epsilon^2} \left(N + 1 + \ln \frac{1}{\delta} \right).$$

Việc giới hạn độ lớn của lớp hàm phân lớp $\mathcal{H}$ là một trong những cách để giảm độ khó của bài toán phân lớp cũng như tăng khả năng tổng quát của mô hình. Định lý 2.8 (Không có bữa trưa miễn phí) cho thấy rằng nếu $\mathcal{H}$ là tập tất cả các hàm phân lớp (không bị giới hạn) thì không có thuật toán huấn luyện nào có thể tìm ra hàm phân lớp h_D với xác suất lỗi $\text{err}_{\mathcal{P}}(h_D)$ nhỏ hơn một giá trị ϵ bất kỳ với xác suất cao $1 - \delta$ cho mọi phân bố $\mathcal{P}$.

Định lý 2.8 (Không có bữa trưa miễn phí). *Cho $\mathcal{H}$ là tập tất cả các hàm phân lớp trên tập đầu vào $\mathcal{X}$. Nếu tập dữ liệu D có n dữ liệu được lấy mẫu độc lập từ phân bố $\mathcal{P}$, sao cho $2n < |\mathcal{X}|$ thì với mọi thuật toán huấn luyện A , luôn tồn tại một phân bố $\mathcal{P}$ sao cho*

$$\mathbb{P}_{D \sim \mathcal{P}^n} [\text{err}_{\mathcal{P}}(A(D)) > 1/8] \geq 1/7.$$

Định lý 2.8 là một trong những kết quả quan trọng nhất trong Học máy. Chứng minh của định lý này có thể tìm thấy trong [8].

2.5 Hàm phân lớp tối ưu Bayes

Trong mục này, chúng ta sẽ tìm hiểu về hàm phân lớp tối ưu h^* còn gọi là *hàm phân lớp tối ưu Bayes*.

Câu hỏi đầu tiên đặt ra đối với bài toán phân lớp trong Định nghĩa 2.5 là “Hàm phân lớp $h^*(x)$ nào là hàm phân lớp tối ưu?”. Về mặt toán học, hàm tối ưu $h^*(x)$, cùng với xác suất lỗi tối ưu R^* được định nghĩa bằng công thức sau:

$$\begin{aligned} h^* &= \arg \min_h \text{err}_{\mathcal{P}}(h) \\ R^* &= \text{err}_{\mathcal{P}}(h^*) = \min_h \text{err}_{\mathcal{P}}(h). \end{aligned}$$

Xét một hàm phân lớp h bất kì, ta có biến đổi sau:

$$\begin{aligned} \text{err}_{\mathcal{P}}(h) &= \mathbb{P}_{\mathcal{P}}[h(x) \neq y] \\ &= \int_{x,y} \mathbb{I}[h(x) \neq y] p(x, y) dx dy \\ &= \int_{x,y} \mathbb{I}[h(x) \neq y] P(y|x) p(x) dx dy \\ &= \int_x \underbrace{\left[\sum_y \mathbb{I}[h(x) \neq y] P(y|x) \right]}_{\text{Xác suất lỗi của } h \text{ trên mẫu dữ liệu } x} p(x) dx \end{aligned}$$

Trong biến đổi trên, $p(x, y)$ là mật độ của mẫu dữ liệu (x, y) trong phân bố $\mathcal{P}$. Theo công thức Bayes, ta có đẳng thức

$$p(x, y) = P(y|x)p(x)$$

với $p(x)$ là mật độ biên của đầu vào x còn $P(y|x)$ là xác suất có điều kiện của nhãn y khi biết đầu vào x trong phân bố $\mathcal{P}$.

Đại lượng $\text{err}_{\mathcal{P}}(h, x) = \sum_y \mathbb{I}[h(x) \neq y]P(y|x)$ là xác suất xảy ra lỗi khi áp dụng hàm phân lớp h trên dữ liệu x . Với nhận xét $\mathbb{I}[h(x) \neq y]$ chỉ khác 0 khi $h(x) \neq y$, ta có thể tính được $\text{err}_{\mathcal{P}}(h, x)$ như sau:

$$\text{err}_{\mathcal{P}}(h, x) = \sum_{y \neq h(x)} P(y|x) = 1 - P(h(x)|x). \quad (2.7)$$

Như vậy, để tối thiểu hóa xác suất lỗi trên từng mẫu dữ liệu x ta phải chọn h^* sao cho $\text{err}_{\mathcal{P}}(h^*, x)$ nhỏ nhất hay $P(h^*(x)|x)$ lớn nhất.

Định nghĩa 2.9 (Hàm phân lớp tối ưu Bayes). Hàm phân lớp tối ưu Bayes là hàm phân lớp thỏa mãn đẳng thức:

$$h^*(x) = \arg \max_y P(y|x), \quad (2.8)$$

trong đó, $P(y|x)$ được gọi là *phân bố hậu nghiệm* của nhãn y khi biết đầu vào x trong phân bố $\mathcal{P}$.

Dễ thấy, do $\text{err}_{\mathcal{P}}(h^*, x) \leq \text{err}_{\mathcal{P}}(h, x), \forall h, x$ nên $\text{err}_{\mathcal{P}}(h^*) \leq \text{err}_{\mathcal{P}}(h)$ với mọi h . Hàm phân lớp h^* đạt giá trị tối ưu R^* là xác suất lỗi phân lớp nhỏ nhất có thể trên phân bố $\mathcal{P}$.

Dựa theo công thức (2.8) để tìm h^* ta phải biết phân bố hậu nghiệm $P(y|x)$. Trong thực tế, việc ước lượng $P(y|x)$ một cách chính xác là không khả thi do chúng ta chỉ biết tập dữ liệu $D = \{(x_i, y_i)\}, i = 1, \dots, n$ mà không biết phân bố $\mathcal{P}$. Tuy nhiên, công thức (2.8) này cũng chỉ ra phương hướng để chúng ta tìm h^* hoặc ít nhất là xấp xỉ h^* . Có hai cách tiếp cận chính trong Học máy xuất phát từ định nghĩa này.

- *Mô hình sinh* (Chương 11): Là mô hình Học máy ước lượng trực tiếp xác suất $p(x, y)$ (hoặc mật độ xác suất) là xác suất sinh ra cặp dữ liệu (x, y) từ phân bố $\mathcal{P}$. Từ phân bố này, ta có thể tìm

h^* theo khai triển sau:

$$\begin{aligned} h^*(x) &= \arg \max_y P(y|x) \\ &= \arg \max_y P(y|x)p(x) = \arg \max_y p(x, y). \end{aligned}$$

- *Mô hình phân biệt* (Các chương 3, 4, 7): Là mô hình Học máy ước lượng xác suất $P(y|x)$ hoặc trực tiếp hàm phân lớp $h^*(x)$.

Trong các mục sau của chương này và các chương kế tiếp của giáo trình, chúng ta sẽ tìm hiểu một loạt thuật toán và mô hình Học máy tuân theo một trong hai cách tiếp cận trên.

2.6 Mô hình hồi quy Logistic

Định nghĩa 2.10 (Mô hình hồi quy Logistic nhị phân). Xét bài toán phân lớp nhị phân với $\mathcal{X} = \mathbb{R}^d$ và $\mathcal{Y} = \{\pm 1\}$, mô hình hồi quy Logistic là mô hình ước lượng xác suất hậu nghiệm $P(y|\mathbf{x})$ như sau:

$$\begin{aligned} P(y = +1|\mathbf{x}) &= \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} = \sigma(f(\mathbf{x})), \\ P(y = -1|\mathbf{x}) &= 1 - \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} = \frac{1}{1 + e^{(\mathbf{w}^T \mathbf{x} + b)}} = \sigma(-f(\mathbf{x})). \end{aligned}$$

Trong đó, $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ là hàm tuyến tính (tuyến tính đối với bộ tham số $\mathbf{w}, b$) còn $\sigma(z) = \frac{1}{1+e^{-z}}$ gọi là hàm *sigmoid*. Nếu chúng ta lợi dụng tính chất $y = \pm 1$ thì có thể kết hợp cả hai công thức trên lại thành một công thức rút gọn như sau:

$$P(y|\mathbf{x}) = \sigma(yf(\mathbf{x})).$$

Đại lượng $s = yf(\mathbf{x})$ gọi là điểm số của mẫu dữ liệu $(\mathbf{x}, y)$.

Theo hàm phân lớp tối ưu Bayes, trong pha suy luận, ta sẽ chọn nhãn $y^* = \arg \max_{y=\pm 1} P(y|\mathbf{x})$. Nhưng do tổng các xác suất bằng

1 nên ta chỉ cần tìm y^* sao cho $P(y^*|\mathbf{x}) > 0,5$. Thế vào các công thức trên, trong pha suy luận, ta sẽ dùng luật phân lớp sau:

$$h(\mathbf{x}) = \begin{cases} +1, & f(\mathbf{x}) \geq 0 \\ -1, & f(\mathbf{x}) < 0 \end{cases}.$$

Lỗi của mô hình xảy ra khi $h(\mathbf{x}) \neq y$ hay $s = yf(\mathbf{x}) < 0$, tức là

$$\mathbb{I}(h(\mathbf{x}) \neq y) = \mathbb{I}(yf(\mathbf{x}) < 0).$$

Cũng như mọi mô hình Học máy khác, trước khi sử dụng mô hình Logistic để dự đoán, chúng ta cần phải huấn luyện mô hình sử dụng một tập dữ liệu huấn luyện $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$. Mục tiêu của việc huấn luyện là tìm bộ trọng số $\Theta = (w, b)$ hợp lý nhất (hoặc ít “lỗi” nhất) trên bộ dữ liệu này.

2.6.1 Nguyên lý Ước lượng hợp lý cực đại

Ta xét một mẫu dữ liệu $(\mathbf{x}, y) \sim \mathcal{P}$. Xác suất của nhãn y theo mô hình Logistic là $P(y|\mathbf{x}) = \sigma(yf(\mathbf{x}))$. Rõ ràng, nếu xác suất này càng lớn thì khả năng mô hình phân lớp mẫu dữ liệu này càng chính xác. Như vậy $L = P(y|\mathbf{x})$ có thể coi như là sự hợp lý của mô hình đối với mẫu dữ liệu $(\mathbf{x}, y)$. Bây giờ xét tập dữ liệu D có n mẫu dữ liệu. Nếu giả sử các mẫu dữ liệu của D độc lập xác suất thì sự hợp lý của mô hình đối với D là tích độ hợp lý của các mẫu dữ liệu.

Định nghĩa 2.11 (Độ hợp lý của mô hình Logistic). Đại lượng

$$L(\Theta; D) = \prod_{i=1}^n P(y_i|\mathbf{x}_i) = \prod_{i=1}^n \sigma(y_i f(\mathbf{x}_i))$$

gọi là độ hợp lý (likelihood) của mô hình Logistic đối với dữ liệu D . Ta còn nói $L(\Theta; D)$ là độ hợp lý của bộ trọng số Θ của mô hình.

Lưu ý $L(\Theta; D)$ là một hàm của bộ trọng số Θ , còn D có thể coi là hằng số do D là dữ liệu được cung cấp để huấn luyện mô hình. Sử dụng độ hợp lý của mô hình, ta có thể huấn luyện mô hình bằng cách tìm bộ trọng số Θ cực đại hoá độ hợp lý này theo Nguyên lý ước lượng hợp lý cực đại.

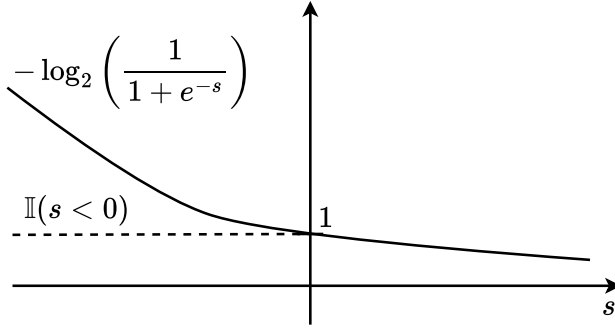
Định nghĩa 2.12 (Nguyên lý Ước lượng hợp lý cực đại). Mô hình tốt nhất là *mô hình có độ hợp lý cực đại* MLE (Maximum Likelihood Estimation) được định nghĩa là:

$$\begin{aligned}\Theta^{MLE} &= \arg \max_{\Theta} L(\Theta; D) \\ &= \arg \max_{\Theta} \log L(\Theta; D) \\ &= \arg \min_{\Theta} -\log L(\Theta; D)\end{aligned}$$

Trong định nghĩa này, ta cũng sử dụng hàm log để tính toán do độ hợp lý thường là tích của độ hợp lý trên các mẫu dữ liệu. Một phương pháp cực tiểu hoá hàm số là cập nhật trọng số theo hướng ngược của đạo hàm. Hay thường được gọi tắt là phương pháp tối ưu xuống đồi bằng đạo hàm GD (gradient descent). Xét hàm âm lô-ga-rít của độ hợp lý như sau:

$$\ell(\Theta; D) = -\log L(\Theta; D) = \sum_{i=1}^n -\log \sigma(y_i f(\mathbf{x}_i)).$$

Hình 2.1 cho thấy hàm $-\log \sigma(s)$ với $s = yf(\mathbf{x})$ là cận trên của hàm lỗi 0-1 do $-\log_2 \sigma(s) \geq \mathbb{I}(s < 0)$. Như vậy, có thể nói nguyên lý ước lượng hợp lý cực đại cực tiểu hóa một cận trên của hàm lỗi 0-1.



Hình 2.1: Hàm lỗi Logistic $-\log \sigma(s)$ với $s = yf(\mathbf{x})$.

2.6.2 Thuật toán huấn luyện

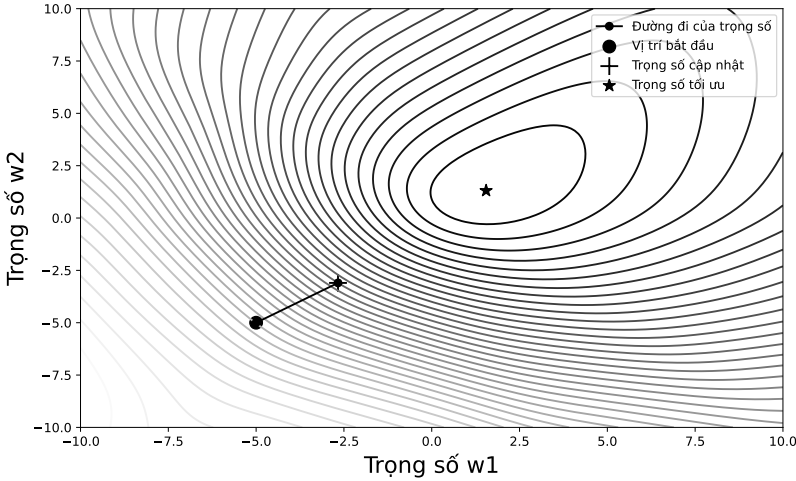
Sử dụng đẳng thức $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, ta có thể biến đổi đạo hàm của hàm ℓ như sau:

$$\begin{aligned} \frac{\partial \ell}{\partial \mathbf{w}} &= - \sum_{i=1}^n \sigma(y_i f(\mathbf{x}_i))^{-1} \sigma(y_i f(\mathbf{x}_i)) (1 - \sigma(y_i f(\mathbf{x}_i))) y_i \mathbf{x}_i \\ &= - \sum_{i=1}^n (1 - \sigma(y_i f(\mathbf{x}_i))) y_i \mathbf{x}_i \\ \frac{\partial \ell}{\partial b} &= - \sum_{i=1}^n (1 - \sigma(y_i f(\mathbf{x}_i))) y_i \end{aligned}$$

Sử dụng một hệ số huấn luyện $\lambda > 0$, ta có thể cập nhật Θ , theo công thức sau:

$$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} - \lambda \frac{\partial \ell}{\partial \mathbf{w}} = \mathbf{w} + \lambda \sum_{i=1}^n (1 - \sigma(y_i f(\mathbf{x}_i))) y_i \mathbf{x}_i \\ b &\leftarrow b - \lambda \frac{\partial \ell}{\partial b} = b + \lambda \sum_{i=1}^n (1 - \sigma(y_i f(\mathbf{x}_i))) y_i. \end{aligned}$$

Hình 2.2 minh họa kết quả cập nhật của trọng số theo hướng của đạo hàm với một bước. Ở bước ban đầu trọng số $\mathbf{w}$ xuất phát



Hình 2.2: Minh họa kết quả cập nhật của trọng số theo hướng của đạo hàm.

từ vùng có lỗi lớn tại vị trí $w^0 = (0.5, 0.5)$. Tại vị trí này, chiều cập nhật đạo hàm là véc-tơ theo hướng đường đi được dấu và sau một bước cập nhật trọng số trọng số w đã đi đến vị trí w^1 (điểm màu đỏ). Trọng số tối ưu trong ví dụ trên được đánh dấu bằng dấu chấm xanh, là mục tiêu cần đạt được của quá trình huấn luyện mô hình.

Nhận xét đại lượng $1 - \sigma(y_i f(\mathbf{x}_i)) = 1 - P(y_i | \mathbf{x}_i)$ chính là phần còn thiếu để xác suất $P(y_i | \mathbf{x}_i) = 1$, mục tiêu của việc huấn luyện. Như vậy các công thức cập nhật trên có xu hướng giúp đẩy các xác suất $P(y_i | \mathbf{x}_i)$ tăng lên. Ngoài ra nếu mỗi lần huấn luyện chúng ta chỉ đưa một mẫu dữ liệu vào huấn luyện (học trực tuyến) thì các công thức trên trở thành:

$$\begin{aligned} w &\leftarrow w + \lambda(1 - \sigma(y_i f(\mathbf{x}_i)))y_i \mathbf{x}_i \\ b &\leftarrow b + \lambda(1 - \sigma(y_i f(\mathbf{x}_i)))y_i. \end{aligned}$$

Công thức trên có một hệ số tuyến tính $(1 - \sigma(y_i f(\mathbf{x}_i)))$ có thể

coi là sai số về xác suất của mô hình trên mẫu dữ liệu $(\mathbf{x}_i, y_i)$. Các thuật toán huấn luyện mô hình Logistic được liệt kê ở Thuật toán 2.1 và Thuật toán 2.2.

Thuật toán 2.1 Thuật toán huấn luyện mô hình hồi quy Logistic

```

1: procedure TRAINLOGISTIC( $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \lambda$ )
2:   Khởi tạo  $\mathbf{w} \leftarrow \mathbf{0}$ ,  $b \leftarrow 0$ 
3:   for epoch = 1, 2, ... do
4:      $\mathbf{w} \leftarrow \mathbf{w} + \lambda \sum_{i=1}^n (1 - \sigma(y_i f(\mathbf{x}_i))) y_i \mathbf{x}_i$ 
5:      $b \leftarrow b + \lambda \sum_{i=1}^n (1 - \sigma(y_i f(\mathbf{x}_i))) y_i$ 
6:     if điều kiện dừng then break
7:   end for
8:   return  $\mathbf{w}, b$ 
9: end procedure

```

Thuật toán 2.2 Thuật toán huấn luyện trực tuyến mô hình hồi quy Logistic

```

1: procedure TRAINLOGISTICONLINE( $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \lambda$ )
2:   Khởi tạo  $\mathbf{w} \leftarrow \mathbf{0}$ ,  $b \leftarrow 0$ 
3:   for epoch = 1, 2, ... do
4:     for all  $(\mathbf{x}_i, y_i) \in D$  do
5:        $\mathbf{w} \leftarrow \mathbf{w} + \lambda (1 - \sigma(y_i f(\mathbf{x}_i))) y_i \mathbf{x}_i$ 
6:        $b \leftarrow b + \lambda (1 - \sigma(y_i f(\mathbf{x}_i))) y_i$ 
7:     end for
8:     if điều kiện dừng then break
9:   end for
10:  return  $\mathbf{w}, b$ 
11: end procedure

```

Điều kiện dừng của các thuật toán huấn luyện trên có thể là

- Khi số epoch đủ lớn;

- Khi log-likelihood $\ell(\Theta; D)$ thay đổi không nhiều;
- Khi hiệu suất phân lớp trên tập dữ liệu kiểm thử không tăng sau nhiều epoch.

Về hệ số huấn luyện λ , ta sẽ xem xét kỹ hơn ở các mục sau trong giáo trình. Thường ta thử chọn λ trong các giá trị $\{10^i\}$ với $i = -5, -4, \dots, 0$.

2.7 Thuật toán K láng giềng gần nhất

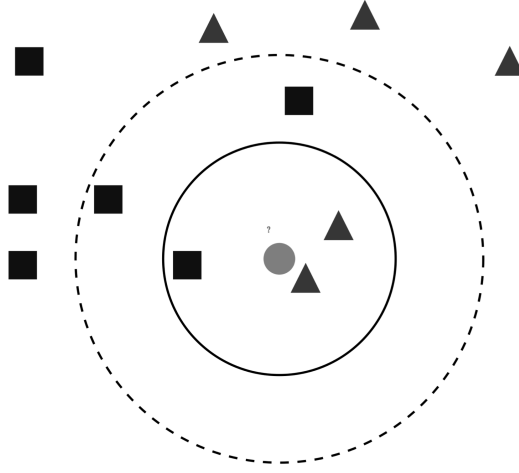
Một trong những thuật toán phân lớp đầu tiên của Học máy là thuật toán K-láng giềng gần nhất, thường được viết tắt là KNN. Thuật toán này hoạt động trên tập đầu vào là không gian số thực m chiều với hàm khoảng cách $d(\mathbf{x}, \mathbf{x}')$. Hàm khoảng cách d có thể là một hàm khoảng cách bất kỳ thỏa mãn bất đẳng thức tam giác (thường dùng khoảng cách Euclid $d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_2$).

Thuật toán KNN là thuật toán theo hướng tiếp cận mô hình phân biệt. Giả sử ta có tập dữ liệu huấn luyện $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$ với $\mathbf{x}_i \in \mathbb{R}^m$ và $y_i \in \mathcal{Y} = \{1, 2, \dots, C\}$. Thuật toán KNN ước lượng xác suất hậu nghiệm $P(y|\mathbf{x})$ đối với mỗi mẫu dữ liệu x bằng cách xét một lân cận V của $\mathbf{x}$ trong không gian $\mathbb{R}^m$ chứa đúng k mẫu dữ liệu trong D . Khi đó, ta ước lượng $P(y|\mathbf{x})$ bằng đại lượng:

$$\hat{P}(y|x) = \frac{|\{(\mathbf{x}_i, y_i) \in D, \mathbf{x}_i \in V, y_i = y\}|}{k},$$

trong đó, tử số là số mẫu dữ liệu có nhãn $y_i = y$ nằm trong lân cận V . Nếu ta dùng $\hat{P}(y|\mathbf{x})$ thay thế cho $P(y|\mathbf{x})$ để tạo ra hàm h^{KNN} , dễ thấy $h^{KNN}(\mathbf{x})$ sẽ là nhãn y xuất hiện nhiều nhất trong các mẫu dữ liệu huấn luyện nằm trong lân cận V (Hình 2.3).

Thuật toán KNN (Thuật toán 2.3) khá đơn giản. Trong pha huấn luyện, thuật toán KNN không làm gì ngoài việc lưu trữ tất



Hình 2.3: Thuật toán KNN với $k = 3$ và $k = 5$.

Thuật toán 2.3 Thuật toán KNN

- 1: **procedure** KNN(x, D, k)
 - 2: Tìm k mẫu dữ liệu gần x nhất trong D theo khoảng cách $d(\mathbf{x}_i, \mathbf{x})$
 - 3: Đếm số lượng các nhãn $c_1, c_2, \dots, c_C$ trong k mẫu dữ liệu gần nhất
 - 4: **return** $\hat{y} \leftarrow \arg \max_y c_y$
 - 5: **end procedure**
-

cả các mẫu dữ liệu huấn luyện trong D . Trong pha suy luận, KNN chọn nhãn xuất hiện nhiều nhất trong k mẫu dữ liệu huấn luyện gần x nhất. Độ phức tạp thuật toán trong pha suy luận chủ yếu nằm ở bước tìm kiếm k “láng giềng” của x . Để tối ưu thời gian suy luận, người ta thường dùng các cấu trúc tìm kiếm không gian như cây kd , cây bóng. Chi tiết cài đặt các cấu trúc dữ liệu tìm kiếm không gian nằm ngoài phạm vi của giáo trình. Tuy nhiên thời gian tìm kiếm có thể rút ngắn còn cỡ $O(mk \log n)$ cho việc tìm kiếm k “láng giềng” gần nhất của x trong không gian m chiều.

Định lý 2.13 (Cận trên tỉ lệ lỗi của KNN với $k = 1$). *Khi số mẫu*

huấn luyện $n \rightarrow \infty$, ta có ước lượng tỉ lệ lỗi $\text{err}_{\mathcal{P}}(h^{\text{KNN}})$ của KNN với $k = 1$ như sau:

$$R^* \leq \text{err}_{\mathcal{P}}(h^{\text{KNN}}) \leq R^* \left(2 - \frac{C}{C-1} R^* \right). \quad (2.9)$$

Trong đó, R^* là xác suất lỗi tối ưu Bayes.

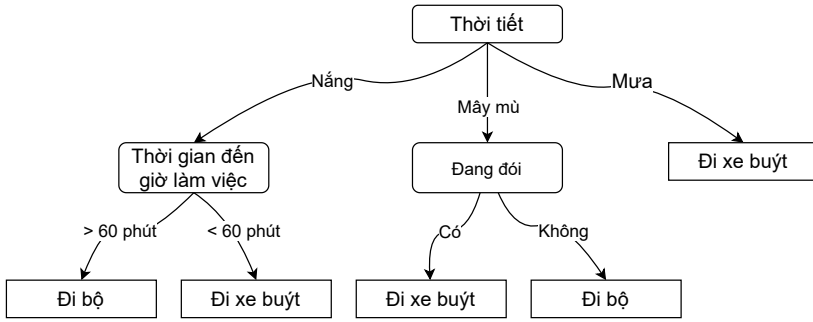
Định lý 2.13 cho thấy chỉ cần với $k = 1$, khi có đủ dữ liệu, xác suất lỗi của hàm phân lớp h^{KNN} không vượt quá hai lần so với xác suất lỗi tối ưu R^* . Với $k > 1$ càng lớn thì $\text{err}_{\mathcal{P}}(h^{\text{KNN}})$ càng tiến đến sát R^* . Đây là một kết quả thú vị nhưng nó đòi hỏi số mẫu huấn luyện n rất lớn so với k .

2.8 Thuật toán Cây quyết định

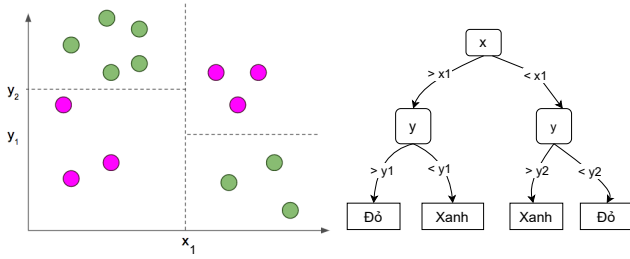
Thuật toán phân lớp dùng Cây quyết định (Decision Tree - DT) có cách tiếp cận tương tự như thuật toán KNN. Theo đó, thuật toán DT cũng ước lượng xác suất $P(y|\mathbf{x})$ trong một lân cận V của $\mathbf{x}$. Tuy nhiên, khác với thuật toán KNN tính toán lân cận V trong pha suy luận (bằng cách tìm lân cận chứa k láng giềng), thuật toán DT tính toán lân cận V trong pha huấn luyện bằng cách phân vùng tập dữ liệu đầu vào $\mathcal{X}$ thành các vùng riêng biệt nhờ chuỗi các quyết định đơn giản (lệnh rẽ nhánh) sao cho việc phân lớp trong mỗi vùng trở nên dễ dàng hơn (Hình 2.4 mô tả cây quyết định “Đi bộ” hay “Đi xe buýt”).

Giả sử tập đầu vào $\mathcal{X}$ gồm các dữ liệu $\mathbf{x} = (x^1, \dots, x^m)$ gồm m thuộc tính, kí hiệu $A = \{A^1, A^2, \dots, A^m\}$, trong đó $x^k \in A^k$. Các thuộc tính A^k có thể rời rạc (ví dụ: thuộc tính màu sắc) hoặc liên tục (ví dụ: thuộc tính cân nặng). Xét các trường hợp sau:

- A^k là thuộc tính rời rạc: thuộc tính A_k chia tập đầu vào $\mathcal{X}$ thành $n_k = |A^k|$ tập con ứng với từng quyết định $x^k = v$ với v là các



Hình 2.4: Cây quyết định phân lớp.



Hình 2.5: Cây quyết định phân chia tập đầu vào cho bài toán phân lớp.

giá trị trong A^k . Ví dụ: thuộc tính Thời tiết trong Hình 2.4 có các giá trị “Nắng”, “Mây mù”, “Mưa”.

$$\mathcal{X} = \bigcup_{v \in A^k} \{\mathbf{x} \in \mathcal{X} : x^k = v\}.$$

- A^k là thuộc tính liên tục, $A^k \subset \mathbb{R}$: nếu ta chọn một ngưỡng $\theta \in \mathbb{R}$ thì tập đầu vào $\mathcal{X}$ được chia thành hai tập con ứng với các quyết định $x^k \leq \theta$ và $x^k > \theta$ (ví dụ: thuộc tính Thời gian đến giờ làm việc trong Hình 2.4 được so sánh với ngưỡng 60 phút).

$$\mathcal{X} = \{\mathbf{x} \in \mathcal{X} : x^k \leq \theta\} \cup \{\mathbf{x} \in \mathcal{X} : x^k > \theta\}$$

Nếu ta gán các quyết định như trên lên một cấu trúc dạng cây, ta đã phân chia tập đầu vào $\mathcal{X}$ một cách đệ quy thành các phân vùng

rời nhau được đại diện bởi các nút lá (ví dụ: Hình 2.5). Tại các nút lá, nếu ta đưa ra quyết định phân lớp thì cây quyết định đó trở thành một mô hình phân lớp trên $\mathcal{X}$.

Thuật toán huấn luyện Cây quyết định có đầu vào là tập dữ liệu $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$, đầu ra là một cây quyết định phân lớp. Lược đồ chung của các thuật toán huấn luyện cây quyết định sử dụng chiến lược đệ quy như sau:

Thuật toán 2.4 Khung thuật toán huấn luyện Cây quyết định

```
1: function TRAINDT( $D$ )
2:   if  $D$  thỏa mãn điều kiện dừng then
3:      $leaf = \arg \max_y c_y$  với  $c_y$  là số lần nhãn  $y$  trong  $D$ .
4:     return  $leaf$ 
5:   else
6:     Tạo nút gốc  $root$ 
7:      $A^k = chooseAttribute(D)$ .
8:      $D_1, D_2, \dots = partition(D, A_k)$ .
9:     for all  $D_i$  do
10:       $root.child[i] = TRAINDT(D_i)$ 
11:    end for
12:    return  $root$ 
13:  end if
14: end function
```

Trong thuật toán huấn luyện Cây quyết định (Thuật toán 2.4), khi ở nút lá, chúng ta ra quyết định phân lớp bằng nhãn xuất hiện nhiều nhất trong các mẫu dữ liệu huấn luyện rơi vào phân vùng tương ứng của nút lá đó. Các làm này tương tự như thuật toán KNN khi tìm kiếm nhãn trong k láng giềng gần nhất. Để có thể cài đặt Thuật toán 2.4, ta còn phải làm rõ hai vấn đề:

- *Điều kiện dừng phân chia*: Trong các gói phần mềm thường gặp, thuật toán huấn luyện sẽ dừng khi tập D nhỏ. Một trong những điều kiện dừng này có thể kể đến như khi $|D|$ ít hơn hằng số

là số lượng mẫu ít nhất ở lá. Tham số này là một tham số của thuật toán sẽ được lựa chọn để tránh hiện tượng học quá. Một điều kiện dừng khác là tập D chỉ còn duy nhất một nhãn phân lớp.

- *Cách chọn thuộc tính phân chia A^k (hàm `chooseAttribute`) và cách chọn ngưỡng θ khi A^k là thuộc tính liên tục:* Đây chính là điểm phân biệt các thuật toán huấn luyện Cây quyết định. Trong bài này, chúng ta xem xét hai cách chọn thuộc tính phân chia của thuật toán ID3 và thuật toán C4.5.

2.8.1 Thuật toán ID3

Thuật toán ID3 lựa chọn thuộc tính dựa trên khái niệm độ tăng thông tin. Trong xác suất thống kê, khái niệm thông tin xuất phát từ định nghĩa về entropy thông tin là một thang đo mức độ ngẫu nhiên. Vì các nhãn phân lớp là rời rạc, ta có thể đo lường mức độ ngẫu nhiên của các nhãn này bằng entropy.

Giả sử trong bài toán phân lớp, ta có một tập dữ liệu huấn luyện $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$ gồm n mẫu dữ liệu với các nhãn $(y_1, y_2, \dots, y_n)$. Entropy của tập các nhãn này được định nghĩa là:

$$H(D) = - \sum_{y=1}^C \frac{c_y}{n} \log_2 \frac{c_y}{n}, \quad (2.10)$$

trong đó c_y là số lần nhãn y xuất hiện trong D . Có thể chứng minh $H(D) = 0$ nếu tồn tại y để $c_y = n$ (dãy các nhãn toàn bằng y , entropy bằng 0, rất dễ đoán ra nhãn y). Còn khi $c_y = \frac{n}{C}$ với mọi y thì $H(D)$ đạt giá trị cực đại (entropy cực đại, rất khó để đoán nhãn y). Entropy $H(D)$ càng nhỏ thì lựa chọn nhãn $y^* = \arg \max_y c_y$ đại diện cho tập D có xác suất lỗi càng nhỏ.

Giả sử ta lựa chọn thuộc tính A^k để phân chia và D bị chia thành n_k tập con $D_1, \dots, D_{n_k}$. Khi đó, mỗi tập D_i sẽ có $H(D_i)$ là

entropy của cây con $T(D_i)$. Tuy nhiên, xác suất để một dữ liệu x nằm trong phân vùng cây $T(D)$ rơi vào phân vùng của cây con $T(D_i)$ là $\frac{|D_i|}{n}$. Do đó, entropy của cây mới (đã phân chia) được tính bằng đại lượng $H(D|A^k)$ theo công thức:

$$H(D|A^k) = \sum_{i=1}^{n_k} \frac{|D_i|}{n} H(D_i). \quad (2.11)$$

Khi đó ta nói, độ tăng thông tin (hay độ giảm entropy) của thuộc tính A^k là $IG(D, A^k)$ được định nghĩa như sau:

$$IG(D, A^k) = H(D) - H(D|A^k). \quad (2.12)$$

Thuật toán ID3 lựa chọn thuộc tính A^k sao cho $IG(D, A^k)$ tính theo các công thức (2.10), (2.11), (2.12) đạt cực đại.

2.8.2 Thuật toán C4.5

Thuật toán ID3 có một nhược điểm là khi chọn $IG(D, A^k)$ cực đại, ID3 sẽ ưu tiên các thuộc tính có nhiều giá trị. Khi đó, số lượng mẫu trong các tập con D_i sẽ giảm đi rất nhanh và khả năng cao là $H(D_i) \rightarrow 0$ do D_i quá nhỏ nên có quá ít nhãn. Ví dụ, nếu thuộc tính A^k là số thẻ tín dụng thì ngay lập tức tập D_i sẽ có kích thước bằng 1 do mỗi thẻ tín dụng chỉ thuộc về một người (và $H(D_i) = 0, \forall i$). Tuy nhiên, cây quyết định nhận được lại chỉ nhớ được tập dữ liệu mà không thể tổng quát hóa cho các dữ liệu chưa biết.

Thuật toán C4.5 khắc phục nhược điểm này bằng cách sử dụng khái niệm tỉ lệ tăng thông tin. Trong đó, độ tăng thông tin được chuẩn hóa bằng cách chia cho đại lượng

$$IV(D, A^k) = - \sum_{i=1}^{n_k} \frac{|D_i|}{n} \log_2 \frac{|D_i|}{n}, \quad (2.13)$$

$$IGR(D, A^k) = \frac{IG(D, A^k)}{IV(D, A^k)}. \quad (2.14)$$

Thuật toán C4.5 lựa chọn thuộc tính A^k sao cho $IGR(D, A^k)$ tính theo các công thức (2.13), (2.14) đạt cực đại.

2.8.3 Thuộc tính liên tục

Khi thuộc tính A^k rời rạc, ta có thể nhanh chóng tính các tập con D_i tương ứng với từng giá trị $v \in A^k$ (độ phức tạp thuật toán $O(n + n_k C)$). Nhưng khi thuộc tính A^k liên tục, ta còn phải xác định ngưỡng θ để phân chia D thành hai tập con D_1, D_2 theo công thức sau:

$$D_1 = \{(x, y) \in D : x^k \leq \theta\}, D_2 = \{(x, y) \in D : x^k > \theta\}. \quad (2.15)$$

Về mặt lý thuyết chúng ta phải tìm θ trên toàn bộ tập số thực $\mathbb{R}$. Tuy nhiên, để ý rằng dù chọn θ thế nào thì cũng chỉ có $n + 1$ khả năng cho D_1, D_2 do chỉ có nhiều nhất n giá trị của x^k trong D . Dựa trên nhận xét này, chúng ta có thể xây dựng một thuật toán hiệu quả để dò ngưỡng θ như sau:

Thuật toán 2.5 Dò ngưỡng θ cho thuộc tính liên tục A^k

```

1: function FINDTHRESHOLD( $D, A^k$ )
2:   Sắp xếp  $D$ :  $x_1 \leq x_2 \leq \dots \leq x_n$ 
3:   Khởi tạo  $n_y = 0, c_y = \sum_{i=1}^n \mathbb{I}(y_i = y), \forall y = 1, 2, \dots, C$ 
4:    $H(D|A^k) = \infty$ 
5:   for  $i = 1$  đến  $n$  do
6:     Cập nhật  $n_{y_i} \leftarrow n_{y_i} + 1$ .
7:      $H(D_1) \leftarrow Entropy(n_y, i = 1 \dots C)$ 
8:      $H(D_2) \leftarrow Entropy(c_y - n_y, i = 1 \dots C)$ 
9:      $H(D|A^k) \leftarrow \min(H(D|A^k), \frac{i}{n}H(D_1) + \frac{n-i}{n}H(D_2))$ 
10:  end for
11:  return ngưỡng  $\theta = x_i$  có giá trị  $H(D|A^k)$  nhỏ nhất
12: end function
```

Thuật toán dò ngưỡng trên có độ phức tạp $O(n \log n + nC)$.

2.9 Tình huống áp dụng: Phân lớp hoa IRIS

Trước khi đi vào các bước triển khai chính, chúng ta sẽ làm quen với dữ liệu đầu vào và các thư viện cần thiết để thực hiện phân lớp với bộ dữ liệu IRIS.

2.9.1 Giới thiệu dữ liệu đầu vào

- **Nguồn dữ liệu:** Bộ dữ liệu IRIS là một tập dữ liệu kinh điển trong học máy, chứa thông tin về ba loại hoa IRIS: Setosa, Versicolor và Virginica. Nhiệm vụ đặt ra là huấn luyện mô hình học máy để phân loại các loại hoa này dựa trên các đặc trưng của chúng. Bộ dữ liệu này có sẵn trong thư viện `scikit-learn` của Python.
- **Cấu trúc dữ liệu:** Bộ dữ liệu gồm 150 mẫu, mỗi mẫu gồm 4 đặc trưng và 1 nhãn lớp.
 - Chiều dài và chiều rộng đài hoa
 - Chiều dài và chiều rộng cánh hoa
 - 1 nhãn lớp: Loại hoa được mã hoá thành các lớp 0, 1, 2 tương ứng với Setosa, Versicolor và Virginica.

2.9.2 Các thư viện sử dụng

Trong phần này chúng ta sẽ làm quen với các thư viện cơ bản để thực hiện phân lớp với bộ dữ liệu IRIS, bao gồm:

- **numpy:** Thư viện toán học cơ bản trong Python, cung cấp các hàm toán học và cấu trúc dữ liệu như mảng. Thư viện numpy cho phép tính toán nhanh với các mảng lớn và thực hiện các phép tính trên véc-tơ.
- **scikit-learn:** Thư viện học máy phổ biến trong Python, cung cấp nhiều công cụ cho phân tích dữ liệu và xây dựng mô hình.

- **pandas**: Thư viện xử lý dữ liệu, giúp dễ dàng thao tác với dữ liệu dạng bảng.
- **seaborn**: Thư viện trực quan hóa dữ liệu.

2.9.3 Các bước triển khai chính

Dựa trên các thư viện đã giới thiệu ở trên, chúng ta sẽ thực hiện các bước sau để xây dựng mô hình phân lớp với bộ dữ liệu IRIS:

- **Phân tích đặc trưng dữ liệu**: ở bước này, chúng ta sẽ sử dụng các hàm trong thư viện **pandas** và **seaborn** để phân tích dữ liệu. Trong đó, chúng ta sẽ vẽ đồ thị để trực quan hóa dữ liệu và tìm hiểu mối quan hệ giữa các đặc trưng, và giữa đặc trưng với nhãn.
- **Tiền xử lý dữ liệu**: Dữ liệu sẽ được chuẩn hoá và phân chia thành 2 tập dữ liệu huấn luyện và kiểm tra.
- **Xây dựng mô hình hồi quy Logistic**: Chúng ta có thể khai báo mô hình hồi quy Logistic, sử dụng thư viện **scikit-learn** với các tham số được lựa chọn. Và tiến hành huấn luyện mô hình với tập dữ liệu huấn luyện.
- **Đánh giá mô hình**: Sử dụng tập dữ liệu kiểm tra, chúng ta sẽ đánh giá độ chính xác của mô hình hồi quy vừa được huấn luyện. Đối với dữ liệu hoa IRIS, chúng ta có thể dựa trên ma trận nhầm lẫn để đánh giá độ chính xác của mô hình.

Người học có thể xem ví dụ về việc sử dụng mô hình hồi quy Logistic để phân lớp trên dữ liệu IRIS tại <https://gist.github.com/cuongtv312/36f280a51c2e15d87a5ea7a5828a177a>

2.10 Tổng kết chương

Chương 2 tập trung vào bài toán phân lớp trong học máy, khởi đầu bằng việc định nghĩa bài toán phân lớp và giới thiệu các khái niệm cơ bản: hàm phân lớp, xác suất lỗi và hàm phân lớp tối ưu Bayes. Các khái niệm này đặt nền tảng lý thuyết cho lĩnh vực Học máy, đi kèm với kết quả quan trọng như Định lý về khả năng học xấp xỉ với xác suất cao, Định lý Không có bữa ăn miễn phí. Chương 2 cũng trình bày các mô hình phân lớp nền tảng, bao gồm hồi quy Logistic, K-láng giềng gần nhất và cây quyết định. Cuối cùng, phương pháp huấn luyện Xuống đồi bằng đạo hàm – một kỹ thuật tối ưu hóa phổ biến trong học máy – được đề cập sơ lược, với phần trình bày chi tiết hơn ở các chương sau.

Bài tập

1. Giải thích ý nghĩa của việc sử dụng hàm sigmoid trong hồi quy Logistic? Tìm một hàm số khác có thể sử dụng cho hồi quy Logistic. So sánh các tính chất của hàm tìm được so với hàm sigmoid thường dùng.
2. Cho tập dữ liệu thể hiện hàm lô-gíc AND sau đây

$$D_1 = \{(0, 0) : \mathbf{0}, (0, 1) : \mathbf{0}, (1, 0) : \mathbf{0}, (1, 1) : \mathbf{1}\}$$

Giả sử mô hình hồi quy Logistic có trọng số khởi tạo là $w = [1.0, -1.0]$ và hệ số tự do $b = 1.0$.

- a) Tính đầu ra của mô hình hồi quy Logistic với các đầu vào $(0, 0)$, $(0, 1)$, $(1, 0)$ và $(1, 1)$.
- b) Tính đạo hàm của hàm lỗi theo trọng số w và hệ số tự do b .
- c) Cập nhật trọng số w và hệ số tự do b bằng phương pháp xuống đồi bằng đạo hàm với tốc độ học $\alpha = 0.1$.

Bảng 2.2: Dữ liệu phân lớp cho cây quyết định

N	x^1	x^2	y
1	1	1	0
2	1	2	0
3	2	1	0
4	2	2	0
5	3	1	0
6	3	4	1
7	4	4	1
8	4	5	1
9	5	4	1
10	5	5	1

3. *[Lập trình]* Cài đặt thuật toán hồi quy Logistic cho bài toán hồi quy Logistic với hàm lỗi là hàm cross-entropy sử dụng dữ liệu D_1 ở câu trên:
 - a) Giả sử số vòng lặp huấn luyện là $N = 10$, tốc độ học $\alpha = 0.1$, in ra các giá trị tìm được của $\theta = [w, b]$ và giá trị hàm lỗi sau mỗi vòng lặp, biết rằng trọng số khởi tạo là $w = [1.0, -1.0]$ và hệ số tự do $b = 1.0$.
 - b) Giả sử các tham số khởi tạo $\theta = [w, b]$ được sinh ngẫu nhiên trong khoảng $[0, 1]$, sử dụng thiết lập tốc độ học $\alpha = 0.1$. Số lần lặp được tăng lên để đảm bảo tính hội tụ của thuật toán. Đánh giá vị trí của điểm hội tụ tìm được khi chạy thuật toán với các tham số khởi tạo khác nhau.
4. Biên quyết định của cây quyết định có hình dạng thế nào? Minh hoạ câu trả lời bằng cho một ví dụ cụ thể trên tập dữ liệu D_2 có 10 mẫu và 2 đặc trưng, như sau
5. *[Lập trình]* Cài đặt thuật toán 2.5 cho tập dữ liệu D_2 ở câu trên.
 - a) In ra cấu trúc cây quyết định tìm được

- b) Cải tiến thuật toán để tìm cây quyết định có độ sâu tối thiểu.
6. *[Tìm hiểu]* Liệt kê một số phương pháp phổ biến để kiểm soát tình trạng phù hợp quá của cây quyết định?
7. *[Tìm hiểu]* Trong thuật toán phân lớp với K-lân cận gần nhất, ta muốn giảm độ nhiễu của mô hình thì giá trị của K thay đổi thế nào?
8. Hãy áp dụng Định lý 2.7 cho mô hình ChatGPT biết rằng mô hình này có khoảng 100 tỉ tham số (số thực). Hãy chọn $\epsilon = 0,01$ và $\delta = 0,05$.

Tài liệu tham khảo

- [1] Vapnik, V. N., *Principles of risk minimization for learning theory*, Advances in Neural Information Processing Systems, 1992.
- [2] Cox, D. R., *The regression analysis of binary sequences*, Journal of the Royal Statistical Society: Series B, 1958.
- [3] Quinlan, J. R., *Induction of decision trees*, Machine Learning, vol. 1, no. 1, pp. 81–106, 1986.
- [4] Cover, T. and Hart, P., *Nearest neighbor pattern classification*, IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.
- [5] Hastie, T., Tibshirani, R., and Friedman, J., *The elements of statistical learning*, Springer, 2001.
- [6] Bishop, C. M., *Pattern recognition and machine learning*, Springer, 2006.
- [7] Wolpert, D. H., and Macready, W. G. (1997). No free lunch theorems for optimization. IEEE transactions on evolutionary computation, 1(1), 67-82.

- [8] Shalev-Shwartz, S., and Ben-David, S., *Understanding machine learning: From theory to algorithms*, Cambridge University Press, 2014.

Chương 3

Mạng nơ-ron nhiều lớp

Từ sau năm 2010, các mô hình học máy “nông” như hồi quy tuyến tính, hồi quy Logistic hay SVM dần trở nên không còn phù hợp để xử lý dữ liệu ngày càng phức tạp và đa dạng. Trong khi đó, mô hình đồ thị xác suất có khả năng biểu diễn tốt nhưng bị giới hạn về hiệu suất do thiếu thuật toán hiệu quả trên nhiều loại đồ thị. Do đó, nhu cầu cấp thiết là tìm kiếm mô hình vừa có khả năng biểu diễn mạnh, vừa đảm bảo hiệu năng tính toán.

Mạng nơ-ron nhân tạo là một cấu trúc tính toán phù hợp, đặc biệt với sự phát triển của Học sâu (Deep Learning) trong thập kỷ qua. Các kiến trúc học sâu hiện đạt độ chính xác cao nhất trong nhiều bài toán, thậm chí vượt cả con người.

Chương này giới thiệu mạng nơ-ron từ cơ bản đến nâng cao, bao gồm cơ chế lan truyền tối, lan truyền ngược và các thuật toán tối ưu như Xuống đồi bằng đạo hàm, đạo hàm ngẫu nhiên và Adam. Những nội dung này là nền tảng cho việc học các kiến trúc mạng sâu hơn trong các chương tiếp theo như mạng tích chập (CNN) (Chương 5) và mạng hồi quy (RNN) (Chương 7).

3.1 Mô hình

Đầu tiên chúng ta sẽ tìm hiểu mô hình của một nơ-ron, sau đó chúng ta sẽ ghép các nơ-ron lại thành một lớp rồi một mạng nơ-ron nhiều lớp.

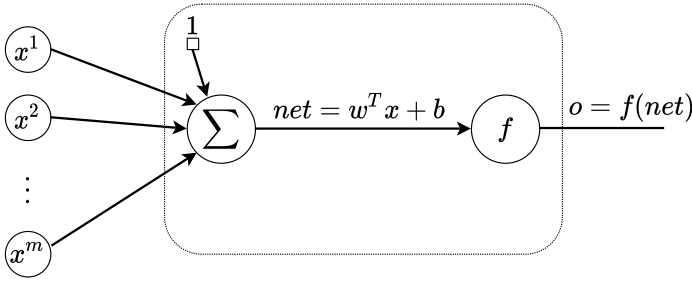
3.1.1 Mô hình nơ-ron đơn vị

Một nơ-ron nhân tạo được tạo nên từ hai bộ phận: một bộ phận so sánh đầu vào với một véc-tơ trọng số thông qua tích vô hướng, một bộ phận phi tuyến hoá kết quả (Hình 3.1).

Định nghĩa 3.1 (Nơ-ron). Mô hình nơ-ron là ánh xạ từ đầu vào $\mathbf{x} \in \mathbb{R}^m$ đến đầu ra $o \in \mathbb{R}$

$$o = f_{\mathbf{w}}(\mathbf{x}) = f(\mathbf{w} \cdot \mathbf{x} + b).$$

Trong đó véc-tơ $\mathbf{w} \in \mathbb{R}^m$ và giá trị ngưỡng $b \in \mathbb{R}$ là các tham số của nơ-ron và hàm $f : \mathbb{R} \rightarrow \mathbb{R}$ gọi là *hàm kích hoạt*.



Hình 3.1: Mô hình nơ-ron.

Phép tính tích vô hướng đạt cực đại khi $\mathbf{x}$ có cùng hướng với $\mathbf{w}$. Vì thế, nơ-ron đã dò biết được $\mathbf{x}$ có cùng cấu tạo như véc-tơ trọng số của nó hay không. Hàm kích hoạt f có mục đích phi tuyến hoá đầu ra, giúp cho mạng nơ-ron có thể tính toán phức tạp hơn. Bảng 3.1 đưa ra công thức một số hàm kích hoạt phổ biến và đạo hàm

Bảng 3.1: Bảng các hàm kích hoạt thông dụng

Hàm số	Công thức	Đạo hàm
Tuyến tính	$f(z) = z$	$f'(z) = 1$
Relu	$f(z) = \max(0, z)$	$f'(z) = \mathbb{I}(z \geq 0)$
Sigmoid	$f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$	$f'(z) = \sigma(z)(1 - \sigma(z))$
Tan-hyperbol	$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$f'(z) = 1 - \tanh^2(z)$

tương ứng của chúng. Theo định nghĩa này, với f là hàm sigmoid và bài toán phân lớp nhị phân, chúng ta sẽ thu được công thức cơ bản của mô hình hồi quy logistic ở Mục 2.6. Trong các chương tiếp theo, chúng ta sẽ gặp các dạng biểu diễn tương tự với mô hình Perceptron ở Chương 4 và mô hình hồi quy tuyến tính ở Chương 7.

Để tiện trình bày và biến đổi các công thức, ta thêm số 1 vào véc-tơ đầu vào $\mathbf{x}$ và đưa b vào trong véc-tơ $\mathbf{w}$

$$o = f(\mathbf{w}^T \mathbf{x}) \quad (3.1)$$

3.1.2 Lớp nơ-ron

Mỗi nơ-ron theo Định nghĩa 3.1 tính toán một kết quả đầu ra dựa trên tổ hợp các thông tin đặc trưng vào. Lớp nơ-ron tiếp tục mở rộng định nghĩa trên thành một tập hợp nơ-ron hoạt động trên cùng một đầu vào $\mathbf{x}$. Mỗi nơ-ron có một đầu ra riêng biệt như thể hiện trong Hình 3.2.

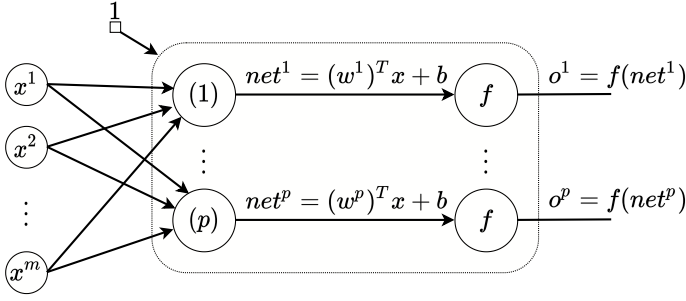
Định nghĩa 3.2 (Lớp nơ-ron). Lớp nơ-ron là ánh xạ từ đầu vào $\mathbf{x} \in \mathbb{R}^m$ đến đầu ra $\mathbf{o} \in \mathbb{R}^p$ với p nơ-ron

$$\mathbf{o} = f_W(\mathbf{x}) = f(W \cdot \mathbf{x}) = \begin{bmatrix} o^1 & o^2 & \dots & o^p \end{bmatrix}^T \in \mathbb{R}^p$$

Trong đó ma trận trọng số W có các hàng là các véc-tơ trọng số của các nơ-ron

$$W = \begin{bmatrix} \mathbf{w}^1 & \mathbf{w}^2 & \dots & \mathbf{w}^p \end{bmatrix}^T \in \mathbb{R}^{p \times m}$$

và hàm kích hoạt f tác động lên từng phần tử của véc-tơ $W\mathbf{x}$.



Hình 3.2: Một lớp nơ-ron có p đầu ra.

Phân lớp nhiều lớp bằng lớp nơ-ron. Sử dụng công thức (3.2), chúng ta có thể mở rộng bài toán phân lớp từ hai lớp thành nhiều lớp hơn. Trước tiên, ta mô tả xác suất $p(y|\mathbf{x})$ với giá trị $y \in \{1, 2, \dots, p\}$ bằng hàm softmax

$$p(c|o) = \frac{\exp\{o^c\}}{\sum_{c'=1}^p \exp\{o^{c'}\}}. \quad (3.2)$$

Đây là hàm tổng quát hoá hàm sigmoid để dành cho phân lớp đa lớp. Sử dụng nguyên lý Ước lượng hợp lý cực đại, ta cần cực tiểu hóa hàm lỗi sau

$$\begin{aligned} \ell(W) &= - \sum_{i=1}^n \ln p(y_i|o_i) \\ &= - \sum_{i=1}^n \sum_{c=1}^p \mathbb{I}[y_i = c] \ln p(c|o_i) \end{aligned} \quad (3.3)$$

$$= \sum_{i=1}^n \ell(o_i, y_i). \quad (3.4)$$

Trong đó $\ell(o, y) = - \sum_{c=1}^p \mathbb{I}[y = c] \ln p(y = c|o)$ là hàm lỗi trên một mẫu dữ liệu (x, y) . Hàm lỗi này được gọi là hàm lỗi entropy chéo (cross-entropy).

Chúng ta tiếp tục tính đạo hàm của *hàm lỗi entropy chéo*. Đặt $MS = \sum_{c=1}^p \exp\{o^c\}$ và dùng đẳng thức $p(y = c|o) = \exp\{o^c\}/MS$,

ta có khai triển sau:

$$\begin{aligned}
 \frac{\partial \ell(o, y)}{\partial o^c} &= - \sum_{c'=1}^p \frac{\mathbb{I}[y = c']}{p(c'|o)} \frac{\mathbb{I}[c = c'] \exp\{o^{c'}\} \text{MS} - \exp\{o^c\} \exp\{o^{c'}\}}{\text{MS}^2} \\
 &= - \sum_{c'=1}^p \mathbb{I}[y = c'] \frac{\mathbb{I}[c = c'] \text{MS} - \exp\{o^c\}}{\text{MS}} \\
 &= - \sum_{c'=1}^p \mathbb{I}[y = c'] (\mathbb{I}[c = c'] - p(c|o)) \\
 &= -(\mathbb{I}[c = y] - p(c|o))
 \end{aligned} \tag{3.5}$$

Sử dụng công thức đạo hàm hàm hợp để tính đạo hàm của hàm lỗi $\ell(o, y)$ với trọng số w^c , ta có:

$$\frac{\partial \ell(o, y)}{\partial w^c} = \frac{\partial \ell}{\partial o^c} \underbrace{\frac{\partial o^c}{\partial w^c}}_{f'(\text{net}^c)x} = -(\mathbb{I}[c = y] - p(y = c|o)) f'(\text{net}^c) \mathbf{x}$$

Và công thức tổng quát đạo hàm của hàm lỗi trên tập dữ liệu D là

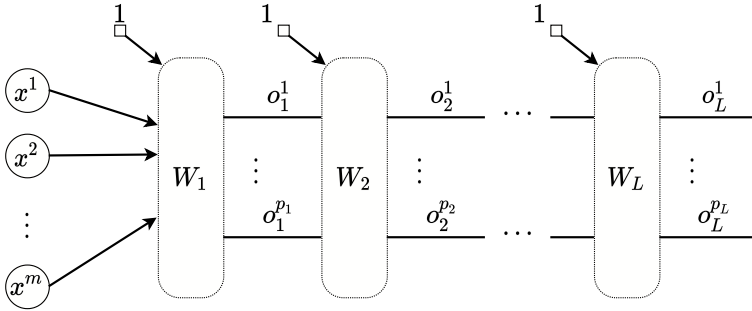
$$\begin{aligned}
 \frac{\partial \ell(W)}{\partial w^c} &= - \sum_{i=1}^n (\mathbb{I}[c = y_i] - p(y_i = c|o_i)) f'(\text{net}_i^c) \mathbf{x}_i \\
 \frac{\partial \ell}{\partial W} &= \left[\frac{\partial \ell}{\partial w^1} \quad \frac{\partial \ell}{\partial w^2} \quad \cdots \quad \frac{\partial \ell}{\partial w^p} \right]^T \in \mathbb{R}^{p \times m}
 \end{aligned}$$

Đây là các đạo hàm có thể dùng để điều chỉnh bộ trọng số W bằng phương pháp xuống đồi bằng đạo hàm, tương tự như phương pháp huấn luyện hàm hồi quy Logistic như trình bày trong phần 2.6.

3.1.3 Mạng nơ-ron nhiều lớp lan truyền tới

Trong phần trình bày ở mục 3.2, chúng ta thấy với một lớp nơ-ron nhất định, đặc trưng bởi ma trận trọng số $W \in \mathbb{R}^{p \times m}$ có thể ánh xạ dữ liệu từ không gian đầu vào $\mathbb{R}^m$ sang một không gian mới $\mathbb{R}^p$. Thay vì ngay lập tức mô hình xác suất đầu ra qua hàm softmax,

chúng ta có thể tiếp tục xây các ánh xạ này cho đến một độ sâu tùy ý bằng cách coi không gian $\mathbb{R}^p$ là không gian đầu vào mới. Đây là cấu trúc xếp lớp cơ bản của mạng học sâu dựa trên nơ-ron. Cụ thể hơn, đầu ra của lớp nơ-ron phía trước là đầu vào của lớp nơ-ron kế tiếp. Như vậy, đầu vào của lớp nơ-ron đầu tiên chính là x , còn đầu ra của lớp nơ-ron cuối cùng là đầu ra của toàn bộ mô hình (Hình 3.3). Cấu trúc này gọi là *mạng nơ-ron nhiều lớp lan truyền tới*, viết tắt là MLP (Multi Layer Perceptron).



Hình 3.3: Mạng nơ-ron nhiều lớp lan truyền tới.

Cụ thể, giả sử ta có L lớp nơ-ron nối tiếp nhau. Lớp thứ i có p_i đầu ra sử dụng bộ trọng số được cho bởi:

$$W_i = \begin{bmatrix} w_i^1 & w_i^2 & \dots & w_i^{p_i} \end{bmatrix}^T \in \mathbb{R}^{p_i \times p_{i-1}}.$$

Ta có công thức tổng quát để tính toán trên đầu ra của lớp $i - 1$ là

$$o_i = f_i(W_i o_{i-1}) \in \mathbb{R}^{p_i}, \quad (3.6)$$

với f_i là hàm kích hoạt ở lớp thứ i và đầu vào của lớp đầu tiên chính là dữ liệu đầu vào x

$$o_0 = \mathbf{x} \in \mathbb{R}^m,$$

và đầu ra của lớp thứ L cuối cùng là đầu ra của toàn bộ mô hình:

$$o = o_L \in \mathbb{R}^{p_L}.$$

Một cách tổng quát chúng ta có định nghĩa về mạng nơ-ron nhiều lớp như sau:

Định nghĩa 3.3 (Mạng nơ-ron nhiều lớp). Một mạng nơ-ron L lớp là ánh xạ từ véc-tơ đầu vào $\mathbf{x}$ đến véc-tơ kết quả o theo công thức

$$o = f_{\theta}(\mathbf{x}) = f_L(W_L f_{L-1}(W_{L-1} f_{L-2}(\dots f_1(W_1 \mathbf{x}))). \quad (3.7)$$

trong đó, $\theta = (W_1, \dots, W_L)$ là bộ tham số và các hàm $f_1, \dots, f_L$ là các hàm kích hoạt tại các lớp nơ-ron.

Trong công thức (3.7), chúng ta tạm thời coi các ma trận tham số có số chiều tương thích. Ngoài không gian đầu vào của $\mathbf{x}$ và không gian đầu ra của o , các kết quả trung gian o_i ở các lớp $1 < i < L$ trong công thức (3.2) được gọi là các miền biểu diễn *đặc trưng ẩn* (latent feature) hoặc gọi tắt là miền ẩn (latent space).

Điểm mạnh của mạng nơ-ron đến từ việc các lớp đơn lẻ khi kết nối với nhau có thể biểu diễn một hàm phi tuyến hết sức phức tạp, có thể xấp xỉ hầu hết các hàm số với độ chính xác cao. Kết quả này được trình bày trong Định lý Xấp xỉ toàn cục của mạng nơ-ron hai lớp do Cybenko [5] đề xuất như sau:

Định lý 3.4 (Định lý xấp xỉ toàn cục). *Với số lượng nơ-ron hữu hạn và hàm kích hoạt phi tuyến thích hợp, mạng nơ-ron hai lớp có thể xấp xỉ một hàm liên tục bất kỳ trên không gian con trù mật của $\mathbb{R}^m$ với độ chính xác $\epsilon > 0$ bất kỳ.*

Việc chứng minh Định lý Xấp xỉ toàn cục cần kiến thức cơ sở về giải tích hàm vượt qua ngoài phạm vi của giáo trình. Kết quả định lý chỉ ra rằng mạng nơ-ron hai lớp đủ linh hoạt để thể hiện một hàm số liên tục thỏa mãn một số điều kiện cho trước. Vì thế, nếu hàm số cần phải học được biết trước, thì chỉ cần tìm kiếm hàm số đó trong không gian tham số của mạng nơ-ron hai lớp với số

lượng nơ-ron đủ lớn. Trong thực tế của bài toán học máy thông kê, hàm số cần học không được biết trước, mà chỉ được ước lượng từ dữ liệu. Vì thế, việc xây dựng một mạng nơ-ron nhiều lớp với số lượng nơ-ron và số lớp lớn hơn được kì vọng là sẽ có kết quả xấp xỉ tốt hơn.

3.2 Huấn luyện mô hình mạng nơ-ron nhiều lớp

Pha huấn luyện của mạng nơ-ron MLP sử dụng một thuật toán nổi tiếng có tên *thuật toán lan truyền ngược* (back propagation). Thuật toán có mục tiêu tính đạo hàm của lỗi $\ell(o, y)$ đối với tất cả các trọng số W_i tại các lớp nơ-ron. Ý tưởng chính của thuật toán này là sử dụng công thức đạo hàm hàm hợp để lan truyền ngược đạo hàm từ lớp cuối về lớp đầu tiên (ngược hướng tính toán đầu ra của mạng). Đầu tiên, ta khai triển từ dạng tổng quát của hàm lỗi theo công thức là:

$$\begin{aligned} \delta_i &= \underbrace{\frac{\partial \ell(o, y)}{\partial o_i}}_{\in \mathbb{R}^{p_i}} = \underbrace{\left[\frac{\partial o_{i+1}}{\partial o_i} \right]^T}_{J_{i+1}^T \in \mathbb{R}^{p_i \times p_{i+1}}} \underbrace{\left[\frac{\partial \ell(o, y)}{\partial o_{i+1}} \right]}_{\delta_{i+1} \in \mathbb{R}^{p_{i+1}}} \\ &= J_{i+1}^T \delta_{i+1} \end{aligned}$$

Trong đó, ma trận Jacobian $J_{i+1} = \left[\frac{\partial o_{i+1}}{\partial o_i} \right] \in \mathbb{R}^{p_{i+1} \times p_i}$ là ma trận đạo hàm của o_{i+1} đối với o_i . Vì $o_{i+1} = f_{i+1}(W_{i+1} o_i)$ nên xét phần tử dòng j , cột k của ma trận J_{i+1} , ta có công thức đạo hàm của o_{i+1}^j đối với o_i^k như sau:

$$\begin{aligned} \frac{\partial o_{i+1}^j}{\partial o_i^k} &= \frac{\partial f_{i+1}([w_{i+1}^j]^T o_i)}{\partial o_i^k} \\ &= f'_{i+1}(\text{net}_{i+1}^j) w_{i+1}^{jk} \end{aligned}$$

Tiếp tục quá trình biến đổi tương tự và nhóm lại các đạo hàm theo công thức sau:

$$\nabla f_i = \text{diag} \left(f'_i(\text{net}_i^1), f'_i(\text{net}_i^2), \dots, f'_i(\text{net}_i^{p_i}) \right) \in \mathbb{R}^{p_i \times p_i}$$

trong đó, ∇f_i là ma trận đường chéo đạo hàm của hàm kích hoạt tại lớp nơ-ron thứ i thì ta có công thức tổng quát cho ma trận Jacobian J_{i+1} được rút gọn lại như sau:

$$J_{i+1} = W_{i+1} \nabla f_{i+1} \quad (3.8)$$

Thuật toán 3.1 Thuật toán lan truyền ngược

```

1: procedure BACKPROPAGATION( $\frac{\partial \ell(o_L, y)}{\partial o_L}, \nabla f_i, i = 1, \dots, L$ )
2:   Khởi tạo  $\delta_L = \frac{\partial \ell}{\partial o_L}$ 
3:   for  $i = L - 1, L - 2, \dots, 0$  do
4:     Tính  $J_{i+1} = W_{i+1} \nabla f_{i+1}$ 
5:     Lan truyền ngược  $\delta_i = J_{i+1}^T \delta_{i+1}$ 
6:   end for
7:   return  $\delta_i, i = 1, 2, \dots, L$ 
8: end procedure

```

Sau khi có giá trị $\delta_i = \frac{\partial \ell(o, y)}{\partial o_i}$ cho tất cả các lớp nơ-ron, ta có thể khai triển và sau đó tính đạo hàm của lỗi theo biến đổi sau

$$\begin{aligned}
 o_i^j &= f_i(\text{net}_i^j) = f_i([w_i^j]^T o_{i-1}) \\
 \frac{\partial \ell(o, y)}{\partial w_i^j} &= \frac{\partial \ell(o, y)}{\partial o_i^j} \frac{\partial o_i^j}{\partial w_i^j} \\
 &= \delta_i^j f'_i(\text{net}_i^j) o_{i-1} \\
 \underbrace{\frac{\partial \ell(o, y)}{\partial W_i}}_{\in \mathbb{R}^{p_i \times p_{i-1}}} &= \underbrace{\nabla f_i}_{\in \mathbb{R}^{p_i \times p_i}} \underbrace{\delta_i}_{\in \mathbb{R}^{p_i}} \underbrace{o_{i-1}^T}_{\in \mathbb{R}^{1 \times p_{i-1}}}
 \end{aligned}$$

Một nhận xét thú vị của thuật toán lan truyền ngược là thuật toán này có thể dùng để tính $\delta_0 = \frac{\partial \ell(o, y)}{\partial x}$. Đây là đạo hàm của hàm

lỗi đối với đầu vào x của mạng. Nhiều thuật toán học máy đã dùng đạo hàm này để tìm các mẫu dữ liệu đầu vào thoả mãn những tính chất nhất định. Ví dụ như tìm ảnh có phong cách giống với phong cách của một hoạ sĩ, tìm dữ liệu có khả năng phá hoại mô hình và nhiều ứng dụng khác.

Để thuật toán lan truyền ngược hoạt động, ta cần cung cấp đầu vào $\frac{\partial \ell}{\partial o}$ là đạo hàm của hàm lỗi đối với đầu ra của lớp cuối cùng. Trong trường hợp tổng quát của bài toán phân lớp, khi đó số đầu ra ở lớp cuối $p_L = C$ là số lớp cần phân loại, ta sử dụng hàm lỗi entropy chéo $\ell(o, y) = - \sum_{c=1}^C \mathbb{I}[y = c] \ln p(c|o)$. Viết lại công thức tính đạo hàm (3.5) ở dạng véc-tơ, ta thu được công thức sau:

$$\frac{\partial \ell}{\partial o} = \begin{bmatrix} p(y = 1|o) \\ p(y = 2|o) \\ \vdots \\ p(y = C|o) \end{bmatrix} - e_y$$

trong đó, e_y là véc-tơ cơ sở chuẩn, gồm toàn giá trị 0 ở những vị trí khác giá trị nhãn mục tiêu y và giá trị 1 ở vị trí nhãn mục tiêu y . Điều này giống với việc chúng ta chỉ cần tính đạo hàm của hàm lỗi đối với nhãn mục tiêu y . Ở pha suy luận, ta chỉ cần chọn $c^* = \arg \max_c o^c$ là nhãn có xác suất $p(y = c|o)$ lớn nhất.

Đạo hàm $\frac{\partial \ell(o, y)}{\partial w}$ ở trên mới chỉ tính trên một mẫu dữ liệu, còn đối với một tập dữ liệu $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$, ta tính đạo hàm của hàm tổng các lỗi trên từng dữ liệu bằng cách cộng từng đạo hàm của từng mẫu dữ liệu lại với nhau. Kết quả ta có thuật toán huấn luyện mạng nơ-ron MLP (Thuật toán 3.2).

Thuật toán 3.2 duyệt qua bộ dữ liệu theo từng epoch. Với mỗi epoch, đầu tiên ta khởi tạo đạo hàm bằng 0. Sau đó, với từng dữ liệu, thực hiện lan truyền tới, lan truyền ngược để tính đạo hàm rồi tích lũy đạo hàm bằng cách cộng dồn. Kết thúc mỗi epoch, ta cập nhật lại trọng số ở các lớp theo phương pháp xuống đồi bằng

Thuật toán 3.2 Thuật toán huấn luyện mạng nơ-ron MLP

```

1: procedure TRAINMLP( $D, \lambda$ )
2:   Khởi tạo  $W_1, W_2, \dots, W_L$  ngẫu nhiên
3:   for epoch = 1, 2, ... do
4:     for  $k = 1$  to  $L$  do
5:        $\frac{\partial \ell}{\partial W_k} \leftarrow 0$ 
6:     end for
7:     for all dữ liệu  $(x_i, y_i)$  trong  $D$  do
8:       Lan truyền tới, tính  $o_1, o_2, \dots, o_L, \nabla f_1, \nabla f_2, \dots, \nabla f_L$ 
9:       Tính đạo hàm hàm lỗi:  $\frac{\partial \ell(o_L, y_i)}{\partial o_L}$ 
        $\delta_L, \delta_{L-1}, \dots, \delta_0$ 
10:      = BACKPROPAGATION  $\left( \frac{\partial \ell(o_L, y_i)}{\partial o_L}, \nabla f_{1 \dots L} \right)$ 
11:      for  $k = 1$  to  $L$  do
12:         $\frac{\partial \ell}{\partial W_k} \leftarrow \frac{\partial \ell}{\partial W_k} + \nabla f_k \delta_k o_{k-1}^T$ 
13:      end for
14:    end for
15:    for  $k = 1$  to  $L$  do
16:       $W_k \leftarrow W_k - \lambda \cdot \frac{\partial \ell}{\partial W_k}$ 
17:    end for
18:  end for
19:  return  $W_1, W_2, \dots, W_L$ 
20: end procedure

```

đạo hàm. Tất nhiên, còn có nhiều phương pháp tối ưu khác, chúng ta sẽ tìm hiểu ở một phần riêng của giáo trình.

3.3 Các thuật toán tối ưu

Thuật toán tối ưu được trình bày trong Thuật toán 3.2 yêu cầu đạo hàm được tính toàn bộ tập dữ liệu, sau đó lỗi này sẽ được lan truyền ngược lại và cập nhật đối với từng trọng số. Trước khi đi vào các thuật toán tối ưu, chúng ta sẽ xem xét đến tính hội tụ của các tham số trong mô hình mạng lan truyền tới với thiết lập cơ bản.

3.3.1 Tính hội tụ của phương pháp xuống đồi bằng đạo hàm

Xét trường hợp mạng nơ-ron một lớp trong định nghĩa 3.1 và được huấn luyện thông qua hàm lỗi entropy chéo (công thức (3.3)) trên tập dữ liệu D cho trước.

Theo Thuật toán huấn luyện mạng nơ-ron MLP, ta có thể thấy rằng, ở bước thứ t ứng với tham số θ_t (gồm tất cả các ma trận trọng số $W_k, k = 1, \dots, L$), ta có thể viết lại công thức cập nhật θ_{t+1} như sau:

$$\theta_{t+1} = \theta_t - \eta \frac{\partial \ell}{\partial \theta_t} \quad (3.9)$$

Sử dụng công thức cập nhật trên, xuất phát từ tham số khởi tạo θ_0 ta có các giá trị hàm lỗi $\ell(\theta_t)$ ứng với giá trị θ_t được cập nhật tại bước thứ t . Chúng ta quan tâm đến có tồn tại giá trị θ^* nào đó sao cho dãy giá trị

$$\ell(\theta_0), \ell(\theta_1), \ell(\theta_2), \dots, \ell(\theta_t), \dots,$$

có hội tụ về giá trị cố định $\ell(\theta^*)$ hay không. Nếu tồn tại giá trị θ^* sao cho dãy giá trị hàm lỗi hội tụ về $\ell(\theta^*)$, thì ta có thể nói rằng thuật toán tối ưu hội tụ về giá trị $\ell(\theta^*)$.

Để trả lời câu hỏi này, chúng ta cần xét đến một trong những đặc điểm quan trọng của hàm lỗi đó là tính trơn của hàm lỗi.

Định nghĩa 3.5 (Tính trơn của hàm lỗi). Một hàm $F : \mathbb{R}^m \rightarrow \mathbb{R}$ được gọi là có hàm trơn theo L nếu F liên tục, khả vi và tồn tại một hằng số $L > 0$ sao cho với mọi $x, y \in \mathbb{R}^m$ ta có:

$$|\nabla F(x) - \nabla F(y)| \leq L \|x - y\|_2.$$

Với định nghĩa hàm lỗi $\ell(\theta)$ theo công thức (3.3) và giá trị đạo hàm $\nabla \ell(\theta) = \frac{\partial \ell}{\partial \theta}$ tương ứng, ta có thể thấy rằng hàm lỗi $\ell(\theta)$ thỏa mãn định nghĩa về tính trơn của hàm lỗi. Bên cạnh đó hàm lỗi $\ell(\theta)$ cũng có thể được xem như là một hàm lỗi dựa theo tính chất của

hàm softmax và hàm entropy chéo. Từ dữ kiện này, chúng ta có thể lựa chọn tốc độ học η để đảm bảo tính hội tụ của dãy $\ell(\theta_t)_{t=0}^{\infty}$ theo định lý sau:

Định lý 3.6 (Tính hội tụ của hàm lỗi). *Giả sử hàm lỗi $\ell(\theta)$ có đạo hàm trơn theo L và tồn tại một giá trị θ^* sao cho $\ell(\theta^*) = \min_{\theta} \ell(\theta)$. Nếu tốc độ học η được chọn sao cho $0 < \eta < \frac{1}{L}$, thì*

$$\min_{t=1}^T \ell(\theta_t) - \ell(\theta^*) = O(1/T) \rightarrow 0$$

và

$$\ell\left(\frac{1}{T} \sum_{t=1}^T \theta_t\right) - \ell(\theta^*) = O(1/T) \rightarrow 0.$$

Chứng minh: Để chứng minh định lý này, ta sẽ sử dụng một số tính chất của hàm lỗi và hàm trơn. Dựa theo khai triển Taylor bậc nhất của hàm lỗi f tại điểm x và với điểm y bất kì ta có:

$$\ell(y) \leq \ell(x) + \nabla \ell(x)^T (y - x) + \frac{L}{2} \|y - x\|^2.$$

Thay $x = \theta_t$ và $y = \theta_{t+1}$, sử dụng khai triển trên ta có:

$$\begin{aligned} \ell(\theta_{t+1}) &\leq \ell(\theta_t) - \eta \|\nabla \ell(\theta_t)\|^2 + \frac{L\eta^2}{2} \|\nabla \ell(\theta_t)\|^2 \\ &= \ell(\theta_t) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla \ell(\theta_t)\|^2 \\ &\leq \ell(\theta_t) - \frac{\eta}{2} \|\nabla \ell(\theta_t)\|^2. \end{aligned}$$

Trong đó, bất đẳng thức cuối cùng do $\eta \leq \frac{1}{L}$, nên $1 - \frac{L\eta}{2} \geq \frac{1}{2}$.

Sử dụng tính chất hàm lỗi, với θ bất kì

$$\ell(\theta) - \ell^* \leq (\theta - \theta^*) \cdot \nabla \ell(\theta),$$

ta có

$$\begin{aligned}
\|\theta_{t+1} - \theta^*\|^2 &= \|\theta_t - \theta^* - \eta \nabla \ell(\theta_t)\|^2 \\
&= \|\theta_t - \theta^*\|^2 - 2\eta(\theta_t - \theta^*) \cdot \nabla \ell(\theta_t) + \eta^2 \|\nabla \ell(\theta_t)\|^2 \\
&\leq \|\theta_t - \theta^*\|^2 - 2\eta(\ell(\theta_t) - \ell^*) + \eta^2 \|\nabla \ell(\theta_t)\|^2 \\
&= \|\theta_t - \theta^*\|^2 - 2\eta \left(\ell(\theta_t) - \ell^* + \frac{\eta}{2} \|\theta_t - \theta^*\|^2 \right) \\
&\leq \|\theta_t - \theta^*\|^2 - 2\eta (\ell(\theta_{t+1}) - \ell^*)
\end{aligned}$$

Cộng các bất đẳng thức trên với $t = 0, 1, \dots, T-1$ ta có:

$$\|\theta_T - \theta^*\|^2 \leq \|\theta_0 - \theta^*\|^2 - 2\eta \sum_{t=0}^{T-1} (\ell(\theta_{t+1}) - \ell^*).$$

Từ bất đẳng thức trên, suy ra:

$$\frac{1}{T} \sum_{t=1}^T (\ell(\theta_t) - \ell^*) = \frac{1}{T} \sum_{t=1}^T \ell(\theta_t) - \ell^* \leq \frac{\|\theta_0 - \theta^*\|^2}{2\eta T} = O(1/T). \quad (3.10)$$

Do giá trị nhỏ nhất luôn bé hơn trung bình cộng nên

$$\min_{t=1}^T \ell(\theta_t) \leq \frac{1}{T} \sum_{t=1}^T \ell(\theta_t),$$

và, do tính chất của hàm lồi

$$\ell \left(\frac{1}{T} \sum_{t=1}^T \theta_t \right) \leq \frac{1}{T} \sum_{t=1}^T \ell(\theta_t)$$

ta có điều phải chứng minh. $\square$

Như vậy hoặc ta lấy trung bình cộng của giá trị tham số trong quá trình huấn luyện hoặc ta lấy tham số tương ứng với giá trị hàm lồi nhỏ nhất trong quá trình huấn luyện. Trong thực hành, người

ta thường sử dụng phép lấy tổng có trọng số mũ của các giá trị tham số trong khi huấn luyện. Cụ thể như sau

$$\bar{\theta}_0 = \theta_0 \quad (3.11)$$

$$\bar{\theta}_t = (1 - \alpha)\bar{\theta}_{t-1} + \alpha\theta_t \quad (3.12)$$

với $0 < \alpha < 1$ là một hằng số nhỏ (ví dụ, 10^{-2} , 10^{-3}).

Để đảm bảo tính hội tụ của thuật toán tối ưu theo công thức (3.10) chúng ta cần đảm bảo tính trơn và lồi của hàm lỗi. Trong trường hợp của mạng nơ-ron với một lớp nơ-ron duy nhất, hàm lỗi được định nghĩa theo công thức (3.3) thỏa mãn điều kiện này. Khi số lớp nơ-ron trong mạng lan truyền tới tăng lên hai hoặc ba lớp, hàm lỗi không còn là hàm lồi nữa. Điều này dẫn đến việc không thể đảm bảo tính hội tụ của thuật toán tối ưu theo công thức (3.10). Chúng ta không đi sâu vào vấn đề hội tụ của thuật toán tối ưu xuống đồi bằng đạo hàm với trường hợp mạng nơ-ron có cấu trúc phức tạp hơn.

Về mặt thực nghiệm, có hai vấn đề xảy ra với thuật toán xuống đồi bằng đạo hàm thông thường:

- Lấy trung bình giá trị cập nhật theo hướng đạo hàm giảm trên toàn tập dữ liệu làm cho các sai số cộng trừ lẫn nhau dẫn đến bước tiến của thuật toán tối ưu sẽ nhỏ, khó tìm được điểm hội tụ thích hợp.
- Mỗi lần lặp cần phải duyệt cả tập dữ liệu, độ phức tạp là $O(K|D|)$ trong đó K là độ phức tạp đối với thuật toán lan truyền ngược trên một điểm dữ liệu. Có thể tính gần đúng là tuyến tính với số tham số trong mô hình mạng lan truyền tới.

Điều này dẫn đến thuật toán không khả thi nếu số lượng mẫu dữ liệu rất lớn. Trên thực tế, chúng ta sẽ tìm một hướng đi dẫn đến cập nhật tham số cho mô hình nơ-ron đủ tốt trong thời gian cho

phép. Để cải thiện hiệu suất của thuật toán xuống đồi bằng đạo hàm thông thường, một số kỹ thuật tối ưu được đề xuất như sau.

3.3.2 Thuật toán xuống đồi bằng đạo hàm ngẫu nhiên

Thuật toán xuống đồi bằng đạo hàm ngẫu nhiên là biến thể dựa trên thuật toán thông thường. Thuật toán này rất phổ biến trong học máy và học sâu và thường được viết tắt là SGD (Stochastic Gradient Descent). Thay vì mỗi lần việc tính lỗi và giá trị đạo hàm lỗi trên toàn bộ tập dữ liệu, chỉ có một tập con của dữ liệu được sử dụng. Kích thước của tập con này thường được chọn là các lũy thừa bậc 2 như: 8, 16, 32, 64. Ví dụ, với tập dữ liệu $|\mathcal{D}|$ có 1000 phần tử, mỗi lượt cập nhật sẽ chọn một tập con có 32 phần tử từ 1000 phần tử. Sau đó thuật toán sẽ tính lỗi, giá trị đạo hàm lỗi trên 32 phần tử này và cập nhật lại tham số W_i theo giá trị này.

Vì có rất nhiều cách chọn tập con có kích thước xác định từ tập dữ liệu ban đầu, dẫn đến giá trị đạo hàm lỗi dùng để cập nhật tham số là một giá trị ngẫu nhiên theo một phân bố nhất định. Nếu đảm bảo quá trình chọn ra tập con là ngẫu nhiên thì với số lượng vòng lặp tiến đến vô cùng, hướng hội tụ của tham số sẽ tiệm cận với hướng đi tối ưu. Trong nhiều trường hợp, do ít hoặc không bị ảnh hưởng của tính loại trừ lẫn nhau, tốc độ hội tụ của thuật toán SGD nhanh hơn nhiều so với thuật toán GD thông thường. Mã giả của thuật toán SGD (Thuật toán 3.3) được phát triển từ Thuật toán 3.2. Ở đây, chúng ta sử dụng thêm tham số b dùng để chỉ ra số lượng mẫu trong mỗi lô dữ liệu (batch) D_e được chọn ngẫu nhiên từ D .

3.3.3 Thuật toán SGD với quán tính

Thuật toán SGD có thể tăng tốc độ tính toán và hội tụ so với thuật toán GD thông thường. Tuy nhiên, trong một số trường hợp hàm

Thuật toán 3.3 Thuật toán SGD huấn luyện mạng nơ-ron MLP

```

1: procedure TRAINMLP( $D, \lambda, b$ )
2:   Khởi tạo  $W_1, W_2, \dots, W_L$  ngẫu nhiên
3:   for  $e = 1, 2, \dots$  do
4:     for  $k = 1$  to  $L$  do
5:        $\frac{\partial \ell}{\partial W_k} \leftarrow 0$ 
6:     end for
7:     Chọn ngẫu nhiên tập con  $\mathcal{D}_e \subset \mathcal{D}$  gồm  $b$  phần tử
8:     for all  $(x_i, y_i) \in \mathcal{D}_e$  do
9:       Lan truyền tới, tính  $o_1, \dots, o_L$  và  $\nabla f_1, \dots, \nabla f_L$ 
10:      Tính đạo hàm lỗi:  $\frac{\partial \ell(o_L, y_i)}{\partial o_L}$ 
11:      Lan truyền ngược, tính  $\delta_L, \dots, \delta_0$ 
12:      for  $k = 1$  to  $L$  do
13:         $\frac{\partial \ell}{\partial W_k} \leftarrow \frac{\partial \ell}{\partial W_k} + \nabla f_k \delta_k o_{k-1}^T$ 
14:      end for
15:    end for
16:    for  $k = 1$  to  $L$  do
17:       $W_k \leftarrow W_k - \lambda \cdot \frac{\partial \ell}{\partial W_k}$ 
18:    end for
19:  end for
20:  return  $W_1, W_2, \dots, W_L$ 
21: end procedure

```

tối ưu là các miền nhấp nhô không đồng đều, thuật toán SGD có thể mất rất nhiều thời gian để đi thoát được khỏi miền tối ưu đầy.

Trong trường hợp này, SGD được thêm vào thành phần quán tính. Sử dụng quán tính giúp cho thuật toán SGD có thể được tăng tốc theo chiều đúng hướng đồng thời giảm độ dao động đối với các tín hiệu đạo hàm bị nhiễu. Chúng ta có thể hiểu quán tính là một đại lượng đặc trưng cho tốc độ của quá trình hội tụ và được cập nhật theo phương trình sau:

$$v_t = \gamma v_{t-1} + \lambda \nabla_{\theta} J(\theta) \quad (3.13)$$

$$\theta = \theta - v_t \quad (3.14)$$

Trong công thức trên, đại lượng λ là tốc độ học. Đại lượng $\nabla_{\theta}J(\theta)$ đặc trưng cho đạo hàm được tính qua tốc độ học thành đại lượng vận tốc v_t được cập nhật vào tham số θ ở phương trình dưới. Đại lượng quán tính γ đặc trưng cho tốc độ đang có, thường được chọn là 0.9 trong thực nghiệm.

Một cách dễ hình dung, việc sử dụng quán tính trong cập nhật trọng số ở công thức (3.13) giống như việc thả một trái bóng lăn xuống dốc. Khi ở đỉnh dốc tốc độ chậm nhưng càng di chuyển tốc độ sẽ càng nhanh và giảm dần khi gần đến vị trí hội tụ tại cục bộ địa phương. Song song với việc tốc độ tăng theo hướng hội tụ, các nhiễu của đạo hàm cũng được loại bỏ. Kết quả là chúng ta sẽ có được một thuật toán cho phép tốc độ hội tụ cao và giảm tính giao động ảnh hưởng bởi nhiễu.

Về mặt tính toán, so với thuật toán SGD, thuật toán SGD với quán tính phải thêm vào một biến phụ để lưu lại vận tốc tại thời điểm t ứng với mỗi tham số trong mô hình θ . Về mặt độ phức tạp tính toán thì cả hai tương đương nhau.

3.3.4 Thuật toán ước lượng quán tính thay đổi - Adam

Thuật toán ước lượng quán tính thay đổi hay viết ngắn gọn là Adam là một phương pháp tối ưu dựa trên thuật toán SGD. Về mặt ý tưởng, thuật toán Adam đưa vào khái niệm ma sát cản trở chuyển động khi quả bóng lăn xuống dốc. Thay vì sử dụng một giá trị quán tính duy nhất γ như trong thuật toán SGD quán tính, Adam sử dụng một ước lượng dựa trên giá trị giảm trung bình theo hàm mũ với gia tốc của v_t . Bên cạnh đó Adam vẫn sử dụng giá trị trung bình theo thời gian của v_t làm đại lượng quán tính tương tự với thuật toán SGD quán tính.

$$m_t = \beta_1^t m_{t-1} + (1 - \beta_1) g_t \quad (3.15)$$

$$v_t = \beta_2^t v_{t-1} + (1 - \beta_2) g_t^2 \quad (3.16)$$

Trong công thức trên, g_t là đại lượng ứng với gia tốc của vận tốc v_t , nói một cách khác là đạo hàm sự thay đổi của đạo hàm theo thời gian ứng với một tham số cần tối ưu. Giá trị m_t và v_t là ước lượng tương ứng của mô-men thứ nhất và mô-men thứ hai của đạo hàm. Hai giá trị này sẽ được sử dụng để cập nhật tham số θ của mô hình

$$\theta_{t+1} = \theta_t - \frac{\lambda}{\sqrt{v_t} + \epsilon} m_t.$$

Trong thực nghiệm, β_1 thường được chọn là 0.9 và β_2 thường được chọn là 0.999. Ở những bước khởi tạo, với $v_0 = 0$ và $g_0 = 0$, giá trị thường bị nhiễu và tiến dần đến 0. Tham số ϵ thường chọn khá bé, khoảng 10^{-8} . Để hạn chế điều này, hàm ước lượng mô-men m_t và v_t có thể được nhân với một hàm chuẩn hóa theo thời gian. So với thuật toán SGD quán tính thì thuật toán Adam phải lưu thêm biến phụ m_t , g_t đối với mỗi trọng số cần tối ưu.

Thuật toán Adam thường được sử dụng rộng rãi trong thực nghiệm vì khả năng tìm được các điểm cực bộ địa phương hiệu quả. Việc lựa chọn tốc độ học cho thuật toán Adam cũng dễ hơn các thuật toán khác ví dụ như SGD.

3.4 Các vấn đề liên quan đến tính hội tụ

Các phương pháp tối ưu ở trên có những cách cập nhật trọng số trực tiếp khác nhau theo giá trị đạo hàm ở thời điểm t . Tuy nhiên tất cả đều dựa trên thuật toán gốc là GD ngẫu nhiên.

Hướng nghiên cứu lý thuyết trong bài toán học sâu thường quan tâm đến tính hội tụ của thuật toán GD. Các mạng học sâu có thể bao gồm vài chục đến vài trăm lớp với hàng triệu tham số cần tối ưu. Các hàm lỗi cần tối ưu thường là các hàm phức tạp, không có tính tuyến tính. Một số hướng nghiên cứu chính về tính hội tụ của thuật toán GD trong mạng học sâu bao gồm:

- Tính hoàn chỉnh: thuật toán SGD có tìm được điểm cực tiểu địa phương hay không
- Tốc độ hội tụ: cần bao nhiêu bước để thuật toán SGD có thể tìm được điểm cực tiểu cần thiết
- Tính tối ưu: độ chênh lệch của điểm hội tụ tìm được của thuật toán SGD so với cực tiểu toàn cục
- Tính tổng quát hóa: điểm cực tiểu tìm được có thể đưa ra hàm lỗi có giá trị thấp trên miền dữ liệu không được huấn luyện hay không

Về mặt thực nghiệm, thuật toán SGD và các biến thể đi kèm với các siêu tham số cần phải được lựa chọn cẩn thận. Tùy vào từng bài toán cụ thể mà chúng ta có thể có những lưu ý sau để nâng cao tốc độ hội tụ của thuật toán tối ưu mạng học sâu lan truyền tới:

- Tốc độ học: tốc độ học cần được lựa chọn cẩn thận. Nếu lớn quá sẽ làm các bước cập nhật bỏ qua các vùng chứa điểm tối ưu. Nếu bé quá sẽ dẫn đến lâu hội tụ, tăng thời gian tính toán không cần thiết.
- Tốc độ học linh hoạt: tốc độ học có thể được thay đổi một cách linh hoạt trong cả quá trình huấn luyện. Thông thường, tốc độ học ở thời điểm ban đầu sẽ lớn và giảm dần theo thời gian khi hàm lỗi tiệm cận giá trị cực tiểu.
- Tốc độ học không đồng nhất: trong phần lớn bài toán các lớp của mạng học sâu có đóng góp khác nhau đến biểu diễn ẩn nổi từ không gian đầu vào và không gian của hàm mục tiêu. Các lớp gần với không gian đầu vào thường có độ ổn định cao hơn so với các lớp gần không gian của hàm mục tiêu. Điều này có thể đưa đến các cải tiến liên quan đến việc giảm dần tốc độ học

theo khoảng cách đến không gian đầu ra. Kỹ thuật này thường được áp dụng trong các bài toán liên quan đến việc tinh chỉnh tham số của một mạng xương sống cho trước.

3.5 Tình huống áp dụng: Nhận dạng chữ số MNIST

Bộ dữ liệu MNIST là tập dữ liệu phổ biến dùng để nhận diện chữ số viết tay (0-9), bao gồm 60.000 hình ảnh cho tập huấn luyện và 10.000 hình ảnh cho tập kiểm tra. Trong phần áp dụng này, thư viện **PyTorch** sẽ được dùng để xây dựng và huấn luyện mạng học sâu với bộ dữ liệu MNIST.

3.5.1 Giới thiệu thư viện Pytorch

PyTorch¹ là một thư viện linh hoạt để xây dựng các mô hình học sâu. Hai chức năng chính của **PyTorch** là hỗ trợ tính toán Xuống đồi bằng đạo hàm và tính toán song song trên các đơn vị tính toán đồ họa (GPU). Các mô hình học sâu trong **PyTorch** được xây dựng dựa trên các lớp trừu tượng *torch.nn.Module*. Người sử dụng có thể dùng các lớp có sẵn trong thư viện hoặc tự định nghĩa các lớp mới bằng cách kế thừa lớp *nn.Module*. Các lớp được định nghĩa sẵn bao gồm các lớp mạng nơ-ron phổ biến, các lớp kích hoạt. **PyTorch** cũng cung cấp các lớp tối ưu hoá như Adam, SGD, Adagrad, RMSprop.

3.5.2 Xây dựng mô hình mạng nơ-ron nhiều lớp

Khi sử dụng mạng nơ-ron nhiều lớp, chúng ta cần thực hiện định nghĩa các lớp mô hình mạng nơ-ron và mô hình tính toán. Trong thư viện **PyTorch**, chúng ta có thể định nghĩa mô hình bằng cách kế thừa lớp *nn.Module*, và định nghĩa các lớp con cho các lớp mạng nơ-ron. Cùng với đó, chúng ta cũng định nghĩa mô hình tính toán, được thể hiện ở phần hàm *forward*. Hàm này sẽ xác định cách mà

¹<https://pytorch.org/>

dữ liệu đầu vào được truyền qua các lớp của mạng nơ-ron và trả về kết quả đầu ra cuối cùng, ứng với véc-tơ đầu ra của mạng nơ-ron theo Định nghĩa 3.3. Trừ lớp ẩn cuối cùng, các lớp ẩn đều sử dụng hàm kích hoạt ReLU. Chúng ta sử dụng hàm kích hoạt softmax cho lớp đầu ra vì chúng ta cần một phân phối xác suất cho các lớp đầu ra. Lớp đầu ra sẽ trả về một véc-tơ có kích thước bằng số lớp cần phân loại.

Bảng 3.2 mô tả cấu trúc của mạng nơ-ron nhiều lớp cho bài toán MNIST. Với khai báo này, mô hình sẽ gồm có 109386 tham số.

Bảng 3.2: Cấu trúc mạng nơ-ron nhiều lớp cho bài toán MNIST

Lớp	Kích thước đầu vào	Kích thước đầu ra
Lớp ẩn 1 - fc1	784	128
Lớp ẩn 2 - fc2	128	64
Đầu ra - fc3	64	10

3.5.3 Các bước triển khai chính

Trong phần này, chúng ta sẽ thực hiện các bước sau để xây dựng mô hình phân lớp với bộ dữ liệu MNIST. Các bước chính bao gồm:

- Chuẩn bị dữ liệu: Tải dữ liệu MNIST, chuẩn hóa và tạo bộ dữ liệu huấn luyện/kiểm tra.
- Xây dựng mô hình mạng nơ-ron nhiều lớp: Định nghĩa các lớp mạng nơ-ron và mô hình tính toán.
- Huấn luyện mô hình: Sử dụng hàm lỗi và bộ tối ưu hoá để huấn luyện mô hình.
- Đánh giá kết quả dự đoán trên tập dữ liệu kiểm thử: Sử dụng tập dữ liệu kiểm thử để đánh giá độ chính xác của mô hình.

- Lưu trữ và sử dụng lại mô hình đã huấn luyện: Lưu trữ mô hình đã huấn luyện để sử dụng lại trong tương lai.

Mô hình có thể đạt độ chính xác khoảng 97-98% trên tập kiểm thử. Người học có thể tham khảo mã nguồn tại <https://gist.github.com/cuongtv312/9a69c1004619be658eca90b3ac8c6bd3>

3.6 Tổng kết chương

Chương 3 đã giới thiệu mạng nơ-ron nhiều lớp (MLP) – nền tảng cốt lõi của Học sâu. Người học được tiếp cận từ kiến trúc mạng, thuật toán huấn luyện, đến các vấn đề về tính hội tụ và hiệu quả mô hình. Ứng dụng phân loại ảnh viết tay MNIST minh họa rõ khả năng vượt trội của MLP so với các mô hình học máy truyền thống.

Chương này cũng nhấn mạnh vai trò trung tâm của thuật toán lan truyền ngược và các kỹ thuật tối ưu như Xuống đồi bằng đạo hàm, đạo hàm ngẫu nhiên, kỹ thuật quán tính, thuật toán Adam. Kỹ năng xây dựng và huấn luyện MLP là nền tảng cần thiết để tiếp cận các mạng học sâu hơn trong các chương kế tiếp.

Bài tập

1. Xét cấu trúc mạng nơ-ron được minh họa ở Hình 3.2 với $p = 2$. Chúng ta thấy rằng trong trường hợp này, có thể huấn luyện trực tiếp bằng cách sử dụng mô hình hồi quy Logistic. So sánh sự giống nhau và khác nhau giữa mô hình ở Hình 3.2 với mô hình hồi quy Logistic về số lượng tham số, cách tính toán đầu ra và cách huấn luyện.
2. Xây dựng một mạng nơ-ron nhiều lớp với 2 lớp ẩn, mỗi lớp có 10 nơ-ron. Dữ liệu đầu vào có 5 chiều và kết quả đầu ra dùng

Bảng 3.3: Dữ liệu phân lớp

N	x	y	N	x	y
1	0.3	1	6	0.9	1
2	1.2	1	7	-0.3	0
3	0.6	0	8	2.4	1
4	0.3	1	9	2.8	1
5	-0.5	0	10	-0.7	1

để nhận dạng giá trị Có/Không. Trừ lớp đầu ra, các lớp ẩn đều sử dụng hàm ReLU là hàm kích hoạt.

- a) Vẽ hình minh hoạ cho mạng nơ-ron này
 - b) Xác định các tham số huấn luyện và viết hàm tính toán đầu ra của mạng nơ-ron này.
3. *[Tìm hiểu]* Tìm một ví dụ mà SGD hội tụ nhưng GD luôn nhận được lượng cập nhật bằng 0.
4. Cho mô hình được cho bởi công thức dưới đây, biết rằng a là hàm ReLU có công thức:

$$o = \phi_0 + \phi_1 a(\theta_{10} + \theta_{11}x) + \phi_2 a(\theta_{20} + \theta_{21}x)$$

$$f(x, \phi, \theta) = \frac{1}{1 + e^{-o}} \tag{3.17}$$

với tham số $\phi = [\phi_0, \phi_1, \phi_2]$ và $\theta = [\theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}]$.

- a) Vẽ hình minh hoạ
 - b) Viết hàm tính toán đạo hàm của hàm hồi quy logistic theo các tham số ϕ và θ .
5. *[Lập trình]* Cho tập dữ liệu phân lớp $D = \{(x_i, y_i)\}_{i=1}^{10}$ như Bảng 3.3. Trong đó x_i là đầu vào của mạng nơ-ron, y_i là đầu ra của mạng nơ-ron. Sử dụng mô hình được định nghĩa theo công thức (3.17) để học dữ liệu trên.

- a) Cài đặt hàm f và tính toán đạo hàm
 - b) Sử dụng phương pháp đạo hàm trên cả tập D để huấn luyện mạng nơ-ron.
 - c) Sử dụng phương pháp đạo hàm ngẫu nhiên SGD để huấn luyện mạng nơ-ron. So sánh với kết quả thu được ở câu trên.
6. Giải thích tại sao trong huấn luyện mạng học sâu cần phải chia ra tập huấn luyện và tập kiểm thử?
7. Nêu vai trò của quán tính trong việc huấn luyện mạng nơ-ron bằng phương pháp Xuống đồi bằng đạo hàm ngẫu nhiên SGD. Làm sao để ước lượng quán tính một cách chính xác dưới sự ảnh hưởng của nhiễu trong SGD?
8. *[Lập trình]* Trong phần tình huống áp dụng, thay đổi số lớp ẩn và số lượng nơ-ron mỗi lớp ẩn. Bạn có nhận xét gì về các kết quả thử nghiệm nhận được?

Tài liệu tham khảo

- [1] Rosenblatt, F., *The perceptron: A probabilistic model for information storage and organization in the brain*, Psychological Review, vol. 65, no. 6, pp. 386–408, 1958.
- [2] Rumelhart, D. E., Hinton, G. E., and Williams, R. J., *Learning representations by back-propagating errors*, Nature, vol. 323, no. 6088, pp. 533–536, 1986.
- [3] Bishop, C. M., *Neural networks for pattern recognition*, Oxford University Press, 1995.
- [4] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P., *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] Cybenko, G. (1989) *Approximation by superpositions of a sigmoidal function*, Mathematics of Control, Signals, and Systems, 2(4), pp. 303–314. Available at: <https://doi.org/10.1007/BF02551274>.

Chương 4

Máy véc-tơ hỗ trợ

Chương này giới thiệu Máy véc-tơ hỗ trợ SVM (Support Vector Machine), một thuật toán học máy cổ điển nhưng vẫn giữ vai trò quan trọng trong các bài toán phân lớp và hồi quy. SVM phát triển từ mô hình Perceptron – một trong những kiến trúc phân lớp đầu tiên trong học máy – với nhiều cải tiến đáng kể về lý thuyết và hiệu quả thực nghiệm.

Nguyên lý hoạt động cốt lõi của SVM là cực đại hoá lề giữa các lớp, nhằm tìm ra siêu phẳng phân tách với khoảng cách tối đa đến các điểm dữ liệu gần nhất thuộc mỗi lớp. Về mặt toán học, SVM là lời giải cho một bài toán tối ưu bậc hai với các ràng buộc tuyến tính, phản ánh sự kết hợp giữa học máy thống kê và tối ưu lồi.

Chương 4 cũng đề cập đến các biến thể quan trọng của SVM như lề mềm và kỹ thuật nhân hoá, cho phép mở rộng mô hình sang các bài toán phân lớp phi tuyến.

4.1 Mô hình nơ-ron nhân tạo Perceptron

Chúng ta bắt đầu chương này với nguyên lý hoạt động của mô hình nơ-ron nhân tạo đầu tiên, mô hình Perceptron. Mô hình Perceptron tiếp cận *bài toán phân lớp nhị phân* bằng cách trực tiếp ước lượng hàm phân lớp $h^*(\mathbf{x})$. Giả sử tập đầu vào $\mathcal{X} = \mathbb{R}^m$, nghĩa là đầu vào $\mathbf{x}$ gồm m số thực $x^1, \dots, x^m$ và $\mathcal{Y} = \{\pm 1\}$. Không mất tính tổng quát, chúng ta sử dụng phương pháp mã hóa nhị phân bằng $+1$ cho lớp dương và -1 cho lớp âm. Mô hình Perceptron tìm cách xấp xỉ hàm quyết định $h^*(\mathbf{x})$ bằng hàm phân lớp tuyến tính $h(\mathbf{x})$ như sau:

$$h(\mathbf{x}) = \begin{cases} +1 & \text{nếu } f(\mathbf{x}) \geq 0 \\ -1 & \text{nếu } f(\mathbf{x}) < 0 \end{cases}$$

trong đó, $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = w^1 x^1 + w^2 x^2 + \dots + w^m x^m + b$. Để cho gọn, ta còn viết $h(\mathbf{x}) = \text{sgn}(f(\mathbf{x}))$ với $\text{sgn}(z)$ là hàm dấu của z . Véc-tơ $\mathbf{w} = (w^1, \dots, w^m)$ và b là các tham số của mô hình Perceptron.

Lỗi của mô hình. Xét một mẫu dữ liệu $(\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}$. Hàm $h(\mathbf{x})$ phân lớp sai nếu $h(\mathbf{x}) \neq y$. Do $y = \pm 1$ và $h(\mathbf{x}) = \text{sgn}(f(\mathbf{x}))$ nên điều kiện này tương đương với $yf(\mathbf{x}) < 0$. Ngược lại, nếu $yf(\mathbf{x}) \geq 0$ thì hàm $h(\mathbf{x})$ đã phân lớp đúng.

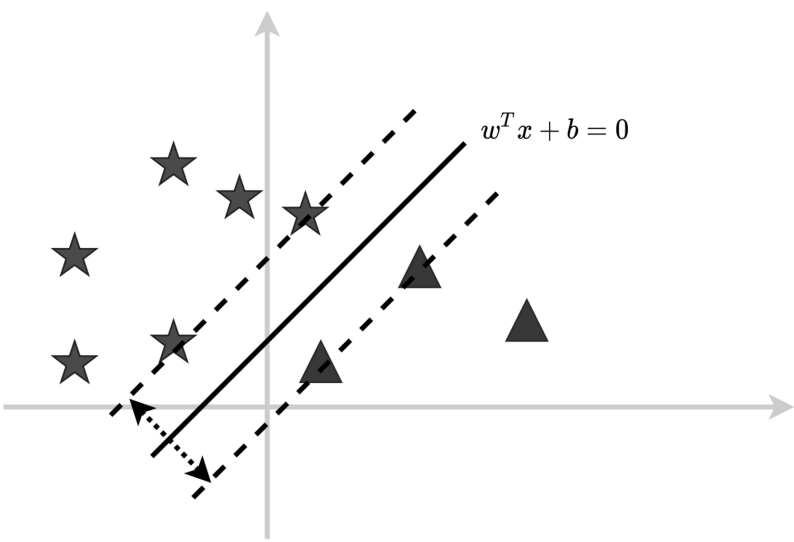
Định nghĩa 4.1 (Khả tách tuyến tính). Xét tập dữ liệu $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$, nếu tồn tại Perceptron phân lớp đúng hoàn toàn trên tập D , tức là,

$$s_i = y_i f(\mathbf{x}_i) > 0, i = 1, 2, \dots, n,$$

thì ta nói tập dữ liệu D *khả tách tuyến tính*.

Định nghĩa 4.2 (Độ rộng của lề). Giả sử Perceptron h phân lớp đúng trên tập dữ liệu D khả tách tuyến tính với bộ trọng số $(\mathbf{w}, b)$. Độ rộng của lề là

$$\delta = \min_{i=1,2,\dots,n} y_i f(\mathbf{x}_i) = \min_{i=1,2,\dots,n} y_i (\mathbf{w}^T \mathbf{x}_i + b)$$



Hình 4.1: Tập dữ liệu khả tách tuyến tính.

4.1.1 Pha huấn luyện

Định nghĩa 4.1 cho thấy phương hướng huấn luyện mô hình Perceptron là tìm bộ trọng số $(w^1, \dots, w^m, b)$ sao cho điểm số s_i của tất cả các mẫu dữ liệu trong D đều không âm. Rõ ràng, việc này chỉ khả thi khi D khả tách tuyến tính. Bây giờ, xét một mẫu dữ liệu $(\mathbf{x}, y)$ mà Perceptron h đang phân lớp sai, tức là $s = yf(\mathbf{x}) < 0$. Nếu ta cộng thêm vào s một đại lượng dương $\mathbf{x}^T \mathbf{x} + 1 > 0$ thì có khả năng s sẽ đỡ “sai” hơn hoặc đỡ “âm” hơn. Ta có

$$\begin{aligned}
 s + \overbrace{\mathbf{x}^T \mathbf{x} + 1}^{>0} &= y(w^T \mathbf{x} + b) + \mathbf{x}^T \mathbf{x} + 1 \\
 &= y(w^T \mathbf{x} + b) + y^2 \mathbf{x}^T \mathbf{x} + y^2 \quad (\text{do } y^2 = 1) \\
 &= y((w + y\mathbf{x})^T \mathbf{x} + (b + y))
 \end{aligned}$$

Nghĩa là, nếu ta làm phép toán cập nhật tham số $w \leftarrow w + y\mathbf{x}$, $b \leftarrow b + y$ thì điểm số s sẽ được tăng thêm một đại lượng dương. Một cách diễn giải khác là nếu coi s là hàm của bộ trọng số w, b thì thay

đổi các trọng số theo hướng của đạo hàm $\frac{\partial s}{\partial \mathbf{w}}$ và $\frac{\partial s}{\partial b}$ làm hàm số s tăng lên. Viết lại theo công thức tính đạo hàm từng phần, ta có:

$$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} + \frac{\partial s}{\partial \mathbf{w}} = \mathbf{w} + \frac{\partial(y(\mathbf{w}^T \mathbf{x} + b))}{\partial \mathbf{w}} = \mathbf{w} + y\mathbf{x} \\ b &\leftarrow b + \frac{\partial s}{\partial b} = b + \frac{\partial(y(\mathbf{w}^T \mathbf{x} + b))}{\partial b} = b + y \end{aligned}$$

Nhận xét này dẫn đến thuật toán huấn luyện Perceptron (Thuật toán 4.1).

Thuật toán 4.1 Thuật toán huấn luyện Perceptron

```

1: procedure TRAINPERCEPTRON( $D$ )
2:   Khởi tạo  $\mathbf{w} \leftarrow 0$ ,  $b \leftarrow 0$ 
3:   repeat
4:      $count \leftarrow 0$ 
5:     for all  $(\mathbf{x}_i, y_i) \in D$  do
6:        $s_i \leftarrow y_i \cdot f(\mathbf{x}_i)$ 
7:       if  $s_i < 0$  then ▷ Phân lớp sai
8:          $\mathbf{w} \leftarrow \mathbf{w} + y_i \cdot \mathbf{x}_i$ 
9:          $b \leftarrow b + y_i$ 
10:       $count \leftarrow mistake + 1$ 
11:     end if
12:   end for
13:   until  $count = 0$ 
14:   return  $(\mathbf{w}, b)$ 
15: end procedure

```

4.1.2 Tính dừng của thuật toán huấn luyện

Đảm bảo tính dừng của thuật toán huấn luyện Perceptron được trình bày trong Định lý 4.3.

Định lý 4.3 (Tính dừng của thuật toán huấn luyện Perceptron).
 Nếu tồn tại bộ trọng số $(\mathbf{w}^*, b^*)$ và $\delta > 0$ sao cho $y_i(\mathbf{w}^{*T} \mathbf{x}_i + b) \geq$

$\delta, \forall i$ thì số lỗi thuật toán 4.1 mắc phải không vượt quá

$$T = \frac{(R^2 + 1)(\|\mathbf{w}^*\|^2 + b^{*2})}{\delta^2} \quad (4.1)$$

với $R = \max_i \|\mathbf{x}_i\|_2$ là chiều dài lớn nhất của các mẫu dữ liệu trong D .

Chứng minh: Gọi $\mathbf{w}_{(t)}, b_{(t)}$ là giá trị trọng số sau lần cập nhật thứ t (hoặc lần mắc lỗi thứ t đối với mẫu dữ liệu $(\mathbf{x}, y)$ nào đó trong D). Ta có

$$\begin{aligned} \mathbf{w}^{*T} \mathbf{w}_{(t)} + b^* b_{(t)} &= \mathbf{w}^{*T} (\mathbf{w}_{(t-1)} + y\mathbf{x}) + b^* (b_{(t-1)} + y) \\ &= \mathbf{w}^{*T} \mathbf{w}_{(t-1)} + b^* b_{(t-1)} + y(\mathbf{w}^{*T} \mathbf{x} + b) \\ [\text{giả thiết}] &\geq \mathbf{w}^{*T} \mathbf{w}_{(t-1)} + b^* b_{(t-1)} + \delta \\ [\text{áp dụng } t \text{ lần}] &\geq t\delta \end{aligned}$$

Mặt khác, ta lại có

$$\|\mathbf{w}_{(t)}\|^2 + b_{(t)}^2 = \|\mathbf{w}_{(t-1)} + y\mathbf{x}\|^2 + (b_{(t-1)} + y)^2.$$

Ta có thể viết lại vế phải (VP) như sau:

$$\begin{aligned} VP &= \|\mathbf{w}_{(t-1)}\|^2 + 2y\mathbf{w}_{(t-1)}^T \mathbf{x} + y^2 \|\mathbf{x}\|^2 + b_{(t-1)}^2 + 2yb_{(t-1)} + y^2 \\ &= \|\mathbf{w}_{(t-1)}\|^2 + b_{(t-1)}^2 + \underbrace{2y(\mathbf{w}_{(t-1)}^T \mathbf{x} + b_{(t-1)})}_{<0 \text{ do mắc lỗi}} + \underbrace{\|\mathbf{x}\|^2 + 1}_{\leq R^2} \\ &< \|\mathbf{w}_{(t-1)}\|^2 + b_{(t-1)}^2 + R^2 + 1 \end{aligned}$$

Ta thu được, bất đẳng thức cho một lần cập nhật trọng số

$$\begin{aligned} \|\mathbf{w}_{(t)}\|^2 + b_{(t)}^2 &< \|\mathbf{w}_{(t-1)}\|^2 + b_{(t-1)}^2 + R^2 + 1 \\ [\text{áp dụng } t \text{ lần}] &< t(R^2 + 1) \end{aligned}$$

Theo bất đẳng thức Bunhiacopxki, ta có

$$t(R^2 + 1) > \|\mathbf{w}_{(t)}\|^2 + b_{(t)}^2 \geq \frac{(\mathbf{w}^{*T} \mathbf{w}_{(t)} + b^* b_{(t)})^2}{\|\mathbf{w}^*\|^2 + b^{*2}} \geq \frac{t^2 \delta^2}{\|\mathbf{w}^*\|^2 + b^{*2}}.$$

Rút gọn ta được cận trên của số lỗi là

$$t < \frac{(R^2 + 1)(\|\mathbf{w}^*\|^2 + b^2)}{\delta^2}.$$

□

Định lý về tính dừng của thuật toán huấn luyện Perceptron là một ví dụ kinh điển về *khả năng học* của một mô hình Học máy. Ý nghĩa của định lý này là khi tập dữ liệu D thỏa mãn một tính chất nhất định thì mô hình Perceptron có thể học được khái niệm từ tập dữ liệu này. Từ công thức (4.1), ta có thể rút ra một số nhận xét:

- Cận trên này không phụ thuộc vào số lượng dữ liệu có trong D mà phụ thuộc vào độ khó của bài toán phân lớp đại diện bởi tham số δ . Tham số này cho biết có thể phân tách các lớp trong D xa nhau với khoảng cách ít nhất $\frac{2\delta}{\|\mathbf{w}^*\|}$.
- Cận trên này cũng phụ thuộc vào độ lớn của các mẫu dữ liệu trong D đại diện bởi R . Nghĩa là khi D bao phủ một không gian rộng thì việc tìm kiếm trọng số $\mathbf{w}, b$ cũng mất nhiều thời gian hơn do thuật toán xuất phát với $\mathbf{w} = 0, b = 0$. Đây cũng là một trong các lý do người ta chuẩn hóa dữ liệu về một vùng không gian nhỏ trước khi thực hiện huấn luyện.

4.2 Máy véc-tơ hỗ trợ lề cứng

Xuất phát từ định lý về tính dừng của thuật toán huấn luyện Perceptron (Định lý 4.3), ta thấy đại lượng $\delta \leq y_i(\mathbf{w}^T \mathbf{x}_i + b)$ mô tả độ khó của tập dữ liệu D vì δ liên quan đến khoảng cách từ các điểm dữ liệu đến siêu phẳng phân lớp $B : \mathbf{w}^T \mathbf{x} + b = 0$. Thật vậy, khoảng cách của điểm dữ liệu $(\mathbf{x}_i, y_i)$ đến siêu phẳng phân lớp được tính bởi công thức:

$$d(\mathbf{x}_i, B) = \frac{|\mathbf{w}^T \mathbf{x}_i + b|}{\|\mathbf{w}\|} = \frac{|y_i(\mathbf{w}^T \mathbf{x}_i + b)|}{\|\mathbf{w}\|} \geq \frac{\delta}{\|\mathbf{w}\|}$$

Như vậy, nếu các điểm dữ liệu có $y_i = +1$ và các điểm dữ liệu có $y_i = -1$ ở hai phía của siêu phẳng phân lớp B thì khoảng cách giữa hai phân lớp trong D sẽ không nhỏ hơn $\frac{2\delta}{\|w\|}$. Khoảng cách này được gọi là lề của hàm phân lớp. So với mô hình Perceptron, mô hình SVM đưa thêm tiêu chí cực đại hóa lề khi tìm kiếm bộ trọng số Θ .

Không mất tính tổng quát, ta có thể chọn $\delta = 1$ (nếu không ta có thể nhân w, b, δ với cùng một đại lượng để $\delta = 1$). Bài toán huấn luyện SVM trở thành bài toán tối ưu một hàm mục tiêu với ràng buộc:

$$\max_{w, b} \frac{2}{\|w\|} \quad (4.2)$$

với ràng buộc $y_i(w^T \mathbf{x}_i + b) \geq 1, \forall i$.

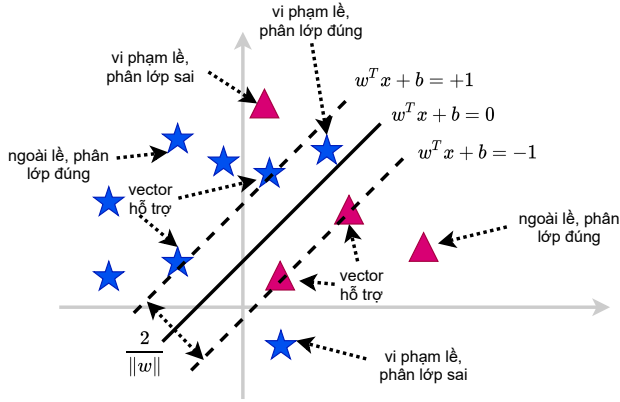
Biến đổi hàm tối ưu ở công thức (4.2), ta thu được bài toán này tương đương, có dạng tìm giá trị tối ưu hàm mục tiêu bậc hai với ràng buộc tuyến tính.

Định nghĩa 4.4 (Máy véc-tơ hỗ trợ SVM lề cứng). Cho tập dữ liệu $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ với $y_i \in \{-1, +1\}$. Bài toán tối ưu SVM lề cứng là bài toán tối ưu

$$\min_{w, b} L(w, b) = \frac{1}{2} \|w\|^2 \quad (4.3)$$

với ràng buộc $y_i(w^T \mathbf{x}_i + b) \geq 1, \forall i$

Bài toán tối ưu được cho bởi (4.3) gọi là bài toán tối ưu SVM với lề cứng do nó đòi hỏi tất cả các mẫu dữ liệu trong D phải được phân lớp đúng. Hiện có nhiều gói phần mềm tối ưu cho phép giải tổng quát dạng tối ưu bậc hai với điều kiện tuyến tính khá tốt. Tuy nhiên, nếu chúng ta chuyển dạng tối ưu bậc hai của SVM lề cứng sang dạng đối ngẫu Lagrange thì sẽ có những cách giải nhanh hơn rất nhiều. Để làm điều này, ta sử dụng hàm Lagrange của bài toán



Hình 4.2: Mô hình Máy véc-tơ hỗ trợ SVM.

gốc được định nghĩa ở công thức (4.3)

$$L(w, b) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \alpha_i [1 - y_i(w^T \mathbf{x}_i + b)], \quad (4.4)$$

với $\alpha_i \geq 0, \forall i$ là các hệ số Lagrange tương ứng với các ràng buộc. Điều kiện cần của w, b tối ưu bài toán gốc là đạo hàm của L với w và b bị triệt tiêu bằng 0. Dựa trên dữ kiện này, chúng ta tiến hành đạo hàm và giải phương trình đạo hàm bằng 0, ta thu được các giá trị tối ưu của w và b như sau:

$$\begin{aligned} \frac{\partial L}{\partial w} &= w - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \Rightarrow w = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \\ \frac{\partial L}{\partial b} &= \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Thế các kết quả trên vào công thức (4.4), ta được công thức đối

ngẫu Lagrange của bài toán SVM lề cứng như sau:

$$\begin{aligned}
 L &= \frac{1}{2} \left\| \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right\|^2 + \sum_{i=1}^n \alpha_i \left[1 - y_i \left(\left(\sum_{j=1}^n \alpha_j y_j \mathbf{x}_j \right)^T \mathbf{x}_i + b \right) \right] \\
 &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\
 &= \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T \mathbf{Q} \alpha.
 \end{aligned}$$

Trong đó $\mathbf{1}$ là véc-tơ n chiều gồm toàn số 1 còn $\mathbf{Q}$ là ma trận kích thước $n \times n$ đối xứng: $\mathbf{Q} = [q_{ij} = y_i y_j \mathbf{x}_i^T \mathbf{x}_j]$.

Định nghĩa 4.5 (Bài toán đối ngẫu SVM lề cứng). Cho bài toán tối ưu SVM lề cứng với hàm mục tiêu và điều kiện ràng buộc theo Định nghĩa 4.4, bài toán đối ngẫu Lagrange SVM lề cứng là bài toán tối ưu:

$$\max_{\alpha} L(\alpha) \triangleq \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T \mathbf{Q} \alpha \quad (4.5)$$

$$\text{với ràng buộc } \sum_{i=1}^n \alpha_i y_i = 0 \text{ và } \alpha_i \geq 0, \forall i$$

với α_i là các hệ số Lagrange tương ứng với các ràng buộc của bài toán gốc.

Định lý 4.6 (Tính chất của bài toán đối ngẫu SVM lề cứng). *Bài toán tối ưu đối ngẫu của SVM lề cứng là một bài toán tối ưu lồi với các ràng buộc tuyến tính.*

Chứng minh: Từ công thức (4.5), chúng ta thấy được rằng hàm mục tiêu $L(\alpha)$ của bài toán đối ngẫu SVM lề cứng là hàm bậc hai với véc-tơ tham số α . Ma trận Hessian của hàm mục tiêu này được tính như sau:

$$\mathbf{H} = \frac{\partial^2 L}{\partial \alpha^2} = -\mathbf{Q} \quad (4.6)$$

Theo định nghĩa của ma trận Q thì phần tử $q_{ij} = y_i y_j \mathbf{x}_i^T \mathbf{x}_j$. Dễ thấy $\forall v \in \mathbb{R}^n$

$$\begin{aligned} v^T Q v &= \sum_{i=1}^n \sum_{j=1}^n v_i v_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ &= \sum_{i=1}^n \sum_{j=1}^n (y_i v_i) (y_j v_j) (\mathbf{x}_i^T \mathbf{x}_j) \end{aligned}$$

Đặt $a_i = y_i v_i$, chúng ta có thể biến đổi tổng trên thành

$$\begin{aligned} v^T Q v &= \sum_{i=1}^n \sum_{j=1}^n a_i a_j (\mathbf{x}_i^T \mathbf{x}_j) \\ &= \left(\sum_{i=1}^n a_i \mathbf{x}_i \right)^T \left(\sum_{j=1}^n a_j \mathbf{x}_j \right) \\ &= \left\| \sum_{i=1}^n a_i \mathbf{x}_i \right\|^2 \geq 0 \end{aligned}$$

Do đó, ma trận Q là ma trận nửa xác định dương. Từ công thức (4.6), ta có thể suy ra $L(\alpha)$ là hàm lõm. Bài toán đối ngẫu tìm cực đại của hàm lõm $L(\alpha)$ với các ràng buộc tuyến tính tương ứng với tìm cực tiểu của một hàm lồi với các ràng buộc tuyến tính. $\square$

Từ định lý trên chúng ta thấy được bài toán đối ngẫu của SVM lề cứng vẫn là bài toán quy hoạch lồi bậc 2 nhưng có các ràng buộc đơn giản hơn rất nhiều so với ràng buộc của bài toán gốc. Các gói phần mềm như *libsvm* [6], *liblinear* [7] được thiết kế và cài đặt chuyên biệt để giải bài toán tối ưu SVM ở dạng đối ngẫu.

Ngoài ra, tại nghiệm tối ưu của bài toán gốc và bài toán đối ngẫu, ta còn có điều kiện bù phần dư trong các điều kiện Karush–Kuhn–Tucker như sau:

$$\alpha_i [1 - y_i (\mathbf{w}^T \mathbf{x}_i + b)] = 0, \forall i \quad (4.7)$$

Sử dụng các điều kiện ở công thức (4.7), ta có thể diễn giải như sau:

- Nếu $y_i(w^T \mathbf{x}_i + b) > 1$ thì $\alpha_i = 0$, tức là mẫu thứ i không tham gia vào việc tính w . Đây là các mẫu dữ liệu nằm ngoài lề của bài toán phân lớp.
- Nếu $\alpha_i > 0$ thì $y_i(w^T \mathbf{x}_i + b) = 1$, tức là nếu mẫu thứ i tham gia vào việc tính w thì mẫu dữ liệu này phải nằm ngay trên lề của bài toán phân lớp. Ta gọi các mẫu dữ liệu này là véc-tơ hỗ trợ vì các véc-tơ này nằm “đỡ” lấy hai bên lề của bài toán phân lớp.

Sau khi tính toán được $\alpha_i, \forall i$, ta có thể tính w và b như sau:

$$w = \sum_{i:\alpha_i>0} \alpha_i y_i \mathbf{x}_i$$

$$b = \frac{1}{|\{i : \alpha_i > 0\}|} \sum_{i:\alpha_i>0} y_i - w^T \mathbf{x}_i$$

4.3 Máy véc-tơ hỗ trợ lề mềm

Do không phải lúc nào tập dữ liệu D cũng khả tách tuyến tính, mô hình SVM có thể sửa để cho phép các mẫu dữ liệu vi phạm ràng buộc lề cứng nhưng cố gắng tối thiểu hóa các vi phạm này. Bài toán SVM lề cứng được trình bày ở Định nghĩa 4.4 sẽ được điều chỉnh thành bài toán lề mềm như sau:

Định nghĩa 4.7 (Máy véc-tơ hỗ trợ SVM lề mềm). Cho tập dữ liệu $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ với $y_i \in \{-1, +1\}$. Bài toán tối ưu SVM lề mềm là bài toán tối ưu

$$\min_{w, b, \xi} L(w, b, \xi) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{với ràng buộc } y_i(w^T \mathbf{x}_i + b) \geq 1 - \xi_i, \forall i,$$

$$\xi_i \geq 0, \forall i.$$

Trong định nghĩa trên, ξ_i là vi phạm của mẫu dữ liệu thứ i và $C > 0$ là trọng số của việc cho phép vi phạm. Giá trị của C là một

tham số điều chỉnh cho bài toán SVM lề mềm. Nếu chúng ta chọn giá trị của C nhỏ đồng nghĩa với việc số lượng các điểm vi phạm điều kiện phân lớp của SVM có thể lớn, và ngược lại giá trị lớn của C sẽ hạn chế lại số lượng điểm vi phạm. Nếu chúng ta tăng $C \rightarrow \infty$ thì bài toán SVM lề mềm sẽ trở thành bài toán SVM lề cứng.

Các ràng buộc đối với ξ_i là $\xi_i \geq 0$ và $\xi_i \geq 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)$ có thể viết gọn lại thành $\xi_i \geq \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$. Nhưng nhận thấy ta đang tối thiểu hóa tổng $\frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$ nên tại nghiệm tối ưu, ta có thể khai triển

$$\xi_i = \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

Do đó, có thể viết lại bài toán tối ưu SVM với lề mềm gọn hơn theo công thức (4.8).

$$\min_{\mathbf{w}, b} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)) \quad (4.8)$$

Trong công thức (4.8), hàm số

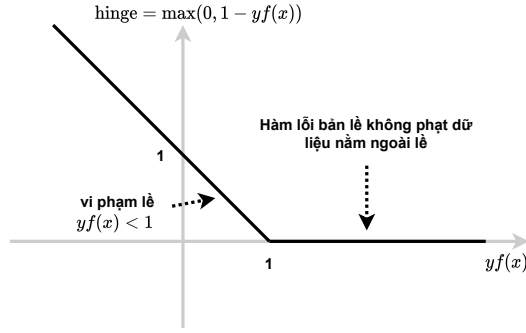
$$\text{hinge}(f(\mathbf{x}), y) = \max(0, 1 - y(\mathbf{w}^T \mathbf{x}_i + b))$$

được gọi là hàm lỗi bản lề. Hàm này là một cách khác để đo độ bất hợp lý của hàm phân lớp so với độ hợp lý mà ta thấy trong mô hình Perceptron và mô hình Logistic.

Để tìm nghiệm tối ưu của Bài toán SVM lề mềm, chúng ta tiếp tục sử dụng khai triển Lagrange dựa trên công thức (4.8):

$$L = \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i [1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) - \xi_i] - \sum_{i=1}^n \beta_i \xi_i,$$

với $\alpha_i \geq 0, \beta_i \geq 0$ là các hệ số Lagrange của các ràng buộc.



Hình 4.3: Hàm lỗi bản lề (hinge loss).

Tiếp tục lấy đạo hàm cho w, b và ξ ta có

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \Rightarrow w = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial L}{\partial b} = \sum_{i=1}^n \alpha_i y_i = 0$$

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \beta_i = 0 \Rightarrow \beta_i = C - \alpha_i.$$

Thế ngược trở lại L , ta thu được

$$L = \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T \mathbf{Q} \alpha$$

với $\mathbf{Q}$ là ma trận Gram của tập dữ liệu D được định nghĩa như trường hợp của hàm đối ngẫu SVM lề cứng. Do $\beta_i \geq 0$, ràng buộc đối với α_i bây giờ trở thành $0 \leq \alpha_i \leq C, \forall i$. Ràng buộc này còn được gọi là ràng buộc dạng hộp do $\alpha \in [0, C]^n$ là một hộp n chiều có kích thước mỗi chiều bằng C .

Điều kiện bù phần dư như sau

$$\begin{aligned} \alpha_i [1 - y_i(w^T \mathbf{x}_i + b) - \xi_i] &= 0, \forall i \\ (C - \alpha_i) \xi_i &= 0, \forall i \end{aligned}$$

Tương tự với khi sử dụng phương pháp SVM lề cứng, chúng ta cũng có hệ quả tương ứng sau:

- Nếu $\alpha_i = 0$ thì $\xi_i = 0$, tức là mẫu thứ i không tham gia vào việc tính w và mẫu này cũng không bị vi phạm lề. Đây là các mẫu dữ liệu nằm ngoài lề của bài toán phân lớp.
- Nếu $0 < \alpha_i < C$ thì $\xi_i = 0$ và $y_i(w^T \mathbf{x}_i + b) = 1$, tức là mẫu thứ i phải nằm ngay trên lề của bài toán phân lớp.
- Nếu $\xi_i > 0$ thì $\alpha_i = C$, tức là khi mẫu dữ liệu thứ i có vi phạm lề, mẫu này sẽ tham gia vào việc tính w với trọng số α_i lớn nhất.

Như vậy, các véc-tơ hỗ trợ là các véc-tơ nằm ngay trên lề phân lớp hoặc các véc-tơ có vi phạm lề.

Tổng kết lại, ta thu được bài toán tối ưu đối ngẫu lề mềm SVM như sau:

Định nghĩa 4.8 (Bài toán đối ngẫu với SVM lề mềm). Cho bài toán tối ưu SVM lề mềm với hàm mục tiêu và điều kiện ràng buộc theo Định nghĩa 4.7, bài toán đối ngẫu SVM lề mềm là bài toán tối ưu

$$\begin{aligned} \max_{\alpha} \quad & \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T \mathbf{Q} \alpha \\ \text{với ràng buộc} \quad & \sum_{i=1}^n \alpha_i y_i = 0, \\ & 0 \leq \alpha_i \leq C, \forall i \end{aligned} \tag{4.9}$$

Sau khi giải bài toán tối ưu đối ngẫu, ta có thể sử dụng phương pháp tương tự lề cứng để tính các trọng số w, b của biên quyết định. Trong trường hợp của bài toán tối ưu đối ngẫu lề mềm SVM chúng ta có thêm sự xuất hiện của trọng số C . Vì thế chúng ta xét thêm điều kiện của $\alpha_i < C$ để tính trọng số b bên cạnh điều kiện trọng

số $\alpha_i > 0$. Kết hợp các điều kiện lại, ta có công thức tính trọng số w và b như sau:

$$w = \sum_{i:\alpha_i>0} \alpha_i y_i \mathbf{x}_i$$

$$b = \frac{1}{|\{i : 0 < \alpha_i < C\}|} \sum_{i:0<\alpha_i<C} y_i - w^T \mathbf{x}_i$$

Ngoài cách tiếp cận giải bài toán SVM lề mềm bằng cách tối ưu bài toán đối ngẫu, chúng ta có thể giải trực tiếp bài toán gốc được trình bày ở Định nghĩa 4.7. Dựa vào nhận xét đây là bài toán tối ưu lồi do $\|w\|^2$ và hàm lỗi hinge là hai hàm lồi, ta có thể tối ưu hóa với phương pháp xuống đồi bằng đạo hàm. Cụ thể, đặt $J(w, b)$ là hàm số cần tối ưu, ta có Thuật toán huấn luyện 4.2. Chú ý là bài toán SVM lề mềm tổng quát hơn bài toán SVM lề cứng nên chúng ta chỉ cần xét trường hợp lề mềm là đủ.

$$\begin{aligned} \frac{\partial J}{\partial w} &= w + C \sum_{i=1}^n \frac{\partial \max(0, 1 - y_i(w^T \mathbf{x}_i + b))}{\partial w} \\ &= w - C \sum_{i:y_i(w^T \mathbf{x}_i + b) < 1} y_i \mathbf{x}_i \\ \frac{\partial J}{\partial b} &= -C \sum_{i:y_i(w^T \mathbf{x}_i + b) < 1} y_i \\ w &\leftarrow w - \lambda \frac{\partial J}{\partial w} = (1 - \lambda)w + \lambda C \sum_{i:y_i(w^T \mathbf{x}_i + b) < 1} y_i \mathbf{x}_i \\ b &\leftarrow b - \lambda \frac{\partial J}{\partial b} = b + \lambda C \sum_{i:y_i(w^T \mathbf{x}_i + b) < 1} y_i \end{aligned}$$

Các công thức cập nhật w và b này chỉ làm việc với các mẫu dữ liệu vi phạm lề phân lớp trong tập dữ liệu D (tức là các mẫu có $y_i(w^T \mathbf{x}_i + b) < 1$). Trong khi mô hình Perceptron chỉ cập nhật khi gặp lỗi (tức là $y_i(w^T \mathbf{x}_i + b) < 0$) thì mô hình SVM mở rộng việc

cập nhật khi gặp cả những mẫu dữ liệu vi phạm lề. Ta có thuật toán huấn luyện mô hình SVM từ bài toán gốc như trong Thuật toán 4.2. Trong mỗi vòng lặp, chúng ta sẽ lấy ngẫu nhiên một gói dữ liệu và cập nhật trọng số w và b . Sau khi tính được đạo hàm dựa theo hàm lỗi của gói dữ liệu, chúng ta sẽ cập nhật lại trọng số w và b theo công thức trên. Thuật toán này dễ dàng có thể chuyển đổi thành thuật toán học trực tuyến bằng cách chỉ đưa một mẫu dữ liệu vào huấn luyện mỗi lần cập nhật. Điều kiện dừng của thuật toán là số vòng lặp đủ lớn, hoặc khi không còn mẫu dữ liệu nào vi phạm lề phân lớp nữa.

Thuật toán 4.2 Thuật toán huấn luyện mô hình SVM từ bài toán gốc

```

1: procedure TRAINSVMPRIMAL( $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \lambda, C$ )
2:   Khởi tạo  $\mathbf{w} \leftarrow 0, b \leftarrow 0$ 
3:   for epoch = 1, 2, ... do
4:     Cập nhật trọng số theo gói (batch):


$$\mathbf{w} \leftarrow (1 - \lambda)\mathbf{w} + \lambda C \sum_{i: y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1} y_i \mathbf{x}_i$$


$$b \leftarrow b + \lambda C \sum_{i: y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1} y_i$$


5:     if điều kiện dừng thoả mãn then
6:       break
7:     end if
8:   end for
9:   return ( $\mathbf{w}, b$ )
10: end procedure

```

Bên cạnh đó ta cũng có thể giải bài toán đối ngẫu được cho bởi Công thức (4.9). Do tính chất đơn giản của ràng buộc dạng hộp, người ta giải bài toán tối ưu đối ngẫu của SVM bằng phương pháp tối ưu cực tiểu tuần tự SMO. Trong phương pháp này, tại mỗi lần

lặp, ta cần tìm một cặp trọng số α_i, α_j vi phạm các ràng buộc bù phần dư như sau:

$$\begin{aligned}\alpha_i = 0 &\Rightarrow y_i(\mathbf{w}^T \mathbf{x}_i + b) > 1 \\ 0 < \alpha_i < C &\Rightarrow y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1 \\ \alpha_i = C &\Rightarrow y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1\end{aligned}$$

Sau đó, giải bài toán tối ưu nhỏ bằng cách giữ nguyên tất cả các trọng số $\alpha_k, k \neq i, k \neq j$ và chỉ tối ưu hai trọng số α_i, α_j . May mắn là khi tối ưu chỉ hai trọng số, ta có công thức tính trực tiếp α_i, α_j tốt nhất. Thuật toán lại tiếp tục tối ưu hóa cặp trọng số khác cho đến khi tất cả các trọng số đều thỏa mãn các ràng buộc trên trong một ngưỡng cho trước.

4.4 Phương pháp nhân hóa

Trong chương này, chúng ta đã tìm hiểu hai mô hình học máy sử dụng hàm phân lớp tuyến tính $h(x) = \text{sgn}(\mathbf{w}^T \mathbf{x} + b)$ là mô hình Perceptron và mô hình SVM. Các công thức cập nhật trọng số $\mathbf{w}$ của các mô hình này đều có một điểm chung là mỗi lần cập nhật $\mathbf{w}$ thay đổi bằng một tổ hợp tuyến tính của các mẫu dữ liệu huấn luyện. Cụ thể, xuất phát từ $\mathbf{w} = 0$, ta có công thức cập nhật như sau:

$$\mathbf{w} \leftarrow \mathbf{w} + \sum_i \alpha_i y_i \mathbf{x}_i \quad (4.10)$$

với $\alpha_i = \mathbb{I}[y_i f(\mathbf{x}_i) < 0]$ cho mô hình Perceptron và $\alpha_i = \mathbb{I}[y_i f(\mathbf{x}_i) < 1]$ cho mô hình SVM.

Như vậy, khi thuật toán dừng, nghiệm $\mathbf{w}$ thu được sẽ là một tổ hợp tuyến tính của các mẫu dữ liệu trong tập $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$. Nghĩa là, trọng số $\mathbf{w}$ có thể được viết dưới dạng:

$$\mathbf{w} = \sum_{i=1}^n \nu_i \mathbf{x}_i \quad (4.11)$$

Nhận xét này dẫn tới một cách tiếp cận khác đối với việc huấn luyện các mô hình tuyến tính. Thay vì trực tiếp tính toán w , ta chỉ cần tính toán các trọng số $\nu_i, i = 1, 2, \dots, n$ là đủ. Khi đó, hàm phân lớp ở pha suy luận được tính toán như sau:

$$f(x) = w^T \mathbf{x} + b = \sum_{i=1}^n \nu_i (\mathbf{x}_i^T \mathbf{x}) + b \quad (4.12)$$

$$h(x) = \text{sgn}(w^T \mathbf{x} + b) = \text{sgn} \left(\sum_{i=1}^n \nu_i (\mathbf{x}_i^T \mathbf{x}) + b \right). \quad (4.13)$$

Ở pha huấn luyện, ta cập nhật các trọng số như sau:

$$\nu_i \leftarrow \nu_i + \alpha_i y_i. \quad (4.14)$$

Đối với mô hình Perceptron ta có:

$$\alpha_i = \mathbb{I}[y_i f(\mathbf{x}_i) < 0] = \mathbb{I} \left[y_i \left(\sum_{j=1}^n \nu_j (\mathbf{x}_j^T \mathbf{x}_i) + b \right) < 0 \right]. \quad (4.15)$$

Đối với mô hình SVM ta có:

$$\alpha_i = \mathbb{I}[y_i f(\mathbf{x}_i) < 1] = \mathbb{I} \left[y_i \left(\sum_{j=1}^n \nu_j (\mathbf{x}_j^T \mathbf{x}_i) + b \right) < 1 \right]. \quad (4.16)$$

Như vậy, ở cả pha suy luận và pha huấn luyện của các mô hình tuyến tính đã học, ta đều có thể tính toán các đại lượng cần thiết nhờ vào phép tính tích vô hướng $\mathbf{x}_i^T \mathbf{x}$ hoặc $\mathbf{x}_i^T \mathbf{x}_j$. Đây là phép tính tích vô hướng ở không gian $\mathbb{R}^m$, không gian dữ liệu đầu vào. Một nhận xét đột phá của ngành Học máy là nếu ta thực hiện một ánh xạ $\phi: \mathbb{R}^m \rightarrow \mathbb{R}^{m'}$ và chạy các thuật toán phân lớp tuyến tính trên không gian $\mathbb{R}^{m'}$ thì chỉ cần tính toán các tích vô hướng

$$\kappa(\mathbf{u}, \mathbf{v}) = \phi(\mathbf{u})^T \phi(\mathbf{v}) \quad (4.17)$$

trên không gian $\mathbb{R}^{m'}$ mà không cần thực sự làm việc trên không gian mới này. Tức là chỉ cần hàm $\kappa(\mathbf{x}, \mathbf{y})$ là đủ. Các công thức từ (4.13) đến (4.16) ở pha huấn luyện và pha suy luận trở thành

$$\begin{aligned} f(\mathbf{x}) &= \sum_{i=1}^n \nu_i \kappa(\mathbf{x}_i, \mathbf{x}) + b \\ h(\mathbf{x}) &= \text{sgn}(f(\mathbf{x})) \\ \nu_i &\leftarrow \nu_i + \alpha_i y_i \\ \alpha_i &= \mathbb{I}[y_i f(\mathbf{x}_i) < 0] \text{ (cho mô hình Perceptron)} \\ \alpha_i &= \mathbb{I}[y_i f(\mathbf{x}_i) < 1] \text{ (cho mô hình SVM).} \end{aligned}$$

Tiếp tục mở rộng ra cho bài toán đối ngẫu SVM lề mềm, ta thu được dạng bài toán tối ưu theo hàm nhân $\kappa(\mathbf{u}, \mathbf{v})$ như sau:

$$\max_{\alpha} \quad \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T \mathbf{Q} \alpha \quad (4.18)$$

$$\text{với ràng buộc} \quad \sum_{i=1}^n \alpha_i y_i = 0, \quad (4.19)$$

$$0 \leq \alpha_i \leq C, \forall i$$

với $Q = [q_{ij}]$ và $q_{ij} = y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j)$. Tức là bài toán đối ngẫu cũng có thể chuyển sang không gian mới sử dụng hàm $\kappa(\mathbf{u}, \mathbf{v})$ mà không tốn thêm nhiều công sức tính toán.

Hàm $\kappa(\mathbf{u}, \mathbf{v}) = \phi(\mathbf{u})^T \phi(\mathbf{v})$ với một ánh xạ ϕ bất kì được gọi là *hàm nhân*. Kỹ thuật sử dụng hàm nhân để chuyển mẫu dữ liệu từ không gian này sang không gian khác đối với các thuật toán tuyến tính được gọi là *phương pháp nhân hoá* (kernelization). Phương pháp này có nhiều ưu điểm được liệt kê dưới đây:

- *Phi tuyến hoá các thuật toán tuyến tính*: Bằng việc sử dụng hàm nhân dựa trên hàm ϕ là hàm phi tuyến, ta đã phi tuyến hoá thuật toán phân lớp ban đầu. Thuật toán mới vẫn là thuật

toán tuyến tính trên không gian $\mathbb{R}^{m'}$ nhưng lại là thuật toán phi tuyến trên không gian ban đầu $\mathbb{R}^m$. Nhờ đó, ta có thể tạo ra các quyết định phức tạp hơn nhiều so với hàm phân lớp tuyến tính ban đầu.

- *Làm bài toán phân lớp trở nên đơn giản hơn*: người ta chứng minh được rằng, khi $m' \gg m$, khả năng có một hàm phân lớp tuyến tính hoàn hảo trên không gian $\mathbb{R}^{m'}$ lớn hơn rất nhiều. Do đó, các thuật toán huấn luyện trên không gian $\mathbb{R}^{m'}$ hoạt động dễ dàng hơn.
- *Không tăng nhiều khối lượng tính toán*: nhớ rằng, chúng ta chỉ cần hàm $\kappa(\mathbf{u}, \mathbf{v})$ mà không thực sự cần tính hàm ϕ . Tức là mọi tính toán vẫn thực hiện ở trên không gian $\mathbb{R}^m$. Có thể coi $\kappa(\mathbf{u}, \mathbf{v})$ như là một cổng giao tiếp với không gian $\mathbb{R}^{m'}$, thông qua $\kappa(\mathbf{u}, \mathbf{v})$ ta làm việc trên không gian $\mathbb{R}^{m'}$.
- *Hàm nhân đa dạng cho các kiểu dữ liệu*: Hàm nhân bản chất là tích vô hướng sau khi áp dụng ánh xạ ϕ . Do đó, không nhất thiết ϕ phải có đầu vào là không gian $\mathbb{R}^m$. Người ta đã nghiên cứu ra vô số hàm nhân rất đa dạng cho các kiểu dữ liệu như chuỗi ký tự, cây, đồ thị, ảnh.

Một số hàm nhân hay sử dụng trong bài toán phân lớp sử dụng mô hình SVM:

- $\kappa(\mathbf{u}, \mathbf{v}) = c + \mathbf{u}^T \mathbf{v}$: hàm nhân tuyến tính.
- $\kappa(\mathbf{u}, \mathbf{v}) = (c + a\mathbf{u}^T \mathbf{v})^b$: hàm nhân đa thức.
- $\kappa(\mathbf{u}, \mathbf{v}) = e^{-\gamma \|\mathbf{u} - \mathbf{v}\|^2}$: hàm nhân Gauss.
- $\kappa(\mathbf{u}, \mathbf{v}) = e^{-\gamma \|\mathbf{u} - \mathbf{v}\|}$: hàm nhân lũy thừa.

- $\kappa(\mathbf{u}, \mathbf{v}) = \tanh(c + \mathbf{a}\mathbf{u}^T\mathbf{v})$: hàm nhân tan-hyperbol. Sử dụng hàm này tương đương với mô hình mạng nơ-ron có một lớp ẩn với hàm kích hoạt tanh.
- $\kappa(\mathbf{u}, \mathbf{v}) = \sum_{i=1}^m \min(u_i, v_i)$: hàm nhân giao histogram (hay dùng trong phân loại ảnh)
- $\kappa(\mathbf{u}, \mathbf{v}) = \sum_{i=1}^m \min(|u_i|^d, |v_i|^d)$: hàm nhân giao histogram tổng quát
- $\kappa(\mathbf{u}, \mathbf{v}) = \frac{1}{1 + \|\mathbf{u} - \mathbf{v}\|^d}$: hàm nhân student
- $\kappa(\mathbf{u}, \mathbf{v}) = \sum_{\ell} \kappa_{\ell}(\mathbf{u}_{\ell}, \mathbf{v}_{\ell})$ với

$$\kappa_{\ell}(a, b) = \sum_{y \in \{0,1\}} P(Y = y | X_{\ell} = a) P(Y = y | X_{\ell} = b)$$

là hàm nhân Bayes hay dùng trong dự đoán tương tác protein.

- $\kappa(\mathbf{u}, \mathbf{v}) = \phi_p(\mathbf{u})^T \phi_p(\mathbf{v})$ với $\mathbf{u}, \mathbf{v}$ là các xâu ký tự và $\phi_p(x)$ là tần suất các đoạn p ký tự trong x . Hàm nhân này dùng cho bài toán dự đoán trên văn bản.

4.5 Tình huống áp dụng: Phân lớp chữ số MNIST bằng SVM

Trong bài toán áp dụng này chúng ta sẽ sử dụng thuật toán SVM lẻ mềm cho bài toán phân lớp chữ số MNIST sử dụng thư viện `scikit-learn`. Như đã đề cập trong Chương 3, bộ dữ liệu MNIST là một tập hợp các hình ảnh chữ số viết tay, được sử dụng rộng rãi trong các bài toán phân loại hình ảnh. Bài toán này bao gồm 60,000 hình ảnh huấn luyện và 10,000 hình ảnh kiểm tra, mỗi hình ảnh có kích thước 28×28 điểm ảnh, tương đương với 784 đặc trưng đầu vào. Các đặc trưng này sẽ đóng vai trò là các véc-tơ trong miền dữ liệu đầu vào. Mục tiêu của chúng ta là xây dựng một mô hình

SVM lẽ mềm có thể phân loại chính xác các hình ảnh này. Các bước triển khai chính bao gồm:

- Chuẩn bị dữ liệu: Chúng ta sử dụng thư viện scikit-learn để tải dữ liệu chữ số MNIST.
- Tiền xử lý dữ liệu: Chúng ta sẽ chuẩn hoá dữ liệu và chia thành 2 tập dữ liệu huấn luyện và kiểm tra. Đồng thời chúng ta cũng tiến hành chuẩn hoá dữ liệu để có được một phân bố có trung bình bằng 0 và phương sai bằng 1.
- Huấn luyện mô hình: Sử dụng thư viện scikit-learn, chúng ta sẽ khai báo mô hình SVM lẽ mềm với các tham số được lựa chọn kèm theo bao gồm: tham số điều chỉnh C , hàm nhân hoá và tham số γ . Tham số C điều chỉnh độ cứng của lẽ, trong khi tham số γ điều chỉnh độ phức tạp của hàm nhân. Trong các bài toán cụ thể, các tham số này có thể được điều chỉnh để đạt được hiệu suất tốt nhất.
- Đánh giá mô hình: Sử dụng tập dữ liệu kiểm tra, chúng ta sẽ đánh giá độ chính xác của mô hình SVM lẽ mềm vừa được huấn luyện.

Kết quả với mô hình SVM lẽ mềm với hàm nhân Gaussian (RBF) cho độ chính xác đạt được khoảng 98%. Phần mã nguồn minh hoạ, người học có thể tham khảo tại <https://gist.github.com/cuongtv312/c7c81f285637e089273cd9749be1fcb8>

4.6 Tổng kết chương

Máy véc-tơ hỗ trợ (SVM) là một kỹ thuật phân lớp hiệu quả, dựa trên việc tìm kiếm siêu phẳng tối ưu phân tách rõ ràng hai lớp dữ liệu. Mô hình sử dụng khái niệm lẽ để chọn siêu phẳng có khoảng cách lớn nhất đến các điểm gần nhất thuộc hai lớp.

Bảng 4.1: Dữ liệu huấn luyện

x^1	x^2	y
1	0	-1
4	5	+1
1	2	-1
0	1	-1
4	1	+1
2	2	-1

SVM được huấn luyện thông qua việc giải bài toán đối ngẫu, trong đó chỉ các điểm nằm trên biên quyết định (gọi là véc-tơ hỗ trợ) mới ảnh hưởng đến vị trí của siêu phẳng tối ưu. Từ đó, mô hình đạt hiệu quả cao về tính khái quát và độ chính xác.

Một ưu điểm nổi bật của SVM là khả năng mở rộng cho các bài toán phân lớp phi tuyến nhờ kỹ thuật nhân hoá, cho phép ánh xạ dữ liệu sang không gian đặc trưng khác mà không tính toán trực tiếp tọa độ trên không gian đó.

Về mặt lý thuyết, SVM dựa trên nền tảng của tối ưu hóa lồi và chương trình bậc hai. Độc giả quan tâm có thể tham khảo thêm trong các tài liệu về tối ưu như [8] và [9].

Bài tập

1. Hãy giải thích tại sao Perceptron không thể giải quyết bài toán phân loại phi tuyến tính.
2. Giải thích tác dụng của phép chuẩn hoá về $\mathcal{N}(0, 1)$ đối với véc-tơ đặc trưng đầu vào khi huấn luyện mô hình Perceptron.
3. Cho dữ liệu trong Bảng 4.1 với nhãn $y_i \in \{-1, 1\}$ như sau Giả sử mô hình Perceptron được khởi tạo với phương trình đường thẳng $x^2 = \frac{3}{2}$.

- a) Xác định các tham số và tính độ chính xác của mô hình Perceptron.
 - b) Phát biểu thuật toán cập nhật và sử dụng thuật toán đó để cập nhật đường thẳng đã cho trong 2 bước cập nhật. Tính toán độ chính xác của mô hình sau mỗi bước cập nhật.
 - c) Tìm một đường thẳng bất kì có thể phân tách được dữ liệu. Tính số lần cập nhật tối đa dựa theo đường phân tách này.
4. *[Lập trình]* Viết chương trình mô phỏng thuật toán Perceptron với dữ liệu trong Bảng 4.1.
- a) Viết hàm cập nhật tham số của mô hình Perceptron.
 - b) Viết hàm tính toán độ chính xác của mô hình Perceptron.
 - c) Khởi tạo ngẫu nhiên các tham số của mô hình Perceptron và sử dụng thuật toán Perceptron để cập nhật tham số. So sánh số lần cập nhật tìm được với số lần cập nhật tối đa tìm được ở câu 3.
5. Hãy so sánh SVM với các mô hình phân loại tuyến tính khác như Perceptron và hồi quy Logistic. Mô hình nào có khả năng xử lý với nhiễu trong dữ liệu ổn định hơn?
6. Hãy giải thích ý nghĩa hình học của siêu phẳng phân chia dữ liệu trong SVM. Tại sao phải cần phép nhân hoá trong huấn luyện mô hình SVM.
7. *[Tìm hiểu]* Hãy tìm hiểu về các hàm nhân phổ biến trong SVM như hàm nhân Gaussian, hàm nhân đa thức, hàm nhân Sigmoid.
8. *[Tìm hiểu]* Phương pháp nhân hoá cho phép mô hình SVM có thể làm việc trên các siêu phẳng với rất nhiều chiều. Ở chiều ngược lại, nếu số chiều của dữ liệu đã rất lớn, ví dụ khoảng 10.000 chiều. Đề xuất phương pháp tối ưu hoá cho mô hình

SVM với dữ liệu như vậy. Nêu sự khác biệt chính giữa SVM tuyến tính và SVM với hàm nhân.

9. *[Lập trình]* Viết chương trình huấn luyện theo thuật toán SVM lề mềm với dữ liệu trong Bảng 4.1. Tính toán độ rộng đường biên và độ chính xác của mô hình với các giá trị khác nhau của tham số C .
10. *[Lập trình]* Dựa trên mã nguồn tham khảo của phần trình bày Tình huống áp dụng, thay đổi các tham số của mô hình SVM lề mềm bao gồm:
 - Thay đổi tham số C trong khoảng $[0.01, 100]$.
 - Thay đổi tham số γ trong khoảng $[0.01, 1]$.
 - Thay đổi hàm nhân hoá từ Gaussian sang hàm nhân đa thức.

So sánh các mô hình huấn luyện được theo độ chính xác.

Tài liệu tham khảo

- [1] Cortes, C., and Vapnik, V., *Support-vector networks*, Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.
- [2] Boser, B. E., Guyon, I. M., and Vapnik, V. N., *A training algorithm for optimal margin classifiers*, Proceedings of the 5th Annual ACM Workshop on Computational Learning Theory, pp. 144–152, 1992.
- [3] Vapnik, V. N., *Statistical learning theory*, Wiley-Interscience, 1998.
- [4] Schölkopf, B., and Smola, A. J., *Learning with kernels: Support vector machines, regularization, optimization, and beyond*, MIT Press, 2001.
- [5] Joachims, T., *Text categorization with support vector machines: Learning with many relevant features*, Proceedings of the 10th European Conference on Machine Learning, pp. 137–142, 1998.
- [6] Chang, Chih-Chung, and Chih-Jen Lin. "LIBSVM: a library for support vector machines." *ACM transactions on intelligent systems and technology (TIST)* 2.3 (2011): 1-27.

- [7] Fan, R. E., Chang, K. W., Hsieh, C. J., Wang, X. R., Lin, C. J. (2008). LIBLINEAR: A library for large linear classification. the Journal of machine Learning research, 9, 1871-1874.
- [8] Boyd, Stephen. "Convex optimization." Cambridge UP (2004).
- [9] Nocedal, Jorge, and Stephen J. Wright, eds. Numerical optimization. New York, NY: Springer New York, 1999.

Chương 5

Mạng nơ-ron tích chập

Thuật toán lan truyền ngược được Paul Werbos đề xuất lần đầu vào năm 1974, và được áp dụng hiệu quả vào mạng nơ-ron năm 1986 bởi Rumelhart, Hinton và Williams. Thuật toán này mở ra hy vọng mới cho Học máy nhờ khả năng biểu diễn phi tuyến mạnh mẽ của mạng nơ-ron nhiều lớp.

Tuy nhiên, mạng nhiều lớp lan truyền tới bộc lộ nhiều hạn chế: số lượng tham số lớn, hiện tượng triệt tiêu đạo hàm, thiếu khả năng tự trích chọn đặc trưng, dễ quá khớp và nhạy cảm với dữ liệu đầu vào. Việc khắc phục các vấn đề này đòi hỏi cải tiến kiến trúc mạng nơ-ron.

Chương 5 giới thiệu một mô hình mạng nơ-ron chuyên biệt cho xử lý ảnh và video: mạng nơ-ron tích chập CNN. CNN xuất phát từ nhu cầu nhận dạng ảnh và đã trở thành nền tảng trong thị giác máy tính.

CNN sử dụng phép toán tích chập để trích xuất đặc trưng cục bộ từ ảnh, với cơ sở toán học từ tích chập trên tín hiệu hai chiều.

Công thức tích chập liên tục được định nghĩa như sau:

$$(x * w) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} x(u, v) \cdot w(x - u, y - v) du dv \quad (5.1)$$

Trong thực tế, ảnh và bộ lọc được biểu diễn dưới dạng ma trận rời rạc. Các đặc trưng như cạnh, góc, khuôn mặt, chữ số hay vật thể được trích xuất qua các lớp tích chập.

Để tăng hiệu quả, CNN hiện đại tích hợp thêm các lớp như liên kết đầy đủ, triệt tiêu ngẫu nhiên và chuẩn hoá loạt. Chương này cũng trình bày các kiến trúc CNN tiêu biểu như VGG, Inception, ResNet và MobileNet.

5.1 Mô hình hoá dữ liệu dạng ảnh

Ảnh và các dữ liệu liên quan như video mang nhiều thông tin thị giác nên là một trong những đối tượng nghiên cứu chính của Học máy. Trong máy tính, ảnh có nhiều cách thể hiện. Trong đó cách thể hiện cơ bản nhất là dùng ma trận nhiều chiều.

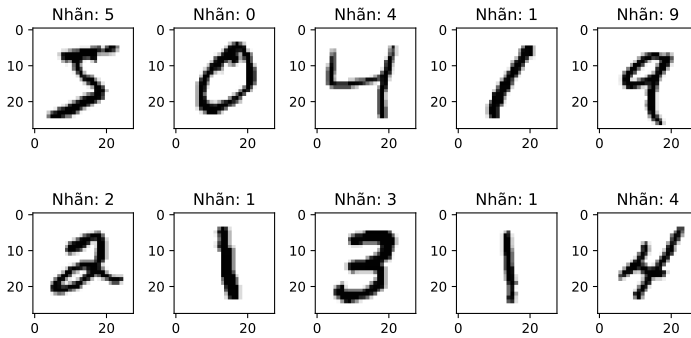
Định nghĩa 5.1 (Ảnh cấp xám). Ảnh cấp xám là ma trận hai chiều $G \in \mathbb{R}^{R \times C}$ với R là chiều cao và C là chiều rộng của ảnh. Mỗi phần tử $a_{i,j}$ trong ma trận là *cường độ* của điểm ảnh tại hàng i và cột j .

Hình 5.1 minh họa ảnh cấp xám với kích thước $R \times C$ trong đó phần tử a_{ij} là phần tử ở hàng i , cột j . Giá trị $a_{i,j}$ có thể được hạn chế là số thực trong đoạn $[0, 1]$ hoặc là số nguyên không âm trong đoạn $[0, 255]$.

Ví dụ 5.2 (Ảnh cấp xám các chữ số MNIST). Một trong các bộ dữ liệu chuẩn về ảnh cấp độ sáng là bộ ảnh MNIST [1] gồm các ảnh có kích thước 28×28 thể hiện các chữ số viết tay từ 0 đến 9.

$$\begin{array}{cccc}
 a_{1,1} & a_{1,2} & \cdots & a_{1,C} \\
 a_{2,1} & a_{2,2} & \cdots & a_{2,C} \\
 \vdots & \vdots & \vdots & \vdots \\
 a_{R,1} & a_{R,2} & \cdots & a_{R,C}
 \end{array}$$

Hình 5.1: Minh họa về ảnh cấp sáng được lưu trữ dưới dạng ma trận hai chiều.

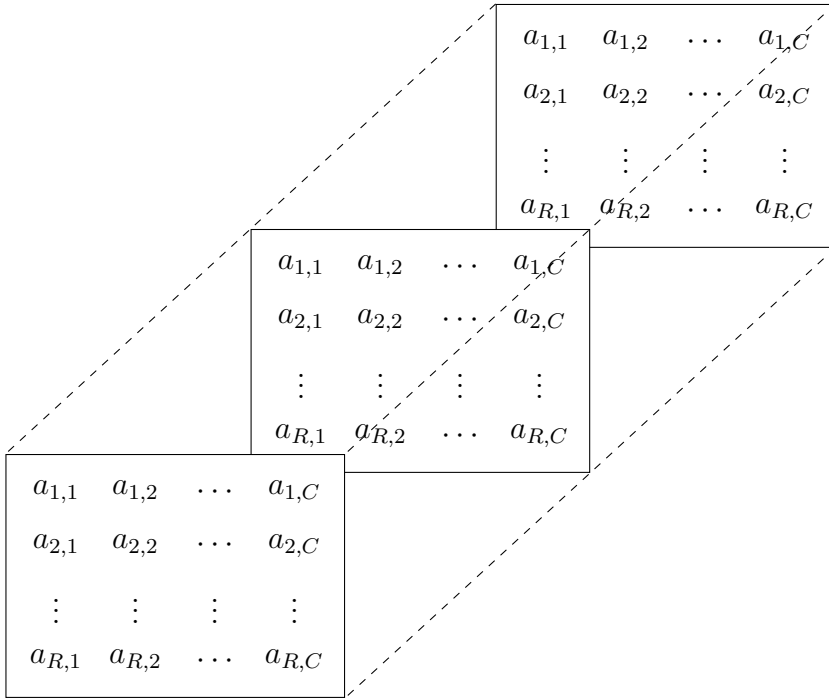


Hình 5.2: Ví dụ về ảnh cấp xám 28×28 được lấy từ tập MNIST với nhãn cần phải học là tập các chữ số từ 0 đến 9.

Định nghĩa 5.3 (Ảnh đa sắc, ảnh đa kênh). Ảnh đa kênh là ma trận ba chiều $I \in \mathbb{R}^{R \times C \times K}$ với R là chiều cao, C là chiều rộng và K là số kênh màu. Có thể hiểu mỗi kênh màu là một ảnh cấp xám đặc trưng cho một kênh màu nhất định. Mỗi phần tử $a_{i,j,k}$ trong ma trận là cường độ của điểm ảnh tại hàng i , cột j và kênh màu thứ k .

Như vậy ảnh số (ảnh màu RGB) là ảnh có 3 kênh: kênh đỏ (R), kênh xanh lá cây (G) và kênh xanh dương (B). Minh họa ba kênh màu cơ bản của ảnh số được lưu trên máy tính theo ma trận (Hình 5.3).

Đa phần các bài toán học máy trong xử lý ảnh sử dụng ảnh màu

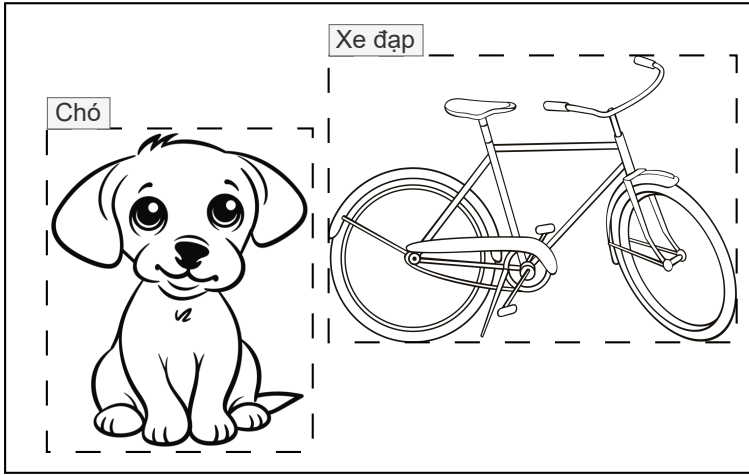


Hình 5.3: Minh họa về ba kênh màu của ảnh RGB.

RGB làm dữ liệu đầu vào. Hình 5.4 [12] minh họa bài toán phát hiện vật thể trong ảnh là một trong những bài toán trọng tâm của Học máy. Ngoài các ảnh màu RGB thông dụng, dữ liệu ảnh dùng trong Học máy còn có các ảnh được chụp chuyên dụng với các cảm biến khác nhau nhận tín hiệu ở các dải tần số khác nhau cũng và có số lượng các kênh màu lớn (ví dụ: ảnh chụp cắt lớp vi tính, ảnh vệ tinh).

Video được thể hiện là một chuỗi ảnh liên tiếp. Ở mức chung nhất có thể xem ảnh là một cách lưu thông tin mang tính không gian. Các thông số kĩ thuật đi kèm dữ liệu ảnh thường có kích thước về chiều cao, chiều rộng, số kênh màu.

Thư viện dùng để xử lí ảnh thường dùng là Opencv. Ngoài ra còn có các thư viện khác như Pillow hoặc Scikit-image. Một điểm

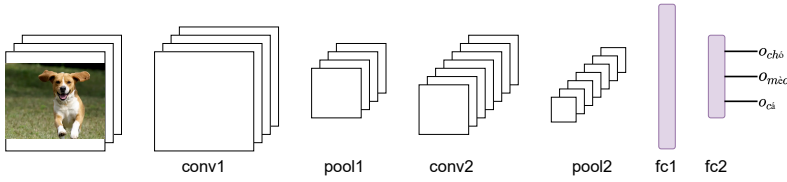


Hình 5.4: Ví dụ về dữ liệu dùng cho bài toán phát hiện vật thể trong ảnh.

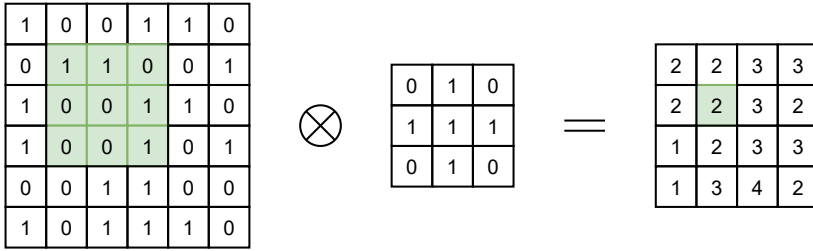
lưu ý là các dữ liệu ảnh thường rất lớn và cần lượng tài nguyên tính toán và lưu trữ thích hợp để có thể xử lý.

5.2 Phép toán tích chập

Mạng nơ-ron tích chập (CNN) có thể nói là mô hình Học sâu phổ biến nhất hiện nay. Mạng CNN là nguồn cảm hứng và động lực cho nhiều nghiên cứu và ứng dụng của Học máy. Bắt đầu gây chú ý với mạng AlexNet năm 2012 [2], việc nghiên cứu và ứng dụng mạng CNN đã bùng nổ suốt từ đó đến nay. Từ mạng Alexnet có tám lớp, các mạng CNN đã phát triển thành mạng ResNet có 152 lớp và còn nhiều mạng có độ sâu hơn thế. Các cấu trúc mới, chuyên biệt cho từng loại ứng dụng Học máy được phát triển dựa trên các mạng xương sống như ResNet [4], Inception và MobileNet nở rộ và đạt nhiều thành tựu, đặc biệt trong các cuộc thi về khả năng nhận dạng. Cấu trúc mạng CNN còn được ứng dụng trong các hệ thống gợi ý, xử lý ngôn ngữ tự nhiên, xử lý âm thanh, v.v..



Hình 5.5: Cấu trúc chung của mạng CNN.



Hình 5.6: Ví dụ phép tích chập trên ảnh cấp xám.

5.2.1 Lớp tích chập

Mạng CNN gồm nhiều loại lớp nơ-ron nối với nhau gồm: lớp tích chập, lớp gộp và lớp liên kết đầy đủ (Hình 5.5). Lớp liên kết đầy đủ chính là lớp nơ-ron của mạng MLP chúng ta đã tìm hiểu ở mục trước, lớp này thường dùng ở phía cuối mạng để cho kết quả dự đoán. Trong mục này, chúng ta tìm hiểu lớp tích chập bao gồm nguyên lý thiết kế và cách cập nhật trọng số theo thuật toán lan truyền ngược.

Ý tưởng của lớp tích chập xuất phát từ kỹ thuật lọc trên tín hiệu hai chiều của xử lý ảnh. Giả sử có một ảnh cấp xám, để tìm một đặc trưng ảnh nhất định, kỹ thuật lọc ảnh *trượt* một ma trận bé (ví dụ cỡ 3×3 , 5×5 , 7×7) - gọi là *bộ lọc* - trên các vị trí trong ảnh to - gọi là *vùng tiếp nhận*. - và sử dụng phép tính tích vô hướng của bộ lọc với vùng tiếp nhận. Kết quả của việc trượt bộ lọc trên ảnh to tạo nên một ảnh cấp xám mới gọi là *bản đồ đặc trưng*. Các kiến thức chuyên sâu hơn về bộ lọc trong xử lý ảnh có thể được tìm hiểu trong các giáo trình về xử lý ảnh [9].

Ví dụ 5.4. Ví dụ phép tích chập Hình 5.6 sử dụng bộ lọc 3×3 (ở giữa) để tìm dấu chữ thập trong ảnh 6×6 (bên trái) cho kết quả là bản đồ đặc trưng (bên phải).

Định nghĩa 5.5 (Tích chập trên ảnh cấp xám). Tích chập của ảnh cấp xám $I \in \mathbb{R}^{H_{in} \times W_{in}}$ với bộ lọc $W \in \mathbb{R}^{k_h \times k_w}$ là một bản đồ đặc trưng $O \in \mathbb{R}^{H_{out} \times W_{out}} = \text{Conv}(I, W)$ được tính như sau:

$$O_{h,w} = \sum_{m=1}^{k_h} \sum_{n=1}^{k_w} W_{m,n} I_{h+m-1, w+n-1} \quad (5.2)$$

với $h \in [1, H_{out}]$, $w \in [1, W_{out}]$. Ở đây $H_{out} = H_{in} - k_h + 1$ và $W_{out} = W_{in} - k_w + 1$ là kích thước của ảnh đầu ra.

Chia sẻ trọng số. Nếu nhìn dưới quan điểm của lớp nơ-ron thì mỗi vị trí trong bản đồ đặc trưng $O_{h,w}$ tương ứng với một nơ-ron. Nơ-ron này chỉ “nhìn” vào một vùng tiếp nhận $k_h \times k_w$ trong ảnh ban đầu bằng cách đặt trọng số cho các vị trí trong vùng này bằng giá trị của bộ lọc còn các vùng ở ngoài nhận trọng số bằng 0. Như vậy các nơ-ron của bản đồ đặc trưng *chia sẻ trọng số* với nhau. Đây là ý tưởng không mới nhưng có tính đột phá đối với mạng nơ-ron vì (i) số lượng tham số của mô hình giảm đi đáng kể và (ii) trong pha huấn luyện, đạo hàm của các tham số có chia sẻ lớn hơn do được cộng dồn. Trong Hình 5.6 có đến $4 \times 4 = 16$ nơ-ron nhưng chỉ có tất cả 3×3 trọng số được chia sẻ.

Cách cộng dồn đạo hàm trên các trọng số được chia sẻ như sau có thể được minh họa qua ví dụ với hai nơ-ron ở công thức (5.3).

$$\begin{aligned} \ell(o^1, o^2, y) &= \ell(o^1(\mathbf{x}, \mathbf{w}), o^2(\mathbf{x}, \mathbf{w}), y) \\ \frac{\partial \ell}{\partial \mathbf{w}} &= \frac{\partial \ell}{\partial o^1} \frac{\partial o^1}{\partial \mathbf{w}} + \frac{\partial \ell}{\partial o^2} \frac{\partial o^2}{\partial \mathbf{w}} \end{aligned} \quad (5.3)$$

Phát hiện đồng thời nhiều đặc trưng. Nếu như mỗi bộ lọc dùng để phát hiện một đặc trưng thì ta có thể thêm nhiều bộ lọc

để phát hiện nhiều đặc trưng ảnh cùng lúc. Kết quả tích chập của ảnh ban đầu với nhiều bộ lọc sẽ cho nhiều bản đồ đặc trưng. Số lượng bản đồ đặc trưng được gọi là *số kênh đầu ra*, được kí hiệu là C_{out} của lớp tích chập. Ví dụ, nếu áp dụng C_{out} bộ lọc, tức là $W \in \mathbb{R}^{k_h \times k_w \times C_{out}}$, trên ảnh đầu vào I thì đầu ra là ảnh đa kênh $O \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$, trong đó mỗi kênh là bản đồ đặc trưng tương ứng với một bộ lọc. Tổng quát hóa cho ảnh đa kênh, ta có định nghĩa tích chập tổng quát như sau.

Định nghĩa 5.6 (Tích chập trên ảnh đa kênh). Tích chập của ảnh đa kênh $I \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$ với bộ lọc $W \in \mathbb{R}^{k_h \times k_w \times C_{in} \times C_{out}}$ là một bản đồ đặc trưng $O \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}} = \text{Conv}(I, W)$ được tính như sau:

$$O_{h,w,c} = \sum_{m=1}^{k_h} \sum_{n=1}^{k_w} \sum_{l=1}^{C_{in}} W_{m,n,l,c} I_{h+m-1, w+n-1, l} \quad (5.4)$$

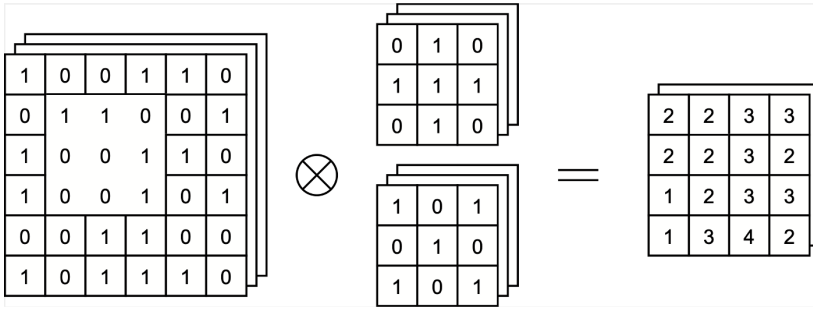
với $h \in [1, H_{out}]$, $w \in [1, W_{out}]$, $c \in [1, C_{out}]$. Ở đây $H_{out} = H_{in} - k_h + 1$ và $W_{out} = W_{in} - k_w + 1$ là kích thước của ảnh đầu ra.

Ví dụ 5.7 (Tích chập trên ảnh 3 kênh). Nếu ảnh đầu vào có kích thước $6 \times 6 \times 3$, lớp tích chập bộ lọc kích thước $3 \times 3 \times 3 \times 2$ thì kết quả ta được ảnh 2 kênh kích thước $4 \times 4 \times 2$ (gồm 2 bản đồ đặc trưng) (Hình 5.7).

Bước nhảy (stride). Để giảm kích thước bản đồ đặc trưng, chúng ta có thể trượt bộ lọc với bước nhảy s lớn hơn 1. Như vậy, ta chỉ tính tích vô hướng giữa bộ lọc và vùng tiếp nhận ở các vị trí cách nhau s theo cả chiều dài và chiều rộng. Công thức tính kích thước khi có bước nhảy được tính như sau:

$$H_{out} = \lceil (H_{in} - k_h + 1) / s \rceil \quad (5.5)$$

$$W_{out} = \lceil (W_{in} - k_w + 1) / s \rceil \quad (5.6)$$



Hình 5.7: Ví dụ phép tích chập trên ảnh ba chiều.

Bù kích thước (padding). Khi dịch chuyển bộ lọc trên ảnh, ở các vị trí cuối cùng bên phải hoặc bên dưới ảnh, một số vùng tiếp nhận bị thiếu so với kích thước bộ lọc. Nếu ta bù một khoảng đệm vào bảng để đủ kích thước ở các vùng tiếp nhận cuối ảnh thì sẽ tận dụng được hết các vùng tiếp nhận này. Thường khi bù, ta thêm các số 0 vào phần bù này. Khi đó công thức tính kích thước sẽ là

$$H_{out} = \lceil (H_{in} + 2p - k_h + 1)/s \rceil \quad (5.7)$$

$$W_{out} = \lceil (W_{in} + 2p - k_w + 1)/s \rceil \quad (5.8)$$

với p là kích thước bù thêm vào ảnh theo các hướng.

Hàm kích hoạt. Sau bước tích chập mà bản chất là một hàm tuyến tính của đầu vào, lớp tích chập lại sử dụng các hàm kích hoạt để phi tuyến hoá kết quả tính toán như hàm sigmoid, hàm tanh, hàm relu.

$$O = f(\text{Conv}(I, W)).$$

Đặc biệt hàm relu và một số biến thể của nó được sử dụng phổ biến nhất do (i) tính toán cực kì đơn giản và (ii) đạo hàm không bị triệt tiêu khi nơ-ron được kích hoạt (đầu ra lớn hơn 0).

5.2.2 Lan truyền ngược qua lớp tích chập

Mục tiêu của thuật toán lan truyền ngược là tính toán $\delta_i = \frac{\partial \ell(o, y)}{\partial o_i}$ với ℓ là hàm lỗi, o là đầu ra của mạng, o_i là đầu ra của lớp thứ i (đầu vào của lớp thứ $i + 1$).

Xét lớp thứ $i + 1$ là lớp tích chập với đầu vào o_i có kích thước $H_{in} \times W_{in} \times C_{in}$, bộ lọc W_{i+1} có kích thước $k_h \times k_w \times C_{in} \times C_{out}$ và đầu ra o_{i+1} có kích thước $H_{out} \times W_{out} \times C_{out}$.

$$o_{i+1} = f(\text{Conv}(o_i, W_{i+1}))$$

với f là hàm kích hoạt. Giả sử rằng khi lan truyền ngược, ta đã có $\delta_{i+1} = \frac{\partial \ell(o, y)}{\partial o_{i+1}}$ với kích thước $H_{out} \times W_{out} \times C_{out}$.

Từ công thức (5.4), chúng ta suy ra được công thức đạo hàm qua từng lớp tích chập như sau:

$$\frac{\partial o_{i+1}^{h,w,c}}{\partial o_i^{p,q,l}} = f'(\text{net}_{i+1}^{h,w,c}) w_{i+1}^{p-h+1, q-w+1, l, c}, \quad (5.9)$$

với $\text{net}_{i+1}^{h,w,c}$ là tổng trọng số của nơ-ron $o_{i+1}^{h,w,c}$ trước khi áp dụng hàm kích hoạt và $p \in [h, h + k_h)$, $q \in [w, w + k_w)$. Trong trường hợp, p và q không thuộc khoảng đề cập thì đạo hàm bằng 0 và chúng ta có thể bỏ qua trường hợp này trong tính toán. Ngoài ra để, đạo hàm theo hàm lỗi phân lớp được cho bởi công thức (5.10):

$$\frac{\partial \ell}{\partial o_i^{p,q,l}} = \sum_{h=\text{lb}(p, k_h)}^p \sum_{w=\text{lb}(q, k_w)}^q \sum_{c=1}^{C_{out}} \delta_{i+1}^{h,w,c} f'(\text{net}_{i+1}^{h,w,c}) w_{i+1}^{p-h+1, q-w+1, l, c} \quad (5.10)$$

Trong công thức (5.10), chúng ta sử dụng hàm $\text{lb}(u, v) = \max(1, u - v + 1)$ để tính cận dưới các chỉ số h và w .

Để ý thấy công thức (5.10) rất giống với công thức (5.4), chỉ khác là bộ lọc W_{i+1} bị xoay 180 độ và đảo chỗ hai chỉ số l, c kết

hợp với bù khoảng đệm. Như vậy, công thức truy hồi để tính δ_i từ δ_{i+1} cũng là một phép tích chập.

Sau khi có $\delta_{i+1} = \frac{\partial \ell(o,y)}{\partial o_{i+1}}$, ta sử dụng đạo hàm này để tính $\frac{\partial \ell(o,y)}{\partial w_{i+1}}$. Từ công thức (5.4), biến đổi tiếp ta thu được công thức tính toán đạo hàm theo trọng số như sau:

$$\begin{aligned} \frac{\partial \ell}{\partial w_{i+1}^{m,n,l,c}} &= \sum_{h=1}^{H_{out}} \sum_{w=1}^{W_{out}} \frac{\partial \ell}{\partial o_{i+1}^{h,w,c}} \frac{\partial o_{i+1}^{h,w,c}}{\partial w_{i+1}^{m,n,l,c}} \\ &= \sum_{h=1}^{H_{out}} \sum_{w=1}^{W_{out}} \delta_{i+1}^{h,w,c} f'(\text{net}_{i+1}^{h,w,c}) o_i^{h+m-1,w+n-1,l} \end{aligned} \quad (5.11)$$

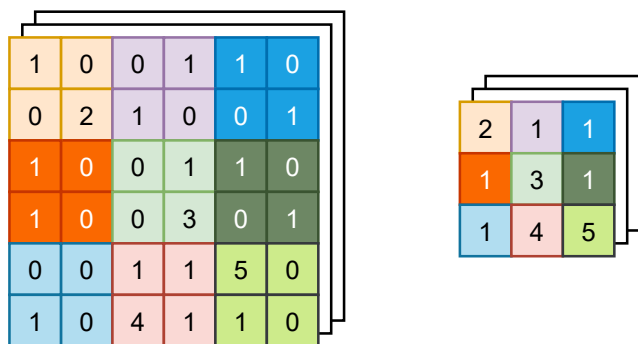
Trong cài đặt trên các thiết bị như GPU, các công thức (5.4), (5.10) và (5.11) được thực hiện song song trên nhiều lõi để tăng hiệu suất tính toán.

5.3 Các lớp phổ biến được sử dụng trong mạng tích chập

Mặc dù lớp tích chập đóng vai trò cơ bản trong việc học ra các đặc trưng cơ bản của ảnh, việc sử dụng các lớp tích chập sẽ thiếu hiệu quả nếu thiếu các phép lấy mẫu và ổn định quá trình huấn luyện mạng học sâu. Trong phần này, chúng ta sẽ làm quen với những lớp cơ bản khác đóng vai trò quan trọng trong cấu trúc một mạng học sâu hiện đại.

5.3.1 Lớp gộp

Sau các lớp tích chập, mạng CNN sử dụng lớp gộp (pooling) để tiếp tục làm giảm kích thước ảnh. Lớp gộp giúp giảm khối lượng tính toán và quan trọng nhất, làm giảm ảnh hưởng của các thay đổi nhỏ từ đầu vào. Lớp gộp hoạt động độc lập trên các bản đồ đặc



Hình 5.8: Ví dụ kết quả của lớp gộp theo giá trị lớn nhất trong vùng tiếp nhận.

trung khác nhau nên chỉ làm thay đổi kích thước chiều dài, chiều rộng mà không làm thay đổi số kênh của ảnh.

Lớp gộp tính toán cũng theo các vùng tiếp nhận như lớp tích chập. Tuy nhiên, lớp tích chập không có trọng số mà nó sử dụng các phép toán quen thuộc như MaxPooling (giá trị lớn nhất trong vùng tiếp nhận) hoặc AveragePooling (giá trị trung bình). Lớp gộp MaxPool được dùng phổ biến nhất trong các mạng CNN. Ngoài ra, ta có thể thay đổi kích thước vùng tiếp nhận cũng như trượt nó với một bước nhảy tùy ý.

Ví dụ 5.8. Hình 5.8 là kết quả của lớp gộp với vùng tiếp nhận 2×2 và bước nhảy 2. Các nơ-ron được tô cùng màu với vùng tiếp nhận của nó. Ta thấy kích thước ảnh giảm hai lần theo cả chiều dài và chiều rộng (giảm bốn lần diện tích) nhưng giữ nguyên số kênh. Nếu sau lớp gộp là lớp liên kết đầy đủ thì số lượng trọng số của lớp liên kết đầy đủ giảm đi 4 lần so với không có lớp gộp. Trong một mạng CNN có hàng chục triệu, trăm triệu tham số, tỉ suất này có tác dụng rất lớn đối với hiệu năng huấn luyện và suy luận của mạng.

5.3.2 Lớp liên kết đầy đủ

Lớp liên kết đầy đủ bản chất là lớp nơ-ron của mạng MLP được trình bày ở chương trước. Trong mạng CNN, sau một loạt các lớp tích chập, lớp gộp, khi độ lớn của ảnh giảm xuống đến kích thước thiết kế, tất cả các nơ-ron này được nối vào một số lớp liên kết đầy đủ nối tiếp nhau để tính đầu ra mong muốn của mô hình. Các tính toán ở các lớp liên kết đầy đủ cuối cùng giống hệt như những gì chúng ta tìm hiểu về mạng MLP ở mục trước. Nhiệm vụ của lớp liên kết đầy đủ trong mạng CNN là kết hợp thông tin từ các bản đồ đặc trưng cuối cùng của mạng CNN để đưa ra dự đoán cuối cùng. Tùy vào từng cấu trúc cụ thể mà, một số mô hình CNN sử dụng một hay nhiều lớp liên kết đầy đủ. Một lưu ý nhỏ nữa là, lớp liên kết đầy đủ thường có số tham số nhiều hơn so với các lớp tích chập và gộp. Do đó, việc sử dụng lớp liên kết đầy đủ cần cân nhắc để tránh tình trạng phù hợp quá mức.

5.3.3 Lớp triệt tiêu ngẫu nhiên

Một kỹ thuật đặc trưng, rất phổ biến trong mạng CNN là kỹ thuật triệt tiêu ngẫu nhiên. Trong nhiều trường hợp, kể cả các mô hình có hiệu suất cao nhất (ví dụ độ chính xác 95%) có thể nhận thêm được 1-2% độ chính xác khi áp dụng kỹ thuật này. Kỹ thuật triệt tiêu ngẫu nhiên giúp mạng CNN tránh hiện tượng học quá bằng một ý tưởng khá đơn giản. Trong quá trình huấn luyện, các nơ-ron ở lớp trước bị triệt tiêu giá trị thành 0 ở lớp triệt tiêu ngẫu nhiên với một xác suất p cho trước (ví dụ: $p = 0,5$ nghĩa là khoảng 50% số nơ-ron lớp trước sẽ bị triệt tiêu).

Khi chúng ta cố tình triệt tiêu giá trị của các nơ-ron trong pha huấn luyện, các nơ-ron khác phải “gánh” trách nhiệm tạo ra đầu ra mong muốn. Do đó, bất cứ nơ-ron nào cũng phải “học” thay vì dựa vào một số nơ-ron nhất định trong mạng. Lớp triệt tiêu ngẫu nhiên có thể đứng sau tất cả các lớp (lớp đầu vào hoặc các lớp ẩn)

ngoại trừ lớp đầu ra. Ngoài ra, lớp triệt tiêu ngẫu nhiên cũng không hoạt động trong pha suy luận mà chúng ta chỉ triệt tiêu giá trị của nơ-ron trong pha huấn luyện. Do đó, trong pha suy luận, đầu ra của lớp triệt tiêu ngẫu nhiên là tích của đầu vào với p để được giá trị có độ lớn phù hợp.

5.3.4 Lớp chuẩn hóa loạt

Vai trò của lớp chuẩn hóa loạt là dùng để ổn định giá trị đầu ra và đạo hàm của các lớp mạng. Lớp chuẩn hóa loạt sử dụng một phép biến đổi tuyến tính để đảm bảo giá trị được cập nhật cho các lớp phía trước tập trung quanh giá trị 0. Một chú ý nhỏ ở đây là giá trị 0 là giá trị đặc biệt của các hàm kích hoạt phổ biến như relu hoặc sigmoid.

Về mặt toán học, lớp chuẩn hóa loạt được đặc trưng bằng hai tham số β, γ . Giả sử tại mỗi lô dữ liệu huấn luyện, ta nhận m điểm dữ liệu $\mathbf{x}_i$ với $i \in [1 \dots m]$, chúng ta có thể tính được giá trị trung bình và phương sai trong lớp đó như sau:

1. Tính trung bình thực nghiệm (μ_B) và phương sai (σ_B^2) của gói trong một lượt cập nhật:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i, \quad (5.12)$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}_i - \mu_B)^2, \quad (5.13)$$

trong đó m là số lượng mẫu của gói.

2. Chuẩn hóa từng đầu vào:

$$\hat{\mathbf{x}}_i = \frac{\mathbf{x}_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad (5.14)$$

với ϵ là hằng số rất nhỏ dùng để tránh chia cho 0.

3. Áp dụng phép biến đổi tuyến tính với các tham số huấn luyện γ và β :

$$\mathbf{y}_i = \gamma \hat{\mathbf{x}}_i + \beta. \quad (5.15)$$

Lớp chuẩn hoá loạt được sử dụng rộng rãi trong các mạng CNN hiện đại như ResNet, MobileNet và InceptionNet.

5.4 Kiến trúc mạng CNN xương sống

Quá trình phát triển bùng nổ của mạng CNN có sự đóng góp rất lớn của một số kiến trúc *mạng xương sống* (backbone). Các kiến trúc mạng này được huấn luyện sẵn với một số bộ dữ liệu cực lớn như bộ dữ liệu ImageNet [10]. Nhờ đó, các mạng xương sống có khả năng phát hiện nhiều đặc trưng ảnh. Các mô hình mới thường sử dụng một trong các mạng xương sống này cùng bộ trọng số đã được huấn luyện sẵn làm cơ sở để thêm các tính năng mới. Trong mục này, chúng ta cùng tìm hiểu một số kiến trúc mạng CNN xương sống và ý tưởng độc đáo của chúng.

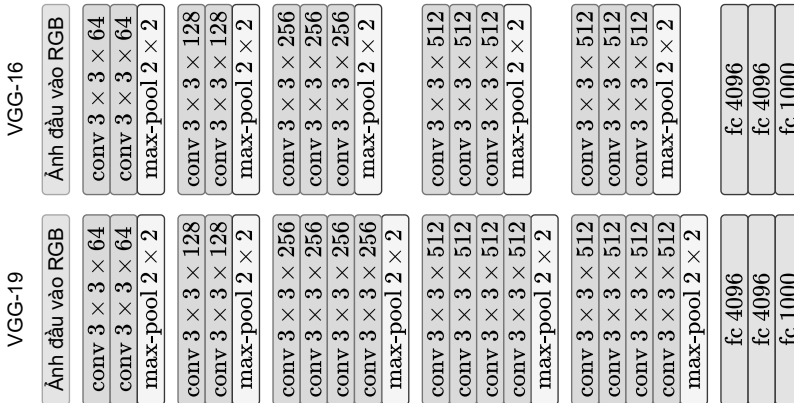
5.4.1 Mạng VGG

Mạng VGG được Karen Simonyan và Andrew Zisserman phát minh năm 2014. Vào năm đó, mạng VGG xếp thứ hai về độ chính xác (tỉ lệ lỗi 7,3%) trong cuộc thi phân loại ảnh với bộ dữ liệu ImageNet. Mạng VGG là một mạng CNN khá đơn giản với các lớp nối tiếp. Ý tưởng chính của mạng VGG là

- Vùng tiếp nhận lớn từ nhiều lớp tích chập liên tiếp: Một khối gồm hai lớp conv 3x3 xếp liền nhau sẽ có vùng tiếp nhận (receptive field) là 5x5. Mạng VGG chỉ dùng các khối nhiều lớp conv3x3 xếp liền nhau.

- Khối nhập chập phía sau có nhiều bộ lọc hơn khối tích chập phía trước.

Ngoài ra, mạng VGG sử dụng hàm kích hoạt Relu và sử dụng lớp gộp 2x2 ngay sau các khối nhân nhập. Một số cải tiến mạng VGG thêm các lớp triệt tiêu ngay sau các lớp tích chập. Kết thúc mạng VGG là các hai lớp fc có 4096 nơ-ron và một lớp liên kết đầy đủ có 1000 nơ-ron đầu ra của mạng do bài toán phân lớp có 1000 lớp. Hai mạng VGG-16 (16 lớp) và VGG-19 (19 lớp) là hai mạng VGG nổi tiếng và hay được sử dụng làm mạng xương sống nhất (Hình 5.9). Kiến trúc mạng VGG không thích hợp cho việc thêm lớp nơ-ron để



Hình 5.9: Các mạng VGG-16 và VGG-19.

mạng sâu hơn do hiện tượng triệt tiêu đạo hàm khi mạng quá sâu. Ngoài ra, số lượng trọng số của mạng VGG cũng rất lớn (VGG-16 có 138 triệu, VGG-19 có 144 triệu trọng số) không thích hợp cho các thiết bị tính toán nhúng hoặc di động.

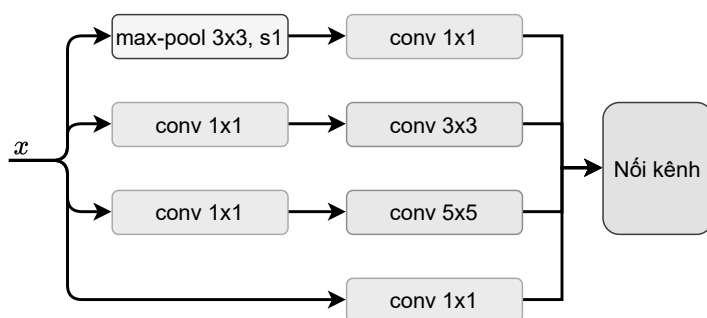
5.4.2 Mạng Inception

Mạng Inception được phát minh để khắc phục các nhược điểm của mạng VGG và các mạng CNN trước đó về khả năng huấn luyện với độ sâu mạng lớn. Điểm yếu của mạng VGG xuất phát từ việc nó có

số kênh quá lớn. Ví dụ, ở các lớp tích chập cuối, để nhận vào 512 kênh và tính ra 512 kênh đòi hỏi $3 \times 3 \times 512 \times 512 = 2.359.296$ tham số cho một lớp tích chập. Các ý tưởng chính của mạng Inception, được mô tả ở Hình 5.10, bao gồm:

- *Nút thắt*: Mạng Inception sử dụng kỹ thuật nút thắt. Cụ thể, để giảm số kênh trước khi thực hiện tích chập 3×3 và 5×5 , mạng Inception thực hiện phép tích chập với kích thước 1×1 để đưa số kênh về mức nhỏ, từ đó giảm số tham số cũng như ép các nơ-ron phải “học” do bị thắt chặt khả năng tính toán.
- *Quét đặc trưng với bộ lọc nhiều kích thước*: mạng Inception sử dụng các lớp conv 1×1 , 3×3 , 5×5 để quét đặc trưng ở các vùng tiếp nhận kích thước khác nhau, từ đó phát hiện các đặc trưng ở các mức độ phân giải khác nhau.
- *Sử dụng lớp gộp toàn cục*: ở cuối mạng thay cho lớp liên kết đầy đủ. Ở lớp tích chập cuối cùng, mạng Inception sử dụng số bộ lọc đúng bằng số lớp cần phân loại. Sau đó, lớp gộp toàn cục tính trung bình cộng từng bản đồ đặc trưng để tính số đầu ra đúng bằng số lớp. Kỹ thuật này cũng làm giảm đáng kể số tham số (Mạng VGG sử dụng khoảng 90% tham số ở 3 lớp liên kết đầy đủ cuối cùng).
- *Sử dụng thêm hàm lỗi bổ trợ*: Để tránh hiện tượng triệt tiêu đạo hàm khi huấn luyện, ở ngay những lớp giữa của mạng Inception có thêm các đầu ra bổ trợ cũng dùng để phân lớp như đầu ra cuối của mạng. Nhờ đó, các lớp phía trước có thêm đạo hàm từ các hàm lỗi đặt tại đây. Khi huấn luyện, hàm lỗi sử dụng là tổng có trọng số của hàm lỗi ở lớp cuối cùng và các hàm lỗi bổ trợ ở các lớp phía trước.

Mạng Inception đến nay có bốn phiên bản từ Inception-v1 đến Inception-v4. Ngoài ra có thêm một phiên bản Inception-ResNet



Hình 5.10: Cấu trúc mô-đun inception.

(56 triệu tham số) sử dụng thêm ý tưởng của mạng ResNet ở mục tiếp sau đây.

Mạng Inception-v1 sử dụng chín mô-đun inception, tổng cộng có 22 lớp, khoảng 5 triệu tham số, đạt tỉ lệ lỗi 6,67% đứng đầu cuộc thi ILSVRC năm 2014. Mạng Inception-v2 (11 triệu tham số), Inception-v3 (23 triệu tham số), Inception-v4 (43 triệu tham số) có một số cải tiến tăng hiệu suất tính toán như (i) bằng cách thay lớp tích chập 5×5 bằng hai lớp tích chập 3×3 ; (ii) thay các lớp tích chập 3×3 bằng hai lớp tích chập 1×3 và 3×1 ; (iii) bố trí các lớp tích chập 1×3 và 3×1 song song thay vì nối tiếp để làm cho mạng rộng thay vì sâu hơn, tránh mất quá nhiều thông tin; (iv) sử dụng lớp tích chập 7×7 (cũng tách như 5×5).

Một kết quả thú vị là khi kết hợp ba mô hình Inception-ResNet-v2 và một mô hình Inception-v4 bằng phương pháp học kết hợp thì tỉ lệ lỗi hạ xuống còn 3,08%.

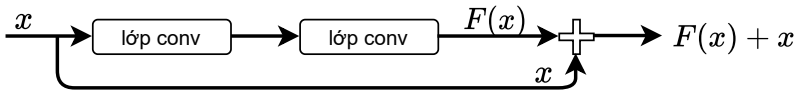
5.4.3 Mạng ResNet

Mạng ResNet, là từ viết tắt của thuật ngữ Residual Network, là mạng nơ-ron đầu tiên chiến thắng tại cuộc thi phân loại ảnh ImageNet vào năm 2015. Mạng ResNet có đặc điểm nổi bật là nó có rất nhiều lớp nơ-ron (mạng ResNet chiến thắng năm 2015 có 152

lớp) mà không bị ảnh hưởng của hiện tượng *triệt tiêu đạo hàm* như kiến trúc VGG. Điểm cải tiến nổi bật của ResNet so với các thế hệ trước là các khối phần dư được thiết lập bởi kết nối dư thừa. Đây được đánh giá là một trong những ý tưởng đột phá nhất về cấu trúc mạng CNN trong những giai đoạn đầu mà nhờ đó ResNet có thể huấn luyện rất dễ dàng, đạt tỉ lệ lỗi 3,57% trên bộ dữ liệu ImageNet năm 2015. So sánh với kiến trúc khác, ý tưởng chính của mạng ResNet bao gồm những điểm chính sau là:

- *Khối phần dư*. Các khối này khắc phục nhược điểm của mạng CNN gồm các lớp tích chập nối tiếp là các khối phía sau không nhìn thấy đầu vào của các khối phía trước mà chỉ nhìn thấy đầu ra của các khối phía trước. Ở pha huấn luyện cũng vậy, đạo hàm cũng phải lan truyền ngược theo một thứ tự nhất định và ngày càng bị triệt tiêu.

Cụ thể, nếu xét x là đầu ra của một lớp tích chập, ta thêm vào sau x một số lớp conv và *cộng* đầu ra với x , kết quả của phép cộng là đầu vào của lớp nơ-ron tiếp theo. Trong Hình 5.11, có thể thấy nếu như x là đầu ra đang đạt được của mạng thì $F(x)$ chính là phần dư còn thiếu để đầu ra này “hoàn hảo” (ví dụ, trích chọn được đặc trưng quan trọng). Ta gọi kiểu kết nối này là *kết nối phần dư*. Kết nối đưa x ra cộng với $F(x)$ gọi là *kết nối đơn vị* hoặc *kết nối tắt*.

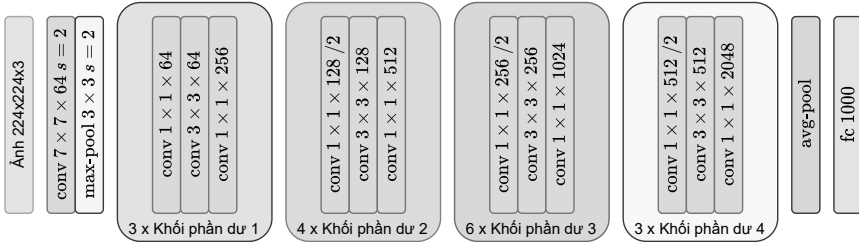


Hình 5.11: Kết nối phần dư.

- *Chuẩn hoá loạt*: mạng ResNet sử dụng các lớp chuẩn hoá loạt sau mỗi lớp tích chập. Các lớp chuẩn hoá loạt giúp pha huấn luyện có thể sử dụng hệ số huấn luyện λ lớn hơn cũng như làm giảm hiện tượng triệt tiêu đạo hàm.

- *Nút thắt*: mỗi khối kết nối phần dư của mạng ResNet gồm ba lớp nhập chập với các kích thước lần lượt là: 1×1 , 3×3 và 1×1 nối tiếp nhau. Hai lớp 1×1 giúp giảm số kênh của ảnh rồi tăng số kênh trở lại. Lớp conv 3×3 đóng vai trò như nút thắt cổ chai đối với thông tin lan truyền, làm việc với số lượng kênh đầu vào và đầu ra nhỏ hơn.

Với các cải tiến kỹ thuật này, mạng ResNet có thể sâu đến 152 lớp mà vẫn có thể huấn luyện một cách dễ dàng. Cấu trúc ResNet ngày nay là cấu trúc xương sống phải tính đến khi xây dựng các mô hình mới.



Hình 5.12: Mạng ResNet-50.

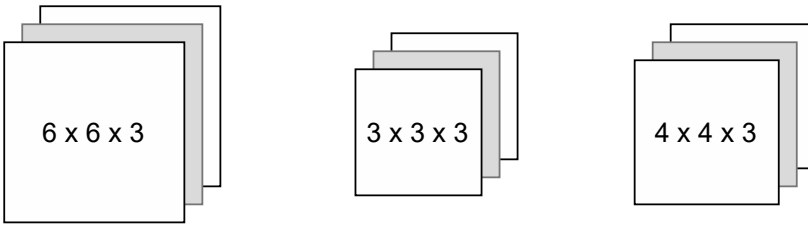
Mạng ResNet-50 (Hình 5.12) gồm 50 lớp, bốn khối phần dư, mỗi khối được lặp lại một số lần. Ở lần lặp đầu tiên, ở các khối phần dư có kí hiệu $/2$, các lớp tích chập 3×3 ở kết nối phần dư và kết nối đơn vị sử dụng bước nhảy bằng 2 làm giảm kích thước ảnh đi hai lần. Mạng ResNet-152 mở rộng mạng ResNet-50 với số lần lặp các khối phần dư thứ hai và thứ ba lần lượt là tám và 36 để đạt độ sâu 152 lớp.

5.4.4 Mạng MobileNet

Mạng MobileNet là mạng CNN được thiết kế với mục đích làm việc trên các thiết bị nhúng, thiết bị di động. Do khả năng tính toán hạn chế, các tính toán trên mạng MobileNet tìm cách mô phỏng

các lớp tích chập thông thường bằng các lớp tích chập nhẹ hơn. Kỹ thuật này gọi là *tích chập phân tách theo kênh*. Cụ thể, kỹ thuật này gồm hai loại tích chập sau để thay thế tích chập bình thường:

- *Tích chập trên từng kênh*: toán tử này thực hiện tích chập trên từng kênh của ảnh đầu vào thay vì quét ảnh đầu vào trên tất cả các kênh như lớp tích chập thông thường. Như vậy, số kênh của bộ lọc bằng với số kênh của đầu vào và số kênh của đầu ra ở bước này. Ví dụ: như hình 5.13, ta tính tích chập từng kênh đầu vào ($6 \times 6 \times 1$) với kênh tương ứng của bộ lọc ($3 \times 3 \times 1$) để được một kênh đầu ra ($4 \times 4 \times 1$), sau đó ghép các kênh kết quả lại để được đầu ra ($4 \times 4 \times 3$).



Hình 5.13: Tích chập trên từng kênh.

- *Tích chập trên từng điểm*: toán tử này chính là phép tích chập với vùng tiếp nhận 1×1 . Như vậy tác dụng của phép tích chập trên từng điểm là thay đổi số kênh của đầu vào.

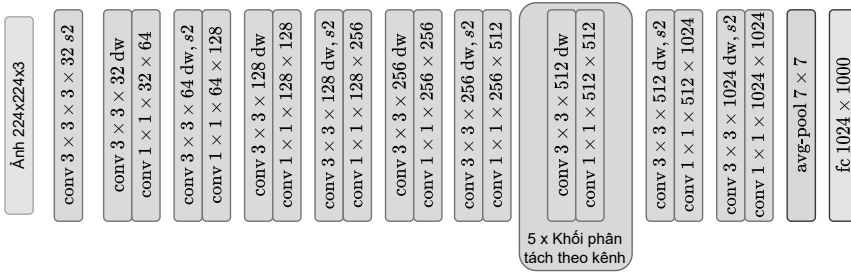
Khi kết hợp với phép tích chập trên từng kênh, hai toán tử này mô phỏng cách tính tích chập bình thường nhưng sử dụng số tham số và số lượng tính toán thấp hơn nhiều. Cụ thể, tổng số tham số của cả hai toán tử là

$$k_h \times k_w \times C_{in} + C_{in} \times C_{out} \quad (5.16)$$

và số lượng tính toán là

$$k_h \times k_w \times C_{in} \times H_{in} \times W_{in} + C_{in} \times C_{out} \times H_{in} \times W_{in} \quad (5.17)$$

So với khối lượng tính toán nếu dùng lớp tích chập thông thường là $k_h \times k_w \times C_{in} \times C_{out} \times H_{in} \times W_{in}$ thì số tính toán giảm đi xấp xỉ $k_h \times k_w$ lần khi C_{out} lớn. Nếu dùng bộ lọc 3×3 thì tỉ số này khoảng gần chín lần.



Hình 5.14: Kiến trúc mạng MobileNet-v1.

Kiến trúc mạng MobileNet-v1 (4,2 triệu tham số) được thể hiện ở Hình 5.14 gồm 27 lớp tích chập, sau lớp tích chập 3×3 đầu tiên là các lớp tích chập trên kênh và tích chập trên điểm xen kẽ nhau, cuối cùng là lớp gộp trung bình và lớp liên kết đầy đủ để phân lớp. Sau mỗi lớp tích chập, mạng MobileNet sử dụng chuẩn hoá loạt và hàm kích hoạt $\text{relu6}(z) = \min(\max(0, z), 6)$ để chặn trên và chặn dưới đầu ra của các lớp.

Đến nay kiến trúc mạng MobileNet đã có thêm các phiên bản MobileNet-v2 (3,5 triệu tham số) và MobileNet-v3 (2,9 triệu tham số). Các mạng MobileNet đều có khả năng chạy trên các thiết bị di động với tốc độ rất cao (xử lý khoảng 100 hình / giây với iPhone 7, khoảng 200 hình / giây với iPhone X).

5.5 Bài toán nhận dạng hình ảnh

Bài toán nhận dạng hình ảnh với mạng học sâu tích chập được giải quyết đầu tiên vào năm 2012 dựa trên kiến trúc mạng AlexNet. Với tập dữ liệu ImageNet, mạng học sâu đã thể hiện khả năng học

vượt trội so với các phương pháp tiếp cận dựa trên đặc trưng được trích chọn của ảnh dựa trên các phương pháp cổ điển.

Dựa trên thành công này, các ứng dụng sử dụng mạng học sâu cho dữ liệu ảnh được áp dụng rộng rãi. Trong đó có hai bài toán chính được đề cập đến như là phép thử đối với các cấu trúc và phương pháp học sâu mới.

5.5.1 Bài toán nhận dạng ảnh

Nhận dạng ảnh hay phân loại hình ảnh là một trong những nhiệm vụ cơ bản nhất trong thị giác máy tính. Các tiến bộ của Học sâu trong bài toán này đã tạo ra một cuộc cách mạng và thúc đẩy những tiến bộ về khoa học và công nghệ trong Trí tuệ nhân tạo những năm gần đây.

Bài toán nhận dạng ảnh có đầu vào là một ảnh và gán cho nó một nhãn từ tập hợp các nhãn được xác định trước trong tập huấn luyện. Cụ thể, với $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ là một ảnh màu với chiều cao H , chiều rộng W và số kênh màu C (ví dụ, $C = 3$ cho ảnh màu RGB), bài toán nhận dạng ảnh yêu cầu dự đoán nhãn $y \in \{1, 2, \dots, K\}$ từ tập hợp K nhãn đã biết trước. Tức là, mô hình cần học một hàm f_θ sao cho:

$$f_\theta : \mathbb{R}^{H \times W \times C} \rightarrow \{1, 2, \dots, K\} \quad (5.18)$$

Trong đó, θ là tập hợp các tham số của mô hình.

Ví dụ 5.9 (Nhận dạng ảnh chó và mèo). Một ví dụ về bài toán nhận dạng ảnh là phân loại ảnh chứa chó hoặc mèo. Trong đó, ảnh là đầu vào có kích thước $H \times W \times 3$ và nhãn là chó hoặc mèo hay $K = 2$.

Ví dụ 5.10 (Nhận dạng chữ số MNIST). Bài toán nhận dạng chữ số từ ảnh là một bài toán cơ bản trong thị giác máy tính. Dữ liệu đầu vào là ảnh xám có kích thước $28 \times 28 \times 1$ và nhãn là một trong các chữ số từ 0 đến 9 hay $K = 10$.

Ví dụ 5.11 (Nhận dạng ảnh ImageNet). ImageNet là một tập dữ liệu lớn chứa hơn 14 triệu ảnh được chia thành 1000 nhãn. Bài toán nhận dạng ảnh ImageNet là một trong những thách thức lớn nhất trong thị giác máy tính. Dữ liệu đầu vào là ảnh màu có kích thước $224 \times 224 \times 3$ và nhãn là một trong 1000 nhãn đã biết trước, $K = 1000$. Trong những thời kì đầu của Học sâu, bài toán nhận dạng ảnh ImageNet đã được giải quyết thành công với kiến trúc AlexNet [2] và sau đó là các kiến trúc khác như VGG, ResNet. Hiện nay tập dữ liệu ImageNet đã trở thành một tiêu chuẩn để đánh giá các mô hình học sâu trong nhận dạng ảnh.

5.5.2 Bài toán nhận dạng vật thể trong ảnh

Bài toán nhận dạng vật thể trong ảnh là một bài toán mở rộng của bài toán nhận dạng ảnh. Trong bài toán này, mục tiêu là xác định vị trí và loại của các vật thể trong ảnh. Mỗi vật thể được xác định bởi một hộp giới hạn và một nhãn. Cụ thể, với ảnh đầu vào $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, bài toán nhận dạng vật thể yêu cầu dự đoán tập hợp các vật thể $\{(y_i, b_i)\}_{i=1}^N$ trong ảnh, trong đó $y_i \in \{1, 2, \dots, K\}$ là nhãn của vật thể thứ i và $b_i = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$ là hộp giới hạn của vật thể thứ i .

Ví dụ 5.12 (Nhận dạng vật thể trong ảnh). Một ví dụ về bài toán nhận dạng vật thể trong ảnh là phân loại và xác định vị trí của các vật thể trong ảnh. Ví dụ, với ảnh đầu vào có kích thước $H \times W \times 3$, mục tiêu là xác định vật thể là chó hoặc mèo và vị trí của chúng trong ảnh.

Ví dụ 5.13 (Nhận dạng vật thể trong ảnh COCO). Tập dữ liệu ảnh COCO (Common Objects in COntext) là một tập dữ liệu lớn chứa hơn 200,000 ảnh với hơn 1.5 triệu nhãn vật thể. Bài toán nhận dạng vật thể trong ảnh COCO yêu cầu xác định vị trí và loại của các vật thể trong ảnh. Dữ liệu đầu vào là ảnh màu có kích thước $224 \times 224 \times 3$ và nhãn là một trong 80 nhãn vật thể đã biết trước.

5.5.3 Bài toán phân vùng ngữ nghĩa ảnh

Bài toán phân vùng ngữ nghĩa ảnh là một bài toán quan trọng trong thị giác máy tính. Mục tiêu của bài toán này là phân loại từng pixel trong ảnh thành các lớp khác nhau. Cụ thể, với ảnh đầu vào $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, bài toán phân vùng ngữ nghĩa ảnh yêu cầu dự đoán một ma trận $\mathbf{Y} \in \{1, 2, \dots, K\}^{H \times W}$, trong đó K là số lớp cần phân loại.

Ví dụ 5.14 (Phân vùng ngữ nghĩa ảnh). Một ví dụ về bài toán phân vùng ngữ nghĩa ảnh là phân loại từng pixel trong ảnh thành các lớp khác nhau. Ví dụ, với ảnh đầu vào có kích thước $H \times W \times 3$, mục tiêu là phân loại từng pixel thành các lớp như đường, cây, nhà, xe, người, v.v.

Ví dụ 5.15 (Phân vùng ngữ nghĩa ảnh Cityscapes). Cityscapes [13] là một tập dữ liệu lớn chứa hơn 5000 ảnh với hơn 20 nhãn khác nhau. Bài toán phân vùng ngữ nghĩa ảnh Cityscapes yêu cầu phân loại từng pixel trong ảnh thành các lớp khác nhau như đường, cây, nhà, xe, người, v.v. Dữ liệu đầu vào là ảnh màu có kích thước $1024 \times 2048 \times 3$ và nhãn là một trong 20 nhãn đã biết trước.

Ví dụ 5.16 (Bài toán nhận dạng khối u trong ảnh CT). Bộ dữ liệu LIDC (Lung Image Database Consortium) chứa hơn 1000 ảnh CT với nhãn khối u. Bài toán nhận dạng khối u trong ảnh CT yêu cầu phân loại từng pixel trong ảnh thành hai lớp: khối u và không phải khối u. Dữ liệu đầu vào là ảnh CT xám có kích thước $512 \times 512 \times 1$ và nhãn là một trong hai lớp: nốt phổi / bình thường.

5.6 Một số chiến lược huấn luyện

Mạng nơ-ron tích chập là một trong những mô hình quan trọng nhất trong học sâu, đặc biệt trong bài toán xử lý ảnh và thị giác máy tính. CNN được huấn luyện thông qua việc tối ưu hàm lỗi trên

tập dữ liệu huấn luyện để học các đặc trưng của dữ liệu. Tuy nhiên, quá trình huấn luyện CNN gặp nhiều thách thức như *quá khớp*, *độ dốc biến mất* và *thời gian huấn luyện dài*. Để khắc phục những vấn đề này, các chiến lược huấn luyện hiệu quả như học chuyển đổi, thuật toán tối ưu và điều chỉnh hệ số học đã được đề xuất và áp dụng rộng rãi.

5.6.1 Học chuyển đổi

Học chuyển đổi là phương pháp sử dụng trọng số hoặc một phần trọng số từ một mô hình đã được huấn luyện trên một bộ dữ liệu lớn (ví dụ: ImageNet) để khởi tạo mô hình mới trên một bài toán khác. Với các bài toán có dữ liệu huấn luyện nhỏ, học chuyển đổi giúp cải thiện hiệu suất và giảm thời gian huấn luyện đáng kể. Hàm loss được tối ưu với khởi tạo trọng số dựa trên (một phần) trọng số Φ của một mô hình có sẵn:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f_{\theta}(\mathbf{x}_i; \Phi, W)),$$

trong đó, Φ là trọng số của mô hình đã được huấn luyện và W là trọng số mới cần tối ưu. Tham số của mô hình cần huấn luyện sẽ là $\theta = [\Phi, W]$. Trong các bài toán lý thuyết và ứng dụng thực tế, chúng ta hay gặp một số phương pháp học chuyển đổi cơ bản như:

- **Tinh chỉnh toàn bộ mô hình:** Tất cả các trọng số trong mô hình gốc đều được điều chỉnh để phù hợp với tập dữ liệu mới.
- **Sử dụng mạng ban đầu như bộ trích chọn đặc trưng:** Giữ nguyên các tầng tích chập đã học và chỉ huấn luyện lại các tầng kết nối đầy đủ trên dữ liệu mới.
- **Đóng băng các tầng của mạng ban đầu:** Đóng băng một số tầng để giữ nguyên trọng số và chỉ cập nhật các tầng trên cùng.

5.6.2 Điều chỉnh hệ số học

Trong cách tiếp cận cơ bản, CNN được huấn luyện thông qua việc tối ưu hàm lỗi bằng thuật toán xuống đồi bằng đạo hàm GD. Công thức tổng quát để cập nhật lại trọng số W theo hàm lỗi $\mathcal{L}$ được cho bởi công thức sau:

$$W^{(t+1)} = W^{(t)} - \eta \nabla_W \mathcal{L}, \quad (5.19)$$

trong đó, η là hệ số học và $\nabla_W \mathcal{L}$ là đạo hàm của hàm lỗi. Như được trình bày ở mục 3.3, sau khi tính toán được đạo hàm của tham số W theo hàm lỗi, việc lựa chọn thuật toán tối ưu như SGD hoặc Adam cũng ảnh hưởng đến hiệu suất của mô hình.

Bên cạnh đó, hệ số học (η) quyết định tốc độ cập nhật trọng số. Hệ số học quá lớn dẫn đến mất ổn định còn quá nhỏ sẽ làm chậm hội tụ. Một số cách điều chỉnh tốc độ học phổ biến như:

- **Giảm theo bước:** Hệ số học giảm dần theo các bước:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/k \rfloor}. \quad (5.20)$$

- **Giảm khi không cải thiện:** Giảm η khi độ chính xác trên tập kiểm thử không cải thiện.
- **Khởi động hệ số học:** Tăng dần η trong các epoch đầu tiên.
- **Lặp theo chu kỳ:** Hệ số học dao động theo chu kỳ, giúp thoát khỏi cực tiểu cục bộ.

5.6.3 Các chiến lược huấn luyện nâng cao khác

Bên cạnh hai phương pháp chính là học chuyển đổi và điều chỉnh hệ số học, một số chiến lược huấn luyện khác cũng được sử dụng để cải thiện hiệu suất của mô hình CNN, bao gồm:

- **Điều chỉnh giảm:** Sử dụng lớp triệt tiêu ngẫu nhiên và hàm điều chỉnh $L2 = \frac{1}{2}\|W\|_2^2$ để tránh quá khớp. Trong đó, $\|W\|_2^2$ là kí hiệu cho tổng bình phương của các trọng số trong mô hình có tham số W .
- **Dừng sớm:** Dừng huấn luyện khi mô hình không cải thiện trên tập kiểm thử.
- **Giới hạn đạo hàm:** Giới hạn độ lớn của đạo hàm để xử lý vấn đề bùng nổ đạo hàm.
- **Chuẩn hoá theo lô:** Giúp chuẩn hóa đầu ra giữa các tầng để tăng tốc độ huấn luyện.

Các chiến lược huấn luyện như học chuyển đổi, thuật toán tối ưu và điều chỉnh hệ số học đóng vai trò quan trọng trong việc nâng cao hiệu suất của mạng CNN. Việc lựa chọn và kết hợp các chiến lược phù hợp với từng bài toán cụ thể sẽ giúp quá trình huấn luyện trở nên hiệu quả và ổn định hơn.

5.7 Tình huống áp dụng: Phân loại ảnh CIFAR10

Bộ dữ liệu CIFAR10 bao gồm 60.000 hình ảnh màu kích thước 32x32 được chia thành 10 lớp, bao gồm: máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu thủy và xe tải. Mỗi lớp có 6.000 hình ảnh, trong đó 5.000 hình ảnh được sử dụng để huấn luyện và 1.000 hình ảnh được sử dụng để kiểm tra. Tương tự với tình huống áp dụng ở Chương 3, chúng ta cũng dựa trên thư viện PyTorch để khai báo và huấn luyện mô hình. Tuy nhiên, thay vì chỉ sử dụng các mạng nơ-ron kết nối đầy đủ, chúng ta sẽ sử dụng thêm các lớp tích chập và lấy mẫu trên hai chiều để học các đặc trưng từ ảnh. Chúng ta cũng sử dụng thêm thư viện `torchvision`, là thư viện cung cấp các hàm hỗ trợ việc tính toán trên ảnh. Các bước triển khai chính bao gồm:

- Chuẩn bị dữ liệu: Dữ liệu CIFAR10 được cung cấp sẵn trong thư viện `torchvision`. Đối với dữ liệu ảnh, chúng ta có thể khai báo thêm bước chuẩn hoá - biến đổi trước khi đưa ảnh vào tính toán với mạng học sâu.
- Khai báo mô hình học sâu tích chập: Để nhận dạng ảnh từ bộ dữ liệu CIFAR10, chúng ta sẽ sử dụng một mô hình học sâu tích chập đơn giản gồm 2 lớp tích chập chính và 2 lớp lấy mẫu. Đặc trưng đầu ra sẽ được học bằng một lớp nơ-ron kết nối đầy đủ có 128 chiều, sau đó sẽ dùng để học ra xác suất tương ứng cho 10 lớp nhãn mục tiêu. Bảng 5.1 mô tả các lớp của mô hình và các tham số được sử dụng tương ứng.
- Huấn luyện mô hình: Chúng ta sẽ sử dụng hàm lỗi Entropy chéo và bộ tối ưu hoá Adam. Với mỗi lượt huấn luyện, chúng ta sẽ tính toán hàm lỗi và cập nhật các tham số của mô hình theo các tập mẫu được lấy ngẫu nhiên. Thư viện `PyTorch` cung cấp các hàm hỗ trợ việc này. Đối với dữ liệu CIFAR10, chúng ta sẽ huấn luyện trong khoảng 10 lượt. Vì dữ liệu CIFAR10 có kích thước lớn, chúng ta sẽ sử dụng thêm bộ tính toán song song (GPU) để tăng tốc độ huấn luyện.
- Đánh giá mô hình: Chúng ta sẽ sử dụng tập dữ liệu kiểm tra để đánh giá độ chính xác của mô hình. Với thiết lập cơ bản như trên, mô hình có thể đạt độ chính xác khoảng 70-72% trên tập kiểm thử.

Người học có thể tham khảo mã nguồn tại <https://gist.github.com/cuongtv312/79a0992dcabaa17df633fc08aa949314>

5.8 Tổng kết chương

Chương này trình bày nguyên lý tích chập và cách xây dựng mạng nơ-ron tích chập (CNN) cho bài toán nhận dạng ảnh. So với mạng

Bảng 5.1: Cấu trúc mô hình học sâu tích chập cho bài toán nhận dạng ảnh màu CIFAR10

Tên lớp	Loại lớp	Tham số	Kích thước đầu ra
conv1	Lớp tích chập 2D	Số kênh vào: 3, Số kênh ra: 32, Kích thước: 3x3, Bước nhảy: 1, Kích hoạt: ReLU	(32, 32, 32)
pool1	Lớp lấy mẫu cực đại 2D	Kích thước: 2x2, Bước nhảy: 2	(32, 16, 16)
conv2	Lớp tích chập 2D	Số kênh vào: 32, Số kênh ra: 64, Kích thước kernel: 3x3, Bước nhảy: 1, Kích hoạt: ReLU	(64, 16, 16)
pool2	Lớp lấy mẫu cực đại 2D	Kích thước: 2x2, Bước nhảy: 2	(64, 8, 8)
flatten	Làm phẳng	Chuyển đổi tensor (64, 8, 8) thành véc-tơ	(4096)
fc1	Lớp kết nối đầy đủ	Số nút vào: 4096, Số nút ra: 128. Kích hoạt: ReLU	(128)
fc2	Lớp kết nối đầy đủ	Số nút vào: 128, Số nút ra: 10. Kích hoạt: Không (đầu ra logits)	(10)

neuron truyền thông, CNN giúp giảm số lượng tham số và tận dụng cấu trúc không gian của ảnh.

Các thành phần cơ bản của CNN gồm lớp tích chập, lớp gộp và lớp kết nối đầy đủ. Từ đó, các kiến trúc CNN nổi bật như VGG, ResNet, MobileNet được xây dựng và chứng minh hiệu quả trên tập ImageNet.

CNN đóng vai trò quan trọng trong sự phát triển của thị giác máy tính và học sâu, nhờ vào khả năng tự động học đặc trưng mà không cần thiết kế thủ công.

Các lớp nâng cao như chuẩn hoá loạt và kết nối phần dư đã góp phần quan trọng trong việc mở rộng kiến trúc mạng sâu. Chiến lược huấn luyện như học chuyển đổi và tối ưu hoá hệ số học đã được áp dụng hiệu quả cho CNN và lan rộng sang các mô hình học sâu khác.

Một ưu điểm lớn của CNN là khả năng tính toán hiệu quả và dễ song song hoá. Các toán tử tích chập và gộp được cài đặt hiệu quả trên CPU, GPU và cả vi xử lý di động, góp phần nâng cao hiệu năng mô hình.

Với khả năng biểu diễn mạnh mẽ, học đặc trưng tự động và khả năng đạt độ chính xác cao, CNN đã giải quyết nhiều vấn đề của mạng MLP và trở thành trụ cột của học sâu trong nhận dạng hình ảnh hiện nay.

Bài tập

- Giả sử ảnh đầu vào có kích thước $6 \times 6 \times 3$ (chiều cao $\times$ chiều rộng $\times$ số kênh), lớp tích chập có $C_{out} = 2$ bộ lọc với kích thước $3 \times 3 \times 3$. Tính kích thước bản đồ đặc trưng khi sử dụng bước nhảy $s = 1$ và không có phần bù.
- Cho ảnh đầu vào có kích thước $28 \times 28 \times 1$ (ảnh xám), sử dụng

một bộ lọc kích thước $5 \times 5 \times 1$ với bước nhảy $s = 2$ và không dùng phân bù. Tính kích thước bản đồ đặc trưng ở đầu ra ($H_{out} \times W_{out} \times C_{out}$). Giả sử $C_{out} = 10$.

3. Cho một vùng tiếp nhận kích thước 3×3 của ảnh đầu vào với $C_{in} = 1$:

$$o_i = \begin{bmatrix} 1 & 2 & 0 \\ -1 & 3 & 1 \\ 0 & 2 & -2 \end{bmatrix},$$

và bộ lọc (kernel) có kích thước $3 \times 3 \times 1$ với các trọng số:

$$w_{i+1} = \begin{bmatrix} 0 & 1 & -1 \\ 2 & 0 & 1 \\ -1 & 2 & 0 \end{bmatrix}.$$

Giả sử hàm kích hoạt f là hàm ReLU, tính giá trị của đầu ra $o_{i+1}^{h,w}$ theo công thức:

$$o_{i+1}^{h,w} = f \left(\sum_{m=1}^3 \sum_{n=1}^3 w_{i+1}^{m,n} o_i^{h+m-1, w+n-1} \right).$$

4. Mô tả kiến trúc cơ bản của một mạng nơ-ron tích chập. Liệt kê các thành phần chính và chức năng của từng thành phần.
5. Giải thích ý nghĩa của cơ chế chia sẻ trọng số trong lớp tích chập. Tại sao việc chia sẻ trọng số lại giúp giảm số lượng tham số và cải thiện khả năng tổng quát của mô hình?
6. Tại sao sau khi thực hiện tích chập (một phép tính tuyến tính) lại cần sử dụng các hàm kích hoạt như sigmoid, tanh hay ReLU? Trong đó, hãy phân tích ưu điểm của hàm ReLU và các biến thể của nó so với các hàm kích hoạt khác.

7. [*Tìm hiểu*] Tính công thức lan truyền ngược của lớp chuẩn hóa loạt. Công thức tìm được thay đổi thế nào nếu chúng ta có kích thước lấy mẫu là 1.
8. [*Tìm hiểu*] Trong mạng dư thừa ResNet, khối dư thừa (Residual Block) hoạt động như thế nào? Tại sao nó lại giúp huấn luyện các mạng rất sâu?
9. [*Lập trình*] Dựa theo tình huống áp dụng đã trình bày, đề xuất một số các kỹ thuật tăng cường dữ liệu như: xoay ảnh, lật ảnh ngang, dịch chuyển ảnh, thay đổi độ sáng để huấn luyện mạng nơ-ron tích chập cho dữ liệu CIFAR-10. So sánh với kết quả mô hình ban đầu.

Tài liệu tham khảo

- [1] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P., *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] Krizhevsky, A., Sutskever, I., and Hinton, G. E., *ImageNet classification with deep convolutional neural networks*, Advances in Neural Information Processing Systems, vol. 25, pp. 1097–1105, 2012.
- [3] Simonyan, K., and Zisserman, A., *Very deep convolutional networks for large-scale image recognition*, arXiv preprint arXiv:1409.1556, 2014.
- [4] He, K., Zhang, X., Ren, S., and Sun, J., *Deep residual learning for image recognition*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778, 2016.
- [5] Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., et al., *Going deeper with convolutions*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1–9, 2015.

- [6] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H., *MobileNets: Efficient convolutional neural networks for mobile vision applications*, arXiv preprint arXiv:1704.04861, 2017.
- [7] LeCun, Y., Bengio, Y., and Hinton, G., *Deep learning*, Nature, vol. 521, no. 7553, pp. 436–444, 2015.
- [8] Tan, M. and Le, Q., 2019, May. Efficientnet: Rethinking model scaling for convolutional neural networks. In International conference on machine learning (pp. 6105-6114). PMLR.
- [9] Jähne, Bernd. Digital image processing. Springer Science & Business Media, 2005.
- [10] Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. "Imagenet: A large-scale hierarchical image database." In 2009 IEEE conference on computer vision and pattern recognition, pp. 248-255. Ieee, 2009.
- [11] Zhang, Haoran, Tiansheng Chen, Yang Liu, Yuxi Zhang, and Jiong Liu. "Automatic seismic facies interpretation using supervised deep learning." Geophysics 86, no. 1 (2021): IM15-IM33.
- [12] Redmon, Joseph, and Ali Farhadi. "Yolov3: An incremental improvement." arXiv preprint arXiv:1804.02767 (2018).
- [13] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The Cityscapes Dataset for Semantic Urban Scene Understanding," in Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016

Chương 6

Mạng nơ-ron cho dữ liệu dạng chuỗi

Dữ liệu chuỗi như văn bản, âm thanh và chuỗi thời gian đòi hỏi mô hình có khả năng ghi nhớ ngữ cảnh trong quá khứ. Mạng nơ-ron hồi quy (Recurrent Neural Networks - RNN) là cách tiếp cận cơ bản để xử lý loại dữ liệu này thông qua trạng thái ẩn h_t tại mỗi thời điểm t , được cập nhật theo công thức:

$$h_t = f_h(W_h h_{t-1} + W_x x_t) \quad (6.1)$$

Trạng thái ẩn h_t đóng vai trò là bộ nhớ động, tích lũy thông tin từ các bước thời gian trước đó, giúp mô hình có khả năng xử lý ngữ cảnh và quan hệ phụ thuộc trong chuỗi.

Chương 6 trình bày các kiến trúc mạng nơ-ron cho dữ liệu dạng chuỗi, bao gồm RNN, LSTM, GRU và mô hình mạng chuyển đổi (Transformer). Đây là các kiến trúc quan trọng đã đạt hiệu quả cao trong các bài toán như xử lý ngôn ngữ tự nhiên, dịch máy, nhận

dạng giọng nói và phân tích chuỗi thời gian. Những mô hình này thể hiện bước tiến quan trọng của học sâu khi mở rộng năng lực biểu diễn sang không gian dữ liệu có quan hệ theo thời gian và là nền tảng cho nhiều ứng dụng hiện đại trong AI.

6.1 Mô hình hoá dữ liệu dạng chuỗi

Một trong những ví dụ cơ bản nhất về dữ liệu dạng chuỗi chính là dữ liệu văn bản, một đối tượng nghiên cứu quan trọng trong lĩnh vực học máy. Với sự phát triển của Internet và kỷ nguyên 4.0, hầu hết các bài toán xử lý dữ liệu văn bản có thể thu thập thông tin từ nhiều nguồn khác nhau như sách, báo chí, trạng thái trên mạng xã hội, diễn đàn, v.v. Các nguồn này thường cung cấp một lượng lớn dữ liệu thô nhưng lại có rất ít dữ liệu gán nhãn. Ngoài ra, dữ liệu văn bản thường cần được xử lý để loại bỏ các ký tự nhiễu, nhằm thu được dữ liệu sạch và phù hợp với yêu cầu của bài toán.

Khác với dữ liệu dạng bảng hay hình ảnh, dữ liệu văn bản yêu cầu một bước mã hóa bắt buộc để chuyển đổi từ dạng ký tự sang biểu diễn số. Nhờ đó, các mô hình học máy mới có thể tiếp nhận và học từ dữ liệu văn bản một cách hiệu quả. Một trong những nguồn dữ liệu văn bản được sử dụng phổ biến hiện nay là các văn bản được trích xuất từ Wikipedia bao gồm các lĩnh vực khác nhau và các ngôn ngữ khác nhau.

Ví dụ 6.1 (Dữ liệu thu thập thông tin bất động sản). Bảng 6.1 minh họa về dạng dữ liệu văn bản được thu thập từ mạng internet. Đặc điểm của các dữ liệu văn bản là cấu trúc không thống nhất, xuất hiện nhiều kí tự, kí hiệu đa dạng. Trong ví dụ trên, việc phân chia ý được dùng xen lẫn dấu cộng hoặc dấu câu.

Ví dụ 6.2 (Dữ liệu âm thanh). Dữ liệu âm thanh là một chuỗi các tín hiệu một chiều của âm thanh với tần số lấy mẫu nhất định. Tùy vào định dạng của tệp âm thanh mà ta có tần số lấy mẫu khác

Bảng 6.1: Ví dụ về dữ liệu dạng văn bản được thu thập trên mạng internet về các thông tin bất động sản.

Mô tả
DT 70 m2 (tầng 1) + tầng hai 35 m2 với 1 phòng 1 phòng ngủ, mặt tiền 4 m Khu đô thị Văn Khê, Hà Nội, khu đông dân cư, nhiều văn phòng
Chạy xe 2p tới đường Thanh Niên, Hồ Tây. Trước nhà 2 ô tô tránh, mặt tiền 5m cực đẹp, ô tô vô nhà. Nhà xây mới, thiết kế hiện đại, mỗi tầng đều có ban công. DT 45m ² , 6 tầng, 5m mặt tiền
Nhà cách mặt phố chỉ khoảng 100m. Diện tích là 50m ² rộng rãi và thoáng mát, mặt tiền rộng 5,6m có thể kinh doanh tạp hóa. Rất thuận tiện di chuyển các tuyến đường chính

nhau, nhưng phổ biến nhất là $f = 44.1kHz$ với mỗi mẫu là 16 bit dữ liệu. Đầu ra của quá trình này là một chuỗi theo thời gian:

$$s = (s_0, s_1, \dots, s_t, \dots, s_{N-1}) \quad (6.2)$$

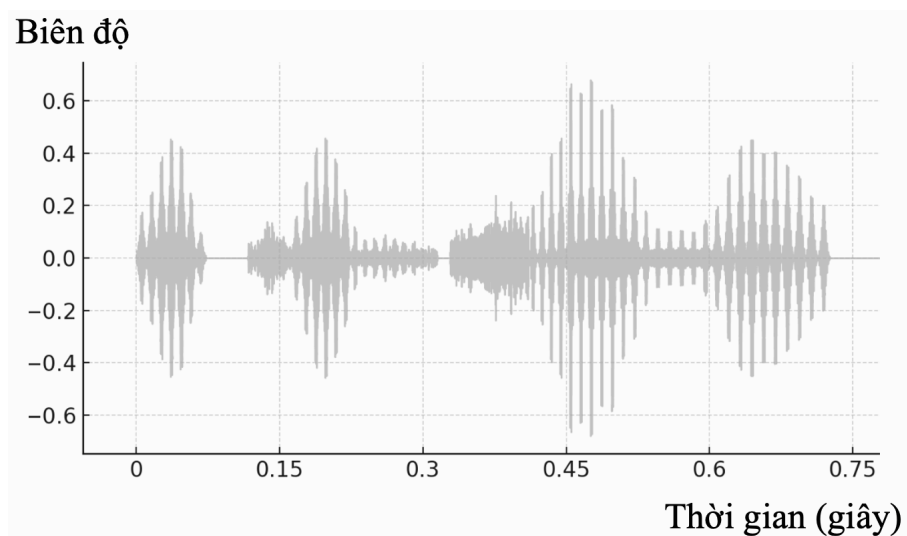
trong đó, t là chỉ số thời gian, N là độ dài của dãy. Dữ liệu âm thanh được dùng chủ yếu trong các bài toán liên quan đến giọng nói con người và âm nhạc. Ví dụ, chúng ta có thể gặp trong bài toán nhận diện giọng nói hoặc bài toán xây dựng văn bản từ tiếng nói.

Định nghĩa 6.3 (Chuỗi thời gian). Chuỗi thời gian là một ánh xạ từ tập chỉ số $\{1, 2, \dots\}$ đến không gian đặc trưng $\mathcal{X}$.

$$\mathbf{x} = (x_1, x_2, \dots, x_t, \dots, x_T), x_t \in \mathcal{X} \quad (6.3)$$

trong đó T là độ dài của chuỗi thời gian, x_t là giá trị của chuỗi tại thời điểm t .

Bên cạnh dữ liệu văn bản và dữ liệu âm thanh, có rất nhiều loại dữ liệu dạng chuỗi thời gian khác cũng được ứng dụng rộng rãi



Hình 6.1: Ví dụ về mẫu tín hiệu âm thanh

trong các bài toán học máy. Đặc điểm chung của các loại dữ liệu này là chúng đều có thứ tự và phụ thuộc vào các mốc thời gian lấy mẫu. Một số ví dụ trong thực tế có dữ liệu chuỗi bao gồm:

- **Dữ liệu tài chính:** Các chuỗi giá cổ phiếu, giao dịch và các chỉ số kinh tế theo thời gian thường được phân tích để dự đoán xu hướng thị trường hoặc phát hiện các bất thường. Tùy vào thiết lập của bài toán thực tế mà các khung thời gian có thể là hàng ngày, hàng giờ hoặc thậm chí hàng phút.
- **Dữ liệu cảm biến và IoT:** Dữ liệu thu thập từ các cảm biến trong nhà máy, xe cộ hoặc các thiết bị đeo thông minh, được sử dụng để dự đoán sự cố, tối ưu hóa quy trình sản xuất hoặc theo dõi sức khỏe.
- **Dữ liệu thời tiết và khí hậu:** Chuỗi dữ liệu về nhiệt độ, độ ẩm, lượng mưa và các yếu tố khí tượng khác được dùng để dự báo thời tiết, phân tích biến đổi khí hậu hoặc tối ưu hóa quản lý tài nguyên thiên nhiên.

6.2 Mạng nơ-ron hồi quy

Điểm yếu cố hữu của mạng nơ-ron MLP nói chung và mạng nơ-ron CNN nói riêng là phải hoạt động trên đầu vào có kích thước cố định. Đặc tính này khiến chúng gặp khó khăn khi làm việc với đầu vào có kích thước biến động như chuỗi thời gian. Ví dụ như khi xử lý một đoạn văn bản, một đoạn sóng âm thanh, các mô hình này cần bước tiền xử lý để chuyển hoá đầu vào về một kích thước cố định cho trước. Khi đó, thứ tự giữa các từ cũng như ngữ cảnh của đoạn văn bị mất rất nhiều thông tin.

Khi làm việc với chuỗi thời gian, có một giả thuyết thường xuyên được các mô hình sử dụng là “các dữ liệu quan sát được x_t là các biến ngẫu nhiên phụ thuộc vào trạng thái ẩn h_t nhất định”. Ví dụ, khi viết một câu văn, các từ trong câu là các biến ngẫu nhiên quan sát được, nhưng các từ trong câu lại phụ thuộc vào trạng thái ẩn là ngữ cảnh của câu văn, trạng thái cảm xúc của người viết.

Mạng nơ-ron hồi quy RNN (Recurrent Neural Network), được thiết kế để làm việc với dữ liệu có kích thước biến động. Mạng RNN cố gắng biểu diễn được (i) ngữ cảnh (trạng thái ẩn) của dữ liệu đầu vào và (ii) quan hệ giữa các từ, các đoạn trong đó, từ đó đưa ra các dự đoán chính xác hơn mạng nơ-ron thông thường. Cụ thể, trong pha suy luận, mạng RNN có phần bộ nhớ mô tả nén thông tin ngữ cảnh của chuỗi quá khứ, kết hợp với dữ liệu hiện tại để cập nhật bộ nhớ cũng như đưa ra dự đoán. Ta gọi phần bộ nhớ này là *trạng thái ẩn*. Cụ thể, thuật toán lan truyền tiến trong mạng RNN được mô tả trong Thuật toán 6.1 và Hình 6.2.

Trong đó, x_t là dữ liệu đầu vào tại bước $t = 1, 2, \dots, T$, h_t là trạng thái ẩn tại bước t được tính từ x_t và trạng thái ẩn ở bước trước h_{t-1} và y_t là đầu ra của mạng tại bước t được tính từ trạng thái ẩn h_t . Các hàm kích hoạt f_h, f_y được chọn từ các hàm kích hoạt như sigmoid, tanh, relu.

Thuật toán 6.1 Lan truyền tới trong mạng RNN

-
- ```

1: procedure RNNFORWARD($\{x_t\}_{t=1}^T, W_x, W_h, W_y, f_h, f_y$)
2: Khởi tạo trạng thái ẩn h_0
3: for $t = 1$ to T do
4: Tính trạng thái ẩn:

```

$$h_t \leftarrow f_h(W_h h_{t-1} + W_x x_t) \quad (\text{Eq. 6.4}) \quad (6.4)$$

- ```

5:     Tính đầu ra:

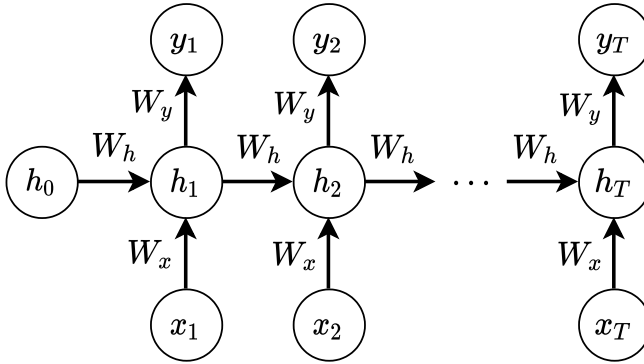
```

$$y_t \leftarrow f_y(W_y h_t) \quad (\text{Eq. 6.5}) \quad (6.5)$$

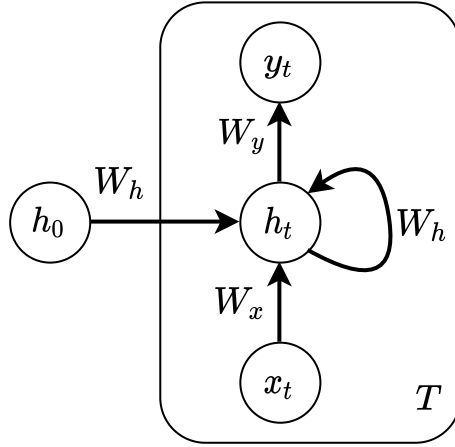
- ```

6: end for
7: return $\{y_t\}_{t=1}^T$
8: end procedure

```
- 

**Hình 6.2:** Tính toán của RNN trên chuỗi dữ liệu.

Hình 6.2 cho thấy  $h_t$  hoạt động giống như một bộ nhớ lưu trữ trạng thái của chuỗi dữ liệu đầu vào đến thời điểm  $t$ . Trạng thái  $h_t$  mô tả mối quan hệ giữa các đầu vào  $x_u, u = 1, 2, \dots, t$ . Từ trạng thái  $h_t$  ta đưa ra dự đoán  $y_t$  như một lớp nơ-ron bình thường. Hàm lỗi của RNN tính trên các đầu ra  $y_t$ . Các thuật toán lan truyền ngược vẫn hoạt động như bình thường với lưu ý cộng dồn đạo hàm của hàm lỗi đối với  $W_x, W_h$  và  $W_y$  do chúng xuất hiện nhiều lần



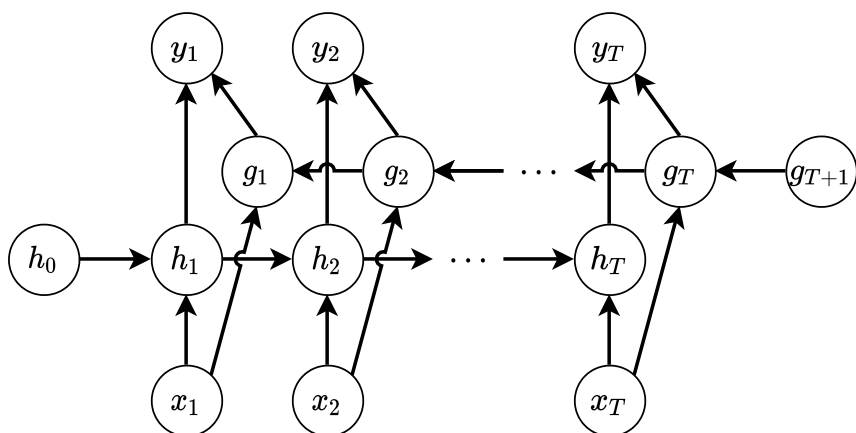
**Hình 6.3:** Biểu diễn rút gọn của mạng RNN.

như trong hình. Một cách hiểu khác là mạng RNN đã chia sẻ trọng số trong việc tính toán đầu ra tại các bước khác nhau nên số lượng đầu ra không bị hạn chế. Cách tiếp cận này rất giống cách mạng CNN chia sẻ trọng số tại lớp tích chập. Về mặt thiết kế tổng quát, việc chia sẻ trọng số giúp cho mạng RNN học được các đặc trưng chung của dữ liệu đầu vào mà không cần phải sử dụng quá nhiều tham số.

### 6.2.1 Mạng nơ-ron hồi quy sâu

Dựa trên các công thức cơ sở (6.4) và (6.5), có bốn cách để làm mạng nơ-ron RNN sâu hơn, trích rút được các đặc trưng trừu tượng hơn:

- Xếp chồng các trạng thái ẩn lên trên nhau, đầu ra của trạng thái ẩn này là đầu vào của trạng thái ẩn kế tiếp như mạng nơ-ron MLP. Đây là cách làm thông dụng nhất.
- Tăng số lớp mạng giữa kết nối các trạng thái ẩn. Ví dụ mạng hai lớp  $h_t = f_2(W_h^2 f_1(W_h^1 h_{t-1} + W_x x_t))$ .



Hình 6.4: Mạng nơ-ron RNN hai chiều.

- Tăng số lớp mạng giữa kết nối đầu vào và trạng thái ẩn
- Tăng số lớp mạng giữa kết nối trạng thái ẩn và đầu ra.

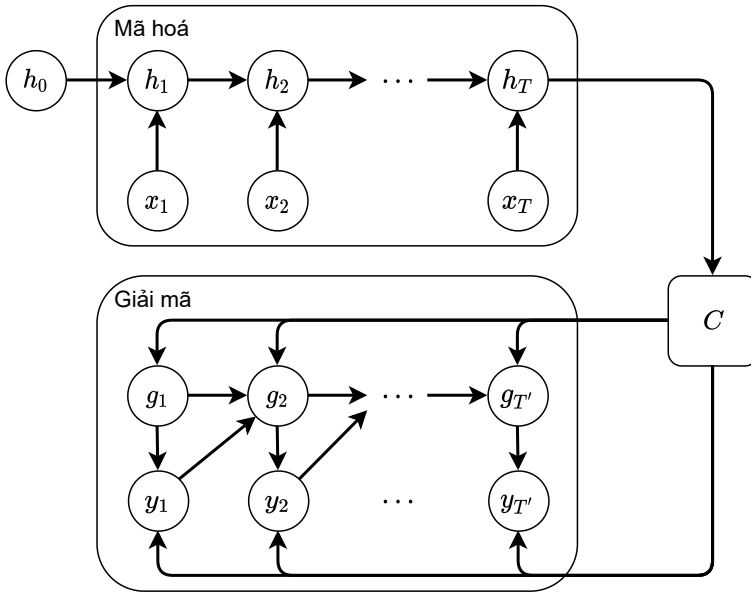
Nói chung, mạng RNN sâu có hiệu suất dự đoán cao hơn các mạng RNN nông.

### 6.2.2 Mạng nơ-ron hồi quy hai chiều

Ngữ cảnh của đầu vào tại bước  $t$  nhất định không những phụ thuộc vào đầu vào ở các bước trước (trong quá khứ) mà còn phụ thuộc vào đầu vào ở các bước sau (trong tương lai). Dự đoán dựa trên cả thông tin trong quá khứ và trong tương lai sẽ chính xác hơn dự đoán dựa trên thông tin từ một phía. Mạng nơ-ron hồi quy hai chiều bao gồm hai mạng RNN đi ngược chiều nhau, dự đoán  $y_t$  được tính từ trạng thái quá khứ  $h_t$  và trạng thái tương lai  $g_t$  (Hình 6.4).

### 6.2.3 Kiến trúc Mã hoá – Giải mã

Kiến trúc này dùng trong bài toán *biến đổi chuỗi sang chuỗi*. Kiến trúc gồm hai RNN hoạt động nối tiếp, trong đó: (i) RNN thứ nhất



**Hình 6.5: Kiến trúc Mã hoá – Giải mã.**

sử dụng chuỗi đầu vào để tính ra  $C$  là ngữ cảnh đầu vào từ trạng thái ẩn cuối cùng  $h_T$  và (ii) RNN thứ hai sử dụng ngữ cảnh  $C$  để sinh ra chuỗi đầu ra mong muốn (Hình 6.5). Điểm đặc biệt của kiến trúc Mã hoá – Giải mã là chuỗi đầu vào và chuỗi đầu ra không cần có cùng độ dài. Do đó kiến trúc này thường được dùng trong các bài toán dịch máy giữa hai ngôn ngữ, bài toán nhận dạng và tổng hợp tiếng nói.

#### 6.2.4 Những vấn đề trong mạng RNN

Trong cách tiếp cận cơ bản, mạng RNN được thiết kế để làm việc với các dữ liệu có độ dài khác nhau, chẳng hạn như văn bản hay tín hiệu âm thanh mà không cần phải tiền xử lý để đưa về kích thước cố định. Việc chia sẻ các trọng số  $W_h, W_x, W_y$  ở mỗi bước thời gian không chỉ giúp giảm số lượng tham số cần học mà còn giúp mô hình học được các đặc trưng chung của dữ liệu chuỗi, tương tự như việc kiến trúc CNN chia sẻ trọng số trong lớp tích chập. Nhờ vào

biến diễn trạng thái ẩn ( $h_t$ ), RNN có khả năng lưu trữ thông tin về các đầu vào trước đó, giúp mô hình hiểu được ngữ cảnh và mối quan hệ giữa các phần tử trong chuỗi.

Tuy nhiên, khi huấn luyện một mạng RNN trên dữ liệu dạng chuỗi trong thực tế, một số vấn đề có thể nảy sinh dẫn đến sự mất ổn định của mô hình, bao gồm:

- *Vấn đề triệt tiêu đạo hàm*: Khi chuỗi dữ liệu quá dài, quá trình lan truyền ngược trong RNN có thể gặp phải hiện tượng triệt tiêu đạo hàm, dẫn đến việc huấn luyện khó khăn và hiệu suất dự đoán giảm.
- *Khả năng ghi nhớ thông tin dài hạn hạn chế*: Do tính chất của mạng RNN cơ bản, việc ghi nhớ các thông tin từ quá khứ xa thường gặp khó khăn, dẫn đến việc mất mát thông tin ngữ cảnh quan trọng trong chuỗi.
- *Tính chất tuần tự của tính toán*: Quá trình tính toán của RNN phụ thuộc vào trạng thái của bước trước đó, điều này làm cho quá trình huấn luyện không thể được thực hiện song song hoàn toàn, gây ra thời gian tính toán lâu hơn so với các mô hình khác có thể xử lý song song.

Điều này thúc đẩy các nghiên cứu tiếp theo để có thể mô hình hoá thông tin của chuỗi dữ liệu một cách hiệu quả hơn. Một trong những mô hình được phát triển để giải quyết các vấn đề trên là mạng LSTM.

### 6.3 Bộ nhớ ngắn hạn dài

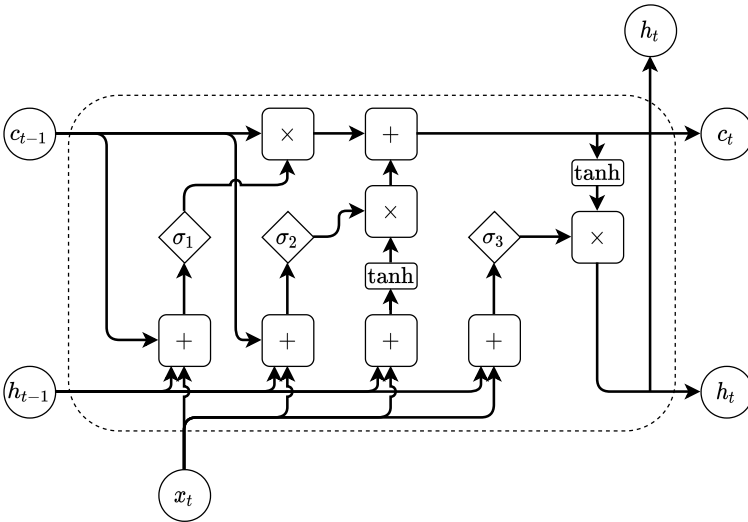
Mạng Bộ nhớ ngắn hạn dài, được viết tắt là LSTM (Long Short-Term Memory), cải tiến cách tính toán trạng thái ẩn trong mạng cơ sở RNN để mô tả được quan hệ ngắn hạn và quan hệ dài hạn



giữa các phần tử trong chuỗi đầu vào. Mạng RNN như mô tả ở trên rất khó học các mối quan hệ dài quá 5 - 10 bước do hiện tượng triệt tiêu đạo hàm. Trong khi đó, mạng LSTM có thể học mối quan hệ cách xa hơn nhờ các cải tiến sau

- *Trạng thái ô* được tính toán tại mỗi bước sao cho đạo hàm có thêm một đường lan truyền ngược ngắn hơn, trực tiếp hơn từ hàm lỗi.
- *Cổng nhân* có tác dụng tránh nhiễu, biến thiên ngẫu nhiên ảnh hưởng đến tính toán trạng thái ẩn và trạng thái ô.

Hình 6.6 mô tả tính toán trạng thái ẩn và trạng thái ô trong một bước của mạng LSTM có sử dụng các cổng kích hoạt sigmoid ( $\sigma$ ) và cổng kích hoạt tanh (3.1). Cụ thể, cổng  $\sigma_1$  (cổng quên) tính



**Hình 6.6:** Một bước tính toán trong mạng LSTM.

toán trọng số của trạng thái ô phía trước  $c_{t-1}$ . Cổng này “linh hoạt” lựa chọn có cần trạng thái ô phía trước không. Cổng  $\sigma_2$  (cổng vào) tính toán tác động (trọng số) của trạng thái ẩn  $h_{t-1}$  và đầu vào

$x_t$  trong việc tính trạng thái ô hiện tại  $c_t$ . Cổng  $\sigma_3$  (cổng ra) tính trọng số cho việc tính toán trạng thái ẩn hiện tại  $h_t$  từ trạng thái ô  $c_t$ . Kết quả tính toán thông tin đầu ra ở mỗi cổng và trạng thái ẩn mô tả bởi Thuật toán 6.2.

---

**Thuật toán 6.2** Lan truyền tới trong mạng LSTM

---

- 1: **procedure** LSTMFORWARD( $\{x_t\}_{t=1}^T, \Theta$ )     $\triangleright \Theta$  gồm các trọng số:  $W_{x*}, W_{h*}, W_{c*}, W_y$
  - 2:     Khởi tạo  $h_0 \leftarrow 0, c_0 \leftarrow 0$
  - 3:     **for**  $t = 1$  **to**  $T$  **do**
  - 4:         $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1}),$   
            $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1}),$   
            $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1}),$   
            $c_t = f_t \times c_{t-1} + i_t \times \tanh(W_{xc}x_t + W_{hc}h_{t-1}),$   
            $h_t = o_t \times \tanh(c_t)$   
            $y_t = \sigma(W_{hy}h_t)$
  - 5:     **end for**
  - 6:     **return**  $\{h_t, y_t\}_{t=1}^T$
  - 7: **end procedure**
- 

Do công thức (6.6), mạng LSTM có thể nhớ dài hạn hơn mạng RNN thuần túy. Mạng LSTM có thể mở rộng thành mạng LSTM sâu có nhiều lớp trạng thái ẩn hoặc mạng LSTM hai chiều như đã mô tả ở trên. So sánh với mạng RNN, mạng LSTM có thể học được ngữ cảnh dài hơn, phù hợp với các bài toán xử lý chuỗi dữ liệu như dịch máy, nhận diện giọng nói, nhận diện văn bản. Tuy nhiên, mạng LSTM cũng có cùng nhược điểm với mạng RNN là không thể tối ưu hoá được hiệu quả tính toán do việc tính toán trạng thái ẩn và trạng thái ô phức tạp và có tính tuần tự. Trong trường hợp độ dài chuỗi lớn, mạng LSTM cũng gặp khó khăn trong việc liên kết thông tin giữa các phần tử trong chuỗi dữ liệu một cách hiệu quả nếu khoảng cách giữa chúng quá xa.

## 6.4 Mạng nơ-ron biến đổi Transformer

Để khắc phục các nhược điểm của mạng hồi quy RNN và mạng LSTM, ý tưởng của Transformer là sử dụng cơ chế chú ý để nắm bắt quan hệ giữa các phần tử trong chuỗi một cách song song, giúp mô hình học được các tương quan xa và cũng tăng tốc quá trình xử lý. Mô tả đầu ra  $\mathbf{y}_t$  bằng các đầu vào  $\mathbf{x}_{1:t}$  và đầu ra trước đó  $\mathbf{y}_{1:t-1}$  được cho bởi công thức xác suất có điều kiện như sau:

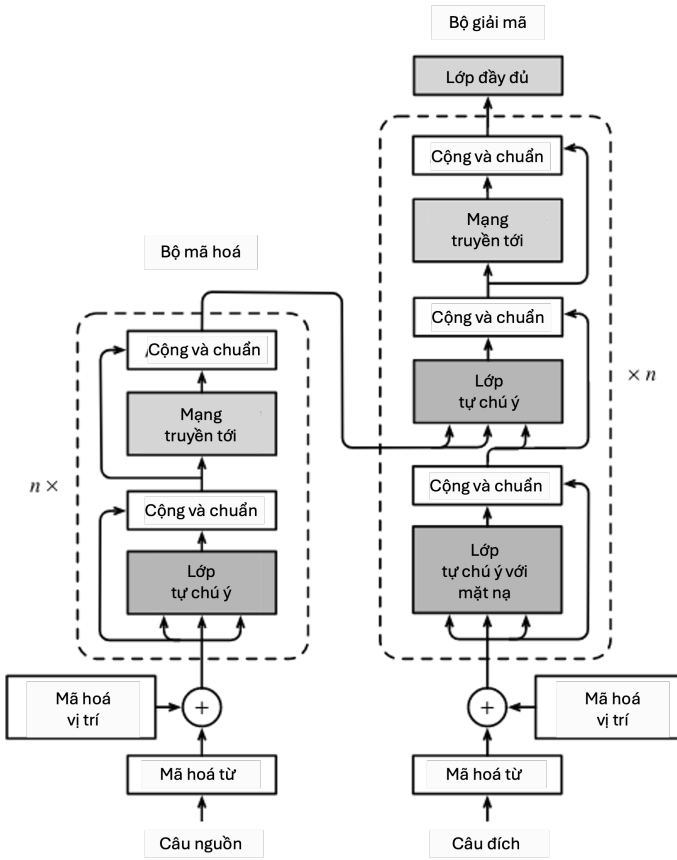
$$\mathbb{P}(\mathbf{y}_t | \mathbf{x}_{1:t}, \mathbf{y}_{1:t-1}) \quad (6.7)$$

Để có thể ánh xạ một chuỗi thông tin bất kỳ về miền ngữ nghĩa, mô hình Transformer sử dụng kiến trúc mã hoá - giải mã. Đầu tiên, ta mã hoá chuỗi đầu vào bằng cách nhúng (embedding) vào miền biểu diễn ẩn. Ở chiều giải mã ngược lại, véc-tơ trong biểu diễn ẩn được sử dụng để ánh xạ từ miền biểu diễn ẩn thành chuỗi đầu ra. Hình 6.7 mô tả cấu trúc này. Trong ứng dụng thực tế, các phần tử chuỗi đầu vào này được lấy từ một tập từ vựng và được mã hoá thành các chỉ số số nguyên. Số lượng phần tử trong tập từ vựng được lựa chọn phù hợp với dữ liệu cần được học.

Trong phép toán tự chú ý, các phần tử trong chuỗi được xử lý song song và bỏ qua các thông tin vị trí trong chuỗi thứ tự đầu vào. Đây là ưu điểm chính giúp cho mạng Transformer có thể tính toán nhanh trên GPU tuy nhiên nó cũng làm mất đi thông tin về vị trí trong dữ liệu đầu vào cũng như là sắp xếp lại thông tin cho dữ liệu đầu ra. Để giải quyết vấn đề này về mặt kỹ thuật, các tác giả của mạng Transformer mã hoá thông tin vị trí (Positional Encoding) thành một đặc trưng trong biểu diễn ẩn của dữ liệu bằng cách tạo ra các véc-tơ có cùng số chiều với không gian của miền nhúng như sau:

$$\text{PE}_{pos,2i} = \sin(pos/10000^{2i/d}) \quad (6.8)$$

$$\text{PE}_{pos,2i+1} = \cos(pos/10000^{2i/d}) \quad (6.9)$$



**Hình 6.7: Cấu trúc tổng quát của mô hình Transformer**

trong đó, các vị trí chẵn được mã hoá bằng hàm sin, các vị trí lẻ được mã hoá bằng hàm cos. Véc-tơ này được cộng với véc-tơ nhúng để tạo ra đầu vào của các khối Mã hoá và Giải mã dưới đây.

**Khối Mã hoá:** Khối mã hoá gồm hai mô-đun chính là mô-đun Chú ý và một mạng MLP. Kiến trúc mã hoá còn sử dụng kết nối phần dư tương tự như trong mạng tích chập ( $\mathbf{F}(\mathbf{x}) + \mathbf{x}$ ) đi cùng lớp chuẩn hoá. Lớp chuẩn hoá trong mạng Transformer thực hiện theo lớp được đề xuất theo [8], giúp giảm bớt sự phụ thuộc giữa các mẫu. Điều này đặc biệt hữu ích trong các mô hình có cấu trúc

phức tạp như Transformer, nơi sự nhất quán của chuẩn hóa cho từng mẫu là rất quan trọng. Các khối mã hoá có thể xếp chồng lên nhau để mô tả các mối quan hệ phức tạp giữa các phần tử đầu vào.

**Khối Giải mã:** gồm một mô-đun Chú ý chạy trên các đầu ra phía trước ( $\mathbf{y}_{1:t-1}$ ), một mô-đun Chú ý chạy trên cả đầu ra của khối mã hoá và một mạng kết nối đầy đủ. Các mô-đun này cũng sử dụng kết nối phần dư và chuẩn hoá, tương tự như khối mã hoá.

Điểm khác biệt chính của Khối Mã hoá và Giải mã vừa trình bày là hai mô-đun này chỉ sử dụng cơ chế Chú ý (Attention) để lan truyền thông tin giữa các phần tử trong chuỗi, cho phép mạng Transformer có bộ nhớ rất dài. Cụ thể hơn, mỗi phần tử trong chuỗi đầu vào có thể truy cập tới tất cả các phần tử khác trong chuỗi, không giống như mạng RNN hay LSTM chỉ có thể truy cập tới phần tử trước đó. Các bước tính toán chính trong cơ chế Chú ý được mô tả trong Thuật toán 6.3.

Mô-đun Chú ý trong mạng Transformer mở rộng cơ chế Chú ý thành nhiều đầu bằng cách chia các ma trận  $Q, K, V$  làm nhiều phần, mỗi phần đi qua một cấu trúc Chú ý đơn lẻ rồi tổng hợp lại. Trong bước tổng hợp lại, một hàm tuyến tính được sử dụng để kết hợp các kết quả từ các đầu ra Chú ý đơn lẻ.

Trên đây là những mô-đun cơ bản trong kiến trúc về mạng Transformer được đề xuất trong bài báo "Attention is All You Need" [5]. Kể từ khi được đề xuất, mạng Transformer đã thể hiện được khả năng mô hình hoá ngữ cảnh vượt trội so với thể hệ các mô hình học sâu trước đó như RNN và LSTM. Mạng Transformer đã được ứng dụng rộng rãi trong nhiều bài toán xử lý ngôn ngữ tự nhiên, dịch máy, nhận dạng giọng nói và phân tích chuỗi thời gian.

---

**Thuật toán 6.3** Tính toán của lớp Chú ý (Attention)

---

- 1: **procedure** COMPUTEATTENTION( $X, W_Q, W_K, W_V$ )
- 2:   Đầu vào  $X$ : ma trận nhúng đã tích hợp mã hóa vị trí
- 3:   Tính các ma trận truy vấn, khóa, giá trị:

$$Q \leftarrow XW_Q$$

$$K \leftarrow XW_K$$

$$V \leftarrow XW_V$$

- 4:   Tính ma trận điểm số (score matrix):

$$S \leftarrow QK^T$$

- 5:   Chuẩn hóa điểm số theo kích thước  $d_k$ :

$$\tilde{S} \leftarrow \frac{S}{\sqrt{d_k}}$$

- 6:   Tính ma trận attention weights bằng softmax:

$$A \leftarrow \text{Softmax}(\tilde{S})$$

- 7:   Tính đầu ra attention:

$$\text{Attention}(Q, K, V) \leftarrow AV$$

- 8:   **return** Attention( $Q, K, V$ )
  - 9: **end procedure**
-

## 6.5 Một số bài toán nhận thức kiểu chuỗi

### 6.5.1 Bài toán dịch máy thông kê

Bài toán dịch máy thông kê có thể được mô hình hoá như sau  $\mathbf{y} = y_1, y_2, \dots, y_{T'}$  với  $y_t \in V$  là các từ trong ngôn ngữ đích. Ở đây độ dài câu đích  $T' \neq T$  (độ dài câu nguồn). Trong ứng dụng thực tế, các phần tử đầu vào  $x_i$  và phần tử đầu ra là  $y_i$  là các từ. Tuy nhiên, các mô hình tính toán không thể tính toán trực tiếp trên kí tự nên cần phải có một bước trung gian là nhúng các phần tử đầu vào  $x_i$  về miền biểu diễn. Giả sử tập từ vựng của chuỗi đầu vào  $\mathbf{x}$  là  $V_x$ , ánh xạ nhúng  $g_x$  được định nghĩa như Công thức (6.10).

$$g_x : V_x \rightarrow \mathbb{R}^{d_x} \quad (6.10)$$

với  $d_x$  là số chiều của không gian nhúng cho chuỗi đầu vào.

Ở chiều ngược lại, với  $V_y$  là tập từ vựng ứng với chuỗi đầu ra  $\mathbf{y}$  thì ánh xạ nhúng  $g_y$  phải tồn tại một ánh xạ ngược từ không gian nhúng về tập từ vựng. Công thức ánh xạ ngược được định nghĩa như sau:

$$g_y^{-1} : \mathbb{R}^{d_y} \rightarrow V_y \quad (6.11)$$

với  $d_y$  là số chiều của không gian nhúng đầu ra.

Từ các dữ liệu thực tế, chúng ta có hai cách chính để khởi tạo ánh xạ nhúng, bao gồm:

1. Khởi tạo ngẫu nhiên: mỗi một từ vựng trong tập từ điển  $V_x$  được khởi tạo bằng một véc-tơ ngẫu nhiên có  $d_x$  chiều.
2. Huấn luyện trước: các từ vựng thường có tính ngữ nghĩa nhất định. Dựa trên cách này, các mô hình nhúng phổ biến có thể được học bằng cách tiền huấn luyện trên ngữ cảnh là câu hoặc các cụm từ có độ dài xác định. Dữ liệu để huấn luyện mô hình nhúng sẽ sử dụng toàn bộ các tập dữ liệu được thu thập. Trong

**Bảng 6.2:** Ví dụ về các chủ đề và dữ liệu chuỗi cần phân loại chủ đề

| Chủ đề    | Dữ liệu chuỗi đầu vào                          |
|-----------|------------------------------------------------|
| Thời sự   | Tình hình mưa lớn gây ngập lụt trên diện rộng. |
| Thời tiết | Bộ phim mới được khán giả đón nhận nồng nhiệt. |
| Giáo dục  | Kì thi THPT sắp diễn ra.                       |

bài toán dịch máy tổng quát, thì mô hình nhúng được huấn luyện trên tập tất cả văn bản tương ứng với ngôn ngữ nhất định. Trong ứng dụng thực tế thì những mô hình này được huấn luyện trước và có thể được sử dụng luôn giống như các cấu trúc mạng xương sống VGG và ResNet trong bài toán nhận dạng hình ảnh. Điểm khác biệt chính giữa mô hình nhúng trên các từ vựng và mô hình nhúng trên các ảnh là mô hình nhúng trên từ vựng thường được huấn luyện trên một tập dữ liệu lớn hơn và có thể học được bằng cách huấn luyện không giám sát.

### 6.5.2 Bài toán phát hiện chủ đề

Bài toán phát hiện chủ đề có thể mô hình hoá như một bài toán phân loại chuỗi với đầu vào là dãy  $\mathbf{x} = \{x_1, x_2, \dots, x_N\}$  và đầu ra nhãn  $y$  tương tự với bài toán phân lớp  $\mathbf{y} = y \in \{1, 2, \dots, C\}$ . Bảng 6.2 mô tả một số ví dụ về các chủ đề và dữ liệu chuỗi cần phân loại chủ đề.

Tùy vào cách tiếp cận cụ thể của bài toán mà các phần tử  $x_i$  có thể là kí tự, từ hoặc cụm từ. Và tương tự như bài toán dịch máy, mô hình mạng nơ-ron học trên chuỗi cần phải có một bước mã hoá để chuyển các phần tử  $x_i$  về miền nhúng.

Để triển khai bài toán theo mô hình này, ứng với chuỗi đầu vào  $x$ , chúng ta không cần thiết phải học hàm ánh xạ về chuỗi đầu ra  $y$  như trong các kiến trúc mô hình mạng hồi quy hoặc mạng bộ nhớ dài ngắn hạn thông thường. Thay vào đó, một ánh xạ từ miền



không gian ẩn của chuỗi hồi quy có thể được sử dụng kết hợp với một mạng nơ-ron lan truyền tối để học qua hàm Softmax. Công thức tổng quát được có thể được định nghĩa như sau:

$$h_{out} = AGG(h_0, h_1, \dots, h_T) \quad (6.12)$$

$$o = \text{Softmax}(FC(h_{out})) \quad (6.13)$$

với  $T$  là số trạng thái ẩn và AGG là hàm tổng hợp như lấy giá trị lớn nhất hoặc lấy giá trị trung bình. Ngoài ra, trong các kiến trúc mới hơn như máy biến đổi Transformer, phần tử đầu hoặc phần tử cuối có thể được lấy để đại diện cho không gian ẩn.

## 6.6 Tình huống áp dụng: học phép toán cộng

### 6.6.1 Mô tả bài toán

Trong phần Tình huống áp dụng của chương này, chúng ta sẽ thực hiện một bài toán về dự đoán chuỗi trong đó dữ liệu sẽ được sinh ra theo một qui trình đã được định trước. Nhiệm vụ của mô hình là tìm cách dự đoán kết quả mà không cần biết cụ thể qui trình sinh ra dữ liệu. Để minh họa cho cách tiếp cận này, chúng ta sẽ thực hiện một bài toán đơn giản là dự đoán phép cộng của hai số tự nhiên có độ dài cho trước.

Dữ liệu minh họa sẽ là có dạng chuỗi thể hiện các phép toán dưới dạng:

$$A + B = C$$

Trong đó  $A$ ,  $B$  và  $C$  là các chuỗi kí tự thể hiện các số tự nhiên có độ dài  $L$  xác định. Ví dụ:

1. Với  $A = 123$ ,  $B = 100$  thì  $C = 223$ , ta có phép toán được biểu diễn bằng một xâu kí tự:

$$123 + 100 = 223$$

2. Với  $A = 500$ ,  $B = 500$  thì  $C = 1000$ , ta có phép toán được biểu diễn bằng một xâu kí tự:

$$500 + 500 = 1000$$

Độ dài của các số tự nhiên  $A$  và  $B$  được quy định trước, ví dụ như  $L=3$  trong trường hợp này. Độ dài của  $C$  thì không được quy định trước, và dựa trên kết quả của phép cộng tương ứng.

Dựa trên cách thiết lập này, chúng ta sẽ có tập dữ liệu minh hoạ  $D = (X_i, Y_i)_{i=1}^N$  với  $N = 3$  được minh hoạ ở Bảng 6.3.

**Bảng 6.3: Ví dụ về bộ dữ liệu  $D$**

| N | X       | Y    |
|---|---------|------|
| 1 | 123+100 | 223  |
| 2 | 500+500 | 1000 |
| 3 | 999+111 | 1110 |

Chúng ta sẽ sử dụng mô hình hồi quy RNN để dự đoán kết quả của phép cộng dựa trên dữ liệu tập dữ liệu  $D$ .

### 6.6.2 Các bước triển khai

Để thực hiện bài toán này, chúng ta vẫn dựa trên các thư viện đã được giới thiệu trong các phần trước. Ngoài ra, chúng ta cần phải có thêm một số thư viện hỗ trợ cho việc sinh dữ liệu  $D$ . Các bước triển khai cụ thể bao gồm:

- Chuẩn bị dữ liệu: Chúng ta sẽ tự sinh dữ liệu  $D$  với các phép cộng đơn giản. Độ dài của các số tự nhiên  $A$  và  $B$  sẽ được quy định trước. Trong phần trình bày này, chúng ta sẽ sử dụng  $L = 3$ . Dựa trên qui trình sinh dữ liệu được cài đặt, chúng ta sẽ lần lượt được sinh ra tập dữ liệu huấn luyện và kiểm thử.

- Xây dựng mô hình hồi quy RNN: Chúng ta sẽ xây dựng một mô hình hồi quy RNN đơn giản có một lớp và sau đó là một lớp kết nối đầy đủ để dự đoán kết quả. Tham số nơ-ron ẩn của lớp RNN là 256.
- Huấn luyện mô hình: Chúng ta sẽ sử dụng bộ dữ liệu  $D$  đã được sinh ra để huấn luyện mô hình hồi quy RNN. Trong phần áp dụng này, chúng ta vẫn sử dụng bộ tối ưu Adam với tốc độ học là 0.001. Chúng ta sẽ huấn luyện mô hình trong 10 vòng lặp.
- Đánh giá mô hình: Chúng ta sẽ sử dụng mô hình hồi quy RNN đã được huấn luyện để dự đoán kết quả trên tập kiểm thử. Trong trường hợp này có nhiều cách để đánh giá độ chính xác mô hình. Chúng ta sẽ sử dụng độ đo về các kí tự chính xác và khoảng cách tuyệt đối giữa hai kết quả trên miền số tự nhiên.

Người học có thể tham khảo phần mã nguồn minh họa được trình bày ở đường dẫn <https://gist.github.com/cuongtv312/13bf5ef46cbbae0dc0d0e2e03b667df5>

## 6.7 Tổng kết chương

Chương này tập trung vào các kiến trúc mạng hồi quy cho dữ liệu chuỗi, làm rõ cách khai thác thông tin theo thứ tự thời gian để giải quyết các nhiệm vụ phức tạp. Các mô hình như RNN và LSTM duy trì trạng thái ẩn, cho phép ghi nhớ ngữ cảnh và học các phụ thuộc trong dữ liệu tuần tự. Nhờ đó, chúng thể hiện hiệu quả trong các tác vụ như biểu diễn ngôn ngữ và dự đoán. Việc lựa chọn kiến trúc phù hợp đóng vai trò quan trọng trong hiệu năng mô hình. Trong nhiều bài toán, LSTM và GRU cho kết quả tốt hơn RNN nhờ khả năng kiểm soát dòng thông tin và khắc phục hiện tượng triệt tiêu đạo hàm. Các mô hình hiện đại như Transformer mang lại bước tiến lớn nhờ khả năng xử lý song song và cơ chế tự chú

ý, mở ra hướng tiếp cận mới cho các bài toán chuỗi dài. Hiệu quả của mô hình còn phụ thuộc vào quá trình tiền xử lý dữ liệu, lựa chọn siêu tham số và các kỹ thuật chống quá khớp để nâng cao khả năng tổng quát hóa. Nghiên cứu mạng học sâu cho dữ liệu chuỗi đã tạo nền tảng cho nhiều ứng dụng quan trọng trong học máy và trí tuệ nhân tạo như dịch máy, nhận dạng giọng nói và phân tích chuỗi thời gian.

## Bài tập

1. Tính toán đạo hàm của mô hình LSTM, RNN và mô hình máy biến đổi Transformer với 1 phần tử theo hàm lỗi  $\delta$  và dữ liệu đầu vào
2. Giải thích tại sao mô hình LSTM lại có khả năng huấn luyện ổn định hơn trong trường hợp triệt tiêu đạo hàm.
3. Nêu nguyên lý của việc xây dựng cấu trúc biến đổi Transformer. Điểm khác biệt chính của mô hình Transformer so với các kiến trúc mạng hồi quy cổ điển là gì.
4. *[Tìm hiểu]* Việc sử dụng hàm sin/cos trong mô hình Transformer có ý nghĩa gì? Đề xuất một số phương pháp thay thế phù hợp.
5. *[Tìm hiểu]* Giải thích điểm khác biệt của chuẩn hoá theo lớp trong mạng Transformer và lớp chuẩn hoá theo tập huấn luyện của mạng CNN thông thường.
6. *[Lập trình]* Dựa trên tình huống áp dụng trong phần trước, hãy xây dựng một mô hình mạng nơ-ron biến đổi Transformer để học phép cộng trong hai trường hợp:
  - a) Phép cộng hai số tự nhiên có độ dài xác định.
  - b) Phép cộng hai số tự nhiên có độ dài không xác định nhưng tổng độ dài không vượt quá 10.

---

# Tài liệu tham khảo

---

- [1] Rumelhart, D. E., Hinton, G. E., and Williams, R. J., *Learning representations by back-propagating errors*, Nature, vol. 323, no. 6088, pp. 533–536, 1986.
- [2] Elman, J. L., *Finding structure in time*, Cognitive Science, vol. 14, no. 2, pp. 179–211, 1990.
- [3] Hochreiter, S., and Schmidhuber, J., *Long short-term memory*, Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y., *Learning phrase representations using RNN encoder-decoder for statistical machine translation*, arXiv preprint arXiv:1406.1078, 2014.
- [5] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I., *Attention is all you need*, Advances in Neural Information Processing Systems, vol. 30, pp. 5998–6008, 2017.
- [6] Devlin, J., Chang, M. W., Lee, K., and Toutanova, K., *BERT: Pre-training of deep bidirectional transformers for language understanding*, arXiv preprint arXiv:1810.04805, 2018.

- [7] Graves, A., Mohamed, A. R., and Hinton, G., *Speech recognition with deep recurrent neural networks*, Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing, pp. 6645–6649, 2013.
- [8] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.

# Chương 7

---

## Mô hình hồi quy

---

Trong Chương 2, chúng ta đã tìm hiểu về khả năng phân lớp của các mô hình Học máy – mô phỏng cách con người học và phân biệt các khái niệm. Ngoài việc học khái niệm, con người còn có khả năng học mối quan hệ định lượng giữa các đại lượng, nhằm dự đoán giá trị của một biến dựa trên các biến khác, như giá nhà, nhiệt độ, hay độ ẩm. Trong Học máy, bài toán này được gọi là bài toán hồi quy.

Về lý thuyết, hồi quy chỉ khác phân lớp ở hàm mất mát. Ta thay hàm lỗi phân lớp bằng hàm lỗi bình phương sai số như sau:

$$L(w) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; w))^2.$$

Chương 7 trình bày các khái niệm cơ bản về hồi quy tuyến tính, phân tích sự đánh đổi giữa độ lệch và phương sai, cùng các mô hình hồi quy phi tuyến như mạng nơ-ron. Các kỹ thuật này đóng vai trò nền tảng trong việc xây dựng mô hình dự báo và ước lượng trong nhiều bối cảnh ứng dụng thực tế.

## 7.1 Bài toán hồi quy

Chúng ta bắt đầu với định nghĩa cơ bản về bài toán hồi quy theo mô hình cực tiểu sai số lỗi thực nghiệm.

**Định nghĩa 7.1** (Bài toán hồi quy). Cho  $\mathcal{X}$  là tập đầu vào,  $\mathcal{Y} = \mathbb{R}$  tập dữ liệu huấn luyện  $D = \{(x_i, y_i)\}, i = 1, 2, \dots, n$  với  $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$  được lấy mẫu theo phân bố  $\mathcal{P}$ , hãy tìm một mô hình (hàm)  $h : \mathcal{X} \rightarrow \mathcal{Y}$  dự đoán tốt trên dữ liệu  $(x, y)$  bất kì cũng lấy mẫu theo  $\mathcal{P}$ .

Để đánh giá khả năng dự đoán của hàm  $h$ , chúng ta sử dụng kì vọng của một loại lỗi giữa nhãn  $y$  và dự đoán  $h(x)$ . Hàm lỗi thường dùng nhất là bình phương sai số. Tương tự như trong bài toán phân lớp, chúng ta định nghĩa hàm lỗi dựa trên phân bố của dữ liệu  $\mathcal{P}$ .

**Định nghĩa 7.2** (Lỗi của mô hình hồi quy). Cho hàm hồi quy  $h$  và phân bố  $\mathcal{P}$  của dữ liệu  $(x, y)$ , lỗi của mô hình hồi quy  $h$  được định nghĩa là:

$$\begin{aligned}\text{err}_{\mathcal{P}}(h) &= \mathbb{E}_{\mathcal{P}}[(y - h(x))^2] \\ &= \int_{x,y} (y - h(x))^2 p(x, y) dx dy\end{aligned}$$

trong đó  $p(x, y)$  là hàm mật độ xác suất của phân bố  $\mathcal{P}$ .

Trong ứng dụng thực tế, chúng ta không biết cụ thể phân bố dữ liệu  $\mathcal{P}$  nên phải xấp xỉ kì vọng ở Định nghĩa 7.2 bằng trung bình bình phương sai số trên tập dữ liệu huấn luyện, được định nghĩa là:

**Định nghĩa 7.3** (Trung bình bình phương sai số). Cho tập dữ liệu huấn luyện  $D = \{(x_i, y_i)\}_{i=1}^n$  với  $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$  được lấy



mẫu theo phân bố  $\mathcal{P}$ , *trung bình bình phương sai số* MSE (mean squared error) của mô hình hồi quy  $h$  được định nghĩa là:

$$\begin{aligned}\text{err}_D(h) &= \frac{1}{n} \sum_{i=1}^n (y_i - h(x_i))^2 \\ &= \mathbb{E}_D[(y - h_D(x))^2]\end{aligned}$$

Nếu có đủ dữ liệu huấn luyện, theo luật số lớn, chúng ta kì vọng lỗi thực nghiệm sẽ hội tụ về lỗi của mô hình hồi quy

$$\text{err}_D(h) \xrightarrow{P} \text{err}_{\mathcal{P}}(h).$$

### 7.1.1 Hàm hồi quy tối ưu

Chúng ta có thể xây dựng hàm hồi quy tối ưu dựa trên phân phối có điều kiện của  $y$  khi biết  $x$  theo định lý sau:

**Định lý 7.4** (Hàm hồi quy tối ưu). *Giả sử dữ liệu  $(x, y) \in \mathcal{X} \times \mathcal{Y}$  và được lấy mẫu theo phân bố  $\mathcal{P}$  với hàm lỗi ở Định nghĩa 7.1. Hàm hồi quy tối ưu thoả mãn*

$$\text{err}_{\mathcal{P}}(h^*) = \min_h \text{err}_{\mathcal{P}}(h)$$

là hàm hồi quy  $h^*$  được định nghĩa như sau:

$$h^*(x) = E_{\mathcal{P}}[y|x] = \int_{\mathcal{Y}} yp(y|x)dy.$$

*Chứng minh:* Ta có

$$\begin{aligned}\text{err}_{\mathcal{P}}(h) &= \mathbb{E}_{\mathcal{P}}[(y - h(x))^2] \\ &= \mathbb{E}_{\mathcal{P}}[(y - h^*(x) + h^*(x) - h(x))^2] \\ &= \mathbb{E}_{\mathcal{P}}[(y - h^*(x))^2] + \mathbb{E}_{\mathcal{P}}[(h^*(x) - h(x))^2] \\ &\quad + \mathbb{E}_{\mathcal{P}}[2(y - h^*(x))(h^*(x) - h(x))] \\ &= \text{err}_{\mathcal{P}}(h^*) + \mathbb{E}_{\mathcal{P}}[(h^*(x) - h(x))^2] \\ &\quad + 2\mathbb{E}_{\mathcal{P}}[(y - h^*(x))(h^*(x) - h(x))]\end{aligned}$$

Xét kì vọng  $\mathbb{E}_{\mathcal{P}}[(y - h^*(x))(h^*(x) - h(x))]$

$$\begin{aligned}
 &= \int_{x,y} (y - h^*(x))(h^*(x) - h(x))p(x,y)dx dy \\
 &= \int_x \underbrace{\left[ \int_y (y - h^*(x))p(y|x)dy \right]}_{= 0 \text{ theo định nghĩa của } h^*} (h^*(x) - h(x))p(x)dx \\
 &= 0
 \end{aligned}$$

Vậy ta có

$$\text{err}_{\mathcal{P}}(h) = \text{err}_{\mathcal{P}}(h^*) + \mathbb{E}_{\mathcal{P}}[(h^*(x) - h(x))^2]$$

và hàm  $h$  tốt nhất chính là  $h^*$  để số hạng thứ 2 bằng 0.  $\square$

### 7.1.2 Sự tráo đổi giữa độ lệch và phương sai

Trong các ứng dụng học máy thống kê, thường không tìm được chính xác  $h^*$  vì chúng ta chỉ có tập dữ liệu  $D = \{(x_i, y_i)\}, i = 1, \dots, n$  thay vì biết chính xác phân bố  $\mathcal{P}$ . Nếu coi các mẫu dữ liệu  $(x_i, y_i)$  là các biến ngẫu nhiên từ phân bố  $\mathcal{P}$  thì bất cứ thuật toán Học máy nào sẽ cho một ước lượng hàm  $h_D$  phụ thuộc vào tập dữ liệu  $D$ .

Xét trung bình lỗi của thuật toán của trên mẫu dữ liệu  $(x, y)$  bất kì, ta có

$$\begin{aligned}
 &\mathbb{E}_D[(y - h_D(x))^2] \\
 &= \mathbb{E}_D[(y - \mathbb{E}_D[h_D(x)] + \mathbb{E}_D[h_D(x)] - h_D(x))^2] \\
 &= \underbrace{\mathbb{E}_D[(y - \mathbb{E}_D[h_D(x)])^2]}_{\text{Bias}_D^2[y, h_D(x)]} + \underbrace{\mathbb{E}_D[(\mathbb{E}_D[h_D(x)] - h_D(x))^2]}_{\text{Var}_D[h_D(x)]} \\
 &\quad + 2\mathbb{E}_D[(y - \mathbb{E}_D[h_D(x)])(\mathbb{E}_D[h_D(x)] - h_D(x))]
 \end{aligned}$$

do  $y - \mathbb{E}_D[h_D(x)]$  là hằng số đối với  $D$ , tiếp tục khai triển thu được

$$\begin{aligned} &= \text{Bias}_D^2[y, h_D(x)] + \text{Var}_D[h_D(x)] \\ &\quad + 2(y - \mathbb{E}_D[h_D(x)]) \underbrace{\mathbb{E}_D[\mathbb{E}_D[h_D(x)] - h_D(x)]}_0 \\ &= \text{Bias}_D^2[y, h_D(x)] + \text{Var}_D[h_D(x)]. \end{aligned}$$

Trong đó, ta có

$$\text{Bias}_D[y, h_D(x)] = y - \mathbb{E}_D[h_D(x)] \quad (7.1)$$

là độ lệch giữa giá trị mong muốn  $y$  và giá trị kì vọng của  $h_D(x)$  khi ta chạy thuật toán nhiều lần trên các tập dữ liệu  $D$  ngẫu nhiên. Còn

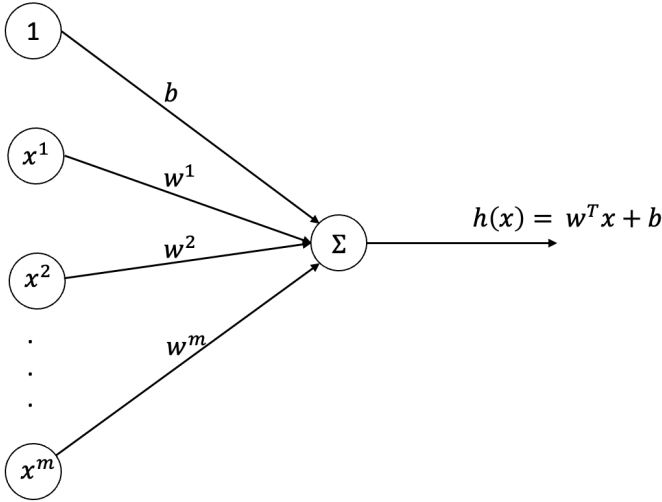
$$\text{Var}_D[h_D(x)] = \mathbb{E}_D[(\mathbb{E}_D[h_D(x)] - h_D(x))^2] \quad (7.2)$$

là phương sai của  $h_D(x)$  khi ta chạy thuật toán nhiều lần trên các tập dữ liệu  $D$  ngẫu nhiên.

Bây giờ cho  $(x, y)$  lấy mẫu từ phân bố  $\mathcal{P}$ , ta có công thức đưa ra mối liên hệ giữa độ lệch và phương sai của một hàm hồi quy  $h_D$  như sau:

$$\mathbb{E}_{\mathcal{P}}[\mathbb{E}_D[(y - h_D(x))^2]] = \mathbb{E}_{\mathcal{P}} [\text{Bias}_D^2[y, h_D(x)] + \text{Var}_D[h_D(x)]] .$$

Đây là khai triển kì vọng về lỗi mô hình hồi quy bất kì, còn gọi là *sự tráo đổi giữa độ lệch và phương sai* (bias-variance tradeoff). Lưu ý  $h_D$  là kết quả của thuật toán Học máy khi chạy trên dữ liệu  $D$  ngẫu nhiên. Do về trái bị chặn dưới bởi  $\text{err}_{\mathcal{P}}(h^*)$ , nếu ta tìm được cách giảm độ lệch (ví dụ: sử dụng mô hình mạnh, có thể nhớ được toàn bộ tập dữ liệu  $D$ ) thì đến một lúc nào đó phương sai của hàm  $h_D$  sẽ phải tăng. Nghĩa là, thuật toán cho kết quả là một hàm dự đoán rất không ổn định, học quá trên tập dữ liệu  $D$ .



Hình 7.1: Mô hình hồi quy tuyến tính.

## 7.2 Mô hình hồi quy tuyến tính

Trong lớp các mô hình hồi quy, hồi quy tuyến tính là một trong những mô hình đơn giản và có ứng dụng rộng rãi nhất. Mô hình Hồi quy tuyến tính ước lượng hàm hồi quy  $E[y \mid \mathbf{x}]$  bằng một hàm tuyến tính theo định nghĩa sau.

**Định nghĩa 7.5** (Hồi quy tuyến tính). Mô hình hồi quy tuyến tính  $h : \mathbb{R}^m \rightarrow \mathbb{R}$  là hàm

$$\begin{aligned} h(\mathbf{x}) &= \mathbf{w}^\top \mathbf{x} + b \\ &= w^1 x^1 + w^2 x^2 + \dots + w^m x^m + b \end{aligned}$$

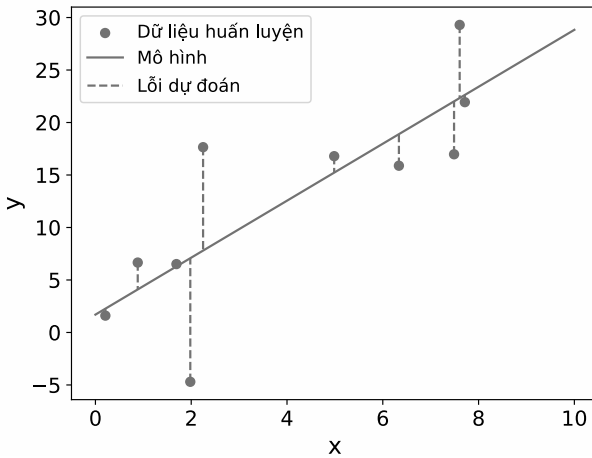
trong đó  $\mathbf{x} \in \mathbb{R}^m$  là véc-tơ đầu vào,  $\mathbf{w} \in \mathbb{R}^m, b \in \mathbb{R}$  là các trọng số của mô hình.

Hình 7.1 minh họa mô hình tính toán của hồi quy tuyến tính trong đó các hệ số  $w^1, w^2, \dots, w^m$  thể hiện đóng góp của các thuộc tính  $x^1, x^2, \dots, x^m$  vào dự đoán  $y$ . Hệ số  $b$  giúp cân bằng lại độ lệch

của hàm hồi quy với các giá trị mục tiêu. Lỗi của mô hình hồi quy tuyến tính trên tập dữ liệu huấn luyện  $D$  là

$$\text{err}_D(h) = \frac{1}{n} \sum_{i=1}^n (y_i - (\mathbf{w}^\top \mathbf{x}_i + b))^2. \quad (7.3)$$

Trong trường hợp đơn giản, với  $x \in \mathbb{R}^1$ , chúng ta có thể minh họa mô hình hồi quy tuyến tính là một đường thẳng và lỗi của mô hình hồi quy tuyến tính là khoảng cách từ các điểm trên tập huấn luyện đến đường thẳng như trong Hình 7.2.



**Hình 7.2: Minh họa lỗi của mô hình hồi quy tuyến tính**

Chúng ta thấy rằng có sự tương đồng giữa các mô hình hồi quy tuyến tính, hồi quy Logistic và perceptron. Các mô hình đều là tổng các trọng số nhân với các thuộc tính đầu vào cộng với một hệ số tự do. Điểm khác biệt chỉ nằm ở hàm kích hoạt và cách tham số được cập nhật để tối ưu hóa hàm lỗi. Bảng 7.1 đưa ra so sánh các khác biệt cơ bản giữa ba mô hình này.

Mô hình hồi quy tuyến tính được quan tâm nhiều trong thực tế và hướng đến việc trả lời được các câu hỏi về mối quan hệ giữa các biến đầu vào và biến đầu ra, cụ thể là:

**Bảng 7.1:** So sánh ba mô hình hồi quy tuyến tính, hồi quy Logistic và perceptron

| Mô hình            | Hàm kích hoạt       | Hàm lỗi                       |
|--------------------|---------------------|-------------------------------|
| Hồi quy Logistic   | $\text{sigmoid}(x)$ | Entropy chéo                  |
| Perceptron         | $\text{sgn}(x)$     | So sánh                       |
| Hồi quy tuyến tính | $\text{id}(x) = x$  | Trung bình bình phương sai số |

- Liệu giữa thuộc tính  $x^k$  và  $y$  có mối liên hệ nào không?
- Mức độ chặt chẽ của mối liên hệ giữa  $x^k$  và  $y$  như thế nào?
- Trong các thuộc tính  $x^1, x^2, \dots, x^m$ , cái nào tác động nhiều đến  $y$ ?
- Dự đoán trên dữ liệu chưa biết chính xác đến mức nào?
- Có thực sự mối quan hệ giữa  $\mathbf{x}$  và  $y$  là tuyến tính?

**Ví dụ 7.6** (Dự báo doanh số từ quảng cáo). Ví dụ, xét mô hình tuyến tính dự báo doanh số từng tháng dựa trên các loại hình quảng cáo:

$$\text{Doanh số} = w^1 \times \text{Tivi} + w^2 \times \text{Đài phát thanh} + w^3 \times \text{Báo chí} + b \quad (7.4)$$

Từ hàm hồi quy tuyến tính ở công thức (7.4), chúng ta có thể đưa ra được các nhận xét sau:

- Nếu thêm 1 đồng vào quảng cáo trên Tivi thì tăng được  $w^1$  đơn vị doanh số
- Nếu thêm 1 đồng vào quảng cáo trên Đài phát thanh thì tăng được  $w^2$  đơn vị doanh số
- Nếu thêm 1 đồng vào quảng cáo trên Báo chí thì tăng được  $w^3$  đơn vị doanh số

- Nếu không quảng cáo gì cả thì được  $b$  đơn vị doanh số.

Nhiệm vụ của thuật toán huấn luyện mô hình là từ dữ liệu về quảng cáo và doanh số, tìm ra bộ trọng số  $(w^1, w^2, w^3, b)$  phù hợp nhất và sau đó dùng bộ trọng số này để dự đoán doanh số tương lai. Chúng ta thấy trong bài toán này, thiết lập tương tự như bài toán học máy có giám sát phân lớp mà chúng ta đã làm quen ở các chương trước. Điểm khác biệt duy nhất là giá trị doanh số là một giá trị thực, không phải là một khái niệm.

### 7.2.1 Huấn luyện mô hình

Trong huấn luyện mô hình hồi quy tuyến tính, dựa theo công thức (7.3), chúng ta cần tìm  $\beta = (\mathbf{w}, b)$  dự đoán tốt trên  $\mathcal{P}$ . Giả sử ta có tập dữ liệu huấn luyện  $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ , thì bộ tham số của mô hình hồi quy tuyến tính được xác định bằng cách tối thiểu hóa hàm lỗi thực nghiệm

$$\begin{aligned}\beta^* &= \arg \min_{\beta} \text{err}_D(h) \\ &= \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n (y_i - h(\mathbf{x}_i))^2.\end{aligned}$$

Chúng ta nhận thấy bài toán huấn luyện mô hình hồi quy tuyến tính có thể được xem như bài toán tối ưu hoá hàm bậc hai theo tham số  $\beta$ .

Xét mỗi mẫu huấn luyện, ta có sai số dự đoán

$$\varepsilon_i = y_i - h(\mathbf{x}_i) = y_i - (\mathbf{w}^\top \mathbf{x}_i + b).$$

Hàm mục tiêu ở trên là tương đương với tổng bình phương sai số  $\varepsilon_i$ , viết tắt là RSS, được định nghĩa là

$$L = \text{RSS} = \sum_{i=1}^n \varepsilon_i^2.$$

Để cực tiểu của hàm lỗi  $L$ , chúng ta lấy đạo hàm theo  $\mathbf{w}$  và  $b$  rồi giải hệ phương trình để hai đại lượng này triệt tiêu:

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{w}} &= -2 \sum_{i=1}^n (y_i - (\mathbf{w}^\top \mathbf{x}_i + b)) \mathbf{x}_i = 0, \\ \frac{\partial L}{\partial b} &= -2 \sum_{i=1}^n (y_i - (\mathbf{w}^\top \mathbf{x}_i + b)) = 0.\end{aligned}$$

Để cho dễ trình bày và cài đặt, ta có thể viết lại dưới dạng ma trận:

$$\mathbf{X} = \begin{bmatrix} 1 & \mathbf{x}_1^\top \\ 1 & \mathbf{x}_2^\top \\ \vdots & \dots \\ 1 & \mathbf{x}_n^\top \end{bmatrix}, \quad \boldsymbol{\beta} = \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix},$$

khi đó hàm lỗi  $L$  là

$$L = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}). \quad (7.5)$$

Lấy đạo hàm theo  $\boldsymbol{\beta}$  và tìm nghiệm để đạo hàm bằng 0, ta thu được

$$-2 \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = 0 \quad \Rightarrow \quad \boldsymbol{\beta}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (7.6)$$

Công thức (7.6) có thể dùng để tính trực tiếp bộ tham số tối ưu  $\boldsymbol{\beta}^*$ , còn gọi là *phương trình chuẩn tắc* (normal equation). Tuy nhiên, cách tiếp cận này yêu cầu phải tính phải tính ma trận nghịch đảo  $(\mathbf{X}^\top \mathbf{X})^{-1}$ . Dễ thấy ma trận  $\mathbf{X}^\top \mathbf{X}$  là ma trận vuông có kích thước  $m \times m$ . Nếu số chiều  $m$  lớn, việc tính toán ma trận nghịch đảo này là tốn kém và có nhiều sai số.

Ngoài cách tiếp cận trên, chúng ta có thể sử dụng phương pháp xuống đồi bằng đạo hàm như được trình bày ở Chương 3. Sử dụng công thức (7.5), ta có công thức cập nhật tham số  $\boldsymbol{\beta}_{t+1}$  tại bước cập nhật thứ  $t + 1$  thông qua bước thứ  $t$  như sau:

$$\boldsymbol{\beta}_{t+1} \leftarrow \boldsymbol{\beta}_t + \lambda \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}_t) \quad (7.7)$$



với  $\lambda$  là tốc độ học. Xuất phát từ điểm bắt đầu  $\beta_0$  ngẫu nhiên, thuật toán sẽ lặp lại bước cập nhật này cho đến khi hội tụ. Gọi  $L(\beta_t)$  là hàm lỗi tại bước  $t$ , chúng ta cũng có được tính chất hội tụ tương tự như trong Chương 3, tức là hàm lỗi  $L(\beta_t)$  sẽ giảm dần theo thời gian và tiến về giá trị cực tiểu  $L(\beta^*)$  với  $\beta^*$  là bộ tham số tối ưu.

**Định lý 7.7** (Tính hội tụ của xuống đồi bằng đạo hàm cho hồi quy tuyến tính). *Xét hàm lỗi*

$$L = \|\mathbf{y} - \mathbf{X}\beta\|^2$$

với ma trận đặc trưng  $\mathbf{X} \in \mathbb{R}^{n \times m}$  và  $\mathbf{Y} \in \mathbb{R}^n$ . Thuật toán xuống đồi bằng đạo hàm

$$\beta_{t+1} \leftarrow \beta_t + \lambda \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\beta_t)$$

hội tụ tới nghiệm tối ưu  $\beta^*$  khi hệ số học

$$0 < \lambda < \frac{2}{\lambda_{\max}(\mathbf{X}^\top \mathbf{X})},$$

với  $\lambda_{\max}(\mathbf{X}^\top \mathbf{X})$  là giá trị riêng lớn nhất của ma trận  $\mathbf{X}^\top \mathbf{X}$ .

Để chứng minh Định lý trên, chúng ta chỉ cần chỉ ra hàm lỗi  $L$  thỏa mãn các điều kiện tương tự của hàm lỗi trong Định lý 3.6 ở Chương 3. Phần chứng minh chi tiết được xem như là bài tập ở cuối chương.

## 7.3 Các phương pháp đánh giá mô hình

Một trong những bước quan trọng trong quá trình xây dựng mô hình hồi quy là đánh giá chất lượng của mô hình. Đánh giá mô hình giúp chúng ta hiểu rõ hơn về khả năng dự đoán của mô hình, từ đó có thể cải thiện mô hình hoặc chọn mô hình phù hợp với bài toán. Trong phần này, chúng ta sẽ tìm hiểu về hai phương pháp đánh giá mô hình hồi quy phổ biến.

### 7.3.1    Đánh giá đầu ra mô hình hồi quy

Đầu ra của mô hình hồi quy là một giá trị liên tục, do đó, chúng ta cần các phương pháp đánh giá khác so với mô hình phân loại. Gọi

$$\hat{y} = h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \hat{y}_i = h(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b$$

là các giá trị dự đoán của mô hình hồi quy.

Các phương pháp đánh giá mô hình hồi quy phổ biến bao gồm:

- *Trung bình bình phương sai số*

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (7.8)$$

- *Căn trung bình bình phương sai số*

$$RMSE = \sqrt{MSE} \quad (7.9)$$

$RMSE$  cho phép đánh giá sai số trên thứ nguyên (đơn vị) của đầu ra.

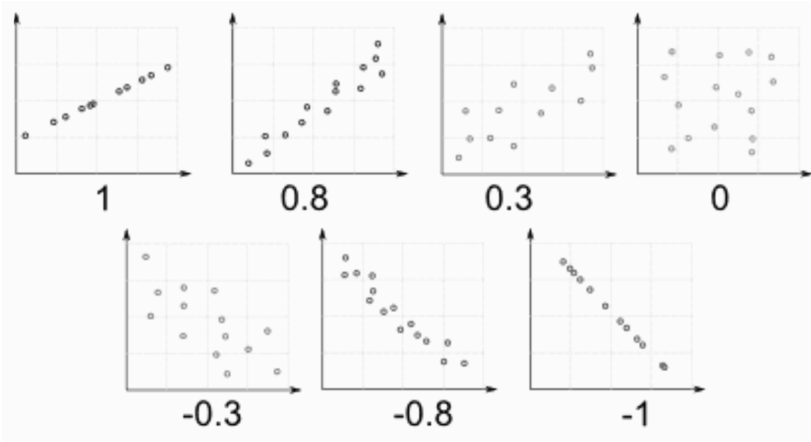
- *Trung bình sai số tuyệt đối*

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (7.10)$$

Giá trị  $MAE$  có đơn vị trùng với đơn vị của đầu ra  $y$  và thường nhỏ hơn độ lệch  $RMSE$  một chút.

- *Độ lệch chuẩn*: Độ lệch chuẩn của dự đoán so với trung bình của nó, được cho bởi công thức sau:

$$\mu = \frac{1}{n} \sum_{i=1}^n \hat{y}_i, \quad SD = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - \mu)^2}{n - 1}}. \quad (7.11)$$



**Hình 7.3: Hệ số  $R$  cho các trường hợp khác nhau.**

Đại lượng  $SD$  trong công thức (7.11) thể hiện tính ổn định của dự đoán và thường được sử dụng cho từng khoảng  $[a, b]$  của đầu ra  $y_i$ .

- *Độ tương quan*: Phương pháp này đo lường mối quan hệ tuyến tính giữa dự đoán  $\hat{y}_i$  và nhân  $y_i$  bằng hệ số tương quan  $R$ , được cho bởi công thức:

$$R = \frac{\sum_{i=1}^n (y_i - \bar{y})(\hat{y}_i - \mu)}{\sqrt{\sum_{i=1}^n (y_i - \bar{y})^2 + \sum_{i=1}^n (\hat{y}_i - \mu)^2}} \quad (7.12)$$

Hệ số  $R$  cho thấy tính xu thế mối quan hệ giữa hai đại lượng  $\hat{y}_i$  và  $y_i$ . Khi  $R = 1$  nghĩa là hai đại lượng cùng tăng, cùng giảm với mối quan hệ tuyến tính. Khi  $R = -1$  nghĩa là hai đại lượng tăng giảm ngược chiều với mối quan hệ tuyến tính. Khi  $R = 0$ , hai đại lượng không có tương quan gì (Hình 7.3).

- *Hệ số  $R^2$* : Bên cạnh Hệ số  $R$ , chúng ta có thể sử dụng hệ số  $R^2$  là tỉ lệ giữa phần phương sai đầu ra được diễn giải bởi mô hình so với phương sai của đầu ra đích.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}. \quad (7.13)$$

### 7.3.2 Đánh giá chất lượng trọng số

Chất lượng trọng số của mô hình hồi quy tuyến tính cũng là một khía cạnh cần xét đến khi đánh giá một mô hình. Chất lượng trọng số thể hiện ở hai điểm sau:

- Mỗi trọng số  $w^k$  có thật sự khác 0? Nghĩa là thuộc tính  $x^k$  có thật sự đóng góp vào việc dự đoán giá trị  $y$ ?
- Giá trị của  $w^k$  có thực sự đáng tin cậy? Nghĩa là nếu tập dữ liệu  $D$  thay đổi (vẫn tuân theo phân bố  $\mathcal{P}$ ) thì  $w^k$  có thay đổi nhiều? Hoặc khoảng tin cậy của  $w^k$  là bao nhiêu?

Các câu hỏi trên được trả lời nhờ *Kiểm tra giả thuyết thống kê* bằng phương pháp bootstrap hoặc phương pháp dựa trên giả thuyết về hiệp phương sai của sai số.

- Phương pháp bootstrap: với phương pháp này, chúng ta lấy mẫu ngẫu nhiên  $B$  lần, mỗi lần  $n$  mẫu có hoàn lại từ tập dữ liệu  $D$ . Giả sử lần lấy mẫu thứ  $b$  ta được bộ dữ liệu  $D_b = \{(\mathbf{x}_{bi}, y_{bi}), i = 1, 2, \dots, n\}$ . Ta tính toán bộ trọng số  $\beta_b$  bằng công thức (7.14):

$$\beta_b = (\mathbf{X}_b^T \mathbf{X}_b)^{-1} \mathbf{X}_b^T \mathbf{y}_b, \quad (7.14)$$

hoặc sử dụng các thuật toán xuống đồi bằng đạo hàm như đã biết. Với  $B$  lần lấy mẫu ta được  $B$  ước lượng  $\beta_1, \beta_2, \dots, \beta_B$ . Đối với từng trọng số  $w^k$ , chúng ta có  $B$  mẫu ước lượng cho từng tham số:

$$w_1^k, w_1^k, \dots, w_B^k. \quad (7.15)$$

Dựa trên tập giá trị này, chúng ta có thể tính được khoảng tin cậy 95% của  $w^k$  là khoảng nằm từ trị số 0,025 đến trị số 0,975 của  $B$  mẫu này. Nếu khoảng tin cậy này chứa 0 thì không thể loại bỏ giả thiết  $H_0 : w_k = 0$ , tức là thuộc tính  $x^k$  không có đóng góp gì vào việc dự đoán giá trị  $y$ .

- Phương pháp dựa trên giả thiết độc lập và hiệp phương sai dạng đường chéo của sai số: Nếu ta giả sử tồn tại  $\beta^* = (w^*, b^*)$  sao cho Đẳng thức (7.16) sau được thoả mãn:

$$y_i = w^{*T} \mathbf{x}_i + b^* + \epsilon_i, \quad (7.16)$$

với  $\epsilon_i$  là biến ngẫu nhiên thoả mãn  $\mathbb{E}[\epsilon_i] = 0$ ,  $\mathbb{E}[\epsilon_i] = \sigma^2$ ,  $\mathbb{E}[\epsilon_i \epsilon_j] = 0, \forall i \neq j$ . Khi đó ta có,

$$\begin{aligned} \text{Var}[\beta] &= \text{Var}[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}] \\ &= \text{Var}[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{X} \beta^* + \boldsymbol{\epsilon})] \\ &= \text{Var}[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \boldsymbol{\epsilon}] \text{ (do } \beta^* \text{ là hằng số)} \\ &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \underbrace{\text{Var}[\boldsymbol{\epsilon}]}_{\sigma^2 \mathbf{I}} \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \\ &= \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1} = [v_{ij}] \in \mathbb{R}^{(m+1) \times (m+1)} \end{aligned} \quad (7.17)$$

Ta có thể ước lượng  $\sigma^2$  bằng ước lượng không chệch như sau:

$$\hat{\sigma}^2 = \frac{\boldsymbol{\epsilon}^T \boldsymbol{\epsilon}}{n - m} \quad (7.18)$$

Từ đó, ta có thể tính khoảng tin cậy 95% của  $w^k$  theo công thức:

$$[w^k - 1, 96\sqrt{v_{k+1,k+1}}; w^k + 1, 96\sqrt{v_{k+1,k+1}}] \quad (7.19)$$

Trong đó,  $v_{k+1,k+1}$  là phương sai của  $w^k$ . Tương tự như phương pháp bootstrap, nếu khoảng tin cậy này chứa số 0, ta nói có thể bỏ qua thuộc tính  $x^k$  khi ước lượng  $y$ .

## 7.4 Mạng nơ-ron hồi quy

### 7.4.1 Hồi quy với hệ cơ sở phi tuyến

Với mô hình hồi quy tuyến tính, các hệ số tuyến tính  $w_1, w_2, \dots, w_d$  tác động trực tiếp lên các biến đầu vào  $x^1, x^2, \dots, x^d$ . Mô hình tuyến

tính có thể không đủ khả năng biểu diễn mối quan hệ giữa đầu vào và đầu ra, nên sai số: có thể lớn. Để khắc phục điều này, ta có thể phi tuyến hoá dữ liệu trước khi sử dụng hàm tuyến tính. Giả sử có  $K$  hàm  $f_1, f_2, \dots, f_K$ , gọi là hệ hàm cơ sở (basis functions), kết quả dự đoán của mô hình như sau:

$$\hat{y} = w_1 f_1(\mathbf{x}) + w_2 f_2(\mathbf{x}) + \dots + w_K f_K(\mathbf{x}). \quad (7.20)$$

Một số lựa chọn phổ biến của hệ cơ sở trong công thức (7.20) bao gồm:

- **Hệ cơ sở đa thức (polynomial basis):**  $f_k(\mathbf{x})$  là một hàm đa thức của  $\mathbf{x}$ .
- **Hệ cơ sở bán kính (radial basis):**  $f_k(\mathbf{x}) = \phi_k(\|\mathbf{x} - \mathbf{x}_k\|)$ , với  $\mathbf{x}_k$  là các mẫu dữ liệu huấn luyện.
- **Mạng nơ-ron:**  $f_k(\mathbf{x}) = f_k(\mathbf{x}; \mathbf{W})$ , với  $f_k$  là mạng nơ-ron có bộ trọng số  $\mathbf{W}$ .

**Lưu ý:** Việc lựa chọn các hàm  $f_k$  còn được gọi là kỹ thuật trích rút đặc trưng, sẽ được thảo luận kỹ hơn trong các phần sau.

### 7.4.2 Mạng nơ-ron

Một trong các phương pháp tăng sức mạnh biểu diễn là sử dụng mạng nơ-ron để tính toán các đặc trưng ở lớp đầu ra. Sau đó, các đặc trưng này được kết hợp lại bằng một hàm tuyến tính. Cụ thể, giả sử ta có mạng nơ-ron được cho bởi công thức sau:

$$\mathbf{f}(\mathbf{x}, \mathbf{W}) : \mathbf{x} \mapsto \mathbf{f}, \quad (7.21)$$

với  $\mathbf{f} \in \mathbb{R}^K$  là véc-tơ đặc trưng tại đầu ra. Kết quả dự đoán theo mô hình hồi quy với mạng nơ-ron như sau:

$$\hat{y} = \mathbf{w}^T \mathbf{f} + b, \quad (7.22)$$

Sử dụng hàm lỗi sai số bình phương trên tập dữ liệu  $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ , ta có công thức tính lỗi  $\ell$  là:

$$\ell = \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{f}_i)^2, \quad \text{với } \mathbf{f}_i = \mathbf{f}(\mathbf{x}_i; \mathbf{W}). \quad (7.23)$$

Để huấn luyện mạng nơ-ron hồi quy, ta tính đạo hàm  $\ell$  đối với các tham số  $\mathbf{W}$  và  $b$  theo quy tắc lan truyền ngược. Đạo hàm của hàm lỗi đối với  $\mathbf{w}$  là:

$$\frac{\partial \ell}{\partial \mathbf{w}} = -2 \sum_{i=1}^n e_i \mathbf{f}_i, \quad (7.24)$$

trong đó  $e_i = y_i - \mathbf{w}^T \mathbf{f}_i$  là sai số. Đạo hàm của hàm lỗi đối với  $\mathbf{W}$  được tính bằng cách lan truyền ngược đạo hàm  $\delta_{\mathbf{f}} = -2e_i \mathbf{w}$  về các lớp phía trước của mạng nơ-ron và cộng dồn qua tất cả các mẫu.

Một số ví dụ về mạng nơ-ron hồi quy ứng dụng trong thực tế bao gồm:

- **Bài toán phát hiện vật thể trong ảnh:** Cần tính vị trí và kích thước của vật thể trong ảnh. Một phương pháp phổ biến là sử dụng các hình chữ nhật bao vật thể. Toạ độ của các hình chữ nhật có thể được dự đoán bằng cách kết hợp đặc trưng trích xuất từ các tầng tích chập của mạng nơ-ron CNN và áp dụng hồi quy để dự đoán toạ độ.
- **Dự đoán nồng độ ô nhiễm không khí:** Sử dụng mạng nơ-ron hồi quy để dự báo mức độ bụi mịn và các chất ô nhiễm khác dựa trên dữ liệu từ các cảm biến thời tiết và môi trường.
- **Dự báo giá bất động sản:** Mô hình có thể ước tính giá trị thị trường của nhà ở dựa trên các đặc trưng như diện tích, vị trí, số phòng ngủ, tiện ích xung quanh, v.v.

- **Dự báo chỉ số kinh tế:** Áp dụng mạng nơ-ron hồi quy để dự đoán các chỉ số kinh tế như GDP, lạm phát dựa trên dữ liệu lịch sử và các biến kinh tế khác.

## 7.5 Hàm lỗi

Quá trình giải bài toán phân lớp và hồi quy có điểm chung như sau:

- Định nghĩa một hàm lỗi:

$$\ell(\mathbf{W}) = \sum_{i=1}^n \ell_i(f(\mathbf{x}_i; \mathbf{W}), y_i), \quad (7.25)$$

trong đó hàm lỗi phụ thuộc vào bài toán cần giải và tính chất mong muốn của nghiệm.

- Tối ưu hóa hàm lỗi bằng phương pháp Xuống đồi bằng đạo hàm hoặc các biến thể của nó.

Hàm lỗi đóng vai trò then chốt trong việc huấn luyện mô hình và nghiên cứu các hàm lỗi mới luôn là vấn đề quan trọng trong học máy. Tuy nhiên, trong thực tế việc tối ưu hàm lỗi có thể gặp nhiều khó khăn do hàm lỗi không phải lúc nào cũng khả vi tối ưu hoặc không phản ánh đúng mục tiêu mong muốn của bài toán. Một trong những ví dụ thường gặp là hàm lỗi bị ảnh hưởng bởi nhiễu trong dữ liệu hoặc bài toán hồi quy bị suy biến khi số chiều dữ liệu lớn.

Để tăng tính ổn định của mô hình trên dữ liệu mới (pha suy luận), ta có thể thêm một đại lượng điều chỉnh vào hàm lỗi. Hàm lỗi điều chỉnh có dạng:

$$\ell(\mathbf{W}) = \lambda \Omega(\mathbf{W}) + \sum_{i=1}^n \ell_i(f(\mathbf{x}_i; \mathbf{W}), y_i) \quad (7.26)$$

trong đó,  $\Omega(\mathbf{W})$  là một hàm điều chỉnh. Các lựa chọn phổ biến cho  $\Omega$  bao gồm:



- $\Omega(\mathbf{W}) = \frac{1}{2}\|\mathbf{W}\|_2^2$ : Hàm điều chỉnh theo chuẩn hoá L2 hay còn gọi là hồi quy Ridge nếu mô hình là tuyến tính. Đạo hàm của hàm điều chỉnh L2 so với  $\mathbf{W}$  cho ta  $\nabla_{\mathbf{W}}\Omega(\mathbf{W}) = \mathbf{W}$ , giúp việc tính toán đạo hàm trở nên trực quan và hiệu quả. Sự khả vi của đạo hàm với hàm điều chỉnh L2 đảm bảo rằng các thuật toán xuống đồi bằng đạo hàm có thể hội tụ ổn định tới nghiệm tối ưu. Việc sử dụng hàm điều chỉnh L2 giúp tăng tính ổn định của mô hình với dữ liệu mới bằng cách làm giảm độ lệch và giảm thiểu vấn đề hay gặp trong trường hợp các dữ liệu đầu vào phụ thuộc tuyến tính.
- $\Omega(\mathbf{W}) = \|\mathbf{W}\|_1$ : Hàm điều chỉnh theo chuẩn hoá L1, còn được gọi là hồi quy Lasso, hay được dùng trong bài toán lựa chọn đặc trưng với hồi quy tuyến tính. Nó có khả năng thiết lập một số trọng số thành 0, qua đó loại bỏ các đặc trưng không quan trọng. Tuy nhiên, hàm điều chỉnh L1 không khả vi tại điểm 0, đòi hỏi áp dụng các kỹ thuật tối ưu đặc biệt. Các thuật toán như đạo hàm dưới có thể được sử dụng để xử lý tính không khả vi của hàm này.

## 7.6 Hồi quy bằng K láng giềng gần nhất

Tương tự như KNN trong bài toán phân lớp, ta có thể sử dụng lân cận là các mẫu dữ liệu gần với đầu vào  $\mathbf{x}$  để ước lượng giá trị của hàm hồi quy. Giả sử, trong tập dữ liệu  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$ , ta lấy ra  $k$  điểm gần với  $\mathbf{x}$  nhất là  $\mathbf{x}_{i_1}, \mathbf{x}_{i_2}, \dots, \mathbf{x}_{i_k}$  theo hàm khoảng cách  $d(u, v)$  (thường chọn  $d(u, v) = \|u - v\|_2$  là khoảng cách Euclid).

Khi đó, ta có thể dùng trung bình cộng nhân của  $k$  láng giềng này để ước lượng kì vọng  $E[y|\mathbf{x}]$  theo công thức (7.27).

$$h(\mathbf{x}) = \frac{1}{k} \sum_{j=1}^k y_{i_j} \quad (7.27)$$

Sử dụng phương pháp hồi quy bằng  $k$  láng giềng gần nhất có pha huấn luyện rất đơn giản: ghi nhớ toàn bộ tập dữ liệu  $D$ . Một hệ quả là, với  $k = 1$ , thuật toán học dữ liệu  $D$  một cách hoàn hảo, nghĩa là  $h(\mathbf{x}_i) = y_i$  với mọi mẫu dữ liệu  $(\mathbf{x}_i, y_i) \in D$ . Vì thế, với  $k = 1$  thuật toán đã bị phù hợp quá. Do đó, thường chúng ta sẽ sử dụng thuật toán này với  $k$  lớn hơn, ví dụ  $k = 3, 5, 7, 9, \dots$ . Khi đó, thuật toán cho hàm dự đoán  $h(\mathbf{x})$  trơn hơn và có khả năng tổng quát hoá trên các đầu vào chưa biết.

---

**Thuật toán 7.1** Thuật toán KNN hồi quy

---

- 1: **procedure** KNNREGRESSION( $x, D, k$ )
- 2:    Tìm  $k$  mẫu dữ liệu gần  $x$  nhất trong  $D$  theo khoảng cách  $d(\mathbf{x}_i, \mathbf{x})$
- 3:    Lưu lại các mẫu gần nhất:

$$(\mathbf{x}_{i_1}, y_{i_1}), (\mathbf{x}_{i_2}, y_{i_2}), \dots, (\mathbf{x}_{i_k}, y_{i_k})$$

- 4:    Tính giá trị dự đoán:

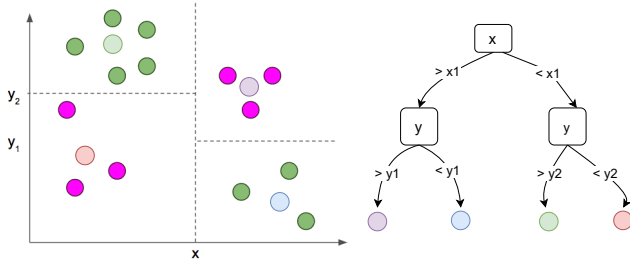
$$h(\mathbf{x}) \leftarrow \frac{1}{k} \sum_{j=1}^k y_{i_j}$$

- 5:    **return**  $h(\mathbf{x})$
  - 6: **end procedure**
- 

Ngoài cách lấy trung bình cộng, có biến thể của Thuật toán 7.1 sử dụng lấy tổng có trọng số cho từng nhãn  $y_{i_j}$  như sau:

$$h(\mathbf{x}) = \frac{1}{\sum_{j=1}^k w_j} \sum_{j=1}^k w_j y_{i_j}, \quad (7.28)$$

với  $w_j = g(d(\mathbf{x}_i, \mathbf{x}))$  với  $g$  là một hàm nghịch biến. Tức là mẫu dữ liệu có khoảng cách nhỏ sẽ có trọng số lớn hơn. Lựa chọn  $g(z) = e^{-z}$  hoặc  $g(z) = 1/z$  là các lựa chọn hay gặp.



Hình 7.4: Cây quyết định hồi quy tập đầu vào  $\mathcal{X}$ .

## 7.7 Mô hình Cây quyết định trong hồi quy

Giống như cây quyết định để phân lớp, ta cũng dùng cây quyết định để phân vùng không gian đầu vào  $\mathcal{X}$  và ước lượng  $E[y|\mathbf{x}]$  thông qua trị số trung bình trong từng phân vùng.

Cụ thể, trong pha suy luận, với mỗi dữ liệu đầu vào  $\mathbf{x}$ , ta tìm phân vùng ở các nút lá  $\text{leaf}(\mathbf{x})$  của cây ứng với  $\mathbf{x}$ , dự đoán bằng giá trị  $h(\mathbf{x})$  được định nghĩa theo công thức sau:

$$h(\mathbf{x}) = \hat{E}_{\text{leaf}(\mathbf{x})}[y|x] = \frac{1}{|\text{leaf}(\mathbf{x})|} \sum_{(\mathbf{x}_i, y_i) \in D} y_i \quad (7.29)$$

là giá trị trung bình các nhãn  $y_i$  của các mẫu dữ liệu trong  $D$  được phân vào lá đó (Hình 7.4).

Trong pha huấn luyện, thay vì tìm kiếm các quyết định phân chia cây sao cho khả năng phân lớp tăng lên, ở cây quyết định hồi quy, chúng ta tìm kiếm quyết định làm giảm phương sai của các nút, làm nhãn của các nút đồng nhất hơn. Cụ thể, phương sai của một nút chứa tập dữ liệu được định nghĩa là:  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$  là

$$\text{Var}(D) = \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n (y_i - y_j)^2 \quad (7.30)$$

Khi phân chia tập dữ liệu  $D$  thành các tập dữ liệu  $D_1, D_2, \dots$

sử dụng thuộc tính  $A^k$ , ta muốn tìm quyết định làm giảm phương sai nhiều nhất có thể, tức là cực đại hoá đại lượng  $I_V(D, A^k)$ , được định nghĩa theo công thức (7.31).

$$I_V(D, A^k) = \mathbb{V}\text{ar}(D) - \sum_i \mathbb{V}\text{ar}(D_i) \quad (7.31)$$

Thuật toán CART (Thuật toán 7.2) sử dụng cùng lược đồ với thuật toán ID3 nhưng dùng tiêu chí  $I_V(D, A^k)$  để phân chia các nút.

---

**Thuật toán 7.2** Thuật toán huấn luyện cây hồi quy CART

---

```
1: procedure TRAINDTREGRESSOR(D)
2: if D thoả mãn điều kiện dừng then
3: Tạo nút lá leaf với nhãn là trung bình nhãn trong D
4: return leaf
5: else
6: Tạo nút gốc root
7: Chọn thuộc tính A^k cực đại hoá $I_V(D, A^k)$
8: Phân chia D thành các tập con $D_1, D_2, \dots$ tùy theo loại
 thuộc tính A^k
9: for all D_i do
10: Gắn nhánh từ root đến TRAINDTREGRESSOR(D_i)
11: end for
12: return root
13: end if
14: end procedure
```

---

## 7.8 Tình huống áp dụng: Dự đoán giá nhà

Trong phần này chúng ta sẽ áp dụng mô hình hồi quy tuyến tính để dự đoán giá nhà dựa trên các đặc trưng như diện tích, số phòng và tuổi nhà. Bài toán hồi quy tuyến tính là một trong những bài toán phổ biến trong học máy. Dữ liệu được sử dụng trong bài toán này là tập dữ liệu giá nhà được cung cấp bởi thư viện `scikit-learn`.

Chúng ta sẽ sử dụng mô hình hồi quy tuyến tính để dự đoán giá nhà dựa trên các đặc trưng đầu vào. Các bước triển khai cơ bản bao gồm:

- Chuẩn bị dữ liệu: Chúng ta sử dụng trực tiếp dữ liệu được cung cấp bởi thư viện `scikit-learn`. Số lượng đặc trưng đầu vào là 8 và 1 nhãn đầu ra. Các đặc trưng bao gồm một số thông số cơ bản như diện tích nhà, số phòng, tuổi nhà cũng như vị trí nhà và các thông tin liên quan. Chúng ta tiếp tục tiền xử lý và chia dữ liệu thành tập huấn luyện và tập kiểm thử.
- Huấn luyện mô hình: Sử dụng thư viện `scikit-learn` để huấn luyện mô hình hồi quy tuyến tính. Trong thư viện được sử dụng, mô hình hồi quy tuyến tính khá đơn giản. Chỉ cần lựa chọn các tham số mặc định là ta có thể thu được mô hình cần huấn luyện.
- Đánh giá mô hình: Sử dụng các chỉ số đánh giá như sai số trung bình bình phương và hàm  $R^2$  để đánh giá mô hình hồi quy tuyến tính.

Người học có thể tham khảo mã nguồn tại đường dẫn <https://gist.github.com/cuongtv312/2bb428d9ed4ab25371978d48b11575cc>

## 7.9 Tổng kết chương

Chương 7 cung cấp tổng quan khá toàn diện về các mô hình hồi quy – phương pháp dự đoán giá trị liên tục trong Học máy. Hồi quy tuyến tính giả định quan hệ tuyến tính giữa biến đầu vào và đầu ra, được huấn luyện qua hàm mất mát bình phương sai số. Phương pháp này đơn giản, hiệu quả với dữ liệu ít chiều và ít nhiễu. Hồi quy phi tuyến mở rộng khả năng biểu diễn bằng cách sử dụng các hàm kích hoạt phi tuyến và tầng ẩn, giúp mô hình học được quan hệ phức tạp hơn giữa các biến. Mạng nơ-ron hồi quy kế thừa cấu

trúc tuyến tính nhưng mạnh mẽ hơn nhờ khả năng học phi tuyến qua nhiều tầng ẩn, đặc biệt phù hợp với các bài toán phức tạp.

Việc triển khai mô hình hồi quy đòi hỏi tiền xử lý dữ liệu như chuẩn hóa, xử lý nhiễu và lựa chọn siêu tham số phù hợp để cải thiện hiệu suất dự đoán. Các chỉ số đánh giá như MAE, MSE và  $R^2$  cho phép định lượng sai số và so sánh hiệu quả giữa các mô hình. Các ứng dụng thực tiễn của hồi quy rất phong phú, từ dự báo kinh tế, phân tích thị trường đến tối ưu hóa hệ thống và tài nguyên. Qua các ví dụ minh họa, chương này khẳng định vai trò thiết yếu của hồi quy trong phân tích và mô hình hóa dữ liệu.

## Bài tập

1. Sử dụng dạng biểu diễn ma trận của hàm lỗi trong hồi quy tuyến tính để chứng minh rằng hàm lỗi là một hàm lồi và trơn trên không gian tham số. Từ đó, khẳng định tính hội tụ khi sử dụng phương pháp Xuống đồi bằng đạo hàm để tìm nghiệm cực tiểu của hàm lỗi.
2. Trong huấn luyện mô hình hồi quy tuyến tính, so sánh phương pháp giải Xuống đồi bằng đạo hàm với phương pháp giải sử dụng công thức tính cực trị trực tiếp. Trong những trường hợp cụ thể dưới đây thì chúng ta nên sử dụng phương pháp nào?
  - a) Số đặc trưng đầu vào nhỏ hơn 10, số dữ liệu lớn hơn 1000.
  - b) Số đặc trưng đầu vào lớn hơn 1000, số dữ liệu lớn hơn 10000.
  - c) Các đặc trưng đầu vào có phụ thuộc tuyến tính với nhau.
3. *[Tìm hiểu]* Trong trường hợp số mẫu huấn luyện  $n$  lớn hơn số biến đầu vào  $m$ , công thức tính cực trị trực tiếp của mô hình hồi quy cho phép tìm được nghiệm tối ưu duy nhất. Tuy nhiên, nếu số mẫu huấn luyện  $n$  nhỏ hơn số biến đầu vào  $m$ , thì nghiệm tối

**Bảng 7.2: Dữ liệu huấn luyện**

| N | $x$ | $y$  | N  | $x$ | $y$ |
|---|-----|------|----|-----|-----|
| 1 | 1.0 | 0.0  | 6  | 0.7 | 0.5 |
| 2 | 1.0 | 1.0  | 7  | 1.5 | 0.8 |
| 3 | 0.1 | -0.4 | 8  | 2.0 | 2.0 |
| 4 | 0.0 | 1.0  | 9  | 1.4 | 1.2 |
| 5 | 0.3 | 0.8  | 10 | 2.0 | 1.0 |

ưu không duy nhất. Đề xuất phương pháp để tìm được nghiệm tối ưu duy nhất trong trường hợp này.

4. *[Lập trình]* Cho dữ liệu  $D = \{(x_i, y_i)\}_{i=1}^{10}$  trong Bảng 7.2 với như sau:
  - a) Viết chương trình huấn luyện theo thuật toán hồi quy tuyến tính với dữ liệu trong Bảng 7.2 có cấu trúc 2 lớp ẩn, mỗi lớp có 5 nơ-ron.
  - b) Thay đổi số lớp ẩn và số nơ-ron trong mỗi lớp ẩn, so sánh kết quả của các mô hình tìm được.
5. *[Tìm hiểu]* Phân tích độ lệch và độ nhiễu mô hình hồi quy K-láng giềng gần nhất
6. *[Tìm hiểu]* Nêu các phương pháp để tăng độ tổng quát hoá của mô hình hồi quy sử dụng cây quyết định.
7. *[Tìm hiểu]* So sánh ưu nhược điểm của mô hình hồi quy tuyến tính, mô hình học sâu và cây quyết định trong việc dự đoán giá trị liên tục.
8. Trình bày các phương pháp đánh giá hiệu năng của mô hình hồi quy, bao gồm: trung bình sai số bình phương, trung bình sai số tuyệt đối và hệ số xác định  $R^2$ .





---

# Tài liệu tham khảo

---

- [1] Legendre, A. M., *Nouvelles méthodes pour la détermination des orbites des comètes*, F. Didot, Paris, 1805.
- [2] Fisher, R. A., *The goodness of fit of regression formulae, and the distribution of regression coefficients*, Journal of the Royal Statistical Society, vol. 85, no. 1, pp. 597–612, 1922.
- [3] Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J., *Classification and regression trees*, Wadsworth International Group, 1984.
- [4] Altman, N. S., *An introduction to kernel and nearest-neighbor nonparametric regression*, The American Statistician, vol. 46, no. 3, pp. 175–185, 1992.
- [5] Bishop, C. M., *Neural networks for pattern recognition*, Oxford University Press, 1995.
- [6] Hastie, T., Tibshirani, R., and Friedman, J., *The elements of statistical learning*, Springer, 2001.



## Chương 8

---

# Học tăng cường

---

Khác với học có giám sát, học tăng cường tập trung vào bài toán ra quyết định tuần tự thông qua tương tác giữa tác nhân và môi trường.

Mục tiêu chính của học tăng cường là cực đại hóa kỳ vọng tổng phần thưởng nhận được trong tương lai. Tác nhân chọn hành động theo chính sách  $\pi$ , nhằm tối ưu hàm mục tiêu:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \right]$$

Trong đó,  $\gamma \in (0, 1)$  là hệ số chiết khấu,  $r_t$  là phần thưởng tại thời điểm  $t$ , và  $\tau$  là một quỹ đạo các trạng thái và hành động theo chính sách  $\pi$ .

Chương này sẽ giới thiệu các khái niệm nền tảng trong học tăng cường, bao gồm môi trường, trạng thái, hành động, chính sách, hàm giá trị và phần thưởng.

Bên cạnh đó, các phương pháp quy hoạch động cổ điển như Value Iteration và Policy Iteration sẽ được trình bày như cơ sở lý thuyết.

Cuối cùng, chương sẽ giới thiệu các tiếp cận hiện đại kết hợp học sâu và học tăng cường – một hướng nghiên cứu nổi bật hiện nay với nhiều ứng dụng trong trò chơi, điều khiển robot và ra quyết định tự động.

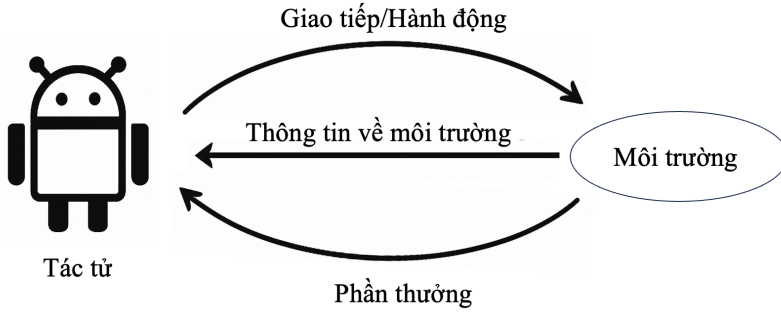
## 8.1 Giới thiệu về quá trình quyết định Markov

Bài toán học tăng cường đóng vai trò trung tâm trong trí tuệ nhân tạo, lĩnh vực tự động hoá. Ứng dụng cơ bản nhất của học tăng cường có thể được minh hoạ bằng Hình 8.1. Học tăng cường sử dụng khái niệm tác tử hoặc rô-bốt thông minh và môi trường mà tác tử có thể tương tác. Trong quá trình tương tác, tác tử nhận được một trạng thái hiện tại  $s$  từ môi trường và xác định hành động  $a$  thông qua chính sách  $\pi$ , và được kí hiệu là

$$a = \pi(s).$$

Dựa trên hành động được đưa ra, tác tử sẽ nhận được các phần thưởng/điểm số thích hợp. Mục tiêu chính của tác tử là tìm cách đưa ra dãy hành động để tối ưu hoá phần thưởng nhận được trong mọi tình huống.

Trong ngữ cảnh của học máy, bài toán học tăng cường phải giải quyết đồng thời hai vấn đề: (i) thu thập dữ liệu về trạng thái-hành động-điểm thưởng và (ii) học trên dữ liệu thu thập được. Tác tử phải liên tục đưa ra hành động tương tác với môi trường để nhận về dữ liệu và đánh giá mức độ hiệu quả của hành động đó. Học tăng cường gần giống bài toán học có giám sát, trong đó mô hình học tăng cường sẽ nhận thông tin từ môi trường và trả về hành động để tương tác với môi trường. Thông tin đầu vào này gần tương tự với



**Hình 8.1:** Mô hình đơn giản minh họa về tác tử thông minh học cách tương tác với môi trường

khái niệm dữ liệu trong học có giám sát. Điểm khác biệt là thông tin về mô hình tối ưu được cho dưới dạng phần thưởng. Đây là thông tin gián tiếp, không phải thông tin trực tiếp từ hàm lỗi trong học có giám sát.

### 8.1.1 Mô hình toán học của học tăng cường

Bài toán học tăng cường được xây dựng dựa trên cơ sở của một quá trình quyết định Markov, viết tắt là MDP (Markov Decision Process).

Tính chất cơ bản của quá trình quyết định Markov đến từ dãy các trạng thái Markov. Xét một chuỗi các trạng thái

$$s_0, s_1, s_2, \dots, s_t$$

trong miền thời gian rời rạc. Tính chất Markov yêu cầu giới hạn lại sự phụ thuộc lẫn nhau giữa trạng thái ở tương lai  $s_{t+1}$  sẽ độc lập với các trạng thái quá khứ  $s_{t-1}, s_{t-2}, \dots, s_0$  theo Định nghĩa 8.1.

**Định nghĩa 8.1** (Tính chất Markov). Dãy  $s_0, s_1, \dots, s_T$  có tính chất Markov nếu với mọi  $t \geq 0$ ,

$$P(s_{t+1}|s_t) = P(s_{t+1}|s_t, s_{t-1}, \dots, s_0).$$

Tính chất này có nghĩa là trạng thái hiện tại  $s_t$  chứa đủ thông tin để dự đoán trạng thái tương lai  $s_{t+1}$  mà không cần biết các trạng thái trước đó. Tính chất này là cơ sở của quá trình quyết định Markov.

**Định nghĩa 8.2** (Quá trình quyết định Markov). Một quá trình quyết định Markov là một bộ  $\langle S, A, T, R, \gamma \rangle$ , trong đó

- Tập  $S$  là tập hợp các trạng thái. Mỗi trạng thái  $s \in S$  đại diện cho một tình huống cụ thể mà tác tử có thể gặp phải.
- Tập  $A$  là tập hợp các hành động. Mỗi hành động  $a \in A$  đại diện cho một hành động mà tác tử có thể thực hiện.
- Hàm chuyển trạng thái  $T(s, a, s')$  là xác suất chuyển sang trạng thái  $s'$  khi tác tử thực hiện hành động  $a$  khi ở trạng thái  $s \in S$ .

$$T(s, a, s') = P(s'|a, s)$$

Hàm chuyển thường được biểu diễn dưới dạng một phân phối xác suất có điều kiện đại diện cho tính không chắc chắn của môi trường.

- Hàm phần thưởng  $R(s, a, s')$  là phần thưởng khi tác tử ở trạng thái  $s$  thực hiện hành động  $a$  và đi đến được trạng thái  $s'$ . Trong hầu hết các bài toán cơ bản, hàm  $R(s, a, s')$  có thể được rút gọn thành  $R(s')$  (chỉ phụ thuộc trạng thái kế tiếp) hoặc  $R(a)$  (chỉ phụ thuộc hành động).
- Hệ số giảm hoặc mất mát  $\gamma$  đặc trưng cho sự giảm dần của giá trị của phần thưởng theo thời gian. Giá trị của  $\gamma$  là 1 số thực trong khoảng  $[0, 1)$ .

Trong một số bài toán cụ thể, chúng ta có thể có thêm định nghĩa về các trạng thái bắt đầu và kết thúc.

**Định nghĩa 8.3** (Chính sách đơn định). Cho một quá trình Markov được đặc trưng bởi  $\langle S, A, T, R, \gamma \rangle$ , chính sách  $\pi$  là ánh xạ từ tập trạng thái  $S$  vào tập hành động  $A$  đại diện cho cách chọn hành động cho tác tử trong môi trường này,

$$\pi : S \mapsto A, \quad (8.1)$$

với  $\pi(s) \in A$  là hành động được chọn tại trạng thái  $s$ .

Chính sách được đề cập trong Công thức (8.1) là chính sách đơn định, trong đó tác tử sẽ chỉ được phép chọn một hành động duy nhất tại mỗi trạng thái. Chúng ta có thể mở rộng định nghĩa này cho trường hợp chính sách ngẫu nhiên.

**Định nghĩa 8.4** (Chính sách ngẫu nhiên). Cho một quá trình Markov được đặc trưng bởi  $\langle S, A, T, R, \gamma \rangle$ , chính sách  $\pi$  là phân phối xác suất lựa chọn hành động tại mỗi trạng thái,

$$\pi(a|s) = P(a|s), \quad (8.2)$$

với  $\pi(a|s)$  là xác suất chọn hành động  $a \in A$  tại trạng thái  $s \in S$ .

Để diễn tả tính ngẫu nhiên của chính sách, chúng ta có thể viết tắt  $a \sim \pi(\cdot|s)$  với ý nghĩa hành động  $a$  được lấy mẫu dựa theo phân phối của chính sách  $\pi$  tại trạng thái  $s$ .

Nhiệm vụ của mô hình là tìm cách tối ưu hoá phần thưởng từ khi xuất phát từ trạng thái bắt đầu đến khi kết thúc (hoặc vô hạn). Ta thấy,  $R(s, a, s')$  chỉ đánh giá được phần thưởng tại một bước riêng biệt. Để đánh giá tổng phần thưởng trên một dãy các trạng thái, còn gọi là *quỹ đạo* (trajectory), có  $L$  bước: xuất phát từ  $s_0$ , thực hiện hành động  $a_0$  đến  $s_1$ ; thực hiện  $a_1$  đến  $s_2$ ; ...; và kết thúc tại trạng thái  $s_L$ , có dạng:

$$\tau = \{s_0, a_0, s_1, a_1, \dots, s_{L-1}, a_{L-1}, s_L\}.$$

Trong trường hợp các hành động  $a_i$  được chọn theo chính sách  $\pi$ , chúng ta có thể viết lại dãy trạng thái như sau:

$$\tau = \{s_0, a_0 = \pi(s_0), s_1, a_1 = \pi(s_1), \dots, s_{L-1}, a_{L-1} = \pi(s_{L-1}), s_L\}.$$

Nếu sử dụng chính sách ngẫu nhiên  $\pi$ , thì dãy trạng thái  $\tau$  có thể được viết lại theo

$$\tau = \{s_0, a_0 \sim \pi(\cdot|s_0), s_1, a_1 \sim \pi(\cdot|s_1), \dots, s_{L-1}, a_{L-1} \sim \pi(\cdot|s_{L-1}), s_L\}.$$

Theo cách diễn đạt này, chúng ta muốn nhấn mạnh việc chính sách  $\pi$  được dùng để sinh ra quỹ đạo  $\tau$ . Khi đó, chúng ta có thể viết tắt là *quỹ đạo  $\tau$  được sinh ra từ chính sách  $\pi$* :

$$\tau \sim \pi.$$

Dựa trên biểu diễn chuỗi trạng thái hành động này, chúng ta có thể định nghĩa giá trị của quỹ đạo  $\tau$ .

**Định nghĩa 8.5** (Giá trị của quỹ đạo). Giá trị của quỹ đạo  $\tau$  là tổng các phần thưởng nhận được sau khi tác tử thực hiện các hành động kèm theo trọng số giảm dần theo thời gian:

$$\begin{aligned} U(\tau) &= R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \dots + \gamma^{L-1} R(s_{L-1}, a_{L-1}, s_L) \\ &= \sum_{t=0}^{L-1} \gamma^t R(s_t, a_t, s_{t+1}). \end{aligned}$$

Trong trường hợp hàm  $R$  chỉ phụ thuộc vào trạng thái đến, tức là  $R(s, a, s') = R(s')$ , ta có công thức rút gọn

$$U(\tau) = \sum_{t=0}^{L-1} \gamma^t R(s_{t+1}).$$

Sự xuất hiện của *hệ số chiết khấu* (discount)  $\gamma$  trong công thức trên có ý nghĩa là các phần thưởng nhận được ở các bước thời gian



sau có độ quan trọng thấp hơn phần thưởng nhận được ở các bước thời gian trước đó. Giá trị của  $\gamma$  được lựa chọn trong khoảng  $[0, 1)$  để đảm bảo rằng tổng phần thưởng  $U(\tau)$  hội tụ về một giá trị hữu hạn. Trong trường hợp  $\gamma = 1$ , tổng phần thưởng  $U(\tau)$  có thể vô hạn khi  $L$  tiến về vô cùng. Việc này sẽ gây khó khăn cho việc tối ưu hoá hàm mục tiêu.

Mục tiêu chính của Học tăng cường là tìm ra chính sách  $\pi$  cực đại hoá giá trị  $U(\tau)$ .

**Định nghĩa 8.6** (Chính sách tối ưu). Cho một quá trình quyết định Markov  $\langle S, A, T, R, \gamma \rangle$ , chính sách  $\pi^*$  là chính sách tối ưu nếu

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} U(\tau),$$

trong đó,  $\mathbb{E}_{\tau \sim \pi} U(\tau)$  là kì vọng tổng điểm thưởng của quỹ đạo  $\tau$  được sinh ra từ chính sách  $\pi$ .

Điểm khác biệt cơ bản giữa học tăng cường và học có giám sát là học tăng cường không cần phải biết giá trị của hàm mục tiêu tại mỗi bước thời gian nên không thể trực tiếp huấn luyện mô hình như trong học có giám sát. Thêm vào đó, các quyết định của mô hình học tăng cường có thể ảnh hưởng đến phân phối các trạng thái dọc theo quỹ đạo.

### 8.1.2 Một số loại môi trường cơ bản của học tăng cường

Dựa vào tính chất riêng của các tập trạng thái  $S$ , tập hành động  $A$  và hàm chuyển  $T$  mà ta có một số loại phân biệt môi trường cơ bản sau:

- *Môi trường tất định và môi trường ngẫu nhiên*: trong môi trường tất định, hàm chuyển  $P(s'|s, a) = 1$  với duy nhất giá trị của trạng thái đến  $s'$  nào đó. Với các giá trị khác, của trạng thái đến

$\hat{s} \neq s'$ ,  $P(\hat{s}|s, a) = 0$ . Tính chất này tồn tại với mọi cặp trạng thái nguồn  $s$  và hành động  $a$ . Trong trường hợp môi trường không thoả mãn điều kiện này, ta gọi là môi trường ngẫu nhiên.

- *Quan sát đầy đủ và quan sát không đầy đủ*: Trong môi trường quan sát đầy đủ, trạng thái quan sát được của tác tử là toàn bộ thông tin về trạng thái của môi trường. Trong trường hợp quan sát không đầy đủ, tác tử chỉ nhìn được một phần của thông tin từ các trạng thái của môi trường và phải dựa vào đó để đưa ra quyết định hợp lý nhất có thể. Các mô hình được xét trong giáo trình này sử dụng định nghĩa về môi trường quan sát đầy đủ.
- *Hành động rời rạc và hành động liên tục*: Trong môi trường hành động rời rạc, tập hành động  $A$  tập hợp rời rạc (ví dụ: điều khiển nhân vật bằng các phím Lên, Xuống, Trái, Phải). Không mất tính tổng quát, ta có thể coi các hành động này là các số nguyên trong khoảng  $[0, N - 1]$ , với  $N = |A|$  là số lượng hành động có thể thực hiện. Trong trường hợp này, các chính sách cũng là các phân bố rời rạc, tương tự như việc huấn luyện các mô hình học máy phân lớp.

Ngược lại, khi tập các hành động có tính liên tục, chính sách là phân bố trên không gian liên tục (có thể một hoặc nhiều chiều). Ví dụ trong bài toán điều khiển xe tự lái, thông tin điều khiển gồm hướng và độ lớn của vận tốc là các số thực.

Bài toán học tăng cường có nhiều ứng dụng như: xe tự lái, các hệ thống trí tuệ nhân tạo cho trò chơi, các hệ thống mà kết quả của mô hình có tác động trực tiếp đến phân bố dữ liệu. Tùy vào bài toán cụ thể mà chúng ta sẽ chọn một trong các mô hình cơ bản trên để mô tả môi trường và xây dựng mô hình học tăng cường phù hợp.

## 8.2 Thuật toán tối ưu quy hoạch động

Học tăng cường có thể xem như là một bài toán tối ưu hóa trên quá trình Markov, thông qua việc đưa ra các hành động tối ưu tại mỗi trạng thái. Trong phần này chúng ta sẽ tìm hiểu về các phương pháp quy hoạch động cổ điển trong học tăng cường.

### 8.2.1 Giá trị tối ưu

Chúng ta bắt đầu bằng các định nghĩa về giá trị trạng thái, từ đó giải quyết bài toán học tăng cường bằng phương pháp lặp giá trị.

**Định nghĩa 8.7** (Giá trị trạng thái trong chính sách). Giá trị của một trạng thái  $s$  trong chính sách  $\pi$  là kì vọng của giá trị quỹ đạo  $\tau$  bắt đầu từ trạng thái  $s$  được sinh ra từ chính sách  $\pi$ :

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{\tau \sim \pi}[U(\tau) | s_0 = s] \\ &= \mathbb{E}_{\tau \sim \pi} \left[ \sum_{t=0, a_t \sim \pi(s_t)}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \middle| s_0 = s \right], \end{aligned}$$

trong đó,  $a_t \sim \pi(s_t)$  là hành động được chọn theo chính sách  $\pi$  tại thời điểm  $t$ ,  $s_t$  là trạng thái tại thời điểm  $t$ .

Ta có

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{\tau \sim \pi} \left[ \sum_{t=0, a_t \sim \pi(s_t)}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \middle| s_0 = s \right] \\ &= \sum_a \pi(a|s) \sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V^\pi(s')). \end{aligned} \quad (8.3)$$

Đây là *phương trình Bellman* cho giá trị trạng thái  $V^\pi(s)$ .

**Định nghĩa 8.8** (Giá trị tối ưu của một trạng thái). Giá trị tối ưu của trạng thái  $s$  là giá trị lớn nhất có thể đạt được của  $V^\pi(s)$

trong tất cả các chính sách  $\pi$ :

$$V^*(s) = \max_{\pi} V^{\pi}(s).$$

Giá trị tối ưu cũng thỏa mãn một phương trình Bellman:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V^*(s')). \quad (8.4)$$

Ta thấy công thức (8.4) có phần tương tự như công thức đệ quy của giá trị  $V^{\pi}(s)$  ở công thức (8.3). Điểm khác biệt duy nhất là trong quá trình tính toán  $V^*(s)$  chúng ta phải duyệt qua tất cả các hành động  $a$  có thể thực hiện từ trạng thái  $s$  và lựa chọn hành động đạt giá trị kì vọng lớn nhất.

Có thể sử dụng định nghĩa tương đương với Định nghĩa 8.6 cho chính sách tối ưu  $\pi^*$  như sau:

**Định nghĩa 8.9** (Chính sách tối ưu). Cho quá trình quyết định Markov với bộ tham số  $\langle S, A, T, R, \gamma \rangle$ , chính sách tối ưu  $\pi^*$  là chính sách đạt được giá trị tối ưu  $V^*(s)$  cho mọi trạng thái  $s \in S$ :

$$\forall s : V^{\pi^*}(s) = V^*(s).$$

### 8.2.2 Giá trị trạng thái - hành động

Tại mỗi trạng thái  $s$ , nếu ta thực hiện hành động  $a$  rồi sau đó tiếp tục sử dụng chính sách  $\pi$  để chọn các hành động tiếp theo, khi đó, giá trị thu được là

$$Q^{\pi}(s, a) = \sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V^{\pi}(s')) \quad (8.5)$$

Ngược lại, chúng ta cũng có đẳng thức liên hệ giữa  $V^{\pi}(s)$  và  $Q^{\pi}(s, a)$

$$V^{\pi}(s) = \sum_a \pi(a|s) Q^{\pi}(s, a).$$

Như vậy, phương trình Bellman của  $Q^\pi(s, a)$  có thể được viết lại như sau:

$$Q^\pi(s, a) = \sum_{s'} T(s, a, s') \left( R(s, a, s') + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s', a') \right). \quad (8.6)$$

Tương tự với hàm giá trị  $V^*$ , dựa trên ta có hàm tối ưu  $Q^*(s, a)$  có thể được tính qua  $V^*(s')$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V^*(s')).$$

Để ý rằng giá trị tối ưu  $V^*(s') = \arg \max_a Q^*(s', a)$ , ta thu được công thức truy hồi của  $Q^*$  như sau:

$$Q^*(s, a) = \sum_{s'} T(s, a, s') \left( R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right) \quad (8.7)$$

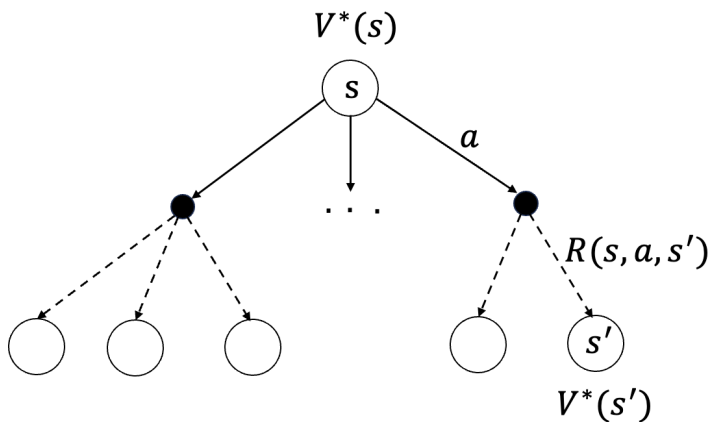
Các hàm  $Q^\pi$  và  $Q^*$  được gọi là hàm giá trị trạng thái - hành động hay *hàm giá trị Q*. Các hàm giá trị  $Q$  đánh giá giá trị của hành động  $a$  tại mỗi trạng thái  $s$ . Trong thực hành, các hàm giá trị  $Q$  thường được sử dụng để dễ dàng tìm chính sách tối ưu  $\pi^*$ .

$$\pi^*(s) = \arg \max_a Q^*(s, a) \quad (8.8)$$

### 8.2.3 Thuật toán lặp giá trị

Trong bài toán tối ưu quyết định Markov với tập tham số cho trước, mục tiêu là tìm chính sách tối ưu  $\pi^*$ , tương đương với việc xác định giá trị tối ưu  $V^*(s)$  cho mọi trạng thái  $s$ . Theo Định nghĩa 8.8, việc tính  $V^*(s)$  yêu cầu xét toàn bộ không gian chính sách  $\pi$ , điều này không khả thi trong thực tế.

Thay vào đó, ta dùng công thức (8.4), cho phép tính  $V^*(s)$  dựa trên giá trị  $V^*(s')$  của các trạng thái kề  $s'$  mà  $s$  có thể chuyển đến.



**Hình 8.2:** Mô hình tính toán giá trị tối ưu  $V^*(s)$  sử dụng công thức đệ quy

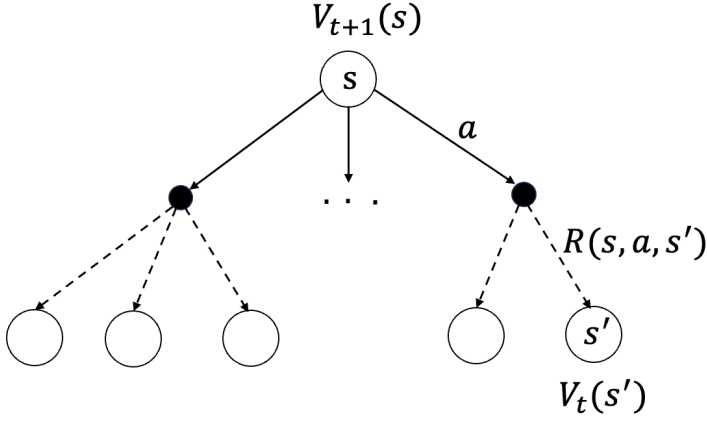
Cụ thể, nếu biết  $V^*(s')$  với mọi  $s'$  liên kề, ta có thể chọn hành động tối ưu bằng cách so sánh kỳ vọng giá trị của các hành động. Mô hình này được minh họa trong Hình 8.2.

Tuy nhiên, để áp dụng công thức đệ quy, cần biết trước  $V^*(s')$  để tính  $V^*(s)$ , và ngược lại, cần  $V^*(s)$  để tính  $V^*(s')$  nếu có thể chuyển ngược từ  $s'$  về  $s$ . Điều này dẫn đến sự phụ thuộc lẫn nhau giữa các trạng thái và giả định đồ thị trạng thái không có chu trình là không thực tế.

Để giải quyết vấn đề phụ thuộc lẫn nhau giữa  $s$  và các trạng thái  $s'$  có thể đi đến từ  $s$ , ta có thể sử dụng phương pháp lặp giá trị. Ý tưởng chính là sử dụng quy hoạch động để xấp xỉ giá trị tối ưu  $V^*(s)$  thông qua quá trình lặp.

Đặt số bước  $t$  là tham số trong tối ưu quy hoạch động. Từ định nghĩa của  $V^*(s)$ , ta mở rộng thành  $V_t(s)$ , là giá trị tối ưu đạt được sau  $t$  bước bắt đầu từ trạng thái  $s$ .

**Định nghĩa 8.10** (Giá trị tối ưu của một trạng thái sau  $t$  bước). Giá trị tối ưu sau  $t$  bước là giá trị kỳ vọng lớn nhất đạt được sau  $t$



**Hình 8.3:** Mô hình tính toán giá trị  $V_{t+1}$  sử dụng giá trị  $V_t$

bước theo một chính sách  $\pi$ :

$$V_t(s) = \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[ \sum_{i=0}^{t-1} \gamma^i R(s_i, a_i = \pi(s_i), s_{i+1}) \mid s_0 = s \right].$$

Tương tự định nghĩa đệ quy của  $V^*$ , ta có công thức đệ quy cho  $V_{t+1}$  dựa trên  $V_t$ :

$$V_{t+1}(s) = \max_a \sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V_t(s')). \quad (8.9)$$

Công thức này gọi là “nhìn về phía trước một bước” vì giá trị  $V_{t+1}(s)$  được tính từ  $V_t(s')$  của các trạng thái  $s'$  có thể đạt được từ  $s$ . Mô hình tính toán minh họa trong Hình 8.3.

Từ Hình 8.3, ta thấy  $V_{t+1}(s)$  được tính mà không cần biết trước giá trị  $V_{t+1}(s')$ , nhờ vậy loại bỏ được sự phụ thuộc lẫn nhau giữa các trạng thái.

Việc chia bài toán tối ưu tổng thể thành các bài toán con đơn giản là nguyên lý cốt lõi của quy hoạch động. Dựa trên công thức (8.9), ta xây dựng được mã giả như trình bày trong Thuật toán 8.1.

**Thuật toán 8.1** Thuật toán lặp giá trị

---

```

1: procedure VALUEITERATION(S, A, T, R, γ)
2: for all $s \in S$ do
3: $V_0(s) \leftarrow 0$
4: end for
5: repeat
6: for all $s \in S$ do
7: $V_{t+1}(s) \leftarrow \max_{a \in A} \sum_{s'} T(s, a, s') \cdot (R(s, a, s') + \gamma \cdot V_t(s'))$
8: end for
9: until $V_{t+1}(s)$ hội tụ với mọi $s \in S$
10: return $V^*(s) \leftarrow V_{t+1}(s)$
11: end procedure

```

---

Điều kiện dừng của vòng lặp tại Bước 5 có thể là số vòng lặp  $t$  đủ lớn hoặc khi chênh lệch giữa hai lần cập nhật nhỏ hơn một ngưỡng  $\epsilon$ , tức là  $\forall s : |V_{t+1}(s) - V_t(s)| < \epsilon$ . Để đảm bảo tính tối ưu, cần bảo đảm rằng  $V_t(s)$  hội tụ về  $V^*(s)$  khi  $t \rightarrow \infty$ . Định lý sau trình bày điều kiện hội tụ của thuật toán lặp giá trị.

**Định lý 8.11** (Tính hội tụ của thuật toán lặp giá trị). *Xét một quá trình quyết định Markov với tập trạng thái hữu hạn  $S$  và tập hành động rời rạc hữu hạn  $A$ . Giả sử tồn tại giá trị tối ưu  $V^*(s)$  hữu hạn với mọi  $s \in S$ . Khi đó, dãy giá trị  $V_t(s)$  sinh ra từ thuật toán lặp giá trị hội tụ về  $V^*(s)$  khi  $t \rightarrow \infty$ .*

*Chứng minh:* Xét trường hợp đơn giản với tập trạng thái  $S$  và hành động  $A$  là rời rạc và hữu hạn. Giả sử hệ số chiết khấu  $\gamma$  thoả mãn  $0 < \gamma < 1$  để đảm bảo tổng phần thưởng hữu hạn. Sắp xếp các trạng thái  $s \in S$  theo thứ tự bất kỳ, khi đó  $V^* \in \mathbb{R}^{|S|}$  là véc-tơ giá trị tối ưu và tương tự  $V_0, V_1, \dots, V_t \in \mathbb{R}^{|S|}$  là dãy véc-tơ giá trị trong quá trình lặp.

Thuật toán lặp giá trị có thể viết dưới dạng toán tử Bellman



$\mathcal{T} : \mathbb{R}^{|S|} \rightarrow \mathbb{R}^{|S|}$  như sau:

$$(\mathcal{T}V)(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]. \quad (8.10)$$

Véc-tơ giá trị tối ưu  $V^*$  là điểm cố định của  $\mathcal{T}$ , tức là  $\mathcal{T}V^* = V^*$ . Thuật toán lặp giá trị được định nghĩa đệ quy bởi:

$$V_{t+1} = \mathcal{T}V_t. \quad (8.11)$$

Để chứng minh hội tụ, ta chứng minh  $\mathcal{T}$  là toán tử co với hệ số  $\gamma$  theo chuẩn  $\infty$ :

$$\|V - V'\|_\infty = \max_{s \in S} |V(s) - V'(s)|. \quad (8.12)$$

Xét  $V, V' \in \mathbb{R}^{|S|}$  bất kỳ và  $s \in S$ , ta có:

$$|(\mathcal{T}V)(s) - (\mathcal{T}V')(s)| \leq \max_a \left| \sum_{s'} T(s, a, s') \gamma (V(s') - V'(s')) \right| \quad (8.13)$$

$$\leq \gamma \max_a \sum_{s'} T(s, a, s') |V(s') - V'(s')| \quad (8.14)$$

$$\leq \gamma \max_{s'} |V(s') - V'(s')| = \gamma \|V - V'\|_\infty. \quad (8.15)$$

Dòng (8.13) dùng tính chất:  $|\max f(a) - \max g(a)| \leq \max |f(a) - g(a)|$ . Dòng (8.14) áp dụng bất đẳng thức tam giác. Dòng (8.15) dùng bất đẳng thức Jensen với  $T(s, a, s')$  là phân phối xác suất. Suy ra:

$$\|\mathcal{T}V - \mathcal{T}V'\|_\infty \leq \gamma \|V - V'\|_\infty.$$

Vậy  $\mathcal{T}$  là toán tử co với hệ số  $\gamma < 1$  trên không gian metric  $(\mathbb{R}^{|S|}, \|\cdot\|_\infty)$ . Theo định lý điểm cố định Banach, tồn tại duy nhất điểm cố

định  $V^*$  của  $\mathcal{T}$  và với mọi  $V_0 \in \mathbb{R}^{|S|}$ , dãy  $V_t$  xác định bởi (8.11) hội tụ về  $V^*$  khi  $t \rightarrow \infty$ .  $\square$

Công thức (8.9) đóng vai trò trung tâm trong các bài toán tối ưu trên các qui trình Markov hoặc học tăng cường. Với lập luận tương tự, ta cũng có thể áp dụng công thức này để tính giá trị tối ưu  $Q^*(s, a)$  cho một trạng thái  $s$  và hành động  $a$ , có xét đến số bước đi  $t$

$$Q_{t+1}(s, a) = \sum_{s'} T(s, a, s') (R(s, a, s') + \gamma \max_{a'} Q_t(s', a')). \quad (8.16)$$

### 8.2.4 Thuật toán lặp chính sách

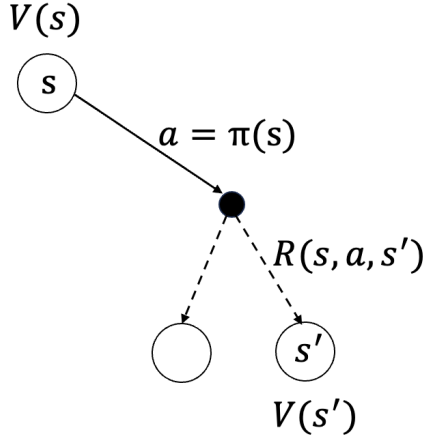
Theo Định lý 8.11, dãy  $V_t$  hội tụ về giá trị tối ưu  $V^*$ . Tuy nhiên, trong công thức cập nhật  $V_{t+1}(s)$ , ta phải duyệt qua toàn bộ các hành động  $a$  khả dĩ tại trạng thái  $s$  theo công thức (8.4). Điều này gây ra chi phí tính toán cao nếu không gian trạng thái  $S$  và hành động  $A$  lớn.

Để giảm chi phí, ta có thể sử dụng một chính sách  $\pi$  để chọn hành động  $a$  thay vì xét tất cả. Cách tiếp cận này cho phép ta đồng thời tối ưu cả giá trị  $V^*$  và chính sách  $\pi^*$ .

Xét trường hợp chính sách  $\pi$  là xác định, tức là với mỗi trạng thái  $s$  tồn tại một hành động duy nhất  $a = \pi(s)$ . Khi thay vào công thức (8.4), ta thu được công thức đệ quy cho giá trị  $V^\pi(s)$  như sau:

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') (R(s, \pi(s), s') + \gamma V^\pi(s')). \quad (8.17)$$

Hình 8.4 thể hiện mô hình tính toán giá trị  $V^\pi(s)$  với chính sách  $\pi$ . So với mô hình tính giá trị tối ưu  $V^*(s)$  trong Hình 8.2, việc tính  $V^\pi(s)$  đơn giản hơn vì không cần tối ưu hóa theo tất cả hành động  $a$ . Tuy nhiên, vẫn cần loại bỏ sự phụ thuộc lẫn nhau giữa  $s$  và  $s'$  bằng cách đưa vào tham số số bước  $t$ .



**Hình 8.4:** Mô hình tính toán giá trị  $V^\pi(s)$  với chính sách  $\pi$

Ta định nghĩa  $V_t^\pi(s)$  là giá trị tại trạng thái  $s$  sau  $t$  bước theo chính sách  $\pi$  và cập nhật theo công thức đệ quy:

$$V_{t+1}^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_t^\pi(s')]. \quad (8.18)$$

Tương tự như thuật toán lặp giá trị, công thức trên cho phép  $V_t^\pi(s)$  hội tụ về  $V^\pi(s)$  khi  $t \rightarrow \infty$ , bắt đầu từ giá trị khởi tạo  $V_0$  bất kỳ. Sau khi đã tính được  $V^\pi(s)$ , ta có thể cập nhật lại chính sách  $\pi$  theo:

$$\pi(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^\pi(s')]. \quad (8.19)$$

Kết hợp hai công thức (8.18) và (8.19), ta thu được thuật toán lặp theo chính sách (Thuật toán 8.2). Thuật toán dừng khi chính sách  $\pi$  không thay đổi sau vòng lặp cập nhật  $V^\pi$ . Trong các điều kiện tương tự Định lý 8.11, thuật toán hội tụ về chính sách tối ưu  $\pi^*$  và giá trị tối ưu  $V^*(s)$ . Phần chứng minh chi tiết được để lại như một bài tập ở cuối chương.

Thuật toán lặp theo giá trị và lặp theo chính sách là hai phương pháp nền tảng cho các thuật toán tối ưu trong học tăng cường. Dựa

**Thuật toán 8.2** Thuật toán lặp chính sách

---

```

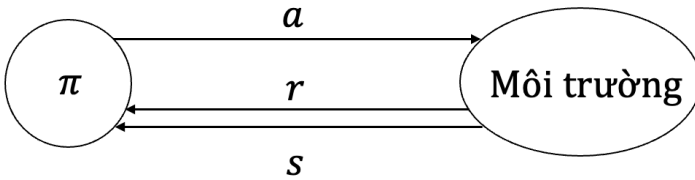
1: procedure POLICYITERATION(S, A, T, R, γ)
2: Khởi tạo chính sách $\pi(s)$ ngẫu nhiên với mọi $s \in S$
3: repeat ▷ Đánh giá chính sách
4: repeat
5: for all $s \in S$ do
6: $V_{t+1}(s) \leftarrow \sum_{a \sim \pi(\cdot|s)} \sum_{s'} T(s, a, s')(R(s, a, s') + \gamma \cdot$
 $V_t(s'))$
7: end for
8: until $V_{t+1}(s)$ hội tụ với mọi s ▷ Cải thiện chính sách
9: $\text{check} \leftarrow \text{true}$
10: for all $s \in S$ do
11: $\pi_{\text{old}} \leftarrow \pi(s)$
12: $\pi(s) \leftarrow \arg \max_{a \in A} \sum_{s'} T(s, a, s')(R(s, a, s') + \gamma \cdot V(s'))$
13: if $\pi(s) \neq \pi_{\text{old}}$ then
14: $\text{check} \leftarrow \text{false}$
15: end if
16: end for
17: until $\text{check} = \text{true}$
18: return π
19: end procedure

```

---

trên quy hoạch động, ta có thể loại bỏ sự phụ thuộc lẫn nhau trong mô hình và xác định thứ tự tính toán hợp lý để xấp xỉ giá trị tối ưu.

Khi số vòng lặp đủ lớn, Định lý 8.11 đảm bảo thuật toán hội tụ về giá trị  $V^*$  cần tìm. Các trình bày trong phần này chỉ áp dụng cho bài toán tối ưu quá trình Markov với miền trạng thái  $S$  và hành động  $A$  là hữu hạn, rời rạc. Với các miền hành động liên tục, cần áp dụng kỹ thuật rời rạc hoá hoặc xấp xỉ để chuyển bài toán về dạng có thể xử lý bằng các thuật toán trên.



**Hình 8.5:** Mô hình tương tác giữa tác tử và môi trường trong học tăng cường

### 8.3 Học tăng cường và phương pháp học hàm Q

Trong mục 8.2, ta đã xây dựng hai thuật toán: lặp giá trị và lặp theo chính sách dựa trên phương pháp quy hoạch động để giải bài toán tối ưu quyết định trên quá trình Markov. Tuy nhiên, trong thực tế, hàm chuyển trạng thái  $T(s, a, s')$  và hàm phần thưởng  $R(s, a, s')$  thường không được biết trước hoặc không gian trạng thái quá lớn để lưu trữ đầy đủ các giá trị này.

Phần này giới thiệu thiết lập tổng quát hơn (Hình 8.5), trong đó tác tử không biết trước  $T$  và  $R$ , mà phải học thông qua tương tác với môi trường. Cụ thể, tác tử thực hiện hành động  $a$ , sau đó nhận được trạng thái mới  $s'$  và phần thưởng riêng lẻ  $r = r(s, a, s')$ . Kể từ đây, ta dùng  $r$  để chỉ phần thưởng thay vì hàm  $R(s, a, s')$  tổng quát.

Nếu tập hành động là rời rạc, tác tử biết trước số lượng hành động và hành động được biểu diễn bởi số nguyên từ 0 đến  $|A| - 1$ . Nếu hành động là liên tục, tác tử biết số chiều  $d$  của hành động và mỗi hành động là một véc-tơ trong  $\mathbb{R}^d$ .

Vì thông tin môi trường—bao gồm hàm chuyển trạng thái  $T$  và hàm phần thưởng  $R$ —không được cung cấp cho tác tử, nên hai Thuật toán 8.1 và 8.2 không thể áp dụng được. Trong thiết lập này, mô hình học tăng cường phải đồng thời thực hiện hai nhiệm vụ: khám phá môi trường và khai thác thông tin thu được để ước

lượng giá trị tối ưu một cách hiệu quả.

Để giải bài toán tối ưu, ta có thể chia các phương pháp tiếp cận thành hai nhóm chính: (i) Học mô hình quyết định Markov và (ii) Học hàm  $Q$  thông qua kinh nghiệm.

### 8.3.1 Học mô hình quá trình quyết định Markov

Ta có thể sử dụng phương pháp thực nghiệm để xấp xỉ hàm chuyển  $T(s, a, s')$  và hàm phần thưởng  $R(s, a, s')$ . Với môi trường có tập hành động rời rạc  $A$ , giả sử tại thời điểm  $t$ , với mỗi cặp trạng thái–hành động  $(s, a)$ , ta thu được  $m$  mẫu kinh nghiệm từ môi trường:

$$H = \{(s, a, s_1, r_1), (s, a, s_2, r_2), \dots, (s, a, s_m, r_m)\}. \quad (8.20)$$

Cách thu thập các mẫu trong  $H$  thường được gọi là chính sách khám phá. Một chính sách cơ bản là chọn hành động theo phân phối đều:

$$a \sim p(a|s) = \frac{1}{|A|}. \quad (8.21)$$

Trong đó, hành động  $a$  được chọn ngẫu nhiên từ tập  $A$  với xác suất như nhau. Chính sách ngẫu nhiên đảm bảo việc khám phá đầy đủ không gian trạng thái–hành động. Không mất tính tổng quát, phần tiếp theo giả sử các chính sách là ngẫu nhiên.

Ta có thể xấp xỉ hàm chuyển và phần thưởng như sau:

$$\hat{T}(s, a, s') = \frac{\sum_j \mathbb{I}(s_j = s')}{m}, \quad (8.22)$$

$$\hat{R}(s, a, s') = \frac{\sum_j \mathbb{I}(s_j = s') r_j}{\sum_j \mathbb{I}(s_j = s')}, \quad (8.23)$$

trong đó  $\mathbb{I}(s_j = s')$  là hàm chỉ thị, bằng 1 nếu  $s_j = s'$  và 0 nếu ngược lại. Khi số lượng mẫu  $m$  đủ lớn,  $\hat{T}$  và  $\hat{R}$  sẽ xấp xỉ gần đúng các giá trị thật.

Tuy nhiên, phương pháp này có hai nhược điểm chính:

- Sai lệch giữa giá trị thật và xấp xỉ có thể gây sai số lớn trong việc tìm chính sách tối ưu.
- Để xấp xỉ chính xác, cần thu thập tập mẫu lớn. Điều này khó đạt được, đặc biệt trong môi trường không quan sát được toàn phần hoặc có miền hành động liên tục.

### 8.3.2 Học hàm Q thông qua kinh nghiệm

Thay vì phải trải qua bước trung gian là ước lượng hàm chuyển trạng thái  $T(s, a, s')$  và hàm phần thưởng  $R(s, a, s')$ , chúng ta có thể cập nhật trực tiếp giá trị  $V$  hoặc  $\pi$  dựa trên các mẫu kinh nghiệm  $(s, a, s', r)$ . Để làm được điều này, chúng ta tiến hành xấp xỉ giá trị  $V^\pi(s)$  mỗi khi nhận được một mẫu kinh nghiệm  $(s, a, s', r)$ . Chúng ta có thể viết lại công thức cập nhật giá trị  $V^\pi(s)$  từ công thức (8.17) theo cách diễn đạt kì vọng theo các mẫu kinh nghiệm như sau:

$$V^\pi(s) = \mathbb{E}_{(s,a,s',r) \sim \pi} [r + \gamma V^\pi(s')] \quad (8.24)$$

Hay nói theo một cách khác, giá trị ước lượng  $x = r + \gamma V^\pi(s')$  trên là một ước lượng không lệch của  $V^\pi(s)$ . Trong trường hợp chúng ta cố định  $s$ , và thực hiện hành động  $a \sim \pi(\cdot|s)$  để thu được dãy kinh nghiệm

$$D_s = \{(s, a_1, s'_1, r_1), (s, a_2, s'_2, r_2), \dots, (s, a_N, s'_N, r_N)\}.$$

Với giả thiết là các giá trị  $V^\pi(s'_i)$  đã được biết trước, ta có thể tính được các giá trị mục tiêu  $x_i$  với  $i = 1, \dots, N$  như sau:

$$x_i = r_i + \gamma V^\pi(s'_i)$$

Từ các giá trị  $x_i$  này, ta sẽ có thể ước lượng giá trị  $V^\pi(s)$  thông qua giá trị trung bình của các  $x_i$

$$V^\pi(s) = \frac{1}{N} \sum_{i=1}^N x_i \quad (8.25)$$

Theo luật số lớn, khi  $N$  đủ lớn, trung bình của các  $x_i$  sẽ hội tụ về giá trị thật  $V^\pi(s)$ . Từ công thức (8.25) ta có thể viết lại để được công thức cập nhật truy hồi cho giá trị  $V^\pi(s)$  thông qua  $x_N$  như sau:

$$V^\pi(s) = \left(1 - \frac{1}{N}\right) \frac{1}{N-1} \sum_{i=1}^{N-1} x_i + \frac{1}{N} x_N$$

Ta có thể đặt  $\alpha = 1/N$  và  $V^\pi(s) = \frac{1}{N-1} \sum_{i=1}^{N-1} x_i$  và viết lại công thức trên thành

$$V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha x_N \quad (8.26)$$

Cách cập nhật truy hồi được trình bày trong công thức (8.26) có thể được hiểu như là một phương pháp lặp giá trị và tương thích với thiết lập học tăng cường ở Hình 8.5. Ngoài ra, chúng ta có thể nhóm (8.26) lại để có dạng tương tự với công thức cập nhật của phương pháp Xuống đồi bằng đạo hàm như sau:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha(x_N - V^\pi(s)) \quad (8.27)$$

với  $\alpha$  là tốc độ học và  $x_N$  là giá trị mục tiêu dựa trên dữ liệu được thu thập.

Tuy nhiên, trong thực tế, chúng ta không thể biết trước giá trị  $V^\pi(s'_i)$ . Để giải quyết vấn đề này, chúng ta có thể sử dụng một cách tiếp cận tương tự như trong quy hoạch động. Giả sử tại thời điểm  $t$ , gọi  $V_t^\pi$  là giá trị ước lượng của hàm  $V^\pi$  tại thời điểm  $t$  và đã



biết với trạng thái  $s \in S$ . Tại thời điểm  $t + 1$  ta nhận được kinh nghiệm  $(s, a, s', r)$  bằng cách thực hiện hành động  $a = \pi(\cdot|s)$ , giá trị  $V_{t+1}^\pi(s)$  có thể được cập nhật theo công thức:

$$v = r + \gamma V_t^\pi(s')$$

$$V_{t+1}^\pi(s) = V_t^\pi(s) + \alpha(v - V_t^\pi(s))$$

Trong đó, giá trị cập nhật  $v$  được xem như là giá trị mục tiêu của  $V^\pi(s)$ . Tham số  $\alpha$  là tốc độ học, xác định mức độ điều chỉnh của giá trị ước lượng  $V_t^\pi(s)$  dựa trên sai số giữa dự đoán  $V_t^\pi(s)$  và giá trị mới  $v$ .

Cách tiếp cận này tuy có thể xấp xỉ được giá trị  $V_\pi(s)$  theo các mẫu kinh nghiệm nhận được nếu số lượng các mẫu kinh nghiệm đủ nhiều. Tuy nhiên, nếu ta dựa theo Thuật toán lặp giá trị 8.2 thì việc trích ra chính sách cập nhật từ  $V$  theo công thức (8.19) còn thiếu dữ kiện là hàm chuyển trạng thái  $T(s, a, s')$  và hàm phần thưởng  $R(s, a, s')$  của môi trường. Do đó, để thực hiện được bước lựa chọn hành động tối ưu tại trạng thái  $s$  chúng ta cần cách học thông tin tổng quát hơn. Thay vì xấp xỉ giá trị  $V^\pi(s)$ , chúng ta sẽ xấp xỉ hàm  $Q^\pi(s, a)$ . Theo định nghĩa, hàm  $Q$  làm việc trên tập trạng thái và hành động, do đó từ giá trị hiện tại của  $Q_t^\pi(s, a)$  chúng ta có thể dễ dàng xác định được giá trị hành động ứng với chính sách hiện tại cũng như cập nhật chính sách mới, tương tự với thuật toán lặp giá trị. Bằng việc sử dụng hàm  $Q_t^\pi(s, a)$ , tại thời điểm  $t$  bất kì, chúng ta có thể lấy ra được giá trị tối ưu và hành động tối ưu nhất tại trạng thái  $s$  theo công thức:

$$V_t^\pi(s) = \max_{a'} Q_t^\pi(s, a') \quad (8.28)$$

$$\pi_t(s) = \arg \max_{a'} Q_t^\pi(s, a') \quad (8.29)$$

Chính sách  $\pi$  được tính theo công thức (8.29) hay được gọi là chính sách tham lam theo hàm  $Q$  hoặc gọi tắt là chính sách tham lam.

Vì hàm  $Q_t^\pi(s, a)$  có thể định nghĩa trực tiếp chính sách  $\pi$  tại trạng thái  $s$ , nên trong trường hợp không có nhầm lẫn ta có thể dùng kí hiệu  $Q_t(s, a)$ .

Giả sử ở bước cập nhật thứ  $t$ , tác tử nhận được kinh nghiệm  $(s, a, s', r)$  từ môi trường, chúng ta có thể áp dụng công thức tham lam (8.28) của  $V_t^\pi(s')$  để cập nhật hàm  $Q_{t+1}(s, a)$  như sau:

$$q = r + \gamma \max_{a'} Q_t(s', a') \quad (8.30)$$

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha (q - Q_t(s, a)) \quad (8.31)$$

Tuy nhiên, trong thực tế thì giá trị  $Q_t(s', a')$  bị ảnh hưởng bởi số lần tác tử đi qua trạng thái  $s'$  và thực hiện hành động  $a'$ . Để mỗi giá trị  $Q_t(s', a')$  không bị sai số nhiều, số lần tác tử đi đến trạng thái  $s'$  và thực hiện hành động  $a'$  cần phải đủ nhiều. Tính chất này gần giống với việc ước lượng hàm chuyển và hàm phần thưởng ở (8.20). Hay nói một cách khác, để ước lượng hàm  $Q(s, a)$  được chính xác thì tác tử phải thu thập thông tin đủ nhiều thông qua các tương tác với môi trường. Để đảm bảo tác tử có thể thu thập đủ thông tin khi tương tác với môi trường, chúng ta tiến hành thêm yếu tố ngẫu nhiên vào chính sách tham lam ở công thức (8.29) qua tham số  $\varepsilon$  như sau:

$$a = \begin{cases} \arg \max_{a'} Q_t(s, a') & \text{với xác suất } 1 - \varepsilon, \\ \text{chọn ngẫu nhiên từ } A & \text{với xác suất } \varepsilon. \end{cases} \quad (8.32)$$

Công thức (8.32) thường viết tắt là  $\varepsilon - Q(s, \cdot)$  được gọi là *chính sách  $\varepsilon$ -tham lam* của hàm  $Q$ . Tính tham lam được thể hiện ở chính sách tác tử sẽ chọn hành động tối ưu nhất hiện tại khi đang ở trạng thái  $s$  với xác suất  $1 - \varepsilon$ . Trong quá trình huấn luyện, khi tác tử chưa thu thập đủ nhiều thông tin từ môi trường, thì việc chọn hành động tối ưu nhất theo hàm  $Q_t(s, a')$  hiện tại đồng nghĩa với việc lựa chọn tham lam. Để đảm bảo tính khám phá của chính sách, tác

tử sẽ luôn có một xác suất nhất định  $\varepsilon > 0$  để lựa chọn hành động ngẫu nhiên.

---

**Thuật toán 8.3** Thuật toán SARSA dựa trên chính sách tham lam  $\varepsilon$

---

```

1: procedure SARSA($\alpha, \gamma, \varepsilon$)
2: for all $(s, a) \in S \times A$ do
3: Khởi tạo $Q(s, a)$ tùy ý
4: end for
5: while chưa đạt điều kiện dừng do
6: Khởi tạo trạng thái ban đầu s
7: Chọn $a \sim \varepsilon - Q(s, \cdot)$
8: while s không phải trạng thái kết thúc do
9: Thực hiện hành động a , nhận được phần thưởng r
 và trạng thái mới s'
10: Chọn $a' \sim \varepsilon - Q(s', \cdot)$
11: Ước lượng giá trị mục tiêu: $q = r + \gamma Q(s', a')$
12: Cập nhật:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (q - Q(s, a))$$

13: $s \leftarrow s', a \leftarrow a'$
14: end while
15: end while
16: end procedure

```

---

Kết hợp các công thức (8.30), (8.31), và (8.32), chúng ta có thể xây dựng Thuật toán 8.3. Thuật toán thường có tên gọi là SARSA vì hàm mục tiêu  $Q$  được cập nhật theo bộ giá trị  $(s, a, r, s', a')$  bao gồm trạng thái hiện tại, hành động hiện tại, phần thưởng nhận được, trạng thái kế tiếp và hành động kế tiếp. Trong phần trình bày ở Thuật toán 8.3, chúng ta bỏ qua tham số  $t$  và chỉ sử dụng ký hiệu  $Q(s, a)$ . Cách tiếp cận này trực tiếp cập nhật hàm  $Q$  theo mỗi mẫu kinh nghiệm  $(s, a, r, s', a')$ . Công thức (8.30) có thể được hiểu như là ước lượng giá trị  $q$  là giá trị mục tiêu hiện

tại của  $Q(s, a)$ . Sau đó, giá trị này được sử dụng để cập nhật giá trị  $Q(s, a)$  với tốc độ học  $\alpha$ . Chúng ta thấy cách tiếp cận này giống với cách tiếp cận của tối ưu hoá theo đạo hàm. Trong phần tiếp theo, cách tiếp cận này sẽ được mở rộng để học xấp xỉ hàm  $Q$  thông qua một mô hình hồi quy tuyến tính.

## 8.4 Các thuật toán học xấp xỉ

Phương pháp học theo lặp giá trị và lặp theo chiến thuật chỉ khả thi khi làm việc với bài toán có không gian nhỏ. Chẳng hạn trong phương pháp lặp theo giá trị, yêu cầu về bộ nhớ cần thiết phải lưu lại được mảng giá trị  $V(s)$  với mọi  $s \in S$ . Điều này là không thực tế với những bài toán có không gian trạng thái lớn như cờ vua hoặc là không gian trạng thái là miền liên tục. Với phương pháp lặp theo chiến thuật hoặc phương pháp SARSA, việc lưu lại mảng giá trị chiến thuật  $\pi$  hoặc hàm  $Q(s, a)$  cũng gặp những khó khăn tương tự.

Để tăng tính ứng dụng của các thuật toán đã trình bày, chúng ta có thể mô hình hoá các hàm chính sách hoặc giá trị qua phương pháp học xấp xỉ. Thay vì phải lưu trữ minh giá trị  $V(s)$  hoặc  $Q(s, a)$ , ta sử dụng một hàm  $f$  để ánh xạ trạng thái  $s$  hoặc cặp trạng thái-hành động  $(s, a)$  về miền giá trị của  $V(s)$  hoặc  $Q(s, a)$  tương ứng.

Có thể có nhiều thể hiện khác nhau của hàm  $f$  nhưng đơn giản nhất là mô hình hồi quy tuyến tính có đầu vào là véc-tơ đặc trưng của trạng thái  $s$  hoặc cặp trạng thái-hành động  $(s, a)$ .

**Định nghĩa 8.12** (Ánh xạ trích chọn đặc trưng). Một ánh xạ trích chọn đặc trưng  $\phi$  là một hàm ánh xạ từ miền trạng thái  $S$  và miền hành động  $A$  về miền véc-tơ đặc trưng có  $d$  chiều  $\mathbb{R}^d$ .

$$\phi : S \times A \rightarrow \mathbb{R}^d. \quad (8.33)$$

Với đầu vào là cặp giá trị  $(s, a)$ , giả sử  $\phi(s, a) = [x_1, x_2, \dots, x_d]$  với  $x_1, x_2, \dots, x_d$  là các đặc trưng được lựa chọn phù hợp. Ở bước học theo thuật toán lặp giá trị, ta có thể mô hình hoá hàm giá trị  $Q(s, a)$  bằng một hàm hồi quy tuyến tính

$$Q_w(s, a) = (f_w \circ \phi)(s, a) = w_0 + w_1 x_1 + \dots + w_d x_d \quad (8.34)$$

được đặc trưng bởi véc-tơ trọng số  $w \in \mathbb{R}^{d+1}$  như sau:

$$w = [w_0, w_1, w_2, \dots, w_d] \quad (8.35)$$

trong đó,  $w_1, w_2, \dots, w_d$  là các trọng số tuyến tính và  $w_0$  là hệ số tự do.

Để cập nhật hàm  $Q_w(s, a)$ , chúng ta sẽ định nghĩa hàm mục tiêu  $\mathcal{L}(w)$  và sử dụng phương pháp Xuống đồi bằng đạo hàm để cập nhật trọng số  $w$ . Dựa trên công thức (8.30), ta có thể viết lại hàm mục tiêu theo mẫu kinh nghiệm  $(s, a, r, s', a')$  như sau:

$$y = r + \gamma \max_{a'} Q_w(s', a') \quad (8.36)$$

$$L(w) = \frac{1}{2} [y - Q_w(s, a)]^2 \quad (8.37)$$

Xem giá trị mục tiêu  $y$  là hằng số đối với tham số  $w$ , ta có đạo hàm của hàm mục tiêu  $L(w)$  theo trọng số  $w$  như sau:

$$\nabla_w L(w) = -[y - Q_w(s, a)] \nabla_w Q_w(s, a) \quad (8.38)$$

Dựa trên kết quả này, chúng ta có thể áp dụng phương pháp Xuống đồi bằng đạo hàm để cập nhật  $w$ . Kết hợp công thức (8.38) với Thuật toán 8.3, ta có được thuật toán SARSA với xấp xỉ hàm giá trị (Thuật toán 8.4).

Phương pháp xấp xỉ giá trị được trình bày có thể áp dụng chung cho các phương pháp lặp giá trị và lặp chính sách. Có một chú ý nhỏ là việc sử dụng giá trị mục tiêu để cập nhật trọng số  $w$  trong

---

**Thuật toán 8.4** Thuật toán SARSA với hàm xấp xỉ tuyến tính  $Q$ 

---

```
1: procedure SARSA($\alpha, \gamma, \varepsilon$)
2: Khởi tạo $Q_w(s, a)$ với w tùy ý
3: while chưa đạt điều kiện dừng do
4: Khởi tạo trạng thái ban đầu s
5: Chọn $a \sim \varepsilon - Q_w(s, \cdot)$
6: while s không phải trạng thái kết thúc do
7: Thực hiện hành động a , nhận được phần thưởng r
 và trạng thái mới s'
8: Chọn $a' \sim \varepsilon - Q_w(s', \cdot)$
9: Tính giá trị mục tiêu y theo (8.36)
10: Tính hàm mục tiêu $L(w)$ theo (8.37)
11: Tính đạo hàm $\nabla_w L(w)$ theo (8.38)
12: Cập nhật trọng số w : $w \leftarrow w - \alpha \nabla_w L(w)$
13: $s \leftarrow s', a \leftarrow a'$
14: end while
15: end while
16: end procedure
```

---

công thức (8.38) thường gặp vấn đề về nhiễu và độ lệch. Các sai số này dẫn đến tính hội tụ của Thuật toán 8.4 không được đảm bảo so với thuật toán lặp giá trị cơ bản. Trong các bài toán thực tế, việc sử dụng hàm tuyến tính khá đơn giản và có thể dẫn đến việc không mô hình được các phân bố phức tạp. Điều này dẫn đến việc mở rộng thành phương pháp học sâu tăng cường với việc sử dụng mạng học sâu để thực hiện đồng thời hai tác vụ trích chọn đặc trưng của  $\phi$  và hàm tuyến tính  $f$ .

## 8.5 Áp dụng học sâu cho học tăng cường

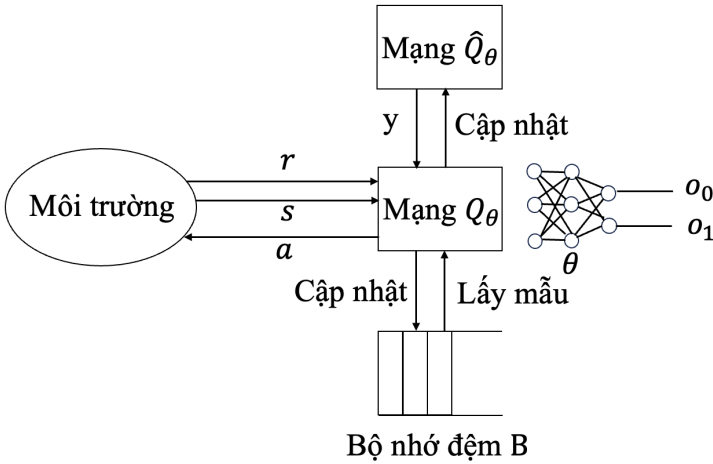
### 8.5.1 Mạng học sâu cho hàm giá trị $Q$

Chúng ta tiếp tục mở rộng cách tiếp cận xấp xỉ bằng việc thay thế hàm trích chọn đặc trưng  $\phi$  và mô hình hồi quy tuyến tính  $Q_w(s, a)$

trong công thức (8.34) bằng một mạng học sâu  $Q_\theta$ . Mạng học sâu  $Q_\theta$  có đầu vào là trạng thái  $s$  và có  $|A|$  đầu ra tương ứng với các giá trị  $Q_\theta(s, a)$  với  $\theta$  là bộ tham số của mạng học sâu.

Mạng học sâu cho hàm giá trị  $Q$  hay thường được viết tắt là DQN (Deep Q-Network). Hình 8.6 mô tả cấu trúc đơn giản của mạng DQN. Trong Hình minh họa 8.6, chúng ta có  $o_0$  sẽ là giá trị của mạng  $Q_\theta$  ứng với hành động  $a_0$  và  $o_1$  ứng với hành động  $a_1$ . Không mất tính tổng quát, chúng ta có thể viết gọn lại  $o_a = Q_\theta(s, a)$  và dựa trên giá trị cụ thể của  $a$  để lấy ra  $o_a$  tương ứng. Ở mỗi bước tương tác với môi trường, khi nhận được trạng thái  $s$ , hành động  $a$  được chọn theo chính sách  $\epsilon$ -tham lam dựa trên giá trị  $o$  như sau:

$$a = \begin{cases} \arg \max_{a'} Q_\theta(s, a') & \text{với xác suất } 1 - \epsilon, \\ \text{chọn ngẫu nhiên từ } A & \text{với xác suất } \epsilon. \end{cases} \quad (8.39)$$



Hình 8.6: Cấu trúc cơ bản của DQN

Ở bước cập nhật, thay vì chỉ cập nhật trọng số  $w$  của hàm hồi quy tuyến tính, chúng ta sẽ cập nhật trọng số  $\theta$  của toàn bộ mạng học sâu DQN. Về mặt mô hình tính toán, mạng DQN khá gần với

một mạng học sâu hồi quy. Tuy nhiên, việc học hồi quy giá trị  $Q_\theta(s, a)$  gặp hai khó khăn chính: sự thiếu ổn định có giá trị mục tiêu và các mẫu dữ liệu có sự tương quan với nhau. Để giải quyết vấn đề này, DQN đưa vào hai kỹ thuật chính là: *Hàm mục tiêu cố định* và *Bộ nhớ đệm*:

- **Hàm mục tiêu cố định  $\hat{Q}$ :** Hàm mục tiêu  $\hat{Q}_{\theta^-}(s, a; )$  dùng để tính toán giá trị mục tiêu  $y$  trong quá trình huấn luyện mạng  $Q_\theta$ . Ở đây, chúng ta cũng sử dụng mạng học sâu có cấu trúc giống như mạng  $Q_\theta$ . Giả sử tại thời điểm  $t$ , tác tử ở trạng thái  $s$ , dựa theo công thức (8.39) chọn hành động  $a$  và nhận được phần thưởng  $r$  và trạng thái tiếp theo  $s'$ . Giá trị mục tiêu  $y$  đối với được tính toán dựa trên giá trị của hàm mục tiêu  $\hat{Q}_{\theta^-}$  theo công thức

$$y = r + \gamma \max_{a'} \hat{Q}_{\theta^-}(s', a'). \quad (8.40)$$

Dựa trên giá trị mục tiêu  $y$ , chúng ta xây dựng được hàm lỗi ứng với tập mẫu kinh nghiệm  $(s, a, r, s')$  như sau:

$$\ell(\theta) = (y - Q_\theta(s, a; \theta))^2. \quad (8.41)$$

Trong quá trình huấn luyện mạng  $Q_\theta$ , trọng số  $\theta^-$  của hàm mục tiêu  $\hat{Q}_{\theta^-}$  được giữ cố định trong một khoảng thời gian nhất định và sau đó sẽ được cập nhật lại bằng trọng số  $\theta$  của mạng  $Q_\theta$ .

- **Bộ nhớ đệm  $B$ :** Để huấn luyện mạng  $Q_\theta$ , chúng ta sử dụng bộ nhớ đệm  $B$  được sử dụng để lưu trữ các mẫu kinh nghiệm  $(s, a, r, s')$  trong quá trình tương tác với môi trường. Tác dụng của bộ nhớ đệm  $B$  là để lấy mẫu ngẫu nhiên các mẫu kinh nghiệm  $(s, a, r, s')$  để huấn luyện mạng  $Q_\theta$ . Kết hợp công thức (8.41) và bộ nhớ đệm  $B$ , ta có công thức hàm lỗi tổng quát là

$$L(\theta) = \mathbb{E}_{(s, a, r, s') \in B} [(y - Q_\theta(s, a))^2]. \quad (8.42)$$



Từ công thức hàm lỗi này, chúng ta có thể sử dụng thuật toán tối ưu hoá theo đạo hàm ngẫu nhiên để cập nhật trọng số  $\theta$  bằng việc lấy mẫu các kinh nghiệm  $(s, a, r, s')$  từ bộ nhớ đệm  $B$ .

Dựa trên các thành phần chính của mạng DQN, chúng ta có thể xây dựng được thuật toán huấn luyện mạng DQN như sau:

---

**Thuật toán 8.5** Thuật toán huấn luyện DQN (Huấn luyện mạng DQN với chính sách tham lam  $\epsilon$ )

---

- 1: Khởi tạo  $\theta$  và  $\theta^-$  với các trọng số ngẫu nhiên
  - 2: Khởi tạo bộ nhớ đệm  $B \leftarrow \emptyset$
  - 3: **for**  $k = 1, 2, \dots$ , **do**
  - 4:     Khởi tạo trạng thái ban đầu  $s$
  - 5:     **for**  $t = 1, 2, \dots, T$  **do**
  - 6:         Chọn hành động  $a \sim \epsilon - Q_\theta(s, a)$
  - 7:         Thực hiện hành động  $a$ ,
  - 8:         Nhận về phần thưởng  $r$  và trạng thái tiếp theo  $s'$
  - 9:         Lưu trữ  $(s, a, r, s')$  vào bộ nhớ đệm  $B$
  - 10:     **end for**
  - 11:     Lấy mẫu ngẫu nhiên một  $(s, a, r, s')$  từ bộ nhớ đệm  $B$
  - 12:     Tính toán giá trị mục tiêu  $y$  theo công thức (8.40)
  - 13:     Tính toán hàm lỗi  $L(\theta)$  theo công thức (8.42)
  - 14:     Cập nhật  $\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$
  - 15:     Sau  $K$  bước, cập nhật  $\theta^- \leftarrow \theta$
  - 16: **end for**
- 

Trong thuật toán trên, việc lựa chọn giá trị của tham số  $T$  và  $K$  đóng vai trò quan trọng với hiệu suất huấn luyện và độ ổn định của mạng DQN. Trong cấu trúc được mô tả ở Hình 8.6, chúng ta có thể thấy rằng mạng DQN có dạng một mạng học sâu nhiều lớp. Với các môi trường khác nhau, chúng ta có thể tích hợp thêm các lớp học sâu tích chập nếu cần trích chọn đặc trưng theo không gian của trạng thái đầu vào hoặc là lớp học sâu hồi quy nếu chúng ta muốn tích hợp thêm các đặc trưng theo thời gian. Mặc dù đã có

thể giải nhiều bài toán giả lập phức tạp như chơi game Atari, mạng DQN còn một số hạn chế, bao gồm:

- Mạng DQN không thể làm việc với không gian hành động liên tục, mà chỉ có thể làm việc với không gian hành động rời rạc.
- Việc huấn luyện mạng DQN có thể không ổn định nếu lựa chọn được các tham số phù hợp.
- Khó khăn trong việc cân bằng giữa quá trình tìm hiểu môi trường và tối ưu.

### 8.5.2 Phương pháp Xuống đồi bằng đạo hàm của chính sách

Trong phần này, chúng ta tiếp tục mở rộng các tiếp cận trong học tăng cường bằng cách tối ưu trực tiếp trên chính sách hiện tại thay vì dùng qua giá trị trung gian như hàm  $V$  hay hàm  $Q$ . Chúng ta đã biết rằng, trong học tăng cường, chính sách là một hàm ánh xạ từ trạng thái  $s$  về tập hành động  $A$ . Hàm ánh xạ này có thể đặc trưng cho một chính sách tất định theo công thức (8.1) hoặc là chính sách ngẫu nhiên theo công thức (8.2). Để có thể thu thập thông tin hiệu quả từ môi trường, chính sách ngẫu nhiên thường được sử dụng. Giả sử, tập các hành động  $A$  là hữu hạn, chính sách ngẫu nhiên có thể được biểu diễn dưới dạng một hàm xác suất  $\pi(a|s)$  được mô hình hoá bằng mạng học sâu như sau:

$$\pi_{\theta} : S \rightarrow \mathbb{R}^{|A|} \quad (8.43)$$

trong đó  $o = \pi_{\theta}(s)$  là véc-tơ xác suất với  $|A|$  chiều. Chúng ta đề ý là cách mô hình hoá này tương tự với việc học một mô hình học sâu phân lớp có số lớp bằng với số hành động có thể trong tập  $A$ . Để thể hiện tính ngẫu nhiên, hành động  $a$  được chọn theo xác suất  $\pi(a|s)$ , được kí hiệu là

$$a \sim \pi_{\theta}(a|s) \text{ hoặc } a \sim \pi_{\theta}(\cdot|s),$$

trong đó xác suất tương ứng với hành động  $a$  được kí hiệu là  $\pi_\theta(a|s)$ . Sử dụng Định nghĩa 8.7 về giá trị của trạng thái theo chính sách, chúng ta có định nghĩa về hàm mục tiêu của chính sách  $\pi_\theta$  như sau:

**Định nghĩa 8.13** (Hàm mục tiêu của chính sách  $\pi_\theta$ ). Hàm mục tiêu của chính sách  $\pi_\theta$  theo tham số  $\theta$  được định nghĩa như sau:

$$\begin{aligned} J(\theta) &= \mathbb{E}[V^{\pi_\theta}|s] \\ &= \mathbb{E}_{\tau \sim \pi_\theta(s)} \left[ \sum_{t=0, a_t \sim \pi_\theta(\cdot|s_t)}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \middle| s_0 = s \right], \end{aligned}$$

với  $\tau \sim \pi_\theta(s)$  là một chuỗi các trạng thái và hành động được sinh ra từ chính sách  $\pi_\theta$  khi bắt đầu từ trạng thái  $s$ .

Với định nghĩa này, chúng ta cần tìm  $\theta^*$  để  $J(\theta^*)$  đạt giá trị cực đại. Điểm khác biệt so với các phương pháp lập chính sách đã trình bày là chúng ta trực tiếp tối ưu hoá trên chính sách  $\pi_\theta$  thay vì tối ưu hoá qua một hàm trung gian thể hiện giá trị  $V$  hay  $Q$ . Tuy nhiên, việc sử dụng trực tiếp Định nghĩa 8.13 trên để tối ưu hoá tham số  $\theta$  là không khả thi vì rất khó để ước lượng  $\nabla_\theta J(\theta)$ . Thay vào đó, chúng ta có thể viết lại định nghĩa trên thông qua giá hàm  $Q^{\pi_\theta}$  như sau. Chúng ta bắt đầu bằng việc đưa ra định nghĩa về phân bố dừng  $d^{\pi_\theta}(s)$  của các trạng thái trong quá trình đi theo chính sách  $\pi_\theta$ .

**Định nghĩa 8.14** (Phân bố dừng của các trạng thái). Phân bố dừng  $d^{\pi_\theta}$  của các trạng thái trong quá trình đi theo chính sách  $\pi_\theta$  được định nghĩa là phân bố thỏa mãn điều kiện:

$$d^{\pi_\theta}(s') = \sum_{s \in S} d^{\pi_\theta}(s) \sum_{a \in A} \pi_\theta(a|s) P(s'|s, a) \quad \forall s' \in S$$

Trong phần trình bày này, chúng ta xét trường hợp đơn giản là tập trạng thái  $S$  và tập hành động  $A$  là rời rạc. Thêm vào đó, với

một chiến thuật  $\pi_\theta$  tùy ý, thì phân bố dừng  $d^{\pi_\theta}$  luôn tồn tại và duy nhất. Sử dụng Định nghĩa 8.14, chúng ta có thể viết lại giá trị hàm mục tiêu  $V^{\pi_\theta}$  ứng với một chính sách  $\pi_\theta$  như sau:

**Định nghĩa 8.15** (Hàm mục tiêu của chính sách theo tham số  $\theta$ ). Giả sử  $d^{\pi_\theta}$  là phân bố dừng của các trạng thái trong quá trình đi theo chính sách  $\pi_\theta$ , hàm mục tiêu  $J(\theta)$  của chính sách  $\pi_\theta$  được định nghĩa như sau:

$$J(\theta) = \sum_{s \in S} d^{\pi_\theta}(s) \sum_{a \in A} \pi_\theta(a|s) Q^{\pi_\theta}(s, a)$$

Để ý rằng, hàm mục tiêu  $J(\theta)$  được định nghĩa như một giá trị kì vọng của hàm  $Q^{\pi_\theta}(s, a)$  và xác suất  $\pi_\theta(a|s)$ . Chúng ta có thể sử dụng biểu diễn này để tính toán giá trị đạo hàm của hàm mục tiêu  $J(\theta)$  theo tham số  $\theta$  theo Định lý đạo hàm chính sách.

**Định lý 8.16** (Định lý đạo hàm chính sách). *Giả sử hàm mục tiêu  $J(\theta)$  được định nghĩa như ở Định nghĩa 8.15, thì đạo hàm của  $J(\theta)$  theo tham số  $\theta$  có thể được xấp xỉ như sau:*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[Q^{\pi_\theta}(s, a) \nabla_\theta \ln \pi_\theta(a|s)]$$

*Chứng minh:* Sử dụng định nghĩa hàm mục tiêu  $J(\theta)$  theo Định nghĩa 8.15, lấy đạo hàm theo  $\theta$  ở hai vế ta có:

$$\begin{aligned} \nabla_\theta J(\theta) &= \sum_{s \in S} d^{\pi_\theta}(s) \sum_{a \in A} Q^{\pi_\theta}(s, a) \nabla_\theta \pi_\theta(a|s) \\ &= \sum_{s \in S} d^{\pi_\theta}(s) \sum_{a \in A} \pi_\theta(a|s) Q^{\pi_\theta}(s, a) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} \\ &= \mathbb{E}_{\pi_\theta}[Q^{\pi_\theta}(s, a) \nabla_\theta \ln \pi_\theta(a|s)] \end{aligned}$$

□

**Thuật toán 8.6** Thuật toán tối ưu chính sách (Policy Gradient)

```

1: procedure POLICYGRADIENT(α, γ)
2: Khởi tạo tham số θ ngẫu nhiên cho chính sách π_θ
3: while chưa đạt điều kiện dừng do
4: Lấy một quỹ đạo theo π_θ :

```

$$(s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T)$$

```

5: for $t = 1$ to T do
6: Tính tổng phần thưởng trả về từ bước t :

```

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

```

7: Cập nhật tham số:

```

$$\theta \leftarrow \theta + \alpha \cdot \gamma \cdot G_t \cdot \nabla_\theta \ln \pi_\theta(a_t | s_t)$$

```

8: end for
9: end while
10: end procedure

```

**8.5.3 Phương pháp tối ưu chính sách**

Dựa trên Định lý 8.16 ta có thể tính gần đúng  $\nabla_\theta J(\theta)$  theo mẫu các kinh nghiệm thu được bằng thuật toán xấp xỉ Monte-Carlo.

$$\begin{aligned} \nabla_\theta J(\theta) &= \mathbb{E}_{\pi_\theta} [Q^{\pi_\theta}(s, a) \nabla_\theta \ln \pi_\theta(a | s)] \\ &= \mathbb{E}_{\pi_\theta} [G_t \nabla_\theta \ln \pi_\theta(a | s)] \end{aligned}$$

trong đó,  $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$  là giá trị trả về tính theo các trạng thái ở tương lai đối với mỗi mẫu kinh nghiệm thu được theo một đường đi  $\tau$  nào đó. Sử dụng quỹ đạo được lựa chọn theo phương pháp lấy mẫu Monte-Carlo, ta có ước lượng không chệch của đạo hàm theo  $\theta$ . Ta có mô tả thuật toán tối ưu chính sách với phương pháp lấy mẫu Monte-Carlo như trong Thuật toán 8.6.

Một lưu ý là, thuật toán này yêu cầu việc tính giá trị  $G_t$  phải dựa trực tiếp trên  $\pi_\theta$  hiện tại. Trong cách cài đặt được trình bày, hàm  $G_t$  được tính theo hàm giá trị  $Q_t(s, a)$  có thể có phương sai cao và độ ổn định thấp. Chúng ta có thể thay bằng hàm lợi ích  $A_t$ , đặc trưng cho độ chênh lệch với giá trị của trạng thái hiện tại và giá trị nhận được sau khi thực hiện hành động  $a \sim \pi(\cdot|s)$

$$A_t(s, a) = Q_t(s, a) - V_t(s).$$

## 8.6 Tình huống áp dụng: Sử dụng mạng DQN để điều khiển thăng bằng

### 8.6.1 Mô tả bài toán

Môi trường CartPole là một bài toán học tăng cường, trong đó yêu cầu giữ cho cây gậy thăng bằng trên một chiếc xe đẩy bằng cách di chuyển nó sang trái hoặc phải. Môi trường được cung cấp bởi thư viện OpenAI Gym. Trạng thái đầu vào gồm có:

- Vị trí của xe đẩy
- Vận tốc của xe đẩy
- Góc của cây gậy
- Vận tốc góc của cây gậy

Hành động có thể thực hiện bao gồm 2 hành động: Di chuyển xe đẩy sang trái hoặc di chuyển xe đẩy sang phải. Hàm phần thưởng nhận được là +1 nếu cây gậy thăng bằng. Lượt chơi hiện tại sẽ dừng nếu cây gậy bị lệch quá  $15^\circ$  hoặc xe đẩy đi quá xa ra khỏi vị trí trung tâm.

### 8.6.2 Các bước triển khai

Để giải quyết bài toán này, chúng ta sẽ sử dụng mạng DQN để học chính sách tối ưu cho bài toán điều khiển thăng bằng. Các bước triển khai chính bao gồm:

- Khởi tạo môi trường mô phỏng: Trong các bài toán học tăng cường, dữ liệu không được cung cấp sẵn mà được thu thập trong quá trình tác tử tương tác với môi trường. Trong trường hợp này, tác tử sẽ điều khiển thông qua hành động và nhận về các tập dữ liệu dưới dạng  $(s, a, s', r)$ . Dữ liệu thu thập được sẽ được lưu trữ trong bộ đệm để huấn luyện mạng DQN.
- Khai báo cấu trúc mạng DQN: Trong bài toán này chúng ta sẽ huấn luyện mạng học sâu DQN có đầu vào là 4 giá trị ứng với trạng thái hiện tại và đầu ra là hai giá trị ứng với phần thưởng nhận được khi thực hiện một trong hai hành động di chuyển sang trái/phải. Chính sách tối ưu được học một cách gián tiếp thông qua hàm học sâu DQN với cấu trúc có 3 lớp. Trong đó, hai lớp ẩn được sử dụng hàm kích hoạt ReLU để ánh xạ trạng thái đầu vào thành các giá trị miền ẩn. Lớp thứ 3 là lớp đầu ra với kích thước bằng số lượng hành động có thể thực hiện tương tự như mô tả ở Hình 8.6. Để có thể khám phá môi trường một cách hiệu quả, hành động sẽ được lấy theo thuật toán tham lam của mạng  $Q$ . Chúng ta cũng có khai báo tương tự cho mạng mục tiêu  $\hat{Q}$ .
- Huấn luyện mạng  $Q$ : Để huấn luyện mạng  $Q$ , trước tiên ta cần khởi tạo các tham số cần thiết cho quá trình tương tác với môi trường và quá trình huấn luyện. Ngoài các tham số tương tự như huấn luyện các mạng học sâu cơ bản, chúng ta có thêm một số tham số như: tham số  $\varepsilon$ , tham số  $K$ . Để cải thiện tính khám phá của mạng, chúng ta sẽ sử dụng phương pháp giảm dần tham số  $\varepsilon$  thông qua giá trị lớn nhất, ứng với các vòng lặp khởi động, và

giá trị nhỏ nhất ứng với các vòng lặp cuối cùng. Trong bước cập nhật hàm mục tiêu  $Q$ , chúng ta sử dụng phương pháp cập nhật mềm thông qua tham số  $\tau$ .

- Đánh giá mô hình: Sử dụng môi trường đã khai báo, chúng ta có thể đánh giá độ tốt của chính sách thông qua tổng số các phần thưởng nhận được.

Phần minh họa sẽ sử dụng môi trường CartPole-v1 trong thư viện OpenAI Gym.

Người đọc có thể tham khảo mã nguồn tại đường dẫn <https://gist.github.com/cuongtv312/de8f60669fd0570e268154dc073f231a>

## 8.7 Tổng kết chương

Chương 8 trình bày các khái niệm nền tảng của học tăng cường, bắt đầu từ định nghĩa bài toán ra quyết định trong môi trường Markov.

Các phương pháp quy hoạch động như lặp giá trị và lặp theo chính sách cho phép tìm giá trị tối ưu và chính sách tối ưu.

Khi không có mô hình môi trường, tác nhân phải đồng thời khám phá và học tối ưu – bài toán được giải thông qua các thuật toán như SARSA và học hàm  $Q$ .

Để mở rộng cho không gian trạng thái lớn, các phương pháp xấp xỉ được dùng nhằm học biểu diễn ẩn của trạng thái, thay vì biểu diễn tường minh. Trên cơ sở đó, các thuật toán học sâu như Deep Q-Network (DQN) và các phương pháp đạo hàm chính sách (policy gradient) đã được phát triển để nâng cao hiệu quả học.

Các kỹ thuật này đã đạt được nhiều thành công thực tế, đặc biệt trong trò chơi, điều khiển tự động và robot học. Người học có



thể tìm hiểu sâu hơn các khía cạnh lý thuyết và ứng dụng của học tăng cường qua các tài liệu liên quan [8].

## Bài tập

1. Nêu các điểm khác biệt chính của học tăng cường và học giám sát. Dưới những điều kiện nào thì nên mô hình bài toán thực tế dưới dạng học tăng cường?
2. Dựa theo chứng minh tính hội tụ của thuật toán lặp theo giá trị, chứng minh tính hội tụ của thuật toán lặp theo chính sách trong trường hợp tập trạng thái và tập hành động là hữu hạn và chính sách  $\pi$  là chính sách xác định.
3. *[Tìm hiểu]* Nếu chúng ta sử dụng chính sách  $\pi$  là chính sách ngẫu nhiên, thì tính hội tụ của thuật toán lặp theo chính sách có còn đúng không?
4. Dựa trên các bước cơ bản của thuật toán quy hoạch động lặp theo giá trị, đề xuất phương pháp quy hoạch động để tính giá trị tối ưu của hàm  $Q$ . Đánh giá độ phức tạp của thuật toán tìm được.
5. *[Tìm hiểu]* Trong thuật toán DQN, chính sách khai phá đóng vai trò quan trọng để thu thập thông tin từ môi trường thông qua kinh nghiệm. Nêu một số chính sách khai phá cho thuật toán học theo mạng học sâu DQN.
6. *[Lập trình]* Dựa trên phần trình bày trong mục Tình huống áp dụng, sử dụng thuật toán đạo hàm theo chính sách giải bài toán CartPole với mạng học sâu có 2 lớp, số lớp ẩn là 128 ở mỗi lớp. So sánh về độ tốt và số lượng mẫu kinh nghiệm cần thiết để học được chính sách tối ưu giữa thuật toán đạo hàm theo chính sách và thuật toán huấn luyện mạng DQN.



---

# Tài liệu tham khảo

---

- [1] Bellman, R. E., *Dynamic programming*, Princeton University Press, 1957.
- [2] Howard, R. A., *Dynamic programming and Markov processes*, MIT Press, 1960.
- [3] Sutton, R. S., *Learning to predict by the methods of temporal differences*, Machine Learning, vol. 3, no. 1, pp. 9–44, 1988.
- [4] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., et al., *Human-level control through deep reinforcement learning*, Nature, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D., *Continuous control with deep reinforcement learning*, arXiv preprint arXiv:1509.02971, 2015.
- [6] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O., *Proximal policy optimization algorithms*, arXiv preprint arXiv:1707.06347, 2017.

- [7] Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., et al., *Mastering the game of Go with deep neural networks and tree search*, Nature, vol. 529, no. 7587, pp. 484–489, 2016.
- [8] Sutton, Richard S. and Barto, Andrew G., *Reinforcement learning: An introduction*, MIT Press, 2018.

## Chương 9

---

# Học kết hợp

---

Chương 9 giới thiệu Học kết hợp (Ensemble Learning) – một hướng tiếp cận mạnh mẽ trong Học máy nhằm nâng cao độ chính xác, khả năng tổng quát hóa và tính ổn định của mô hình.

Học kết hợp xây dựng một tập các mô hình con và sử dụng một hàm kết hợp  $\mathcal{F}$  để đưa ra quyết định cuối cùng

$$H(\mathbf{x}) = \mathcal{F}\{M_1(\mathbf{x}), M_2(\mathbf{x}), \dots, M_K(\mathbf{x})\}. \quad (9.1)$$

Trong đó,  $M_k(\mathbf{x})$  là các mô hình con được huấn luyện độc lập hoặc theo thứ tự.

Hai kỹ thuật học kết hợp phổ biến là Bagging và Tăng cường (Boosting). Bagging sử dụng lấy mẫu ngẫu nhiên có hoàn lại để tạo các mô hình con, điển hình là Rừng ngẫu nhiên (Random Forest). Boosting tạo mô hình con theo trình tự, trong đó mỗi mô hình kế tiếp tập trung vào các mẫu bị dự đoán sai trước đó. Các thuật toán nổi bật gồm AdaBoost, Tăng cường bằng đạo hàm (Gradient Boosting) và XGBoost.

Cốt lõi của học kết hợp là thiết kế mô hình con sao cho đủ đa dạng và kết hợp chúng hiệu quả để giảm thiểu sai số tổng thể. Chương này trình bày nguyên lý hoạt động, ưu điểm, hạn chế và các ứng dụng thực tiễn của từng phương pháp học kết hợp.

## 9.1 Giới thiệu

Ý tưởng chính của *Học kết hợp* (ensemble learning) là sử dụng đồng thời nhiều mô hình với trọng số khác nhau, rồi kết hợp quyết định của chúng để tạo ra một mô hình có hiệu năng cao hơn từng mô hình đơn lẻ. Phương pháp này mô phỏng cách con người thường tham khảo ý kiến từ nhiều nguồn—người thân, bạn bè, đồng nghiệp, chuyên gia—khi cần đưa ra quyết định quan trọng. Mỗi ý kiến được đánh giá theo độ tin cậy và tất cả được kết hợp lại để đưa ra quyết định cuối cùng.

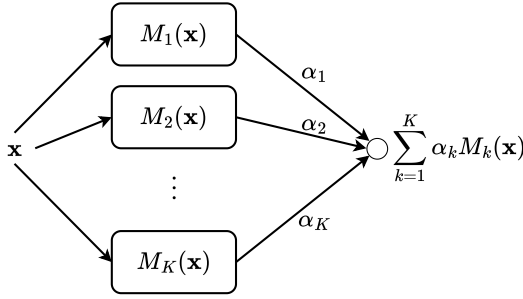
Tương tự, trong học kết hợp, có hai bài toán cần giải quyết:

- (i) Cách tạo ra nhiều mô hình khác nhau cho cùng một nhiệm vụ;
- (ii) Cách tính trọng số để kết hợp các mô hình một cách hiệu quả.

Trong chương này, ta giả sử có  $K$  mô hình được huấn luyện, ký hiệu là  $M_k(\mathbf{x})$ , với  $k = 1, \dots, K$ . Mỗi mô hình được gán trọng số  $\alpha_k$ . Hình 9.1 minh họa nguyên lý hoạt động của phương pháp học kết hợp.

- Bài toán phân lớp nhị phân:  $M_k(\mathbf{x}) \in \{\pm 1\}$ .

$$H(\mathbf{x}) = \text{sgn} \left( \sum_{k=1}^K \alpha_k M_k(\mathbf{x}) \right) \quad (9.2)$$



Hình 9.1: Học kết hợp

- Bài toán phân lớp  $C$  lớp:  $M_k(\mathbf{x}) \in \{1, \dots, C\}$ .

$$H(\mathbf{x}) = \arg \max_y \left( \sum_{k=1}^K \alpha_k \mathbb{I}[M_k(\mathbf{x}) = y] \right) \quad (9.3)$$

- Bài toán hồi quy:  $M_k(\mathbf{x}) \in \mathbb{R}$ .

$$H(\mathbf{x}) = \frac{1}{\sum_{k=1}^K \alpha_k} \left( \sum_{k=1}^K \alpha_k M_k(\mathbf{x}) \right) \quad (9.4)$$

Một trường hợp đặc biệt của các công thức trên là khi  $\alpha_k = 1$  với mọi  $k$ , tức là, các mô hình quan trọng như nhau. Khi đó, mô hình kết hợp đưa ra quyết định giống như bỏ phiếu theo đa số đối với bài toán phân lớp và lấy trung bình cộng đối với bài toán hồi quy.

## 9.2 Phương pháp Bagging

### 9.2.1 Tổng quan phương pháp

Phương pháp tổng hợp mô hình qua lấy mẫu ngẫu nhiên, còn gọi là *Bootstrap Aggregating* hay *Bagging*, là kỹ thuật kết hợp nhiều mô hình cơ sở, được huấn luyện bằng cùng một thuật toán học máy  $\mathcal{A}$  nhưng trên các tập dữ liệu khác nhau. Các tập dữ liệu này được

tạo ra bằng cách lấy mẫu ngẫu nhiên có hoàn lại từ tập dữ liệu gốc, theo phương pháp *bootstrap*.

Phương pháp lấy mẫu ngẫu nhiên có hoàn lại (*bootstrap*) là một kỹ thuật thống kê dùng để ước lượng các đặc trưng của phân phối dữ liệu bằng cách lấy mẫu ngẫu nhiên có hoàn lại từ tập dữ liệu gốc. Ưu điểm nổi bật của phương pháp này là có thể ước lượng khoảng tin cậy cho các tham số mô hình mà không cần giả định phân phối xác suất cụ thể của dữ liệu. Ước đồ bootstrap được mô tả trong Thuật toán 9.1, với  $\hat{\theta} = \hat{\theta}(D)$  là một thống kê bất kỳ được tính từ dữ liệu  $D$ .

---

**Thuật toán 9.1** Phương pháp Bootstrap

---

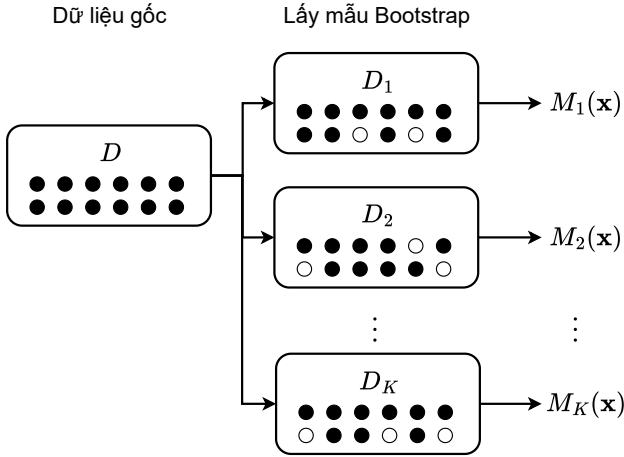
```
1: procedure BOOSTRAP($D = \{x_1, \dots, x_n\}, S, B$)
2: for $b = 1$ to B do
3: Lấy mẫu ngẫu nhiên có hoàn lại $D_b = \{x_{b1}, \dots, x_{bn}\}$ từ
 tập dữ liệu gốc D
4: Tính thống kê $\hat{\theta}_b = \hat{\theta}(D_b)$
5: end for
6: Ước lượng điểm $\bar{\theta} = \frac{1}{B} \sum_{b=1}^B \hat{\theta}_b$
7: Tính độ lệch chuẩn $\sigma = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\theta}_b - \bar{\theta})^2}$
8: Tính khoảng tin cậy 95% $CI = [\bar{\theta} - 1.96 \cdot \sigma, \bar{\theta} + 1.96 \cdot \sigma]$
9: end procedure
```

---

Người ta chứng minh được rằng, dưới đây cho thấy khi kích thước mẫu đủ lớn và thống kê  $\hat{\theta}(\cdot)$  khả vi thì phân phối của ước lượng bootstrap hội tụ về phân phối của ước lượng thống kê gốc.

Phương pháp Bagging (Hình 9.2) sử dụng phương pháp Bootstrap để tạo ra các bộ dữ liệu huấn luyện khác nhau  $D_k$  từ bộ dữ liệu gốc  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$  bằng cách lấy mẫu ngẫu nhiên có hoàn lại. Sau đó, mỗi bộ dữ liệu  $D_k$  được sử dụng để huấn luyện một mô hình con  $M_k(\mathbf{x}) = \mathcal{A}(D_k)$  bằng cách sử dụng thuật toán học máy  $\mathcal{A}$ . Cuối cùng, các mô hình con phối hợp với nhau





**Hình 9.2: Phương pháp Bagging**

để đưa ra quyết định cuối cùng. Lược đồ chung của phương pháp Bagging được mô tả trong Thuật toán 9.2.

---

**Thuật toán 9.2** Phương pháp Bagging

---

```

1: procedure TRAINBAGGING($\mathcal{A}, D, K$)
2: for $k = 1$ to K do
3: Lấy mẫu bootstrap D_k từ tập dữ liệu gốc D
4: Huấn luyện mô hình con: $M_k(\mathbf{x}) \leftarrow \mathcal{A}(D_k)$
5: Gán trọng số: $\alpha_k \leftarrow 1$
6: end for
7: return $M_k(\mathbf{x}), \alpha_k, k = 1, \dots, K$
8: end procedure

```

---

Như vậy phương pháp Bagging sử dụng trọng số  $\alpha_k = 1$  cho mọi mô hình huấn luyện được.

Các phương pháp bagging tuân thủ lược đồ 9.2 nhưng có thể khác nhau ở các bước

- Phép lấy mẫu ngẫu nhiên: việc lấy mẫu có thể tiến hành với cả

tập mẫu và tập đặc trưng.

- Thuật toán huấn luyện: chúng ta có thể sử dụng mọi mô hình và thuật toán học máy ở đây.

### 9.2.2 Mô hình Rừng ngẫu nhiên

Trong mục này chúng ta cùng tìm hiểu Thuật toán *Rừng ngẫu nhiên* là một ví dụ tiêu biểu của phương pháp Bagging. Rừng ngẫu nhiên là mô hình phổ dụng vì rất dễ hiểu và dễ sử dụng. Rừng ngẫu nhiên có hiệu suất cao ngay cả khi không tinh chỉnh tham số quá nhiều.

Thuật toán huấn luyện Rừng ngẫu nhiên là ứng dụng trực tiếp của phương pháp Bagging cho Cây quyết định (Thuật toán 9.3 và 9.4). Cụ thể, tại bước lấy mẫu, bộ dữ liệu  $D_k$  được tạo ra bằng cách lấy mẫu ngẫu nhiên có hoàn lại các mẫu dữ liệu của tập dữ liệu huấn luyện  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$ . Từ mỗi bộ dữ liệu  $D_k$  được lấy mẫu ngẫu nhiên như vậy, chúng ta sử dụng thuật toán huấn luyện cây quyết định (ví dụ: ID3, C4.5) để tạo ra một cây quyết định  $M_k(\mathbf{x})$ . Một cải tiến quan trọng của Rừng ngẫu nhiên là tại mỗi nút phân chia của cây quyết định, thay vì sử dụng tất cả các thuộc tính để tìm thuộc tính tốt nhất để phân chia, chúng ta chỉ sử dụng một tập con ngẫu nhiên  $m$  thuộc tính.

Mô hình Rừng ngẫu nhiên tận dụng khái niệm quen thuộc là “trí tuệ của đám đông”. Khi số lượng cây quyết định lớn được tạo ra bởi phương pháp lấy mẫu trên, mối quan hệ tương quan giữa các quyết định đơn lẻ của các cây này thấp do sử dụng dữ liệu huấn luyện cũng như đặc trưng dữ liệu khác nhau. Đây là đặc điểm quan trọng nhất, là nguyên nhân tại sao mô hình Rừng ngẫu nhiên có hiệu suất cao hơn từng cây quyết định đơn lẻ. Có thể hiểu một cách trực quan là các cây quyết định hỗ trợ nhau tránh khỏi các lỗi của từng cây. Khi có một vài cây bị sai nhưng đa số cây ra quyết

**Thuật toán 9.3** Huấn luyện mô hình Rừng ngẫu nhiên

---

```

1: procedure TRAINRF(D, K, m)
2: for $k = 1$ to K do
3: Lấy mẫu ngẫu nhiên có hoàn lại từ D để tạo tập dữ liệu
 con D_k , với $|D_k| = n$
4: Huấn luyện cây quyết định $M_k(\mathbf{x})$ trên D_k , sử dụng thuật
 toán cây quyết định (ví dụ: ID3, C4.5 hoặc CART), với mỗi nút
 phân chia chọn ngẫu nhiên m đặc trưng từ tổng số đặc trưng
5: Gán trọng số $\alpha_k \leftarrow 1$
6: end for
7: return $M_k(\mathbf{x}), \alpha_k, k = 1, \dots, K$
8: end procedure

```

---

**Thuật toán 9.4** Dự đoán với Rừng ngẫu nhiên

---

```

1: procedure EVALRF($M_k, \alpha_k, \mathbf{x}$)
2: for all $y \in \mathcal{Y}$ do
3: $score[y] \leftarrow \sum_{k=1}^K \alpha_k \cdot \mathbb{I}[M_k(\mathbf{x}) = y]$
4: end for
5: return $H(\mathbf{x}) \leftarrow \arg \max_y score[y]$
6: end procedure

```

---

định đúng thì mô hình kết hợp vẫn cho kết quả đúng. Trên thực tế, để mô hình Rừng ngẫu nhiên cho kết quả tốt ta cần có (i) các đặc trưng có liên quan đến nhiệm vụ cần giải quyết và (ii) các cây quyết định được xây dựng một cách ngẫu nhiên và có tương quan yếu.

**Định lý 9.1.** *Nếu mỗi cây  $M_k(\mathbf{x})$  có phương sai  $Var(M_k(\mathbf{x})) = \sigma^2$  và độ tương quan trung bình giữa các cây là  $\rho$ , thì phương sai của mô hình kết hợp  $H(\mathbf{x}) = \frac{1}{K} \sum_{k=1}^K M_k(\mathbf{x})$  (trong bài toán hồi quy) được cho bởi:*

$$Var(H(\mathbf{x})) = \frac{1}{K} \sigma^2 + \frac{K-1}{K} \rho \sigma^2.$$

Khi  $K \rightarrow \infty$ , nếu  $\rho$  nhỏ (do tính đa dạng của các cây), thì  $Var(H(\mathbf{x})) \rightarrow \rho \sigma^2$ , nhỏ hơn  $\sigma^2$ .

*Chứng minh:* Gọi  $H(\mathbf{x}) = \frac{1}{K} \sum_{k=1}^K M_k(\mathbf{x})$  là mô hình kết hợp trung bình từ  $K$  mô hình cơ sở  $M_k(\mathbf{x})$ . Ta cần tính phương sai của  $H(\mathbf{x})$ :

$$\begin{aligned} \text{Var}(H(\mathbf{x})) &= \text{Var} \left( \frac{1}{K} \sum_{k=1}^K M_k(\mathbf{x}) \right) \\ &= \frac{1}{K^2} \sum_{i=1}^K \sum_{j=1}^K \text{Cov}(M_i, M_j). \end{aligned}$$

Chia tổng trên thành hai phần:

- Trường hợp  $i = j$ : Có  $K$  số hạng  $\text{Var}(M_k) = \sigma^2$ .
- Trường hợp  $i \neq j$ : Có  $K(K - 1)$  số hạng  $\text{Cov}(M_i, M_j) = \rho\sigma^2$ .

Suy ra

$$\begin{aligned} \text{Var}(H(\mathbf{x})) &= \frac{1}{K^2} [K\sigma^2 + K(K - 1)\rho\sigma^2] \\ &= \frac{\sigma^2}{K} + \frac{K - 1}{K} \rho\sigma^2 \\ &\rightarrow \rho\sigma^2, \end{aligned}$$

khi  $K \rightarrow \infty$ .

Nếu  $\rho \ll 1$ , tức các mô hình cơ sở có tính đa dạng cao, thì phương sai của mô hình kết hợp sẽ nhỏ hơn nhiều so với phương sai của từng mô hình đơn lẻ.  $\square$

Một điểm thú vị của mô hình Rừng ngẫu nhiên là nó có thể được sử dụng để ước lượng lỗi tổng quát hóa mà không cần một tập kiểm tra riêng biệt. Điều này được thực hiện thông qua lỗi ngoài túi (Out-of-Bag hay OOB). Cụ thể, lỗi OOB được tính như sau:

- Gọi  $\mathcal{F}_i \subset \{1, 2, \dots, K\}$  là tập hợp các chỉ số  $k$  mà mẫu  $(\mathbf{x}_i, y_i) \notin D_k$ .

- Với mỗi mẫu  $(\mathbf{x}_i, y_i)$  trong tập huấn luyện  $D$ , ta tính

$$H_{\text{OOB}}(\mathbf{x}_i) = \frac{1}{|\mathcal{F}_i|} \sum_{k \in \mathcal{F}_i} M_k(\mathbf{x}_i). \quad (9.5)$$

Tức là kết hợp các mô hình  $M_k$  không được huấn luyện bằng mẫu  $(\mathbf{x}_i, y_i)$ .

- Cuối cùng, ta tính lỗi OOB bằng cách

$$\text{err}_{\text{OOB}} = \frac{1}{n} \sum_{i=1}^n L(H_{\text{OOB}}(\mathbf{x}_i), y_i), \quad (9.6)$$

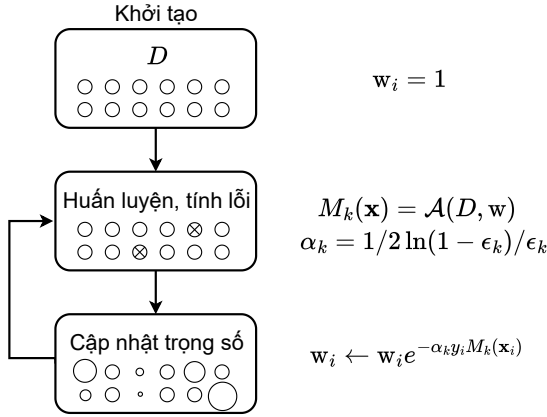
Mỗi mẫu trong tập huấn luyện  $D$  có xác suất khoảng  $(1 - 1/n)^n \approx 1/e \approx 0.368$  không được chọn vào một tập dữ liệu Bootstrap  $D_k$ . Do đó sẽ có khoảng 36,8% mẫu không được chọn vào  $D_k$ . Tận dụng điều này, mô hình Rừng ngẫu nhiên có thể ước lượng lỗi tổng quát hóa mà không cần một tập kiểm tra riêng biệt.

Có thể chứng minh được lỗi  $\text{err}_{\text{OOB}}$  là một ước lượng không chệch của kì vọng lỗi  $\text{err}_{\mathcal{P}}(H(\mathbf{x}))$ .

### 9.3 Phương pháp Boosting

Khác với Bagging coi các mô hình con độ quan trọng như nhau, phương pháp Tăng cường (Boosting) kết hợp các mô hình học máy bằng cách lần lượt huấn luyện các mô hình dựa trên điểm yếu của các mô hình được huấn luyện trước đó. Có hai lược đồ Tăng cường hay được sử dụng nhất là AdaBoost và Tăng cường bằng đạo hàm (Gradient Boosting). Trong mục này, chúng ta cùng tìm hiểu phương pháp AdaBoost.

Đầu tiên, xét bài toán phân lớp nhị phân  $\mathcal{Y} = \{\pm 1\}$ . Ý tưởng chính của phương pháp AdaBoost là lần lượt xây dựng các mô hình phân lớp  $M_k(\mathbf{x})$ ,  $k = 1, 2, \dots$  dựa trên bộ dữ liệu  $D_k$  gồm các mẫu

**Hình 9.3: Phương pháp AdaBoost**

dữ liệu trong  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$  nhưng mỗi mẫu  $(\mathbf{x}_i, y_i)$  có độ quan trọng  $w_k(i)$  được xác định từ điểm yếu của các mô hình học máy trước đó (Hình 9.3).

Ở bước khởi tạo, trọng số các mẫu dữ liệu bằng nhau và bằng  $\frac{1}{n}$  với  $n$  là số lượng mẫu dữ liệu trong tập huấn luyện:

$$w_1(i) = \frac{1}{n}, i = 1, 2, \dots, n. \quad (9.7)$$

Ta dùng bộ trọng số này để huấn luyện mô hình  $M_1(\mathbf{x})$  sao cho tỉ lệ lỗi sau khi huấn luyện là nhỏ nhất có thể được, được mô tả bằng công thức tính  $\epsilon_1$ :

$$\epsilon_1 = \sum_{i=1}^n w_1(i) \mathbb{I}[M_1(\mathbf{x}_i) \neq y_i] \quad (9.8)$$

Phương pháp AdaBoost sau đó cập nhật lại trọng số  $w_2(i)$  bằng cách tăng trọng số cho các mẫu dữ liệu bị  $M_1(\mathbf{x})$  dự đoán sai và giảm trọng số cho các mẫu dữ liệu được  $M_1(\mathbf{x})$  đoán đúng rồi chuẩn

hoá để tổng trọng số bằng 1.

$$\tilde{w}_2(i) = w_1(i)e^{-\alpha_1 y_i M_1(\mathbf{x}_i)} \quad (9.9)$$

$$w_2(i) = \frac{\tilde{w}_2(i)}{\sum_{i'=1}^n \tilde{w}_2(i')} \quad (9.10)$$

với hệ số  $\alpha_1$  được xác định bởi công thức

$$\alpha_1 = \frac{1}{2} \ln \frac{1 - \epsilon_1}{\epsilon_1} \quad (9.11)$$

Công thức (9.11) trên cho thấy, khi  $M_1(\mathbf{x})$  tốt hơn hàm phân lớp ngẫu nhiên một chút (tức là  $1 - \epsilon_1 > 0,5$ ) thì  $\alpha_1$  sẽ là một số dương. Như vậy, trọng số của các mẫu dữ liệu mà  $M_1(\mathbf{x})$  đoán sai sẽ được nhân với một đại lượng  $e^{\alpha_1} > 1$  còn trọng số các mẫu dữ liệu được đoán đúng sẽ được nhân với một đại lượng  $e^{-\alpha_1} < 1$ . Ta lại huấn luyện mô hình phân lớp tiếp theo  $M_2(\mathbf{x})$  để tối thiểu hoá tỉ lệ lỗi

$$\epsilon_2 = \sum_{i=1}^n w_2(i) \mathbb{I}[M_2(\mathbf{x}_i) \neq y_i] \quad (9.12)$$

và phương pháp AdaBoost tiếp tục cho đến khi đạt số lượng mô hình phân lớp  $K$  cho trước hoặc khi một hệ số  $\alpha_k < 0$  (Thuật toán 9.5).

Ta có Định lý 9.2 sau đây về cận trên tỉ lệ lỗi huấn luyện của AdaBoost.

**Định lý 9.2** (Cận trên tỉ lệ lỗi huấn luyện của AdaBoost). *Thuật toán 9.5 cho mô hình phân lớp  $H(x)$  với tỉ lệ lỗi huấn luyện thoả mãn*

$$\text{err}_D(H) \leq \prod_{k=1}^K \sqrt{4\epsilon_k(1 - \epsilon_k)} = \prod_{k=1}^K \sqrt{1 - 4\gamma_k^2}.$$

Trong đó  $\epsilon_k = \sum_{i=1}^n w_k(i) \mathbb{I}[M_k(\mathbf{x}_i) \neq y_i] < \frac{1}{2}$  là tỉ lệ lỗi của mô hình cơ sở thứ  $k$  và  $\gamma_k = \frac{1}{2} - \epsilon_k$ .

---

**Thuật toán 9.5** Phương pháp AdaBoost

---

```
1: procedure TRAINADABOOST($\mathcal{A}, D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, K$)
2: for $i = 1$ to n do
3: $w_1(i) \leftarrow \frac{1}{n}$
4: end for
5: for $k = 1$ to K do
6: Huấn luyện mô hình $M_k \leftarrow \mathcal{A}(D, w_k)$
7: Tính lỗi: $\epsilon_k \leftarrow \sum_{i=1}^n w_k(i) \cdot \mathbb{I}[M_k(\mathbf{x}_i) \neq y_i]$
8: Tính trọng số mô hình: $\alpha_k \leftarrow \frac{1}{2} \ln \frac{1-\epsilon_k}{\epsilon_k}$
9: if $\alpha_k < 0$ then
10: $K \leftarrow k - 1$; break
11: end if
12: for $i = 1$ to n do
13: $\tilde{w}_k(i) \leftarrow w_k(i) \cdot e^{-\alpha_k y_i M_k(\mathbf{x}_i)}$
14: end for
15: $Z_k \leftarrow \sum_{i=1}^n \tilde{w}_k(i)$
16: for $i = 1$ to n do
17: $w_{k+1}(i) \leftarrow \frac{\tilde{w}_k(i)}{Z_k}$
18: end for
19: end for
20: return M_k, α_k với $k = 1, \dots, K$
21: end procedure
```

---

---

**Thuật toán 9.6** Dự đoán với AdaBoost

---

```
1: procedure EVALADABOOST($M_k, \alpha_k, \mathbf{x}$)
2: $F(\mathbf{x}) \leftarrow \sum_{k=1}^K \alpha_k \cdot M_k(\mathbf{x})$
3: return $H(\mathbf{x}) \leftarrow \text{sgn}(F(\mathbf{x}))$
4: end procedure
```

---



*Chứng minh:* Đầu tiên, xuất phát từ công thức ước lượng trọng số của mẫu dữ liệu tại bước  $k + 1$

$$\begin{aligned} w_{K+1}(i) &= \frac{w_K(i)e^{-\alpha_k y_i M_k(\mathbf{x}_i)}}{Z_k} \\ &= \frac{1}{n} \prod_{k=1}^K \frac{e^{-\alpha_k y_i M_k(\mathbf{x}_i)}}{Z_k} \\ &= \frac{1}{n} \left( \prod_{k=1}^K Z_k \right)^{-1} \exp \left\{ -y_i \sum_{k=1}^K \alpha_k M_k(\mathbf{x}_i) \right\} \end{aligned}$$

Nếu  $H(\mathbf{x}_i) \neq y_i$ , tức là  $H(\mathbf{x})$  phân lớp sai trên mẫu dữ liệu  $(\mathbf{x}_i, y_i)$  thì theo định nghĩa của  $H(\mathbf{x})$ , ta có

$$y_i \sum_{k=1}^K \alpha_k M_k(\mathbf{x}_i) < 0$$

Như vậy ta luôn có cận trên của lỗi là

$$\begin{aligned} \mathbb{I}[H(\mathbf{x}_i) \neq y_i] &\leq \exp \left\{ -y_i \sum_{k=1}^K \alpha_k M_k(x_i) \right\} \\ \text{err}_D(H) &= \frac{1}{n} \sum_{i=1}^n \mathbb{I}[H(\mathbf{x}_i) \neq y_i] \leq \frac{1}{n} \sum_{i=1}^n \exp \left\{ -y_i \sum_{k=1}^K \alpha_k M_k(\mathbf{x}_i) \right\} \\ &= \prod_{k=1}^K Z_k \sum_{i=1}^n w_{K+1}(i) \\ &= \prod_{k=1}^K Z_k \end{aligned}$$

Để thấy

$$Z_k = \epsilon_k e^{\alpha_k} + (1 - \epsilon_k) e^{-\alpha_k}$$

Chọn  $\alpha_k$  để tối thiểu hoá  $Z_k$ , lấy đạo hàm của  $Z_k$  đặt bằng 0 để tìm cực trị

$$\epsilon_k e^{\alpha_k} - (1 - \epsilon_k) e^{-\alpha_k} = 0.$$

Giải ra ta được

$$\alpha_k = \frac{1}{2} \ln \frac{1 - \epsilon_k}{\epsilon_k}.$$

Đây chính là công thức tính hệ số  $\alpha_k$  trong thuật toán 9.5. Đặt  $\gamma_k = \frac{1}{2} - \epsilon_k$ , giá trị cực tiểu của  $Z_k$  là

$$\begin{aligned} \min_{\alpha_k} Z_k &= \sqrt{4\epsilon_k(1 - \epsilon_k)} \\ &= \sqrt{4\left(\frac{1}{2} - \gamma_k\right)\left(\frac{1}{2} + \gamma_k\right)} \\ &= \sqrt{1 - 4\gamma_k^2} \end{aligned} \tag{9.13}$$

□

Theo kết quả nhận được từ đánh giá ở công thức (9.13), giá trị này bé hơn 1 nếu  $\epsilon_k < 0,5$ , tức là chỉ cần mô hình  $M_k(\mathbf{x})$  có hiệu suất tốt hơn đoán ngẫu nhiên một chút. Như vậy, cận trên của tỉ lệ lỗi trên tập dữ liệu huấn luyện sẽ tiến đến 0 khi số lượng mô hình cơ sở tăng dần.

Thuật toán AdaBoost có thể mở rộng cho bài toán phân lớp nhiều lớp như trong thuật toán 9.7 có tên là thuật toán SAMME.

## 9.4 Tăng cường bằng đạo hàm

Phương pháp Tăng cường bằng đạo hàm (Gradient boosting) là kết hợp các mô hình học máy trong đó các mô hình được tạo ra sau tìm các sửa lỗi mô hình được tạo ra trước đó. Qua đó, mô hình kết hợp cuối cùng có hiệu suất cao hơn nhiều mô hình ban đầu (Hình 9.4).

Cụ thể, giả sử ta có bài toán học có giám sát với bộ dữ liệu  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$  và hàm lỗi  $L(\hat{y}, y)$  đánh giá lỗi giữa giá trị dự đoán  $\hat{y}$  và giá trị mong muốn  $y$ . Phương pháp Tăng cường bằng đạo hàm bắt đầu với việc tìm mô hình  $H_0(\mathbf{x})$  là hằng số thoả

---

**Thuật toán 9.7** Phương pháp SAMME cho phân lớp  $C$  lớp

---

```

1: procedure TRAINADABOOSTSAMME($\mathcal{A}, D, K, C$)
2: for $i = 1$ to n do
3: $w_1(i) \leftarrow \frac{1}{n}$
4: end for
5: for $k = 1$ to K do
6: Huấn luyện mô hình $M_k \leftarrow \mathcal{A}(D, w_k)$
7: Tính lỗi: $\epsilon_k \leftarrow \sum_{i=1}^n w_k(i) \cdot \mathbb{I}[M_k(\mathbf{x}_i) \neq y_i]$
8: Tính trọng số mô hình:

$$\alpha_k \leftarrow \ln \left(\frac{(C-1)(1-\epsilon_k)}{\epsilon_k} \right)$$

9: if $\alpha_k < 0$ then
10: $K \leftarrow k - 1$; break
11: end if
12: for $i = 1$ to n do
13: $\tilde{w}_k(i) \leftarrow w_k(i) \cdot e^{\alpha_k \cdot \mathbb{I}[y_i \neq M_k(\mathbf{x}_i)]}$
14: end for
15: $Z_k \leftarrow \sum_{i=1}^n \tilde{w}_k(i)$
16: for $i = 1$ to n do
17: $w_{k+1}(i) \leftarrow \frac{\tilde{w}_k(i)}{Z_k}$
18: end for
19: end for
20: return M_k, α_k với $k = 1, \dots, K$
21: end procedure

```

---



---

**Thuật toán 9.8** Dự đoán với SAMME (AdaBoost đa lớp)

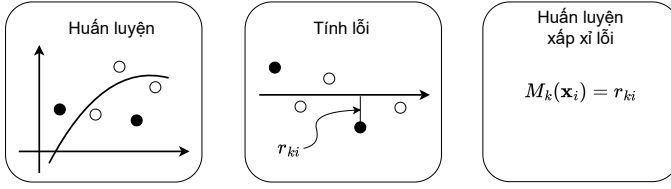
---

```

1: procedure EVALADABOOSTSAMME($M_k, \alpha_k, \mathbf{x}, C$)
2: for $y = 1$ to C do
3: $score[y] \leftarrow \sum_{k=1}^K \alpha_k \cdot \mathbb{I}[y = M_k(\mathbf{x})]$
4: end for
5: return $H(\mathbf{x}) \leftarrow \arg \max_{y=1}^C score[y]$
6: end procedure

```

---



Hình 9.4: Phương pháp Tăng cường bằng đạo hàm

mãn bài toán tối ưu, được mô tả như sau:

$$H_0(\mathbf{x}) = \alpha_0 = \arg \min_{\alpha} \sum_{i=1}^n L(\alpha, y_i)$$

Sau đó, lần lượt tìm các mô hình  $H_1(\mathbf{x}), H_2(\mathbf{x}), \dots, H_K(\mathbf{x})$  có hiệu suất tăng dần bằng bài toán tối ưu theo công thức (9.14)

$$H_k(\mathbf{x}) = H_{k-1}(\mathbf{x}) + \arg \min_{M_k \in \mathcal{H}} \left\{ \sum_{i=1}^n L(H_{k-1}(\mathbf{x}_i) + M_k(\mathbf{x}_i), y_i) \right\}. \quad (9.14)$$

Tức là tại bước thứ  $k$ , ta tìm mô hình  $M_k$  trong không gian các mô hình cơ sở  $\mathcal{H}$  sao cho khi cộng thêm  $M_k$  vào mô hình  $H_{k-1}$  ở bước trước thì tổng số lỗi là nhỏ nhất.

#### 9.4.1 Tăng cường bằng đạo hàm bậc 1

Đặt  $u_i = H_{k-1}(\mathbf{x}_i), i = 1, \dots, n$  và  $v_i = M_k(\mathbf{x}_i)$  rồi xấp xỉ hàm lỗi  $L(u_i + v_i, y_i)$  bằng khai triển Taylor bậc nhất quanh  $u_i$ , bài toán tối ưu trên trở thành

$$\min_{v_i, i=1, \dots, n} \left\{ \sum_{i=1}^n v_i \frac{\partial L(\hat{y}, y_i)}{\partial \hat{y}} \Big|_{\hat{y}=u_i} \right\}. \quad (9.15)$$

Do đó, ta chọn  $M_k$  theo hướng ngược hướng đạo hàm của hàm lỗi trên. Tức là chọn  $M_k$  sao cho điều kiện sau được thỏa mãn:

$$M_k(x_i) \approx r_{ki} = - \frac{\partial L(\hat{y}, y_i)}{\partial \hat{y}} \Big|_{\hat{y}=H_{k-1}(x_i)}, i = 1, 2, \dots, n \quad (9.16)$$

Đại lượng  $r_{ki}$  được gọi là *phần dư* của mẫu dữ liệu thứ  $i$  tại bước  $k$ . Đây chính là *bài toán hồi quy* với đầu vào là các mẫu dữ liệu  $x_i$  và đầu ra là các phần dư  $r_{ki}$ .

Sau đó ta tìm kiếm hằng số  $\alpha_k$  để tối ưu tổng số lỗi sau

$$\alpha_k = \arg \min_{\alpha} \left\{ \sum_{i=1}^n L(H_{k-1}(x_i) + \alpha M_k(x_i), y_i) \right\}. \quad (9.17)$$

Đây là bài toán tối ưu một biến nên dễ dàng giải hơn. Lược đồ tổng quát của phương pháp Tăng cường bằng đạo hàm bậc 1 như liệt kê trong Thuật toán 9.9.

---

**Thuật toán 9.9** Phương pháp Tăng cường bằng đạo hàm bậc 1 (Gradient Boosting)

---

```

1: procedure TRAINGRADIENTBOOST($\mathcal{A}, D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, K$)
2: Đặt $M_0(\mathbf{x}) \leftarrow 1$
3: Tìm $\alpha_0 \leftarrow \arg \min_{\alpha} \sum_{i=1}^n L(\alpha, y_i)$
4: $H_0(\mathbf{x}) \leftarrow \alpha_0$
5: for $k = 1$ to K do
6: for $i = 1$ to n do
7: $r_{ki} \leftarrow - \left. \frac{\partial L(\hat{y}, y_i)}{\partial \hat{y}} \right|_{\hat{y}=H_{k-1}(\mathbf{x}_i)}$
8: end for
9: Tạo tập dữ liệu huấn luyện mới: $D_k = \{(\mathbf{x}_i, r_{ki})\}_{i=1}^n$
10: Huấn luyện mô hình hồi quy: $M_k \leftarrow \mathcal{A}(D_k)$
11: Tìm $\alpha_k \leftarrow \arg \min_{\alpha} \sum_{i=1}^n L(H_{k-1}(\mathbf{x}_i) + \alpha M_k(\mathbf{x}_i), y_i)$
12: Cập nhật mô hình tổng: $H_k(\mathbf{x}) \leftarrow H_{k-1}(\mathbf{x}) + \alpha_k M_k(\mathbf{x})$
13: end for
14: return M_k, α_k với $k = 0, \dots, K$
15: end procedure

```

---

### 9.4.2 Tăng cường bằng đạo hàm bậc 2

Bên cạnh việc sử dụng đạo hàm bậc một như trình bày ở công thức (9.14), chúng ta có thể sử dụng đạo hàm bậc hai của hàm lỗi để cải

**Thuật toán 9.10** Dự đoán với Tăng cường bằng đạo hàm bậc 1

---

```

1: procedure EVALGRADIENTBOOST($M_k, \alpha_k, \mathbf{x}$)
2: $H(\mathbf{x}) \leftarrow \sum_{k=0}^K \alpha_k M_k(\mathbf{x})$
3: return $H(\mathbf{x})$
4: end procedure

```

---

thiện mô hình kết hợp. Phương pháp này được gọi là Tăng cường bằng đạo hàm bậc 2 (Newton boosting).

Cụ thể, ta khai triển hàm lỗi  $L(\hat{y}, y)$  theo khai triển Taylor bậc hai quanh  $u_i$  như sau:

$$\begin{aligned}
 L(u_i + v_i, y_i) &\approx L(u_i, y_i) + v_i \left. \frac{\partial L(\hat{y}, y_i)}{\partial \hat{y}} \right|_{\hat{y}=u_i} + \frac{1}{2} v_i^2 \left. \frac{\partial^2 L(\hat{y}, y_i)}{\partial \hat{y}^2} \right|_{\hat{y}=u_i} \\
 &= L(u_i, y_i) - v_i r_{ki} + \frac{1}{2} v_i^2 w_{ki} \\
 &= C + \frac{1}{2} w_{ki} \left( v_i - \frac{r_{ki}}{w_{ki}} \right)^2
 \end{aligned}$$

với

$$w_{ki} = \left. \frac{\partial^2 L(\hat{y}, y_i)}{\partial \hat{y}^2} \right|_{\hat{y}=H_{k-1}(\mathbf{x}_i)} \quad (9.18)$$

Để cực tiểu hàm lỗi này, ta cần chọn  $v_i$  sao cho

$$v_i = M_k(x_i) \approx \frac{r_{ki}}{w_{ki}}$$

và trọng số của mẫu dữ liệu thứ  $i$  này chính là  $w_{ki}$ .

Tăng cường bằng đạo hàm bậc 2 cải tiến Boosting bằng đạo hàm bậc 1 bằng cách tìm mô hình  $M_k(\mathbf{x})$  cho bài toán hồi quy với bộ dữ liệu có trọng số  $D_k = \{(\mathbf{x}_i, r_{ki}/w_{ki})\}$ , trong đó mẫu dữ liệu thứ  $i$  có trọng số là  $w_{ki}$ , chính là đạo hàm bậc 2 của hàm lỗi theo công thức (9.18). Tăng cường bằng đạo hàm bậc 2 tận dụng cả đạo hàm bậc hai để xác định trọng số của từng mẫu, do đó, thường hội tụ nhanh hơn nhưng chi phí tính toán cao hơn vì phải ước lượng

đạo hàm bậc hai. Tăng cường bằng đạo hàm bậc 1 đơn giản hơn và phổ biến hơn trong thực tế vì dễ triển khai và mở rộng.

---

**Thuật toán 9.11** Tăng cường bằng đạo hàm bậc 2
 

---

```

1: procedure TRAINNEWTONBOOST($\mathcal{A}, D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, K$)
2: Đặt $M_0(\mathbf{x}) \leftarrow 1$
3: Tìm $\alpha_0 \leftarrow \arg \min_{\alpha} \sum_{i=1}^n L(\alpha, y_i)$
4: $H_0(\mathbf{x}) \leftarrow \alpha_0$
5: for $k = 1$ to K do
6: for $i = 1$ to n do
7: $r_{ki} \leftarrow - \left. \frac{\partial L(\hat{y}, y_i)}{\partial \hat{y}} \right|_{\hat{y}=H_{k-1}(\mathbf{x}_i)}$
8: $w_{ki} \leftarrow \left. \frac{\partial^2 L(\hat{y}, y_i)}{\partial \hat{y}^2} \right|_{\hat{y}=H_{k-1}(\mathbf{x}_i)}$
9: end for
10: Tạo tập dữ liệu huấn luyện mới: $D_k = \{(\mathbf{x}_i, r_{ki}/w_{ki})\}_{i=1}^n$
11: Huấn luyện mô hình hồi quy: $M_k \leftarrow \mathcal{A}(D_k, w_k)$
12: Tìm $\alpha_k \leftarrow \arg \min_{\alpha} \sum_{i=1}^n L(H_{k-1}(\mathbf{x}_i) + \alpha M_k(\mathbf{x}_i), y_i)$
13: Cập nhật mô hình tổng: $H_k(\mathbf{x}) \leftarrow H_{k-1}(\mathbf{x}) + \alpha_k M_k(\mathbf{x})$
14: end for
15: return M_k, α_k với $k = 0, \dots, K$
16: end procedure

```

---

## 9.5 Tình huống áp dụng: XGBoost

XGBoost xây dựng mô hình bằng cách kết hợp nhiều cây quyết định, mỗi cây mới được xây dựng để sửa các sai số của cây trước đó. XGBoost sử dụng một hàm lỗi tổng quát để tối ưu hóa mô hình, có thể được hiểu như một dạng tổng quát của các mô hình cây quyết định. Giả sử ở bước thứ  $t$ , mô hình đã học được một hàm dự đoán  $\hat{y}_i^{(t-1)} = \sum_{k=1}^{t-1} f_k(\mathbf{x}_i)$  từ các cây quyết định trước đó

$$\hat{y}_i^{(t-1)} = \sum_{k=1}^{t-1} f_k(\mathbf{x}_i). \quad (9.19)$$

Để huấn luyện cây quyết định thứ  $t$ , ta cần tìm cây  $f_t$  sao cho hàm lỗi sau khi thêm cây  $f_t$  vào mô hình là nhỏ nhất.

Giả sử hàm lỗi  $L(y_i, \hat{y}_i^{(t-1)})$ , với  $g_i$  và  $h_i$  lần lượt là đạo hàm bậc nhất và bậc hai của hàm lỗi tại điểm  $\hat{y}_i^{(t-1)}$ :

$$g_i = \frac{\partial}{\partial \hat{y}_i^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)}), \quad h_i = \frac{\partial^2}{\partial (\hat{y}_i^{(t-1)})^2} L(y_i, \hat{y}_i^{(t-1)}), \quad (9.20)$$

Chúng ta có thể xấp xỉ hàm lỗi khi thêm vào cây mới  $f_t$  là:

$$\begin{aligned} \tilde{\mathcal{L}}^{(t)} &= \sum_{i=1}^n \left[ L(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i (f_t(x_i))^2 \right] + \Omega(f_t) \\ &\approx \sum_{i=1}^n \left[ g_i f_t(x_i) + \frac{1}{2} h_i (f_t(x_i))^2 \right] + \Omega(f_t), \end{aligned} \quad (9.21)$$

với  $\Omega$  là hàm điều chỉnh trên cây mới  $f_t$ . Trong XGBoost, hàm điều chỉnh cho một cây với  $T$  lá và trọng số lá  $w_j$  được định nghĩa là:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2. \quad (9.22)$$

Tổng hợp lại, ta thu được hàm lỗi xấp xỉ khi thêm cây  $f_t$  vào mô hình là:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[ g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i (f_t(\mathbf{x}_i))^2 \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2. \quad (9.23)$$

Cây mới  $f_t$  được lựa chọn để cực tiểu hoá hàm lỗi  $\tilde{\mathcal{L}}^{(t)}$ . Việc sử dụng hàm lỗi  $\tilde{\mathcal{L}}^{(t)}$  để huấn luyện cây  $f_t$  có ý nghĩa là cây mới có thể sửa chữa các lỗi do kết hợp  $t - 1$  các cây trước.

Trong ứng dụng, XGBoost cung cấp một API để huấn luyện mô hình và dự đoán trên dữ liệu. Người học có thể xem ví dụ về việc sử dụng XGBoost trên dữ liệu IRIS tại <https://gist.github.com/tqlong/de320d89aab422a0bf0978f84e5d75ae>



## 9.6 Tổng kết chương

Chương này trình bày các phương pháp học kết hợp nhằm nâng cao hiệu suất dự đoán của mô hình Học máy thông qua việc phối hợp nhiều mô hình con.

Bagging giảm phương sai bằng cách huấn luyện song song nhiều mô hình trên các tập dữ liệu con được lấy mẫu ngẫu nhiên có lặp, và kết hợp đầu ra bằng trung bình hoặc bỏ phiếu đa số. Boosting cải thiện dần hiệu suất qua từng vòng lặp bằng cách tập trung vào các mẫu dữ liệu khó, nhờ cơ chế gán trọng số thích nghi cho lỗi của mô hình trước. XGBoost là một cải tiến mạnh mẽ của Boosting với khả năng tối ưu hóa hiệu quả và hỗ trợ xử lý dữ liệu lớn nhờ cơ chế tính toán phân tán.

Học kết hợp cho phép kết hợp linh hoạt nhiều mô hình con khác nhau, tận dụng thế mạnh riêng của từng mô hình nhằm đạt hiệu suất tổng thể vượt trội. Tuy nhiên, việc sử dụng nhiều mô hình cũng làm tăng chi phí tính toán và gây khó khăn trong việc diễn giải kết quả, đặc biệt trong các hệ thống đòi hỏi minh bạch.

Nhìn chung, học kết hợp là một trong những kỹ thuật quan trọng và hiệu quả trong xây dựng mô hình Học máy hiện đại, và vẫn đang là chủ đề thu hút nhiều nghiên cứu trong cộng đồng khoa học dữ liệu.

### Bài tập

1. Cho một tập dữ liệu nhỏ gồm 10 mẫu:  $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ . Thực hiện phương pháp Bootstrap để tạo ra 5 tập dữ liệu con (mỗi tập có 10 phần tử, lấy mẫu ngẫu nhiên có hoàn lại). Tính giá trị trung bình của từng tập dữ liệu con và ước lượng điểm trung bình cùng khoảng tin cậy 95% cho giá trị trung bình của tập dữ liệu gốc.

2. *[Lập trình]* Sử dụng thư viện `scikit-learn` trong Python, huấn luyện một mô hình Rừng ngẫu nhiên trên tập dữ liệu IRIS. So sánh hiệu suất về mặt độ chính xác của mô hình này với một cây quyết định đơn lẻ trên cùng tập dữ liệu.
3. *[Tìm hiểu]* Giải thích sự khác biệt chính giữa phương pháp Bagging và Boosting. Trong trường hợp nào thì nên sử dụng Bagging, và trong trường hợp nào thì nên sử dụng Boosting?
4. Cho tập dữ liệu sau

$$D = \{(\mathbf{x}_1, y_1 = 1), (\mathbf{x}_2, y_2 = -1), (\mathbf{x}_3, y_3 = 1), (\mathbf{x}_4, y_4 = -1)\}$$

và dự đoán từ mô hình  $M_1(\mathbf{x})$  là

$$M_1(\mathbf{x}_1) = 1, M_1(\mathbf{x}_2) = 1, M_1(\mathbf{x}_3) = -1, M_1(\mathbf{x}_4) = -1$$

Hãy thực hiện một bước của thuật toán AdaBoost

- Tính lỗi  $\epsilon_1$ .
  - Tính trọng số  $\alpha_1$ .
  - Cập nhật trọng số mẫu  $w_1, w_2, w_3, w_4$ .
5. *[Lập trình]* Nghiên cứu và triển khai một biến thể của AdaBoost là SAMME (dành cho phân loại đa lớp) sử dụng `scikit-learn`.
  6. Viết tường minh công thức tính giá trị mục tiêu  $r_{ki}$  trong thuật toán Tăng cường bằng đạo hàm bậc 1 cho các trường hợp
    - Mô hình hồi quy tuyến tính, hàm lỗi là  $L = \frac{1}{2}(\hat{y} - y)^2$ .
    - Mô hình hồi quy logistic, hàm lỗi là  $L = \log(1 + e^{-\hat{y}y})$ .
    - Mô hình SVM, hàm lỗi là  $L = \max(0, 1 - \hat{y}y)$ .
  7. Viết tường minh công thức tính giá trị mục tiêu  $r_{ki}/w_{ki}$  trong thuật toán Tăng cường bằng đạo hàm bậc 2 trong các trường hợp hàm lỗi trên.

8. *[Lập trình]* Sử dụng thư viện **xgboost** trong Python để giải quyết bài toán phân loại trên tập dữ liệu IRIS. Thực hiện các bước sau:

- Chuẩn bị dữ liệu (tải và chia tập huấn luyện/kiểm tra).
- Huấn luyện mô hình XGBoost với các siêu tham số mặc định.
- Tinh chỉnh ít nhất hai siêu tham số (ví dụ: độ sâu lớn nhất, tốc độ học) để cải thiện hiệu suất.
- Đánh giá mô hình bằng độ chính xác và trực quan hóa tầm quan trọng của các đặc trưng.



---

# Tài liệu tham khảo

---

- [1] Breiman, L., *Bagging predictors*, Machine Learning, vol. 24, no. 2, pp. 123–140, 1996.
- [2] Freund, Y., and Schapire, R. E., *A decision-theoretic generalization of on-line learning and an application to boosting*, Journal of Computer and System Sciences, vol. 55, no. 1, pp. 119–139, 1997.
- [3] Breiman, L., *Random forests*, Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [4] Friedman, J. H., *Greedy function approximation: A gradient boosting machine*, Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.
- [5] Chen, T., and Guestrin, C., *XGBoost: A scalable tree boosting system*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.



# Chương 10

## Bài toán trích xuất đặc trưng

Bên cạnh việc thiết kế mô hình, cách trích xuất và biểu diễn đặc trưng là yếu tố then chốt ảnh hưởng đến hiệu suất và tốc độ học của hệ thống Học máy.

Chương 10 tập trung vào các kỹ thuật giảm chiều và trích chọn đặc trưng, bao gồm Phân tích thành phần chính (PCA), các biến thể của bộ tự mã hóa (autoencoder) và các phương pháp học có giám sát như trích chọn dựa trên độ quan trọng của đặc trưng. Các phương pháp này nhằm loại bỏ nhiễu, giảm độ phức tạp mô hình và làm nổi bật các đặc trưng có giá trị thống kê hoặc ý nghĩa phân biệt cao trong dữ liệu.

Nội dung chương có mối liên hệ chặt chẽ với các mô hình học sâu như MLP (Chương 3), CNN (Chương 5) và các mạng cho dữ liệu chuỗi (Chương 6), nơi đặc trưng có thể được học tự động thông qua các tầng biểu diễn trừu tượng.

Chương này cung cấp giúp người học hiểu và cải thiện khả năng biểu diễn dữ liệu trong các mô hình Học máy hiện đại.

## 10.1 Giới thiệu

Trong học máy thống kê, dữ liệu thô thường là một tập hợp các thuộc tính có thể dưới dạng số, véc-tơ, văn bản hoặc các dạng tổng quát hơn. Để các mô hình học máy có thể làm việc hiệu quả với dữ liệu thô, chúng ta cần phải chuyển đổi dữ liệu thô thành các đặc trưng hoặc thuộc tính mà mô hình có thể hiểu được. Quá trình này được gọi là trích xuất đặc trưng. Các đặc trưng mới có thể biểu diễn chính xác dữ liệu thông qua việc rút gọn hoặc hữu ích hơn khi sử dụng trong các nhiệm vụ học máy khác. Chúng ta có thể mô hình hoá quá trình trích xuất đặc trưng thông qua hàm mã hoá  $\mathbf{E}$  như sau:

**Định nghĩa 10.1** (Trích xuất đặc trưng). Trích xuất đặc trưng là quá trình chuyển đổi dữ liệu thô thành các đặc trưng hoặc thuộc tính mới có thể hữu ích hơn trong việc giải quyết một bài toán cụ thể.

$$\mathbf{x} \rightarrow \mathbf{z} = \mathbf{E}(\mathbf{x}), \quad (10.1)$$

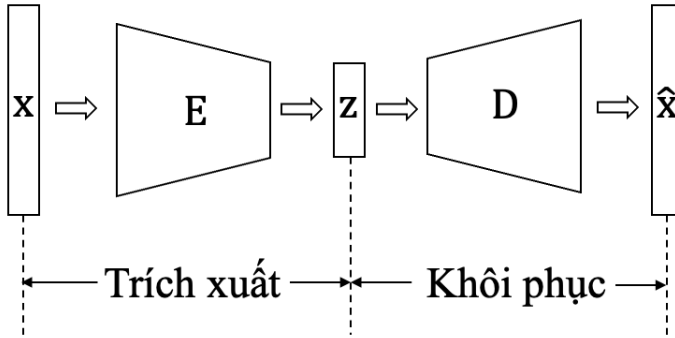
trong đó,  $\mathbf{x}$  là dữ liệu thô,  $\mathbf{z}$  là đặc trưng mới,  $\mathbf{E}$  là hàm trích xuất đặc trưng hay hàm mã hoá.

Đặc trưng mới  $\mathbf{z}$  có thể là một véc-tơ hoặc ma trận hoặc một loại dữ liệu khác. Các đặc trưng mới có thể được sử dụng để xây dựng mô hình học máy hoặc để giải quyết một bài toán cụ thể. Ví dụ, trong bài toán nén dữ liệu, ta có thể sử dụng đặc trưng  $\mathbf{z}$  để khôi phục lại dữ liệu thô  $\mathbf{x}$  thông qua một hàm giải mã  $\mathbf{D}$ :

$$\mathbf{z} \rightarrow \hat{\mathbf{x}} = \mathbf{D}(\mathbf{z}) \approx \mathbf{x} \quad (10.2)$$

Trong đó,  $\hat{\mathbf{x}}$  là dữ liệu khôi phục được từ đặc trưng  $\mathbf{z}$ . Hình 10.1 mô tả quá trình trích xuất đặc trưng và khôi phục lại dữ liệu thô từ đặc trưng bằng cách kết hợp cả hai hàm mã hoá và giải mã ở công thức (10.1) và (10.2).





**Hình 10.1:** Quá trình trích xuất đặc trưng và khôi phục lại dữ liệu thô

Có nhiều cách tiếp cận khác nhau để trích xuất đặc trưng từ dữ liệu, bao gồm: xây dựng đặc trưng thủ công dựa trên tri thức chuyên gia, học đặc trưng thông qua các phương pháp học không giám sát và học đặc trưng tự động bằng các mô hình học sâu. Các đặc trưng thủ công thường dễ hiểu nhưng phụ thuộc nhiều vào kinh nghiệm; học không giám sát có thể khai thác cấu trúc ẩn trong dữ liệu nhưng đôi khi thiếu định hướng cụ thể; học sâu cho phép mô hình hóa các đặc trưng phức tạp nhưng chi phí tính toán cao. Trong thực tiễn, việc lựa chọn phương pháp trích xuất đặc trưng thường phải trải qua quá trình thử nghiệm và hiệu chỉnh dựa trên bản chất của tập dữ liệu cũng như yêu cầu cụ thể của bài toán đặt ra.

## 10.2 Phương pháp phân tích thành phần chính

Phương pháp phân tích thành phần chính PCA (Principal Component Analysis) là một trong các phương pháp trích xuất đặc trưng kinh điển trong lĩnh vực học máy. PCA thường được sử dụng để giảm chiều dữ liệu, nén dữ liệu và để trực quan hóa dữ liệu. PCA làm việc với dữ liệu dạng số, đặc biệt là dữ liệu dạng ma trận.

Cụ thể, giả sử ta có một tập dữ liệu  $\mathbf{X} \in \mathbb{R}^{n \times d}$ , ta có dạng

véc-tơ của  $\mathbf{X}$  là:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_n^T \end{bmatrix} \in \mathbb{R}^{n \times d}$$

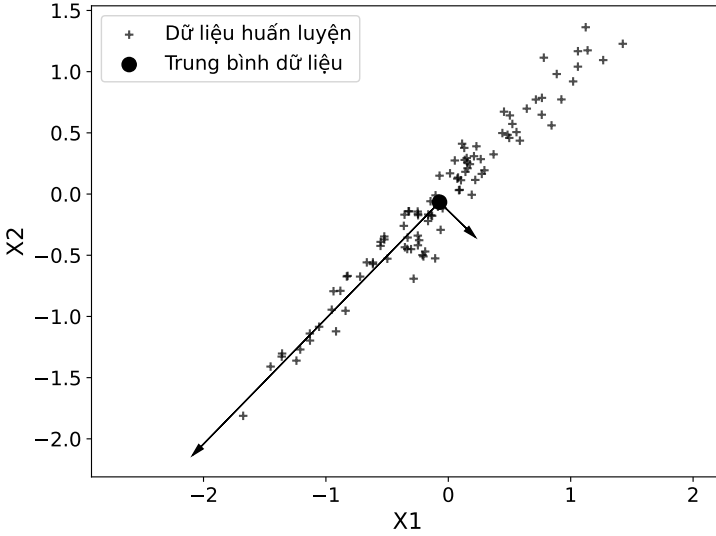
với các mẫu dữ liệu  $\mathbf{x}_i \in \mathbb{R}^d$  nằm trên các dòng của  $\mathbf{X}$ . Mục tiêu của PCA là tìm một ma trận  $\mathbf{W} \in \mathbb{R}^{d \times d'}$  để biến đổi dữ liệu ban đầu  $\mathbf{X}$  thành dữ liệu mới  $\mathbf{Z} = \mathbf{XW} \in \mathbb{R}^{n \times d'}$  với  $d' \leq d$ . Trong đó, các cột của  $\mathbf{Z}$  là các thành phần chính của dữ liệu  $\mathbf{X}$ . Các dòng của  $\mathbf{Z}$  là các biểu diễn mới của các mẫu dữ liệu  $\mathbf{x}_i$ .

Đặc điểm của phương pháp PCA là các thành phần chính của được sắp xếp theo thứ tự giảm dần của phương sai. Trong đó, phương sai được hiểu là sự biến thiên hay thông tin của dữ liệu. Hình 10.2 minh họa cho độ phân tán của dữ liệu trong không gian 2 chiều, được minh họa qua hai véc-tơ đặc trưng cho hai thành phần chính. Độ dài của các véc-tơ thành phần chính này đặc trưng cho độ phân bố của dữ liệu theo các chiều khác nhau.

Khi sắp xếp theo thứ tự giảm dần của phương sai, thành phần chính đầu tiên sẽ là thành phần chính có phương sai lớn nhất, tức là thành phần chứa nhiều thông tin của dữ liệu nhất. Thành phần chính thứ hai sẽ là thành phần chứa nhiều thông tin thứ hai và cứ tiếp tục như vậy. Do đó, ta có thể giảm chiều dữ liệu bằng cách chỉ giữ lại một số thành phần chính đầu tiên. Các thành phần chính còn lại có thể bị loại bỏ hoặc có thể được giữ lại để tăng độ chính xác của mô hình.

### 10.2.1 Thành phần chính đầu tiên

Giả sử dữ liệu huấn luyện gồm  $n$  mẫu dữ liệu  $\mathbf{x}_1, \dots, \mathbf{x}_n$  trong không gian  $\mathbb{R}^d$ . Chúng ta có công thức tính phương sai của dữ liệu



**Hình 10.2:** Minh hoạ phương pháp phân tích thành phần chính PCA trên dữ liệu 2 chiều

ban đầu là:

$$\text{Var}[\mathbf{x}_1, \dots, \mathbf{x}_n] = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \mu\|^2 \quad (10.3)$$

trong đó  $\mu = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$  là trung bình của bộ dữ liệu.

Gọi véc-tơ đơn vị  $\mathbf{w}_1$  ( $\|\mathbf{w}_1\| = 1$ ) là cột đầu tiên của ma trận  $\mathbf{W}$ . Giá trị này cũng đặc trưng cho thành phần chính đầu tiên. Trọng số của thành phần này của dữ liệu  $\mathbf{X}$  được cho bởi công thức (10.4):

$$z_i = (\mathbf{x}_i - \mu)^T \mathbf{w}_1 \quad (10.4)$$

Nếu ta chiếu  $\mathbf{x}_i - \mu$  lên  $\mathbf{w}_1$ , ta sẽ được véc-tơ khôi phục  $\hat{\mathbf{x}}_i = z_i \mathbf{w}_1 + \mu$ . Khi đó phương sai của bộ dữ liệu mới  $\hat{\mathbf{X}}$  được định nghĩa như sau:

$$\text{Var}[\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_n] = \frac{1}{n} \sum_{i=1}^n \|\hat{\mathbf{x}}_i - \mu\|^2 \quad (10.5)$$

Lưu ý, là trung bình của các véc-tơ trong bộ dữ liệu mới cũng bằng  $\mu$ . Theo định lý Pythagoras, ta có đẳng thức sau:

$$\|\hat{\mathbf{x}}_i - \mu\|^2 + \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = \|\mathbf{x}_i - \mu\|^2 \quad (10.6)$$

Kết hợp công thức (10.5) và (10.6), ta thu được:

$$\text{Var}[\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_n] + \underbrace{\frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2}_{\text{Lỗi khôi phục}} = \text{Var}[\mathbf{x}_1, \dots, \mathbf{x}_n] = \text{hằng số}$$

Như vậy, ta vừa chứng minh sự tương đương giữa cực đại hoá phương sai và cực tiểu hoá lỗi khôi phục khi chiếu dữ liệu lên thành phần chính đầu tiên.

**Định lý 10.2** (Phương sai và lỗi khôi phục). *Cực đại hoá phương sai của thành phần chính đầu tiên tương đương với cực tiểu hoá lỗi khôi phục của dữ liệu khi chiếu dữ liệu lên thành phần chính đầu tiên.*

Hơn nữa, kết hợp đẳng thức  $\|\mathbf{w}_1\| = 1$  và  $\|\hat{\mathbf{x}}_i - \mu\|^2 = \|z_i \mathbf{w}_1\|^2 = |z_i|^2$  ta thu được công thức (10.7).

$$\text{Var}[\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_n] = \text{Var}[z_1, \dots, z_n] \quad (10.7)$$

Nghĩa là phương sai của dữ liệu mới bằng phương sai của các hệ số tương ứng với thành phần chính đầu tiên.

Để cực đại phương sai này, ta cần tìm  $\mathbf{w}_1$  là véc-tơ đơn vị,  $\|\mathbf{w}_1\| = 1$ , sao cho  $\mathbf{w}_1^T \Sigma \mathbf{w}_1$  đạt cực đại. Trong đó  $\Sigma$  là ma trận hiệp phương sai của dữ liệu  $\mathbf{X}$ . Để tìm  $\mathbf{w}_1$  thỏa mãn điều kiện trên, ta có thể giải bài toán tối ưu

$$\max_{\|\mathbf{w}_1\|=1} \mathbf{w}_1^T \Sigma \mathbf{w}_1 \quad (10.8)$$

Để tìm nghiệm cực đại được cho bởi hàm mục tiêu ở công thức (10.8), ta sử dụng phương pháp nhân tử Lagrange như sau. Đầu tiên, ta xây dựng hàm Lagrange như sau:

$$\mathcal{L}(\mathbf{w}_1, \lambda) = \mathbf{w}_1^T \Sigma \mathbf{w}_1 + \lambda(1 - \|\mathbf{w}_1\|^2). \quad (10.9)$$

Sử dụng điều kiện triệt tiêu của đạo hàm tại cực trị, ta tính được đạo hàm của hàm Lagrange theo  $\mathbf{w}_1$  và  $\lambda$  như sau:

$$\nabla_{\mathbf{w}_1} \mathcal{L}(\mathbf{w}_1, \lambda) = 2\Sigma \mathbf{w}_1 - 2\lambda \mathbf{w}_1 = 0 \quad (10.10)$$

$$\Rightarrow \Sigma \mathbf{w}_1 = \lambda \mathbf{w}_1. \quad (10.11)$$

tức là  $\mathbf{w}_1$  là véc-tơ riêng của  $\Sigma$  với trị riêng  $\lambda$ . Thế ngược lại, phương sai của thành phần chính đầu tiên là

$$\text{Var}[z_1, \dots, z_n] = \mathbf{w}_1^T \Sigma \mathbf{w}_1 = \lambda \mathbf{w}_1^T \mathbf{w}_1 = \lambda. \quad (10.12)$$

Do đó, ta có thể tìm được  $\mathbf{w}_1$  bằng cách tìm véc-tơ riêng  $\mathbf{w}_1$  ứng với giá trị riêng lớn nhất của  $\Sigma$ . Do ma trận  $\Sigma$  là ma trận đối xứng và toàn phương xác định dương, nên ta có thể tìm được  $d$  giá trị riêng là số thực không âm và sắp xếp chúng giảm dần

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0. \quad (10.13)$$

Như vậy,  $\mathbf{w}_1$  là véc-tơ riêng tương ứng với giá trị riêng lớn nhất  $\lambda_1$ .

Theo lập luận ở trên, véc-tơ  $\mathbf{w}_1$  này cũng tối thiểu lỗi khôi phục dữ liệu khi chiếu lên nó.

### 10.2.2 Các thành phần chính tiếp theo

Để thành phần chính thứ hai không có tương quan với thành phần đầu tiên, ta cần tìm  $\mathbf{w}_2$  sao cho trực giao với  $\mathbf{w}_1$ , hay  $\mathbf{w}_2^T \mathbf{w}_1 = 0$ . Hay  $\mathbf{w}_2$  thuộc vào không gian con của  $\mathbb{R}^d$  trực giao với  $\mathbf{w}_1$ . Vì tính trực giao của các véc-tơ riêng, lập luận tương tự như với thành phần đầu tiên, ta có thể tìm được  $\mathbf{w}_2$  bằng cách tìm véc-tơ riêng

$\mathbf{w}_2$  ứng với giá trị riêng lớn thứ hai của  $\Sigma$ , tức là  $\lambda_2$ . Tiếp tục như vậy, ta có thể tìm được các thành phần chính thứ  $k$  bằng cách tìm véc-tơ riêng  $\mathbf{w}_k$  ứng với giá trị riêng lớn thứ  $k$ , tức là  $\lambda_k$ , của  $\Sigma$ .

Do các thành phần chính không tương quan, thông tin (phương sai) của dữ liệu ban đầu được lưu trữ trong các đặc trưng mới là

$$\lambda_1 + \lambda_2 + \cdots + \lambda_{d'}. \quad (10.14)$$

Do đó, ta có thể chọn  $d'$  sao cho tỉ lệ phương sai được lưu trữ trong các đặc trưng mới vượt qua một ngưỡng cho trước. Ví dụ, ta có thể chọn  $d'$  sao cho tỉ lệ

$$\frac{\lambda_1 + \lambda_2 + \cdots + \lambda_{d'}}{\lambda_1 + \lambda_2 + \cdots + \lambda_d} \geq \theta \quad (10.15)$$

với  $\theta \in (0, 1)$ . Ví dụ, có thể chọn  $\theta = 0.95$  là tỉ lệ phương sai giữ lại được ít nhất 95%. Tỉ lệ 95% này thường được sử dụng trong thực hành PCA.

---

**Thuật toán 10.1** Thuật toán Phân tích thành phần chính (PCA)

---

- 1: **procedure** PCA( $\mathbf{X} \in \mathbb{R}^{n \times d}, \theta$ )
- 2:   Tâm hoá dữ liệu:  $\mathbf{X} \leftarrow \mathbf{X} - \frac{1}{n} \mathbf{1}_n \mathbf{1}_n^T \mathbf{X}$
- 3:   Tính ma trận hiệp phương sai:  $\Sigma \leftarrow \frac{1}{n-1} \mathbf{X}^T \mathbf{X}$
- 4:   Phân tích trị riêng:

Tính các véc-tơ riêng  $\mathbf{w}_1, \dots, \mathbf{w}_d$  và giá trị riêng  $\lambda_1 \geq \dots \geq \lambda_d$

- 5:   Tìm  $d'$  sao cho:  $\frac{\sum_{i=1}^{d'} \lambda_i}{\sum_{i=1}^d \lambda_i} \geq \theta$
  - 6:   Tạo ma trận:  $\mathbf{W} \leftarrow [\mathbf{w}_1, \dots, \mathbf{w}_{d'}] \in \mathbb{R}^{d \times d'}$
  - 7:   Tính các thành phần chính:  $\mathbf{Z} \leftarrow \mathbf{XW}$
  - 8:   **return**  $\mathbf{W}, \mathbf{Z}$
  - 9: **end procedure**
- 

Việc tính các véc-tơ riêng và các giá trị riêng của ma trận  $\Sigma$  có thể được thực hiện thông qua các phân tích trị riêng hoặc phân tích

trị đơn (Singular Value Decomposition - SVD). Các phương pháp này thường được cài đặt sẵn trong các thư viện lập trình của các ngôn ngữ Matlab, Python, R, C/C++.

### 10.2.3 Giảm chiều dữ liệu

Ghép các cột  $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{d'}$  lại thành ma trận  $\mathbf{W}$  có  $d'$  cột. Ta có đẳng thức sau:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}. \quad (10.16)$$

Các cột của ma trận  $\mathbf{Z}$  bao gồm  $\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_{d'}$  là biểu diễn mới của dữ liệu ban đầu  $\mathbf{X}$ .

Lưu ý rằng, nếu ta chọn  $d' = d$ , thì ma trận  $\mathbf{W}$  sẽ là ma trận vuông và trực chuẩn (ma trận của  $d$  véc-tơ riêng). Ta có thể khôi phục chính xác dữ liệu  $\mathbf{X}$  từ dữ liệu đã giảm chiều  $\mathbf{Z}$  bằng cách biến đổi như sau:

$$\mathbf{X} = \mathbf{X} \underbrace{\mathbf{W}\mathbf{W}^T}_{\mathbf{I}} = \mathbf{Z}\mathbf{W}^T = \sum_{k=1}^d \mathbf{z}_k \mathbf{w}_k^T. \quad (10.17)$$

Công thức (10.17) cho thấy rằng, dữ liệu ban đầu  $\mathbf{X}$  có thể được khôi phục bằng tổng có trọng số các thành phần chính  $\mathbf{w}_k$  (trọng số  $\mathbf{Z}$  là biểu diễn mới của dữ liệu).

Khi  $d' < d$ , ta có thể khôi phục một cách xấp xỉ dữ liệu  $\mathbf{X}$  từ dữ liệu đã giảm chiều  $\mathbf{Z}$  bằng đại lượng

$$\tilde{\mathbf{X}} = \mathbf{Z}\mathbf{W}^T = \sum_{k=1}^{d'} \mathbf{z}_k \mathbf{w}_k^T. \quad (10.18)$$

Công thức (10.18) cho thấy rằng, dữ liệu ban đầu  $\mathbf{X}$  có thể được xấp xỉ bằng việc loại bỏ các thành phần chính  $\mathbf{w}_k$  có ít thông tin ( $\lambda_k$  nhỏ) của dữ liệu ban đầu.

Ví dụ: Giả sử ta có dữ liệu  $\mathbf{X}$  gồm  $n = 1.000.000$  ảnh có độ phân giải  $64 \times 64 = 4096$  điểm ảnh. Bộ dữ liệu gốc có kích thước  $1.000.000 \times 4.096 \text{ byte} \approx 3.096MB$ . Nếu ta chọn  $d' = 128$ , thì ma trận  $\mathbf{W}$  có kích thước  $4.096 \times 128 \text{ byte} = 512KB$ . Ta có thể nén dữ liệu gốc thành dữ liệu đã giảm chiều  $\mathbf{Z}$  có kích thước  $1.000.000 \times 128 \text{ byte} \approx 122MB$ , giảm kích thước xuống còn  $122/3.096 \approx 3.9\%$  so với kích thước gốc.

Ở dưới góc độ tối thiểu hoá lỗi khôi phục dữ liệu, thuật toán PCA tìm ra các thành phần chính  $\mathbf{W}$  có lỗi khôi phục nhỏ nhất theo Định lý 10.3 sau.

**Định lý 10.3** (PCA tối ưu lỗi khôi phục). *Phương pháp PCA tìm được nghiệm tối ưu của bài toán sau*

$$\min_{\mathbf{W} \in \mathbb{R}^{n \times d'} : \|\mathbf{w}_1\| = \dots = \|\mathbf{w}_{d'}\| = 1} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^T\|_F^2 \quad (10.19)$$

Trong đó, phép biến đổi tuyến tính  $\mathbf{X}\mathbf{W}$  là một phép chiếu dữ liệu  $\mathbf{X}$  lên không gian con có  $d'$  chiều. Phép biến đổi tuyến tính  $\mathbf{Z}\mathbf{W}^T$  là một phép khôi phục dữ liệu từ không gian con về không gian gốc. Chuẩn Frobenius của lỗi khôi phục  $\|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^T\|_F^2$  đo mức độ sai khác giữa dữ liệu gốc  $\mathbf{X}$  và dữ liệu khôi phục  $\mathbf{X}\mathbf{W}\mathbf{W}^T$ .

## 10.3 Bộ tự mã hóa sâu

### 10.3.1 Giới thiệu

Trong phân tích thành phần chính, chúng ta đã thấy rằng, việc trích xuất đặc trưng (nén dữ liệu) được thực hiện bởi một hàm tuyến tính

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} = \mathbf{E}(\mathbf{x}) \quad (10.20)$$



trong đó,  $\mathbf{W} \in \mathbb{R}^{d \times d'}$  là ma trận các thành phần chính. Hơn nữa, việc khôi phục dữ liệu từ  $\mathbf{z}$  cũng được thực hiện bởi hàm tuyến tính

$$\hat{\mathbf{x}} = \mathbf{W}\mathbf{z} = \mathbf{D}(\mathbf{z}) \quad (10.21)$$

Như vậy, PCA là một phương pháp trích xuất đặc trưng tuyến tính. Tuy nhiên, nó không thể trích xuất được các đặc trưng phi tuyến. Để giải quyết vấn đề này, chúng ta có thể sử dụng các mô hình học sâu để trích xuất đặc trưng phi tuyến và khôi phục dữ liệu từ các đặc trưng này. Trong phần này, chúng ta sẽ giới thiệu một mô hình học sâu được sử dụng phổ biến để trích xuất đặc trưng gọi là *bộ tự mã hóa sâu* DAE (Deep Auto-encoder).

Gọi một mạng nơ-ron sâu  $f_\theta : \mathbf{x} \mapsto \mathbf{z}$  với tham số  $\theta$  là bộ mã hoá và một mạng nơ-ron sâu khác  $g_\theta : \mathbf{z} \mapsto \hat{\mathbf{x}}$  là bộ giải mã. Mô hình tổng quát của DAE được mô tả như sau:

$$\mathbf{z} = f_\theta(\mathbf{x}) \quad (10.22)$$

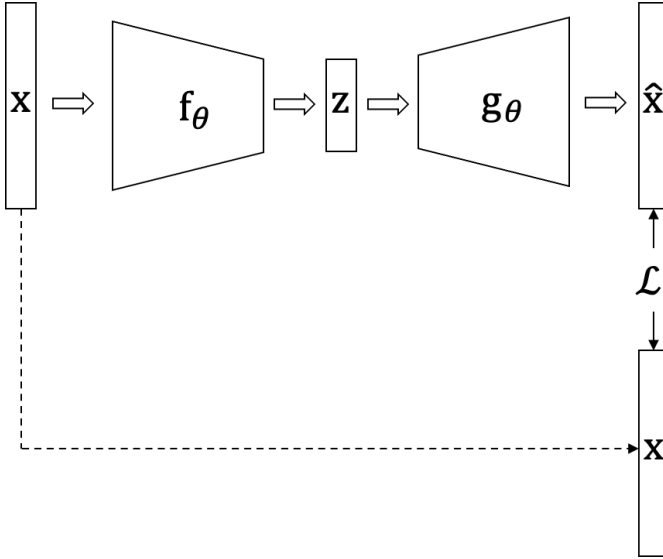
$$\hat{\mathbf{x}} = g_\theta(\mathbf{z}) \quad (10.23)$$

### 10.3.2 Huấn luyện bộ tự mã hóa sâu

Để huấn luyện một bộ tự mã hóa sâu, chúng ta cần xác định hàm lỗi  $\mathcal{L}$  đối với bộ tham số  $\theta$  của mô hình. Giống như phương pháp PCA, chúng ta cũng muốn tối thiểu hóa sai số khôi phục dữ liệu  $\hat{\mathbf{x}} = g_\theta(\mathbf{z})$  từ  $\mathbf{z} = f_\theta(\mathbf{x})$ . Do đó, chúng ta có thể sử dụng hàm lỗi trung bình tổng bình phương sai số khôi phục được cho bởi công thức (10.24).

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - g_\theta(f_\theta(\mathbf{x}_i))\|^2 \quad (10.24)$$

trong đó,  $\mathbf{x}_i$  là mẫu dữ liệu thứ  $i$  và  $\hat{\mathbf{x}}_i$  là kết quả khôi phục dữ liệu của mô hình. Hình 10.3 mô tả quá trình huấn luyện mô hình DAE, gồm hai bước: bước mã hoá và bước giải mã.



**Hình 10.3:** Minh hoạ quá trình mã hoá và giải mã trong bước huấn luyện của bộ tự mã hoá sâu

Như vậy, dữ liệu huấn luyện của bộ tự mã hoá sâu là các mẫu dữ liệu  $\mathbf{x}_i$  và nhãn tương ứng  $\mathbf{y}_i = \mathbf{x}_i$ . Tức là ta sử dụng bộ dữ liệu  $D = (\mathbf{x}_1, \mathbf{x}_1), (\mathbf{x}_2, \mathbf{x}_2), \dots, (\mathbf{x}_n, \mathbf{x}_n)$ . Ta còn gọi phương pháp huấn luyện này là phương pháp huấn luyện *tự giám sát* (self supervised).

### 10.3.3 Bộ tự mã hoá khử nhiễu

Một trong những ứng dụng phổ biến của tự mã hoá sâu DAE là bộ tự mã hoá khử nhiễu. Trong phương pháp này, chúng ta sử dụng một mạng nơ-ron sâu để khôi phục dữ liệu  $\hat{\mathbf{x}}$  từ dữ liệu bị nhiễu  $\mathbf{x} + \epsilon$ . Trong đó  $\epsilon$  là nhiễu được thêm vào dữ liệu  $\mathbf{x}$ . Thường  $\epsilon$  được lấy từ phân phối chuẩn  $\mathcal{N}(0, \sigma^2 \mathbf{I})$  với  $\sigma$  là độ lớn của nhiễu. Khi đó, bộ dữ liệu huấn luyện trở thành

$$D = (\mathbf{x}_1 + \epsilon_1, \mathbf{x}_1), (\mathbf{x}_2 + \epsilon_2, \mathbf{x}_2), \dots, (\mathbf{x}_n + \epsilon_n, \mathbf{x}_n).$$

Khi huấn luyện xong, mạng DAE có khả năng khử nhiễu ở đầu vào của mạng. Ngoài ra, mạng DAE huấn luyện với nhiễu có khả năng trích xuất số đặc trưng nhiều hơn số đầu vào ban đầu ( $d' > d$ ).

### 10.3.4 Bộ tự mã hoá biến phân

Điểm yếu của bộ tự mã hoá sâu DAE truyền thống và bộ tự mã hoá khử nhiễu là:

- Chúng không đảm bảo tính liên tục của không gian mã hoá . Nghĩa chúng không đảm bảo  $g_\theta(f_\theta(\mathbf{x}) + \epsilon) \approx \mathbf{x}$ .
- Chúng không đảm bảo tính đầy đủ của không gian mã hoá. Nghĩa là không đảm bảo  $g_\theta(\mathbf{z})$  là một dữ liệu có nghĩa đối với mã hoá  $\mathbf{z}$  bất kì.

Để đảm bảo tính liên tục và đầy đủ của không gian mã hoá, chúng ta có thể sử dụng *bộ tự mã hoá biến phân* VAE (Variational Auto-encoder). Mô hình VAE được mô tả qua hai công thức (10.25) và (10.26).

$$\mathbf{z} \sim \mathcal{N}(\mu_\theta(\mathbf{x}), \sigma_\theta^2(\mathbf{x})) \quad (10.25)$$

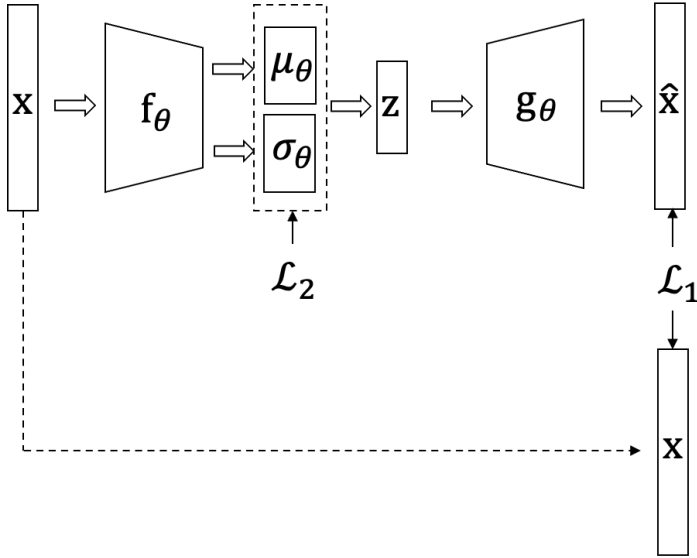
$$\hat{\mathbf{x}} = g_\theta(\mathbf{z}) \quad (10.26)$$

Trong đó,  $\mathbf{z}$  là một biến ngẫu nhiên được lấy từ phân phối chuẩn với kì vọng  $\mu_\theta(\mathbf{x}) \in \mathbb{R}^{d'}$  và độ lệch chuẩn  $\sigma_\theta(\mathbf{x}) \in \mathbb{R}^{d'}$ . Thường  $\mathbf{z}$  được lấy mẫu bằng phương pháp tham số hoá lại như sau. Trước tiên, chúng ta tham số hoá  $\mathbf{z}$  bằng  $\mu_\theta(\mathbf{x})$  và  $\sigma_\theta(\mathbf{x})$  như sau:

$$\mathbf{z} = \mu_\theta(\mathbf{x}) + \sigma_\theta(\mathbf{x}) \cdot \epsilon \quad (10.27)$$

với  $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ .

Hình 10.4 mô tả quá trình tính toán cơ bản của bộ tự mã hoá biến phân, sử dụng phương pháp tham số hoá theo phân phối Gauss



**Hình 10.4:** Minh hoạ quá trình mã hoá và giải mã trong bước huấn luyện của bộ tự mã hoá biến phân

hiều chiều lại để lấy mẫu biến ngẫu nhiên  $\mathbf{z}$  trong miền biểu diễn ẩn. Hàm lỗi của mô hình VAE được tính gồm hai thành phần

$$\mathcal{L}(\theta) = \mathcal{L}_1(\theta) + \mathcal{L}_2(\theta) \quad (10.28)$$

Trong đó, hàm lỗi  $\mathcal{L}_1$  đặc trưng cho tính khôi phục dữ liệu của mô hình

$$\mathcal{L}_1(\theta) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \quad (10.29)$$

được tính như bình thường. Hàm lỗi  $\mathcal{L}_2$  được dùng để chuẩn hóa phân phối trên miền không gian ẩn của  $\mathbf{z}$

$$\mathcal{L}_2(\theta) = \frac{1}{N} \sum_{i=1}^N \|\mu_\theta(\mathbf{x}_i)\|^2 + \|\sigma_\theta(\mathbf{x}_i)\|^2 - \log \|\sigma_\theta(\mathbf{x}_i)\|^2 - 1 \quad (10.30)$$

được sử dụng để đảm bảo tính liên tục và tính đầy đủ của không gian mã hoá. Trong đó,  $\mu_\theta(\mathbf{x}_i)$  và  $\sigma_\theta(\mathbf{x}_i)$  là kết quả mã hoá của mô hình VAE trên dữ liệu  $\mathbf{x}_i$ . Ở đây, hàm lỗi chuẩn hóa  $\mathcal{L}_2(\theta)$  được tính bằng khoảng cách Kullback-Leibler giữa phân phối  $\mathcal{N}(\mu_\theta(\mathbf{x}), \sigma_\theta^2(\mathbf{x}))$  và phân phối chuẩn. Nhờ đó, các phân phối  $\mathcal{N}(\mu_\theta(\mathbf{x}), \sigma_\theta^2(\mathbf{x}))$  được đưa về gần phân phối chuẩn  $\mathcal{N}(0, \mathbf{I})$ . Điều này ngăn mô hình mã hóa dữ liệu cách xa nhau trong không gian mã hoá và khuyến khích các phân phối “chồng lấp” lên nhau, từ đó đáp ứng các điều kiện về tính liên tục và tính đầy đủ của không gian mã hoá.

## 10.4 Trích chọn đặc trưng trong bài toán học có giám sát

Các phương pháp đã giới thiệu trong chương này không sử dụng thông tin về nhãn của dữ liệu. Các thuật toán PCA, bộ tự mã hoá sâu là các phương pháp không giám sát. Chúng tìm cách nén dữ liệu và giữ các thông tin quan trọng nhất với mục tiêu khôi phục lại dữ liệu ban đầu. Tuy nhiên, trong một số trường hợp, nếu ta muốn tìm ra các đặc trưng quan trọng nhất với mục tiêu dự đoán nhãn của dữ liệu, ta có thể sử dụng các phương pháp trích chọn đặc trưng trong bài toán học có giám sát. Cụ thể, bài toán trích chọn đặc trưng như sau: Cho dữ liệu  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$  thuật toán học máy  $\mathcal{A}$  và hàm lỗi  $\mathcal{L}(f) = \mathbb{E}[\ell(f(\mathbf{x}), y)]$ , tìm đặc trưng  $z = \mathbf{E}(\mathbf{x})$  sao cho lỗi trên tập dữ liệu đã trích chọn đặc trưng

$$D_E = \{(\mathbf{E}(\mathbf{x}_i), y_i)\}, i = 1, 2, \dots, n$$

là nhỏ nhất

$$\mathbf{E}^* = \min_{\mathbf{E}} \mathcal{L}(\mathcal{A}(D_E))$$

Có ba hướng tiếp cận chính để trích xuất đặc trưng trong trường hợp này: phương pháp lọc, phương pháp gói, phương pháp nhúng.

### 10.4.1 Phương pháp lọc

Phương pháp lọc đánh giá mối quan hệ, mức độ tương quan giữa các đặc trưng và nhãn bằng điểm số nhận được từ các phép tính thống kê. Tùy thuộc vào loại đặc trưng và kiểu nhãn, có rất nhiều cách đánh giá mức độ tương quan giữa các đặc trưng và nhãn. Một số phương pháp phổ biến để đánh giá độ tương quan giữa đặc trưng và nhãn bao gồm:

**Độ tương quan Pearson** là phép tính độ tương quan giữa hai biến liên tục. Độ đo này được định nghĩa như sau

$$\begin{aligned}\rho_{x,y} &= \frac{\mathbb{E}[(x - \mathbb{E}[x])(y - \mathbb{E}[y])]}{\sqrt{\mathbb{E}[(x - \mathbb{E}[x])^2]\mathbb{E}[(y - \mathbb{E}[y])^2]}} \\ &\approx \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}\end{aligned}$$

Độ đo này có giá trị trong khoảng  $[-1, 1]$ . Nếu giá trị của  $\rho_{x,y}$  gần với 1, thì có nghĩa rằng hai biến liên tục này có mối quan hệ tuyến tính đồng biến. Nếu giá trị của  $\rho_{x,y}$  gần với -1, thì có nghĩa rằng hai biến liên tục này có mối quan hệ tuyến tính nghịch biến. Nếu giá trị của  $\rho_{x,y}$  gần với 0, thì có nghĩa rằng hai biến liên tục này không có mối quan hệ tuyến tính. Giá trị của độ đo này không phụ thuộc vào độ lớn của hai biến liên tục. Độ đo này được sử dụng để loại bỏ các đặc trưng ít có mối quan hệ tuyến tính với nhãn hoặc giữ lại các đặc trưng có mối quan hệ tuyến tính mạnh với nhãn.

**Độ tương quan thứ hạng Spearman** là phép tính độ tương quan về thứ hạng của hai biến liên tục. Độ đo này đánh giá quan hệ giữa hai biến có thể mô tả tốt thông qua một *hàm đồng biến* hay không. Độ tương quan thứ hạng Spearman được định nghĩa bằng độ tương quan Pearson giữa thứ hạng của hai biến đó.

$$r_{x,y} = \rho_{\text{rank}(x), \text{rank}(y)}$$

Cách dùng độ đo Spearman để đánh giá mức độ tương quan giữa đặc trưng và nhãn tương tự, từ đó loại bỏ hoặc giữ lại đặc trưng tương tự như độ tương quan Pearson.

**Phân tích phương sai ANOVA** là phương pháp sử dụng phép kiểm tra giả thuyết thống kê để đánh giá mối quan hệ về phương sai giữa các nhóm dữ liệu. Ý tưởng chính ở phương pháp này là khi biến  $x$  có phương sai thấp (ít biến thiên) thì nó sẽ có ít ảnh hưởng đến nhãn  $y$ .

**Thông tin tương hỗ** là phép tính định lượng thông tin của một biến thay đổi khi biết thông tin của một biến khác. Ta đã biết đến khái niệm này trong phương pháp Cây quyết định. Tương tự như vậy ta định nghĩa thông tin tương hỗ của hai biến  $x, y$  là lượng thông tin thêm được về  $x$  khi biết  $y$ .

**Định nghĩa 10.4** (Thông tin tương hỗ). Thông tin tương hỗ giữa hai biến  $x$  và  $y$  được định nghĩa như sau:

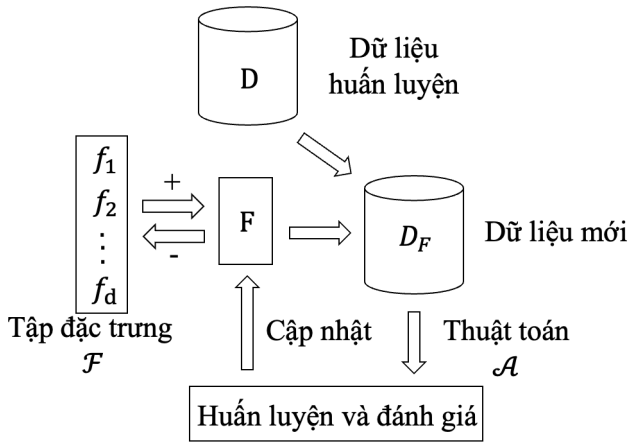
$$I(x, y) = H(x) - H(x|y)$$

trong đó,  $H(x)$  là entropy của  $x$  và  $H(x|y)$  là entropy của  $x$  khi biết  $y$ .

Trong trường hợp đặc trưng  $x$  và nhãn mục tiêu  $y$  là biến rời rạc, chúng ta có thể tính thông tin tương hỗ giữa  $x$  và  $y$  theo  $H(x)$  và  $H(x|y)$  bằng cách sử dụng các tần suất xuất hiện của các giá trị của  $x$  và  $y$  trong tập dữ liệu. Cụ thể, ta có

$$H(x) = - \sum_{x_i} P(x_i) \log P(x_i)$$

$$H(x|y) = - \sum_{y_j} P(y_j) \sum_{x_i} P(x_i|y_j) \log P(x_i|y_j)$$



Hình 10.5: Lược đồ phương pháp gói

#### 10.4.2 Phương pháp gói

Phương pháp gói là phương pháp tìm kiếm các đặc trưng tốt nhất bằng cách thử nghiệm các đặc trưng khác nhau và đánh giá chúng bằng chính thuật toán học máy có giám sát đang quan tâm. Cho tập dữ liệu  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$  với các đặc trưng ban đầu  $\mathcal{F} = \{f_1, f_2, \dots, f_d\}$  và thuật toán học máy  $\mathcal{A}$ , lược đồ cơ bản của phương pháp gói như sau được minh họa trong Hình 10.5. Xuất phát từ tập đặc trưng  $\mathcal{F}$ , phương pháp gói bắt đầu chọn một tập con  $F$  của  $\mathcal{F}$  và đánh giá hiệu suất của thuật toán  $\mathcal{A}$  trên tập dữ liệu  $D_F$ . Tập dữ liệu này được sinh ra bằng cách giới hạn tập  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$  về miền của các đặc trưng  $F$ . Sau đó, thuật toán  $\mathcal{A}$  sẽ được huấn luyện và đánh giá kết quả. Dựa trên kết quả này, tập  $F$  sẽ được thêm (+) hoặc loại bỏ (-) một số đặc trưng theo các chiến thuật:

- **Lấy thêm đặc trưng** là phương pháp gói trong đó đặc trưng mới được thêm vào tập đặc trưng đang xét là đặc trưng đem lại hiệu suất tốt nhất.



- **Loại bỏ đặc trưng** là phương pháp gói trong đó đặc trưng được bỏ ra khỏi tập đặc trưng đang xét sao cho tập đặc trưng mới đem lại hiệu suất tốt nhất.

### 10.4.3 Phương pháp nhúng

Phương pháp nhúng là phương pháp tìm kiếm các đặc trưng tốt nhất bằng cách tận dụng khả năng đánh giá các đặc trưng của chính thuật toán học máy có giám sát đang quan tâm. Lược đồ của phương pháp này gần giống với lược đồ Phương pháp gói ở Hình 10.5. Tuy nhiên, trong phương pháp nhúng, chúng ta có thể trực tiếp đánh giá được độ quan trọng của các đặc trưng. Dựa trên đánh giá này, việc lựa chọn thêm đặc trưng hoặc bỏ đặc trưng sẽ dễ dàng hơn. Có nhiều thuật toán học máy cho phép đánh giá trực tiếp độ quan trọng của đặc trưng như thuật toán học máy sử dụng mô hình cây quyết định hoặc mô hình hồi quy tuyến tính.

**Cây quyết định:** Khi sử dụng mô hình cây quyết định để huấn luyện trên tập dữ liệu, mỗi lần phân chia tại một nút sử dụng đặc trưng  $f_i$ , thuật toán sẽ cộng thêm vào độ quan trọng của đặc trưng  $f_i$  đại lượng bằng đúng tiêu chí phân chia tại nút đó. Độ quan trọng của đặc trưng  $f_i$  được tính bằng tổng các tiêu chí tại các nút của cây sử dụng đặc trưng đó.

**Hồi quy:** Trong hồi quy, độ quan trọng của đặc trưng  $f_i$  được tính bằng trọng số tương ứng với đặc trưng. Các trọng số này được điều chỉnh bởi thuật toán hồi quy kết hợp với các hàm điều chỉnh như. Ví dụ, mô hình LASSO, được cho bởi công thức (10.31)

$$\min_{\mathbf{w}} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_1. \quad (10.31)$$

Hàm  $l$  được điều chỉnh có tác dụng ép các trọng số tương ứng với các đặc trưng không quan trọng tiến về 0. Có thể dùng các hàm

điều chỉnh này trong cả mô hình hồi quy tuyến tính và mô hình hồi quy Logistic.

## 10.5    Tình huống áp dụng: Phân tích thành phần chính trên dữ liệu CIFAR-10

Bộ dữ liệu CIFAR-10 bao gồm 60.000 ảnh màu kích thước  $32 \times 32$ , được chia thành 10 lớp khác nhau (ví dụ: ô tô, chim, chó, mèo, tàu, v.v.). Mỗi ảnh có 3 kênh màu (RGB), tổng cộng 3072 đặc trưng ( $32 \times 32 \times 3$ ) cho mỗi ảnh. Với số chiều lớn như vậy, việc huấn luyện các mô hình học máy trực tiếp trên dữ liệu gốc có thể tốn nhiều tài nguyên tính toán và dễ dẫn đến hiện tượng học quá, đặc biệt khi số lượng mẫu huấn luyện không đủ lớn so với số chiều.

Quá trình giảm chiều dữ liệu có thể tiến hành như sau:

1. [Phẳng hóa] Dữ liệu bao gồm 50.000 ảnh trong tập huấn luyện và 10.000 ảnh trong tập kiểm tra. Để chuẩn bị cho PCA, các ảnh được phẳng hóa (flatten) từ kích thước  $32 \times 32 \times 3$  thành các véc-tơ kích thước 3072.
2. Sử dụng phương pháp PCA, giảm chiều dữ liệu từ 3072 xuống còn 100 thành phần chính. Quá trình này tạo ra hai sản phẩm: (i) Ma trận  $\mathbf{W} \in \mathbb{R}^{3072 \times 100}$  chứa các thành phần chính và (ii) ma trận  $\mathbf{Z} \in \mathbb{R}^{50000 \times 100}$  chứa các hệ số thành phần chính của các ảnh trong tập huấn luyện.
3. Dữ liệu đã giảm chiều (các vectơ 100 chiều) được sử dụng để huấn luyện mô hình học máy hồi quy Logistic để phân loại các ảnh vào 10 lớp của CIFAR-10.
4. Mô hình được huấn luyện trên tập huấn luyện (50.000 mẫu) và đánh giá trên tập kiểm tra (10.000 mẫu).

Việc sử dụng dữ liệu đã giảm chiều giúp giảm thời gian huấn luyện và yêu cầu bộ nhớ, so với việc sử dụng dữ liệu gốc 3072 chiều. Mã ví dụ có thể xem tại <https://gist.github.com/tqlong/88c494e70cee0ac0c22b0e08d48d9c48>

## 10.6 Tổng kết chương

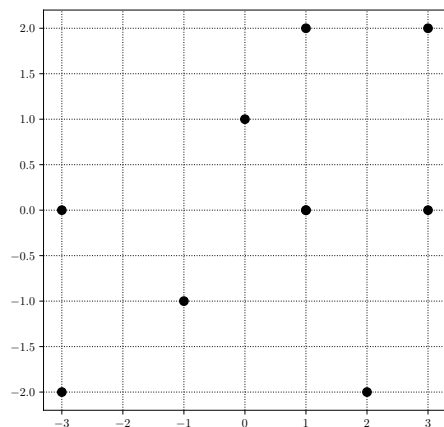
Chương 10 giới thiệu các phương pháp trích xuất đặc trưng hiệu quả, góp phần nâng cao hiệu suất huấn luyện và giảm nguy cơ quá khớp trong mô hình Học máy.

Phân tích thành phần chính PCA là một công cụ ổn định để giảm chiều, giúp làm nổi bật cấu trúc ẩn trong dữ liệu và hỗ trợ trực quan hóa hoặc tiền xử lý. Bộ mã hoá - giải mã AE (autoencoder) ứng dụng học sâu để học các biểu diễn nén của dữ liệu, từ đó tái tạo lại thông tin đầu vào một cách hiệu quả. Đây là phương pháp mạnh mẽ để học đặc trưng không giám sát. Một phiên bản của AE là bộ mã hoá biến phân VAE, cho phép học phân bố ẩn của dữ liệu và sinh mẫu mới từ không gian đặc trưng đã học, mở rộng khả năng biểu diễn và tổng quát hóa của mô hình.

Những phương pháp này có vai trò quan trọng trong thiết kế mô hình, đặc biệt trong các bài toán với số chiều lớn và dữ liệu thực tế có số chiều ẩn thấp hoặc có nhiễu.

## Bài tập

1. Chứng minh Định lý 10.3.
2. Cho tập dữ liệu trong  $\mathbb{R}^2$  như Hình 10.6. Hãy tính (i) ma trận hiệp phương sai  $\Sigma$ , (ii) các véc-tơ riêng  $\mathbf{v}_1$  và  $\mathbf{v}_2$ , (iii) các thành phần chính  $\mathbf{z}_1$  và  $\mathbf{z}_2$  của các điểm dữ liệu trong tập huấn luyện.



**Hình 10.6: Tập dữ liệu trong  $\mathbb{R}^2$**

3. *[Lập trình]* Sử dụng thư viện `scikit-learn` trong Python, áp dụng PCA để giảm chiều dữ liệu tập IRIS từ 4 chiều xuống 2 chiều. Trực quan hóa dữ liệu đã giảm chiều bằng biểu đồ phân tán và so sánh với dữ liệu gốc.
4. *[Tìm hiểu]* Giải thích sự khác biệt giữa PCA và bộ tự mã hóa sâu trong trích xuất đặc trưng. Tại sao bộ tự mã hóa sâu có khả năng trích xuất đặc trưng phi tuyến?
5. Cho một bộ tự mã hóa sâu đơn giản với hàm kích hoạt tuyến tính và hàm lỗi là trung bình bình phương sai số

$$L(\theta) = \frac{1}{2n} \sum_{i=1}^n \|\mathbf{x}_i - g_{\phi}(f_{\theta}(\mathbf{x}_i))\|^2.$$

Tính đạo hàm của hàm lỗi đối với các tham số  $\theta, \phi$  của bộ tự mã hóa sâu.

6. *[Lập trình]* Sử dụng thư viện `Keras` hoặc `Pytorch`, triển khai một bộ tự mã hóa khử nhiễu sử dụng bộ dữ liệu MNIST. Đánh giá khả năng khôi phục ảnh của bộ tự mã hóa.

7. *[Tìm hiểu]* Giải thích vai trò của hàm lỗi Kullback-Leibler (KL) trong bộ tự mã hóa biến phân (VAE). Tại sao nó đảm bảo tính liên tục và đầy đủ của không gian mã hóa?
8. *[Lập trình]* Sử dụng thư viện `scikit-learn`, triển khai áp dụng PCA trên tập dữ liệu CIFAR-10.
  - Giảm chiều dữ liệu xuống còn  $\theta = 95\%$  tổng phương sai,
  - Huấn luyện một mô hình Logistic Regression trên dữ liệu đã giảm chiều.
  - So sánh hiệu suất với mô hình trên dữ liệu gốc.
  - Trực quan hóa hiệu suất của mô hình và tỉ lệ phương sai  $\theta$  được sử dụng. Hãy thử với nhiều giá trị khác nhau của  $\theta$ .



---

# Tài liệu tham khảo

---

- [1] Pearson, K., *On lines and planes of closest fit to systems of points in space*, Philosophical Magazine, vol. 2, no. 11, pp. 559–572, 1901.
- [2] Hotelling, H., *Analysis of a complex of statistical variables into principal components*, Journal of Educational Psychology, vol. 24, no. 6, pp. 417–441, 1933.
- [3] Hinton, G. E., and Salakhutdinov, R. R., *Reducing the dimensionality of data with neural networks*, Science, vol. 313, no. 5786, pp. 504–507, 2006.
- [4] Kingma, D. P., and Welling, M., *Auto-encoding variational Bayes*, arXiv preprint arXiv:1312.6114, 2013.
- [5] Guyon, I., and Elisseeff, A., *An introduction to variable and feature selection*, Journal of Machine Learning Research, vol. 3, pp. 1157–1182, 2003.
- [6] Hall, M. A., *Correlation-based feature selection for machine learning*, Proceedings of the 17th International Conference on Machine Learning, pp. 359–366, 1999.

- [7] Tibshirani, R., *Regression shrinkage and selection via the Lasso*, Journal of the Royal Statistical Society: Series B, vol. 58, no. 1, pp. 267–288, 1996.
- [8] Fisher, R. A., *The correlation between relatives on the supposition of Mendelian inheritance*, Transactions of the Royal Society of Edinburgh, vol. 52, no. 2, pp. 399–433, 1919.



# Chương 11

---

## Mô hình sinh

---

Chương 11 giới thiệu các mô hình sinh (Generative Models) – một nhánh quan trọng và đặc biệt của Học máy, tập trung vào việc tạo ra dữ liệu mới dựa trên phân phối thống kê tiềm ẩn của dữ liệu quan sát.

Khác với các mô hình phân biệt chỉ học ranh giới giữa các lớp, mô hình sinh tìm cách học toàn bộ cấu trúc phân phối xác suất của dữ liệu, cho phép sinh mẫu mới tương tự như dữ liệu huấn luyện.

Chương sẽ trình bày các phương pháp ước lượng phân phối như cực đại độ hợp lý MLE (Maximum Likelihood Estimation), thuật toán cực đại kỳ vọng EM (Expectation Maximization) và các tiếp cận học phân phối sử dụng mạng nơ-ron sâu.

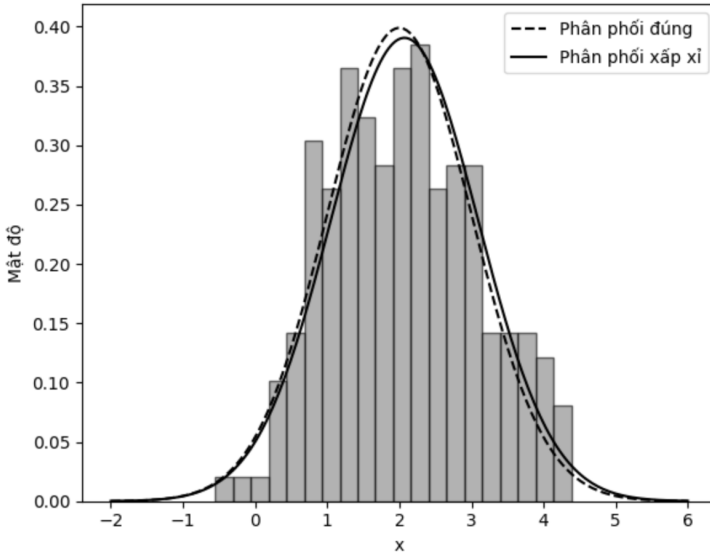
Các phương pháp được giới thiệu không chỉ mang giá trị lý thuyết mà còn đóng vai trò nền tảng trong các ứng dụng hiện đại như tạo dữ liệu tổng hợp, phục hồi ảnh, dịch phong cách và sáng tạo nội dung.

## 11.1 Bài toán sinh dữ liệu

Một trong các khả năng mà con người tự hào nhất là khả năng tạo ra dữ liệu mới có độ tương thích nhất định từ dữ liệu đã có. Ví dụ, chúng ta có thể tạo ra một bức tranh mới sau khi học cách vẽ tranh hoặc chúng ta có thể tạo ra một câu mới từ một mẫu câu. Một trong các hướng tiếp cận trong Học máy thống kê nhằm cung cấp khả năng này cho máy tính là Mô hình sinh (generative models). Mô hình sinh cho phép giải quyết các bài toán sinh dữ liệu, bao gồm:

- **Bổ sung dữ liệu:** sinh thêm dữ liệu để bổ sung một tập dữ liệu sẵn có, giúp cải thiện hiệu suất của các mô hình học máy khi có thêm nhiều dữ liệu.
- **Phát hiện ngoại lệ:** phát hiện các bất thường hoặc ngoại lệ trong một bộ dữ liệu bằng cách so sánh dữ liệu được sinh ra bởi mô hình sinh và dữ liệu thực tế.
- **Bổ sung dữ liệu thiếu:** bổ sung các dữ liệu thiếu trong bộ dữ liệu sẵn có bằng cách sinh ra dữ liệu mới hoặc đặc trưng dữ liệu bị thiếu.
- **Sinh dữ liệu:** sinh ra một bộ dữ liệu mới hoàn toàn giống với một bộ dữ liệu đã cho hoặc sử dụng các thông tin bổ sung sinh dữ liệu có định hướng.

Với cách tiếp cận sử dụng xác suất - thống kê, bài toán sinh dữ liệu được quy về (i) bài toán *ước lượng phân bố* của dữ liệu và (ii) bài toán *lấy mẫu* từ phân bố. Cụ thể, ta ước lượng phân bố xác suất của dữ liệu  $P(x)$  hay  $P(x, y)$ , rồi dùng các kỹ thuật lấy mẫu từ các phân bố này để sinh ra dữ liệu mới. Trong chương này, chúng ta sẽ tìm hiểu các mô hình sinh dữ liệu dựa trên phân bố xác suất.



Hình 11.1: Minh hoạ phương pháp ước lượng mật độ tham số

## 11.2 Ước lượng mật độ bằng mô hình tham số

Sử dụng mô hình tham số để mô tả phân bố hoặc mật độ phân bố là phương pháp cổ điển trong thống kê. Trong cách tiếp cận này, ta giả định rằng có thể mô tả phân bố thật sự  $p^*(x)$  của dữ liệu bằng một mô hình phân bố có tham số  $p_\theta(x)$ . Sau đó, ta tìm các tham số của hàm số đó sao cho phân phối có tham số đó phù hợp nhất với phân phối thực nghiệm của dữ liệu. Cụ thể bài toán ước lượng mật độ bằng mô hình tham số như sau

**Định nghĩa 11.1** (Ước lượng mật độ bằng mô hình tham số). Cho tập dữ liệu  $D = \{x_1, \dots, x_n\}$  với  $x_i \sim p^*(x)$  và lớp mô hình phân bố có tham số  $\{p_\theta(x) : \theta \in \Theta\}$ . Tìm tham số  $\theta$  để mô hình  $p_\theta(x)$  “xấp xỉ”  $p^*(x)$ .

Hình 11.1 là một ví dụ về ước lượng mật độ bằng mô hình tham số. Trong đó, ta giả định rằng dữ liệu  $D$  được sinh ra từ một phân bố chuẩn. Đường liền nét là kết quả thuật toán ước lượng và đường

đứt nét là phân phối thực tế cần ước lượng. Tuy nhiên, dữ liệu chỉ có các điểm rời rạc được lấy mẫu và được thể hiện bằng các cột hít-tô-gram trong hình.

Như vậy để tìm được tham số  $\theta$ , ta phải chỉ ra thể nào là phù hợp. Trong lý thuyết xác suất, có nhiều cách để so sánh hai phân bố, ở đây là mô hình  $p_\theta$  và phân bố  $p^\star$  của các mẫu dữ liệu trong  $D$ .

### 11.2.1 Ước lượng hợp lý cực đại

Nếu sử dụng khoảng cách Kullback-Leibler (KL) để so sánh  $p^\star$  và  $p_\theta$ , ta có công thức khoảng cách sau:

$$D_{KL}(p^\star \| p_\theta) = \mathbb{E}_{p^\star} \left[ \log \frac{p^\star(x)}{p_\theta(x)} \right] \geq 0 \quad (11.1)$$

Dựa trên công thức (11.1), nếu  $D_{KL}(p^\star \| p_\theta) = 0$ , ta có thể suy ra được  $p^\star = p_\theta$ . Tuy nhiên, khoảng cách KL không tính được do không biết  $p^\star$ . Sử dụng Luật số lớn, xấp xỉ khoảng cách KL bằng giá trị trung bình (thực nghiệm) trên tập dữ liệu  $D$  được lấy mẫu từ phân bố  $p^\star$ , ta có biến đổi sau:

$$\begin{aligned} D_{KL}(p^\star \| p_\theta) &\approx \mathbb{E}_D \left[ \log \frac{p^\star(x)}{p_\theta(x)} \right] \\ &= \frac{1}{n} \sum_{i=1}^n \log \frac{p^\star(x_i)}{p_\theta(x_i)} \\ &= C - \frac{1}{n} \sum_{i=1}^n \log p_\theta(x_i) \\ &= C - \log \prod_{i=1}^n p_\theta(x_i) \\ &= C - \log p(D|\theta) \end{aligned}$$

trong đó,  $C$  là hằng số không phụ thuộc vào  $\theta$ ,  $p(D|\theta)$  là xác suất của dữ liệu trong mô hình  $p_\theta(x)$ . Như vậy ta có thể tối thiểu khoảng cách KL bằng cách tối đa hóa hàm mật độ xác suất của dữ liệu  $p(D|\theta)$ , còn gọi là *độ hợp lý* (likelihood).

**Định nghĩa 11.2** (Ước lượng hợp lý cực đại). Gọi

$$L(\theta) = P(D|\theta) = P(x_1, x_2, \dots, x_n|\theta)$$

là độ hợp lý của  $\theta$  đối với tập dữ liệu  $D$ . *Ước lượng hợp lý cực đại* (MLE) của  $\theta$  là nghiệm tối ưu của bài toán cực đại hóa  $L(\theta)$

$$\theta^{MLE} = \arg \max_{\theta} L(\theta).$$

**Ví dụ 11.3** (Ước lượng mô hình Bernoulli). Mô tả một đồng xu không cân bằng với xác suất mặt ngửa là  $\theta$ . Giả sử ta quan sát được  $n$  lần tung đồng xu và thu được  $k$  lần mặt ngửa. Hỏi  $\theta$  hợp lý nhất là bao nhiêu?

Ta đặt  $X_i = 1$  nếu mặt ngửa xuất hiện và  $X_i = 0$  nếu mặt sấp xuất hiện ở lần tung thứ  $i$ . Có thể mô tả phân bố của  $X_i$  bằng một biến ngẫu nhiên nhị thức  $X_i \sim \text{Bernoulli}(x|\theta) = \theta^x(1-\theta)^{1-x}$

Giả sử  $n$  lần tung độc lập với nhau thì xác suất của dữ liệu trong mô hình có thể được tính theo công thức

$$P(D|\theta) = \prod_{i=1}^n \theta^{x_i}(1-\theta)^{1-x_i} = \theta^{\sum_{i=1}^n x_i} (1-\theta)^{n-\sum_{i=1}^n x_i} = \theta^k (1-\theta)^{n-k}$$

do các biến  $x_i \in \{0, 1\}$ .

Do log là hàm đồng biến, cực trị của  $L(\theta)$  là cực trị của  $\log L(\theta)$ . Do đó, ta có thể tìm  $\theta^{MLE}$  bằng cách giải bài toán tối ưu

$$\theta^{MLE} = \arg \max_{\theta} \log L(\theta) = \arg \max_{\theta} \sum_{i=1}^n k \log \theta + (n-k) \log(1-\theta) \quad (11.2)$$

Lấy đạo hàm và đặt bằng 0 để tìm cực trị ta được

$$\theta^{MLE} = \frac{k}{n}.$$

Như vậy, tỉ số giữa số lần mặt ngửa và tổng số lần tung là ước lượng hợp lý cực đại của mô hình đồng xu nói trên.

**Ví dụ 11.4** (Ước lượng mô hình Gauss). Các bạn nam trong lớp có chiều cao là  $x_1, x_2, \dots, x_n$ . Hỏi phân bố chiều cao của các bạn nam trong lớp như thế nào ?

Nếu ta giả định rằng chiều cao của các bạn nam trong lớp tuân theo phân bố Gauss thì ta có thể mô tả chiều cao của một bạn nam bất kì bằng một biến ngẫu nhiên Gauss có phân phối như sau:

$$X \sim \mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (11.3)$$

với  $\mu$  là trung bình và  $\sigma^2$  là phương sai.

Giả sử các biến  $X_i$  là độc lập với nhau thì xác suất của dữ liệu trong mô hình là

$$L(\mu, \sigma^2) = P(D|\mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}}$$

$$\ell(\mu, \sigma^2) = \log L(\mu, \sigma^2) = n \log \frac{1}{\sqrt{2\pi\sigma^2}} - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Tương tự như Ví dụ 11.3, ta lấy đạo hàm  $\ell(\mu, \sigma^2)$  theo  $\mu$  và  $\sigma^2$  và đặt bằng 0 được

$$\frac{\partial \ell(\mu, \sigma^2)}{\partial \mu} = -\frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) = 0 \Rightarrow \mu = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\frac{\partial \ell(\mu, \sigma^2)}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^n (x_i - \mu)^2 = 0 \Rightarrow \sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

### 11.2.2 Ước lượng hậu nghiệm cực đại

Trong thực tế, ước lượng hợp lý cực đại có thể dẫn đến các ước lượng vô lý do nhiễu và khả năng học quá. Ví dụ, nếu ta tung đồng xu năm lần mà không ra mặt ngửa lần nào thì ước lượng hợp lý cực đại của  $\theta$  là 0. Tuy nhiên, ta có thể thấy rằng đây là một ước lượng vô lý vì đồng xu bình thường không thể không bao giờ tung ra mặt ngửa. Hoặc ngược lại, nếu cả 5 lần đều ra mặt ngửa thì ước lượng của  $\theta$  là 1, đây cũng là một ước lượng vô lý.

Một ước lượng thay thế phổ biến là ước lượng hậu nghiệm cực đại (Maximum a Posteriori - MAP). Ước lượng này được tính bằng cách tối đa hóa hàm hậu nghiệm  $p(\theta|D)$ , còn gọi là hàm hậu nghiệm.

**Định nghĩa 11.5** (Ước lượng hậu nghiệm cực đại). Giả sử  $p(\theta)$  là xác suất tiên nghiệm của tham số  $\theta$ . Ước lượng hậu nghiệm cực đại của  $\theta$  là nghiệm tối ưu của bài toán cực đại hóa hàm  $p(\theta|D)$

$$\theta^{MAP} = \arg \max_{\theta} p(\theta|D) = \arg \max_{\theta} p(\theta)p(D|\theta).$$

Theo công thức Bayes, ta có biến đổi sau để tối đa xác suất hậu nghiệm

$$\begin{aligned} \log p(\theta|D) &= \log \frac{p(D|\theta)p(\theta)}{p(D)} \\ &= \log p(D|\theta) + \log p(\theta) - \underbrace{\log p(D)}_{\text{const}} \\ \Rightarrow \theta^{MAP} &= \arg \max_{\theta} \log p(D|\theta) + \log p(\theta) \end{aligned}$$

trong đó,  $\log p(D)$  là hằng số không phụ thuộc vào  $\theta$ ,  $p(D|\theta)$  là hàm hợp lý (likelihood) còn  $p(\theta)$  là xác suất tiên nghiệm của tham số  $\theta$ .

Như vậy, ước lượng hậu nghiệm cực đại là sự điều chỉnh của ước lượng hợp lý cực đại bằng cách thêm vào hàm mục tiêu hàm xác suất tiên nghiệm  $p(\theta)$ . Xác suất tiên nghiệm là “niềm tin” của

ta về giá trị của tham số khi chưa biết dữ liệu. Khi có dữ liệu, ta điều chỉnh “niềm tin” này bằng giá trị ước lượng hậu nghiệm cực đại. Đây là một cách để tránh hiện tượng học quá do các ước lượng được điều chỉnh bởi xác suất tiên nghiệm.

Để tính toán đơn giản và dễ hiểu, ta thường giả định xác suất tiên nghiệm có một dạng hàm số phù hợp với xác suất hậu nghiệm theo định nghĩa sau

**Định nghĩa 11.6** (Xác suất tiên nghiệm liên hợp). Xác suất tiên nghiệm liên hợp là xác suất tiên nghiệm có cùng dạng hàm số với xác suất hậu nghiệm.

Dưới đây là một số ví dụ về xác suất tiên nghiệm liên hợp của một số phân phối phổ biến.

**Ví dụ 11.7** (Phân phối nhị phân). Xét thí nghiệm sau, tung đồng xu  $N$  lần và đếm số lần tung ra mặt ngửa. Thực hiện  $n$  thí nghiệm như vậy ta thu được các dữ liệu  $X_i, i = 1, 2, \dots, n$  là số lần ra mặt ngửa trong  $n$  lần thí nghiệm.

Sử dụng tính chất của phân phối nhị phân ta có thể ước lượng được xác suất  $p(X_i|\theta)$  theo công thức sau:

$$p(X_i|\theta) = B(X_i|N, \theta) = \binom{N}{X_i} \theta^{X_i} (1 - \theta)^{N - X_i}$$

Từ đó, ta có thể tính được hàm hợp lý trên toàn bộ dữ liệu  $D$  là

$$\begin{aligned} p(D|\theta) &= \prod_{i=1}^n p(X_i|\theta) = \prod_{i=1}^n \binom{N}{X_i} \theta^{X_i} (1 - \theta)^{N - X_i} \\ \log p(D|\theta) &= \sum_{i=1}^n [X_i \log \theta + (N - X_i) \log(1 - \theta)] + C \end{aligned}$$



Phân phối hậu nghiệm của  $\theta$  có thể được tính bằng công thức Bayes như sau:

$$\begin{aligned}\log p(\theta|D) &= \log p(D|\theta) + \log p(\theta) + C_1 \\ &= \left( \sum_{i=1}^n X_i + \alpha - 1 \right) \log \theta \\ &\quad + \left( nN - \sum_{i=1}^n X_i + \beta - 1 \right) \log(1 - \theta) + C_2\end{aligned}$$

với  $p(\theta) = \text{Beta}(\theta|\alpha, \beta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}$  và  $C_1, C_2$  là hằng số không phụ thuộc vào  $\theta$ . Do đó,  $p(\theta|D)$  cũng là phân phối Beta với tham số  $\text{Beta}(\theta|\alpha_n, \beta_n)$  được cho bởi công thức sau:

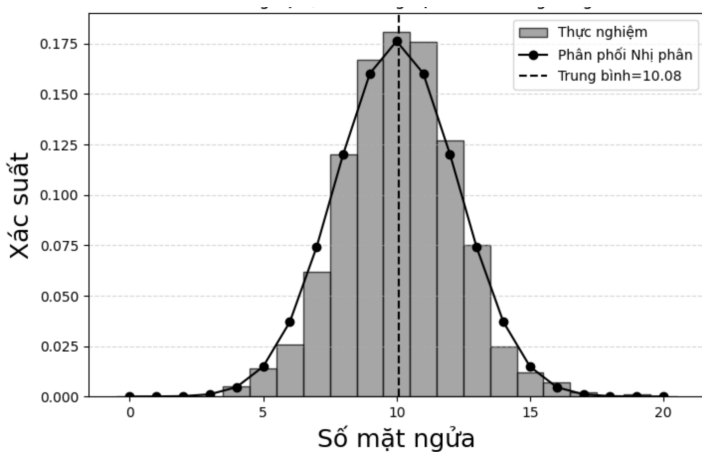
$$\begin{aligned}\alpha_n &= \sum_{i=1}^n X_i + \alpha \\ \beta_n &= nN - \sum_{i=1}^n X_i + \beta.\end{aligned}$$

Từ đó ta có thể dễ dàng tính được, phân bố  $p(\theta|D)$  này đạt cực đại với giá trị  $\theta = \frac{\sum_{i=1}^n X_i + \alpha - 1}{nN + \alpha + \beta - 2}$ . Hình 11.2 là một ví dụ về phân phối Beta với  $N = 20$  và số lần thí nghiệm  $n = 10$ . Đường liền nét là phân phối hậu nghiệm của  $\theta = 0.5$ . Do việc lấy mẫu là ngẫu nhiên nên giá trị trung bình thực nghiệm đạt 10.08, sẽ bị lệch so với giá trị lý thuyết. Khi số lần thí nghiệm  $n$  tăng lên, giá trị trung bình thực nghiệm sẽ tiến gần hơn đến giá trị lý thuyết.

**Ví dụ 11.8** (Phân phối đa thức (multinomial)). Xét thí nghiệm sau, tung xúc xắc  $K$  mặt tất cả  $N$  lần và đếm số lần ra từng mặt  $X^1, X^2, \dots, X^K$ . Thực hiện  $n$  thí nghiệm như vậy ta thu được các dữ liệu

$$X_i = (X_i^1, X_i^2, \dots, X_i^K), i = 1, 2, \dots, n$$

là số lần ra từng mặt xúc xắc trong  $n$  lần thí nghiệm.



**Hình 11.2: Phân phối Nhị phân với  $\theta = 0.5$ ,  $N = 20$  và  $n = 10$**

Nếu dữ liệu tuân theo phân phối đa thức

$$p(X_i|\theta_1, \dots, \theta_K) = Mult(X_i|N, \theta) = \binom{N}{X_i^1, X_i^2, \dots, X_i^K} \prod_{k=1}^K \theta_k^{X_i^k}$$

thì hàm hợp lý của dữ liệu là

$$p(D|\theta) = \prod_{i=1}^n p(X_i|\theta) = \prod_{i=1}^n \binom{N}{X_i^1, X_i^2, \dots, X_i^K} \prod_{k=1}^K \theta_k^{X_i^k}$$

$$\log p(D|\theta) = \sum_{i=1}^n \sum_{k=1}^K X_i^k \log \theta_k + C$$

Phân phối liên hợp của phân phối đa thức là phân phối Dirichlet.

$$p(\theta) = \text{Dir}(\theta|\alpha_1, \dots, \alpha_K) \propto \prod_{k=1}^K \theta_k^{\alpha_k-1}$$

$$\log p(\theta) = \sum_{k=1}^K (\alpha_k - 1) \log \theta_k + C$$

Phân phối này đạt cực đại tại  $\theta_k = \frac{\alpha_k - 1}{\sum_{k=1}^K \alpha_k - K}$ .

Phân phối hậu nghiệm của  $\theta$  là phân phối Dirichlet với tham số

$$p(\theta|D) = \text{Dirichlet}(\theta|\alpha_{1n}, \dots, \alpha_{Kn})$$

$$\alpha_{kn} = \sum_{i=1}^n X_i^k + \alpha_k$$

Phân bố này đạt cực đại tại  $\theta_k = \frac{\sum_{i=1}^n (X_i^k + \alpha_k) - 1}{\sum_{i=1}^n \sum_{k=1}^K (X_i^k + \alpha_k) - K}$ .

Trên đây là một số ví dụ về ước lượng hậu nghiệm cực đại của một số phân phối phổ biến. Chúng ta có thể áp dụng phương pháp này để ước lượng tham số cho các phân phối phổ biến khác ví dụ như Phân phối Poisson, Phân phối Đa thức, Phân phối chuẩn, v.v..

### 11.3 Thuật toán EM - Cực đại hoá kì vọng

Ước lượng tham số của mô hình phân bố có tham số như ở mục trên chỉ thực hiện được khi ta có mô hình phân bố tường minh của dữ liệu được quan sát. Tuy nhiên, trong thực tế, ta thường không có mô hình phân bố tường minh của dữ liệu được quan sát mà chỉ có mô hình phân bố liên hợp của cả dữ liệu được quan sát và phần dữ liệu ẩn. Khi đó việc ước lượng tham số của mô hình trở nên khó khăn hơn nhiều vì ta không thể tìm được chính xác phân phối xác suất của dữ liệu được quan sát.

Cụ thể như sau, giả sử có biến ngẫu nhiên  $X$  và  $Z$  với mô hình phân phối liên hợp, được cho bởi công thức sau:

$$p_{\theta}(X, Z) = p_{\theta}(X|Z)p_{\theta}(Z) = p_{\theta}(Z|X)p_{\theta}(X) \quad (11.4)$$

Giả sử ta quan sát được  $X$  và muốn ước lượng tham số  $\theta$  của mô hình phân bố  $p_{\theta}(X, Z)$ . Giả sử ta tuân theo nguyên tắc ước lượng hợp lý cực đại

$$\theta^{MLE} = \arg \max_{\theta} \log p_{\theta}(X) = \arg \max_{\theta} \log \sum_Z p_{\theta}(X, Z) \quad (11.5)$$

trong đó, tổng trên  $Z$  được tính theo mọi khả năng có thể có của biến ẩn  $Z$  (do ta không biết  $Z$ ). Tính toán này cực kì khó khăn vì phải duyệt qua mọi khả năng của  $Z$  cũng như do hàm log-tổng.

**Ví dụ 11.9** (Hai đồng xu). Giả sử có 2 đồng xu với xác suất ra mặt ngửa lần lượt là  $\theta_1$  và  $\theta_2$ . Xét thí nghiệm sau: Với xác suất  $p$  ta chọn đồng xu thứ nhất và với xác suất  $1 - p$  ta chọn đồng xu thứ hai để tung. Thực hiện  $n$  thí nghiệm như vậy thu được  $x_1, \dots, x_n \in \{0, 1\}$ , trong đó  $x_i = 1$  nếu được mặt ngửa,  $x_i = 0$  nếu được mặt sấp. Hãy ước lượng bộ tham số  $\theta = (p, \theta_1, \theta_2)$ .

Ở đây biến ẩn là biến  $Z_i \in \{0, 1\}$ , biến  $Z_i$  chỉ ra đồng xu được chọn trong thí nghiệm ( $Z_i = 1$  nếu chọn đồng xu thứ nhất,  $Z_i = 0$  nếu chọn đồng xu thứ hai). Biến quan sát là biến  $X_i \in \{0, 1\}$ . Ta có mô hình phân bố liên hợp như sau:

$$p_\theta(Z_1, X_1, \dots, Z_n, X_n) = \prod_{i=1}^n p_\theta(X_i, Z_i)$$

$$p_\theta(X, Z) = p_\theta(X|Z)p_\theta(Z)$$

$$p_\theta(Z) = p^Z(1-p)^{1-Z}$$

$$p_\theta(X|Z) = \theta_1^{XZ}(1-\theta_1)^{(1-X)Z}\theta_2^{X(1-Z)}(1-\theta_2)^{(1-X)(1-Z)}$$

Như vậy, ta có dạng tường minh (công thức) của xác suất dữ liệu đầy đủ (cả biến quan sát được và biến ẩn). Tuy nhiên, do biến ẩn không được biết (không biết chọn đồng xu nào để tung), ta phải tính tổng trên tất cả các khả năng có thể có của biến ẩn để tính hàm hợp lý (likelihood) của dữ liệu  $D = \{X_1, \dots, X_n\}$ :

$$p_\theta(X_1, \dots, X_n) = \sum_{Z_1, \dots, Z_n} p_\theta(X_1, \dots, X_n, Z_1, \dots, Z_n)$$

Cực đại hoá đại lượng trên không hề dễ dàng mà thường chúng ta phải sử dụng một đại lượng thay thế (cận dưới) để tính hơn để cực đại hoá.

### 11.3.1 Cận dưới bằng chứng

*Cận dưới bằng chứng* (evidence lower bound - ELBO) là một đại lượng thay thế cho hàm hợp lý để tính toán hơn và tối ưu hơn. Trong trường hợp biến quan sát được là  $X$  và biến ẩn là  $Z$  có mô hình xác suất liên hợp  $p_\theta(X, Z)$ . Cận dưới bằng chứng được xác định như sau: Theo nguyên lý ước lượng hợp lý cực đại, ta cần tìm  $\theta^*$  cực đại hàm log-hợp lý

$$\ell(\theta) = \ln p_\theta(X) = \ln \sum_Z p_\theta(X, Z)$$

Xét một phân bố bất kì của biến ẩn  $q(Z)$ , ta có

$$\begin{aligned} \ell(\theta) &= \ln \sum_Z q(Z) \frac{p_\theta(X, Z)}{q(Z)} \\ &= \ln \mathbb{E}_{q(Z)} \left[ \frac{p_\theta(X, Z)}{q(Z)} \right] \end{aligned}$$

Do hàm  $\ln$  là hàm lõm,  $\ln \mathbb{E}[\cdot] \geq \mathbb{E}[\ln(\cdot)]$  nên

$$\begin{aligned} \ell(\theta) &\geq \mathbb{E}_{q(Z)} \left[ \ln \frac{p_\theta(X, Z)}{q(Z)} \right] \\ &= \mathbb{E}_{q(Z)} [\ln p_\theta(X, Z)] + \mathbb{E}_{q(Z)} [-\ln q(Z)] \\ &= \mathbb{E}_{q(Z)} [\ln p_\theta(X, Z)] + H(q) \end{aligned} \tag{11.6}$$

$$\begin{aligned} &= \underbrace{\mathbb{E}_{q(Z)} [\ln p_\theta(X)]}_{=\ln p_\theta(X)=\ell(\theta)} + \underbrace{\mathbb{E}_{q(Z)} [\ln p_\theta(Z|X)] + \mathbb{E}_{q(Z)} [-\ln q(Z)]}_{=-\text{KL}[q||p_\theta(Z|X)]} \\ &= \ell(\theta) - \text{KL}[q||p_\theta(Z|X)]. \end{aligned} \tag{11.7}$$

$= \text{ELBO}$

Trong đó kì vọng được tính trên phân bố  $q(Z)$ ,  $H(q)$  là entropy của  $q(Z)$ ,  $\text{KL}[q||p_\theta(Z|X)]$  là khoảng cách Kullback–Leibler giữa phân bố  $q(Z)$  và phân bố hậu nghiệm  $p_\theta(Z|X)$ .

Bất đẳng thức tại công thức (11.6) đúng với mọi phân bố  $q(Z)$ , cho thấy cận dưới ELBO của  $\ell(\theta)$  gồm hai phần:

- Kỳ vọng của log-hợp-lý đầy đủ  $\mathbb{E}_{q(Z)}[\ln p_\theta(X, Z)]$ : nếu lựa chọn  $q(Z)$  hợp lý, ta có thể tính toán được đại lượng này vì ta đã biết dạng hàm số tường minh của  $p_\theta(X, Z)$ .
- Entropy của  $H(q) = \mathbb{E}_{q(Z)}[-\ln q(Z)]$ : đại lượng này là hằng số đối với  $\theta$  nếu ta cố định  $q(Z)$

Công thức (11.7) cho thấy cận dưới ELBO là chặt nếu ta chọn  $q(Z)$  chính là xác suất hậu nghiệm  $p_\theta(Z|X)$ . Với phân bố này thì  $\text{KL}[q||p_\theta(Z|X)]$  triệt tiêu bằng 0.

### 11.3.2 Thuật toán cực đại hoá kỳ vọng

Kết hợp lại ta có hai bước E và bước M trong thuật toán cực đại hoá kỳ vọng EM (Expectation-Maximization) (Thuật toán 11.1)

- Bước E: Chọn phân bố  $q(Z) = p_\theta(Z|X)$  để tính kỳ vọng

$$ELBO(\theta) = \mathbb{E}_{q(Z)}[\ln p_\theta(X, Z)]$$

- Bước M: Cực đại hóa  $ELBO(\theta)$  để cập nhật tham số  $\theta$ . Bước này đảm bảo rằng log-hợp lý sẽ tăng lên.

**Ví dụ 11.10.** Áp dụng cho bài toán ở ví dụ 11.9, ta có  $X = X_1, \dots, X_n$  và  $Z = Z_1, \dots, Z_n$  là các biến ngẫu nhiên thuộc  $\{0, 1\}$  với xác suất liên hợp

$$p_\theta(Z_1, X_1, \dots, Z_n, X_n) = \prod_{i=1}^n p_\theta(X_i, Z_i)$$

**Bước E:** Tính xác suất hậu nghiệm

$$q(Z_i|X_i) = p_{\theta^{(t-1)}}(Z_i|X_i) = \frac{p_{\theta^{(t-1)}}(X_i, Z_i)}{\sum_{z_i=0}^1 p_{\theta^{(t-1)}}(X_i, z_i)} = \mathbb{E}[Z_i|X_i] = r_i$$

---

**Thuật toán 11.1** Thuật toán Cực đại hoá Kỳ vọng (EM Algorithm)
 

---

- 1: **procedure** TRAINEM( $\mathbf{X}$ )
- 2:    Khởi tạo tham số  $\theta^{(0)}$  ngẫu nhiên
- 3:    **for**  $t = 1, 2, \dots$  cho đến khi hội tụ **do** ▷ Bước E
- 4:     Tính phân phối hậu nghiệm:

$$q(Z) \leftarrow p_{\theta^{(t-1)}}(Z \mid X)$$

▷ Bước M

- 5:     Thiết lập hàm mục tiêu:

$$Q(\theta, \theta^{(t-1)}) = \mathbb{E}_{q(Z)}[\ln p_{\theta}(X, Z)]$$

- 6:     Cập nhật tham số:

$$\theta^{(t)} \leftarrow \arg \max_{\theta} Q(\theta, \theta^{(t-1)})$$

- 7:    **end for**
  - 8:    **return**  $\theta^{(t)}$
  - 9: **end procedure**
- 

Các đại lượng này tính được do ta biết xác suất liên hợp  $p_{\theta}(X_i, Z_i)$ . Đại lượng  $r_i$  còn được gọi là *trách nhiệm* của đồng xu thứ nhất đối với kết quả tung  $X_i$ . Đồng xu thứ hai có trách nhiệm là  $1 - r_i$ .

**Bước M:** Cập nhật tham số  $\theta$  bằng cách tối ưu hoá hàm mục tiêu

$$\begin{aligned} Q(\theta, \theta^{(t-1)}) &= \mathbb{E}_q[\ln p_{\theta}(X, Z)] \\ &= \mathbb{E}_q[\log \theta_1 \sum_{i=1}^n X_i Z_i + \log(1 - \theta_1) \sum_{i=1}^n (1 - X_i) Z_i \\ &\quad + \log \theta_2 \sum_{i=1}^n X_i (1 - Z_i) + \log(1 - \theta_2) \sum_{i=1}^n (1 - X_i) (1 - Z_i) \\ &\quad + \log p \sum_{i=1}^n Z_i + \log(1 - p) \sum_{i=1}^n (1 - Z_i)] \end{aligned}$$

Lấy kì vọng theo phân bố  $q(Z) = p_\theta(Z|X)$  ta được

$$\begin{aligned} Q(\theta, \theta^{(t-1)}) &= \log \theta_1 \sum_{i=1}^n X_i r_i + \log(1 - \theta_1) \sum_{i=1}^n (1 - X_i) r_i \\ &\quad + \log \theta_2 \sum_{i=1}^n X_i (1 - r_i) + \log(1 - \theta_2) \sum_{i=1}^n (1 - X_i) (1 - r_i) \\ &\quad + \log p \sum_{i=1}^n r_i + \log(1 - p) \sum_{i=1}^n (1 - r_i) \end{aligned}$$

Áp dụng phương pháp tìm cực trị như trong Ví dụ 11.3, ta có cập nhật

$$\begin{aligned} \theta_1^{(t)} &= \frac{\sum_{i=1}^n X_i r_i}{\sum_{i=1}^n r_i} \\ \theta_2^{(t)} &= \frac{\sum_{i=1}^n X_i (1 - r_i)}{\sum_{i=1}^n (1 - r_i)} \\ p^{(t)} &= \frac{\sum_{i=1}^n r_i}{n} \end{aligned}$$

### 11.3.3 Thuật toán EM trên phân bố xác suất họ mũ

Phân bố xác suất họ mũ là một lớp các phân bố thường dùng trong xác suất thống kê cũng như Học máy. Hầu hết các phân bố xác suất ta thường dùng như phân bố nhị phân, phân bố nhị thức, phân bố chuẩn, v.v. ... đều thuộc họ phân bố này. Phân bố họ mũ cho biến ngẫu nhiên  $X$  là

$$p_\theta(X) = \frac{1}{Z(\theta)} G(X) \exp \{ \theta^T F(X) \}.$$

Trong đó  $F(X)$  gọi là thống kê đầy đủ của phân bố,  $Z(\theta)$  là hằng số chuẩn hoá để công thức trên tạo thành một xác suất. Đặc điểm của hằng số  $Z(\theta)$  là:

$$\begin{aligned} \nabla \ln Z(\theta) &= \mathbb{E}[F(X)] \\ \nabla^2 \ln Z(\theta) &= \mathbb{V}\text{ar}[F(X)] \end{aligned}$$



Nếu áp dụng nguyên lý Ước lượng hợp lý cực đại lên phân bố xác suất họ mũ, ta có công thức sau:

$$\ell(\theta) = \ln p_\theta(X) = \underbrace{\ln G(X)}_{\text{const}} + \theta^T F(X) - \ln Z(\theta). \quad (11.8)$$

Do đó, cực trị của  $\ell(\theta)$  chỉ phụ thuộc vào thống kê đầy đủ  $F(X)$

$$\theta^* = \mathcal{A}(F(X)).$$

Trong đó,  $\mathcal{A}$  là thuật toán ước lượng hợp lý cực đại dựa trên thống kê đầy đủ  $F(X)$ .

**Thuật toán EM trên xác suất họ mũ.** Nếu xác suất liên hợp  $p_\theta(X, Z)$  thuộc họ mũ thì thuật toán EM hoạt động thế nào? Ta sẽ cực đại hoá kì vọng

$$\begin{aligned} \mathbb{E}_q[\ln p_\theta(X, Z)] &= \mathbb{E}_q[\ln G(X, Z) + \theta^T F(X, Z) - \ln Z(\theta)] \\ &= \underbrace{\mathbb{E}_q[\ln G(X, Z)]}_{\text{const}} + \theta^T \mathbb{E}_q[F(X, Z)] - \underbrace{\mathbb{E}_q[\ln Z(\theta)]}_{\ln Z(\theta)} \\ &= \text{const} + \theta^T \mathbb{E}_q[F(X, Z)] - \ln Z(\theta) \end{aligned} \quad (11.9)$$

Dễ thấy công thức (11.9) trên không khác gì công thức (11.8) ngoài việc thay thống kê đầy đủ  $F(X)$  bằng kì vọng  $\mathbb{E}_q[F(X, Z)]$ . Do đó, để có được thuật toán EM, ở bước M, ta chỉ cần áp dụng thuật toán ước lượng hợp lý cực đại  $\mathcal{A}$  với đầu vào là kì vọng  $\mathbb{E}_q[F(X, Z)]$  là cập nhật được tham số  $\theta^{(t)} = \mathcal{A}(\mathbb{E}_q[F(X, Z)])$ .

**Ví dụ 11.11** (Phân bố trộn Gauss). Phân bố Gauss có dạng

$$p_{\mu, \Sigma}(\mathbf{x}) = \mathcal{N}(\mathbf{x} | \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} e^{-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)}$$

với  $\mu, \Sigma$  lần lượt là kì vọng và ma trận hiệp phương sai.

Theo nguyên tắc ước lượng hợp lý cực đại, với dữ liệu  $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ , ta có

$$\boldsymbol{\mu}^{MLE} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \quad (11.10)$$

$$\boldsymbol{\Sigma}^{MLE} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \boldsymbol{\mu}^{MLE})(\mathbf{x}_i - \boldsymbol{\mu}^{MLE})^T \quad (11.11)$$

Ta có thể trộn  $K$  phân bố Gauss lại với nhau để được phân bố trộn Gauss có dạng

$$p_{\theta}(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

trong đó, bộ tham số  $\theta = (\pi_{1:K}, \boldsymbol{\mu}_{1:K}, \boldsymbol{\Sigma}_{1:K})$  với điều kiện  $\pi_k \geq 0$  và  $\sum_k \pi_k = 1$ . Tức là mẫu dữ liệu  $\mathbf{x}$  được lấy mẫu bằng cách chọn ngẫu nhiên một phân bố trong  $K$  phân bố Gauss với xác suất  $\pi_{1:K}$ .

Để ước lượng bằng thuật toán EM, gọi  $z_{ik} \in \{0, 1\}$  là biến ngẫu nhiên (biến ẩn - hidden variable) chỉ ra  $\mathbf{x}_i$  (biến quan sát được - observed variable) được lấy mẫu từ phân bố thứ  $k$ . Đặt  $\mathbf{e}_k = 0, \dots, 0, 1, 0, \dots, 0$  là véc-tơ cơ sở chuẩn tắc thứ  $k$ . Ta có

$$\begin{aligned} p(\mathbf{z}_i = \mathbf{e}_k) &= \pi_k \\ p(\mathbf{z}_i) &= \prod_{k=1}^K \pi_k^{z_{ik}} \\ p(\mathbf{x}_i | \mathbf{z}_i = \mathbf{e}_k) &= \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \\ p(\mathbf{x}_i | \mathbf{z}_i) &= \prod_{k=1}^K \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_{ik}} \\ p(\mathbf{x}_i, \mathbf{z}_i) &= \prod_{k=1}^K \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_{ik}} \pi_k^{z_{ik}} \\ p(\mathbf{z}_i = \mathbf{e}_k | \mathbf{x}_i) &= r_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_k \pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)} \end{aligned}$$

Xác suất cuối cùng trong các công thức trên được gọi là trách nhiệm của phân bố thứ  $k$  với mẫu dữ liệu  $\mathbf{x}_i$ .

Hàm hợp lý của dữ liệu đầy đủ  $X = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$ ,  $Z = (z_{ik})$  là

$$p_\theta(\mathbf{X}, Z) = \prod_{i=1}^n \prod_{k=1}^K \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_{ik}} \pi_k^{z_{ik}}$$

với log-hợp lý

$$\log p_\theta(\mathbf{X}, Z) = \sum_{i=1}^n \sum_{k=1}^K z_{ik} \log \pi_k + z_{ik} \log \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Lấy kì vọng theo phân bố hậu nghiệm được (Bước E)

$$\mathbb{E}_{Z|\mathbf{X}}[\log p_\theta(\mathbf{X}, Z)] = \sum_{i=1}^n \sum_{k=1}^K r_{ik} \log \pi_k + r_{ik} \log \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Giải cực trị của hàm trên ta được cập nhật (Bước M)

$$\begin{aligned} \pi_k &\leftarrow \frac{1}{n} \sum_{i=1}^n r_{ik} \\ \boldsymbol{\mu}_k &\leftarrow \frac{\sum_{i=1}^n r_{ik} \mathbf{x}_i}{\sum_{i=1}^n r_{ik}} \\ \boldsymbol{\Sigma}_k &\leftarrow \frac{\sum_{i=1}^n r_{ik} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^T}{\sum_{i=1}^n r_{ik}} \end{aligned}$$

**Ví dụ 11.12** (Thuật toán K-means). Thuật toán K-means (Thuật toán 11.2) là một thuật toán phân cụm dựa trên phân bố trộn Gauss với hai điểm riêng sau:

- Cố định các ma trận hiệp phương sai  $\boldsymbol{\Sigma}_k$  là ma trận đơn vị.
- Xác định trách nhiệm của các cụm bằng cách tính khoảng cách và chọn cụm gần nhất với dữ liệu.

**Thuật toán 11.2** Thuật toán K-means

---

```

1: procedure TRAINKMEANS($\mathbf{X}, K$)
2: Khởi tạo các tâm cụm $\boldsymbol{\mu}_1^{(0)}, \dots, \boldsymbol{\mu}_K^{(0)}$ ngẫu nhiên
3: for $t = 1, 2, \dots$ cho đến khi hội tụ do
4: for all $\mathbf{x}_i \in \mathbf{X}$ do
5: Gán cụm gần nhất: ▷ Bước E: Gán cụm

$$r_{ik} \leftarrow \begin{cases} 1 & \text{nếu } k = \arg \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j^{(t-1)}\| \\ 0 & \text{ngược lại} \end{cases}$$

6: end for
7: for $k = 1$ to K do
8: Cập nhật: ▷ Bước M: Cập nhật tâm cụm

$$\boldsymbol{\mu}_k^{(t)} \leftarrow \frac{\sum_{i=1}^n r_{ik} \cdot \mathbf{x}_i}{\sum_{i=1}^n r_{ik}}$$

9: end for
10: end for
11: return $\theta = (\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K)$
12: end procedure

```

---

**11.4 Ước lượng mật độ phi tham số**

Ước lượng mật độ phi tham số là phương pháp xây dựng mô hình dựa trên dữ liệu huấn luyện nhưng không có số lượng tham số cố định. Có hai phương pháp ước lượng mật độ cơ bản là: phương pháp ước lượng mật độ bằng cách đếm và phương pháp ước lượng mật độ bằng hàm nhân. Phương pháp ước lượng bằng cách đếm được sử dụng phổ biến đối với dữ liệu đơn giản, số chiều đặc trưng ít. Do đó trong phần này, chúng tôi xin trình bày về phương pháp sử dụng hàm nhân.

Phương pháp ước lượng mật độ bằng hàm nhân (kernel density estimation - KDE) dựa trên ý tưởng của sô Parzen. Theo đó, mỗi

điểm dữ liệu huấn luyện có "ảnh hưởng" đến vùng không gian xung quanh nó (cửa sổ) thông qua một hàm nhân.

Xét một lân cận  $V_{\mathbf{x}}$  quanh điểm dữ liệu  $\mathbf{x}$ . Mật độ xác suất trung bình của lân cận  $V_{\mathbf{x}}$  là

$$\bar{p}_{\mathbf{x}} = \frac{\mathbb{P}(V_{\mathbf{x}})}{v(V_{\mathbf{x}})}$$

với  $v(V_{\mathbf{x}})$  là thể tích của lân cận  $V_{\mathbf{x}}$  còn  $\mathbb{P}(V_{\mathbf{x}})$  là tổng xác suất của  $V_{\mathbf{x}}$ . Xác suất  $\mathbb{P}(V_{\mathbf{x}})$  có thể ước lượng bằng tỉ lệ số điểm dữ liệu rơi vào lân cận  $V_{\mathbf{x}}$

$$\mathbb{P}(V_{\mathbf{x}}) = \frac{\sum_{i=1}^n \mathbb{I}(\mathbf{x}_i \in V_{\mathbf{x}})}{n} \implies \bar{p}_{\mathbf{x}} = \frac{\sum_{i=1}^n \mathbb{I}(\mathbf{x}_i \in V_{\mathbf{x}})}{nv(V_{\mathbf{x}})}$$

Xét trường hợp đơn giản trong không gian  $\mathbb{R}^d$ , ta chọn lân cận  $V_{\mathbf{x}}$  là hình hộp (cửa sổ Parzen) có kích thước  $s$  và có tâm tại  $\mathbf{x}$ , ta có

$$\begin{aligned} \mathbb{I}(\mathbf{x}_i \in V_{\mathbf{x}}) &= \kappa\left(\frac{\mathbf{x} - \mathbf{x}_i}{s}\right) \\ v(V_{\mathbf{x}}) &= s^d \end{aligned}$$

với  $\kappa(\mathbf{u})$  là hàm nhân có các tính chất sau:

- $\kappa(\mathbf{u})$  thể hiện hình hộp có kích thước bằng 1 có tâm ở gốc tọa độ.
- $\kappa(\mathbf{u}/s)$  hình hộp có kích thước  $s$  có tâm ở gốc tọa độ
- $\kappa((\mathbf{u} - \mathbf{x})/s)$  hình hộp có kích thước  $s$  có tâm ở  $\mathbf{x}$

Dựa trên các tính chất trên, chúng ta có thể chọn hàm nhân  $\kappa(\mathbf{u})$  sao cho

$$\kappa(\mathbf{u}) = \begin{cases} 1 & \text{nếu } |u_k| \leq 1/2, \forall k = 1, 2, \dots, d \\ 0 & \text{nếu có } k : |u_k| > 1/2 \end{cases}$$

Ta có ước lượng phân phối  $p_{\mathbf{x}}$  tại điểm  $\mathbf{x}$  là:

$$\bar{p}_{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \frac{1}{s^d} \kappa\left(\frac{\mathbf{x} - \mathbf{x}_i}{s}\right) = \frac{1}{n} \sum_{i=1}^n \kappa_s(\mathbf{x} - \mathbf{x}_i)$$

với  $\kappa_s(\mathbf{u}) = s^{-d} \kappa\left(\frac{\mathbf{u}}{s}\right)$ . Một tính chất của các hàm  $\kappa, \kappa_s$  là chúng có tích phân bằng 1 giống như một hàm mật độ xác suất và đối xứng.

$$\int_{\mathbf{u}} \kappa_s(\mathbf{u}) d\mathbf{u} = 1$$
$$\kappa(\mathbf{u}) = \kappa(-\mathbf{u})$$

Các lập luận trên mở ra một hướng tiếp cận ước lượng mật độ theo lược đồ Thuật toán 11.3.

---

**Thuật toán 11.3** Ước lượng mật độ KDE (Kernel Density Estimation)

---

- 1: **procedure** KDE( $\mathbf{x}, \{\mathbf{x}_1, \dots, \mathbf{x}_n\}, \kappa, s$ )  $\triangleright \kappa$  là hàm nhân,  $s$  là băng thông (bandwidth)
- 2:     Tính hàm mật độ co giãn:

$$\kappa_s(\mathbf{u}) \leftarrow \frac{1}{s^d} \cdot \kappa\left(\frac{\mathbf{u}}{s}\right)$$

- 3:     Tính ước lượng mật độ tại điểm  $\mathbf{x}$ :

$$\bar{p}_{\mathbf{x}} \leftarrow \frac{1}{n} \sum_{i=1}^n \kappa_s(\mathbf{x} - \mathbf{x}_i)$$

- 4:     **return**  $\bar{p}_{\mathbf{x}}$
  - 5: **end procedure**
- 

Hàm nhân  $\kappa_s(\mathbf{u})$  được điều chỉnh bởi tham số kích thước  $s$ . Khi  $s$  lớn, hàm mật độ  $\kappa_s(\mathbf{u})$  bao phủ rộng hơn (của số Parzen lớn) và ngược lại. Một số hàm nhân hay được sử dụng trong lý thuyết và thực tế như:

- Hàm nhân Gauss (Gaussian kernel):

$$\kappa(\mathbf{u}) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{1}{2}\mathbf{u}^T\mathbf{u}\right)$$

- Hàm nhân Epanechnikov (Epanechnikov kernel):

$$\kappa(\mathbf{u}) = \frac{d+2}{c_d} \max(1 - \|\mathbf{u}\|^2, 0)$$

với  $c_d = \int_{\|\mathbf{u}\| \leq 1} 1 d\mathbf{u}$  là thể tích của vòng tròn đơn vị trong  $\mathbb{R}^d$ .

## 11.5 Biểu diễn phân bố bằng mạng nơ-ron

### 11.5.1 Biểu diễn mật độ bằng hàm năng lượng

Các mô hình biểu diễn phân bố bằng hàm năng lượng là một lớp mô hình sinh tổng quát hoá xác suất họ mũ như sau:

$$p_\theta(x) = \frac{1}{Z(\theta)} \exp(-E_\theta(x)) = \frac{1}{Z(\theta)} \exp(f_\theta(x)),$$

$$Z(\theta) = \int \exp(f_\theta(x)) dx.$$

trong đó

- $x$  là một điểm dữ liệu.
- $E_\theta(x)$  là hàm năng lượng (thế năng) của điểm dữ liệu  $x$ .
- $Z(\theta)$  là hàm chia.
- $f_\theta(x) = -E_\theta(x)$  là mô hình cần huấn luyện.
- $\theta$  là tham số của mô hình cần huấn luyện.

Về bản chất, hàm  $p_\theta(x)$  có công thức rất giống với hàm softmax dùng trong các mô hình phân lớp: mỗi điểm dữ liệu  $x$  được gán một điểm số  $f_\theta(x)$  và sử dụng softmax để tính mật độ xác suất của  $x$ .

**Ví dụ 11.13.** Giả sử  $x$  là một ảnh màu  $x \in [0, 1]^{H \times W \times 3}$ ,  $f_\theta(x)$  là một mạng nơ-ron (ví dụ: mạng CNN) có một đầu ra là số thực. Khi đó, ta có thể huấn luyện mạng này để biểu diễn được phân bố của ảnh màu từ bộ dữ liệu  $D = \{x_1, x_2, \dots, x_n\}$ .

**Lấy mẫu.** Việc lấy mẫu  $x \sim p_\theta(x)$  có thể thực hiện bằng phương pháp Langevin Monte Carlo như sau:

$$\begin{aligned}\eta^{(t)} &\sim \mathcal{N}(0, I) \\ x^{(t+1)} &= x^{(t)} + \epsilon \nabla_x f_\theta(x^{(t)}) + \sqrt{2\epsilon} \eta^{(t)}\end{aligned}$$

Bước lấy mẫu được lặp lại  $T$  lần để đảm bảo phân phối của  $x^{(T)}$  gần với phân phối  $p_\theta(x)$ .

**Huấn luyện mô hình biểu diễn mật độ bằng hàm năng lượng.** Để huấn luyện mô hình biểu diễn mật độ bằng hàm năng lượng, sử dụng nguyên tắc ước lượng hợp lý cực đại MLE, ta cần cực đại hóa hàm hợp lý theo tham số  $\theta$  như sau:

$$\begin{aligned}\theta^{MLE} &= \arg \max_{\theta} \sum_{i=1}^n \log p_\theta(x_i) \\ &= \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n f_\theta(x_i) - \log Z(\theta).\end{aligned}$$

**Định lý 11.14** (Đạo hàm hàm chia). *Đạo hàm của hàm chia  $\log Z(\theta)$  là*

$$\nabla \log Z(\theta) = \mathbb{E}_{p_\theta(x)}[\nabla f_\theta(X)].$$



*Chứng minh:*

$$\begin{aligned}
 \nabla \log Z(\theta) &= \frac{1}{Z(\theta)} \nabla Z(\theta) \\
 &= \frac{1}{Z(\theta)} \nabla \int \exp(f_\theta(x)) dx \\
 &= \int \frac{1}{Z(\theta)} \exp(f_\theta(x)) \nabla f_\theta(x) dx \\
 &= \int p_\theta(x) \nabla f_\theta(x) dx \\
 &= \mathbb{E}_{p_\theta(x)}[\nabla f_\theta(X)].
 \end{aligned}$$

□

Sử dụng phương pháp leo đồi bằng đạo hàm thì cần tính đạo hàm như sau:

$$\begin{aligned}
 \nabla \ell(\theta) &= \frac{1}{n} \sum_{i=1}^n \nabla f_\theta(x_i) - \mathbb{E}_{p_\theta(x)}[\nabla f_\theta(X)] \\
 &\approx \frac{1}{n} \sum_{i=1}^n \nabla f_\theta(x_i) - \frac{1}{n_s} \sum_{i=1, z_i \sim p_\theta(x)}^{n_s} \nabla f_\theta(z_i)
 \end{aligned}$$

Số hạng đầu tiên trong công thức trên có thể tính bằng phương pháp lan truyền ngược và số hạng thứ hai có thể tính bằng phương pháp Monte Carlo thông qua việc lấy mẫu  $x \sim p_\theta(x)$ . Do đó, ta có thể huấn luyện mô hình biểu diễn mật độ bằng hàm năng lượng bằng cách sử dụng phương pháp lan truyền ngược và phương pháp lấy mẫu Monte Carlo.

Một cách nhìn khác của thuật toán huấn luyện mô hình năng lượng ở trên là bài toán cực tiểu hoá một hàm lỗi  $V(Z, \theta)$  như sau:

$$\min_Z \max_\theta V(Z, \theta) = \frac{1}{n} \sum_{i=1}^n f_\theta(x_i) - \frac{1}{n_s} \sum_{i=1}^{n_s} f_\theta(z_i)$$

trong đó,  $Z = (z_1, \dots, z_{n_s})$  cần cực tiểu  $V(z, \theta)$  hay cực đại

$$\frac{1}{n_s} \sum_{i=1}^{n_s} f_\theta(z_i),$$

---

**Thuật toán 11.4** Huấn luyện mô hình năng lượng (EBM Training)

---

```
1: procedure TRAINEBM($\{\mathbf{x}_1, \dots, \mathbf{x}_n\}, \eta$)
2: Khởi tạo tham số $\theta^{(0)}$
3: for $t = 1, 2, \dots$ cho đến khi hội tụ do $\triangleright$ Lấy mẫu từ mô
 hình hiện tại
4: for $i = 1$ to n_s do
5: Lấy mẫu $z_i \sim p_{\theta^{(t-1)}}(x)$
6: end for $\triangleright$ Cập nhật tham số theo chênh lệch đạo hàm
7: Tính đạo hàm:

$$\theta^{(t)} \leftarrow \theta^{(t-1)} + \eta \left[\frac{1}{n} \sum_{i=1}^n \nabla f_{\theta^{(t-1)}}(\mathbf{x}_i) - \frac{1}{n_s} \sum_{i=1}^{n_s} \nabla f_{\theta^{(t-1)}}(z_i) \right]$$

8: end for
9: return θ
10: end procedure
```

---

còn  $\theta$  cần cực đại  $V(z, \theta)$  hay cực đại điểm số của dữ liệu

$$\frac{1}{n} \sum_{i=1}^n f_{\theta}(x_i).$$

### 11.5.2 Biểu diễn phân bố bằng hàm sinh

Một cách tiếp cận khác để biểu diễn phân bố (một cách gián tiếp) là sử dụng hàm sinh. Phương pháp này mô tả cách sinh ra dữ liệu một cách ngẫu nhiên từ một phân bố. Cụ thể, để sinh ra một điểm dữ liệu  $x$  thuộc phân bố  $p(x)$ , ta xây dựng một mạng nơ-ron có đầu vào là một biến ngẫu nhiên  $z$  và đầu ra là một điểm dữ liệu  $x$  như sau:

$$\begin{aligned} z &\sim p_z = \mathcal{N}(0, I) \in \mathbb{R}^d \\ x &= G_{\theta}(z) \sim p(x). \end{aligned}$$

trong đó,  $z$  là véc-tơ ngẫu nhiên có phân phối chuẩn,  $G_{\theta}(z)$  là một mạng nơ-ron biến đổi không gian ngẫu nhiên của  $z$  thành

không gian dữ liệu  $x$  có phân bố  $p(x)$ . Mạng nơ-ron  $G_\theta(z)$  được gọi là mô hình sinh dữ liệu (generator). Gọi  $p_g$  là phân bố của  $X = G_\theta(Z)$ ,  $Z \sim \mathcal{N}(0, I)$ .

**Lấy mẫu.** Mạng  $G_\theta(z)$  được huấn luyện sao cho đầu ra của nó tuân theo phân bố mong muốn  $p(x)$  khi  $z \sim \mathcal{N}(0, I)$ . Do đó, việc lấy mẫu từ mô hình sinh dữ liệu rất đơn giản, theo công thức sau:

$$\begin{aligned} z &\sim p_z = \mathcal{N}(0, I) \\ x &= G_\theta(z) \end{aligned}$$

**Khoảng cách Kullback–Leibler và Jensen–Shannon.** Bản chất của ước lượng hợp lý cực đại là tối thiểu khoảng cách Kullback–Leibler (KL) giữa mô hình phân bố  $q(x) = p_\theta(x)$  (phân bố của  $G_\theta(Z)$ ) và phân bố của dữ liệu  $p(x)$  (không biết nhưng biết bộ dữ liệu  $D = \{x_1, x_2, \dots, x_n\}$ ).

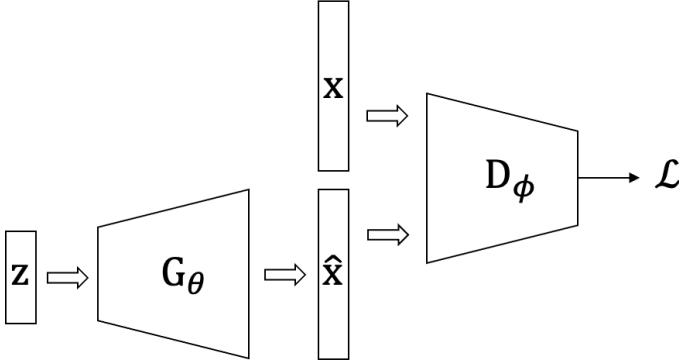
$$D_{KL}(p||q) = \int p(x) \log \frac{p(x)}{q(x)} dx$$

Khoảng cách KL bằng không khi và chỉ khi  $p(x) = q(x), \forall x$ . Tuy nhiên, việc sử dụng khoảng cách KL trong tính toán gặp một số khó khăn sau

- Khoảng cách KL không đối xứng  $D_{KL}(p||q) \neq D_{KL}(q||p)$ .
- Khi tính trên các vùng có xác suất  $p(x)$  thấp (ít dữ liệu) nhưng xác suất  $q(x)$  lại lớn, đặc biệt trong giai đoạn đầu của quá trình huấn luyện.

Khoảng cách Jensen–Shannon có thể được sử dụng để giảm khó khăn tính toán trên bằng việc đối xứng hoá tính toán khoảng cách như sau:

$$D_{JS}(p||q) = \frac{1}{2} \left[ D_{KL} \left( p \left\| \frac{p+q}{2} \right\| \right) + D_{KL} \left( q \left\| \frac{p+q}{2} \right\| \right) \right]$$



**Hình 11.3: Minh hoạ mô hình GAN**

Khoảng cách Jensen–Shannon có tính chất đối xứng và trơn hơn so với khoảng cách Kullback–Leibler.

**Huấn luyện hàm sinh.** Để hướng dẫn việc huấn luyện hàm sinh  $G_\theta(z)$ , ta sử dụng một mô hình phân biệt  $D_\phi(x)$  có đầu ra là đánh giá xác suất  $x$  thuộc vào phân bố  $p(x)$ . Mô hình  $D_\phi(x)$  đóng vai trò của một “nhà phê bình” phân biệt các mẫu dữ liệu “thật” (real) khỏi những mẫu dữ liệu “giả”. Mô hình  $G_\theta(z)$  đóng vai trò là kẻ tấn công tìm cách đánh lừa “nhà phê bình”  $D_\phi(x)$ . Hai mô hình cạnh tranh nhau nên mô hình này còn được gọi là *mô hình sinh đối nghịch* GAN (Generative Adversarial Networks). Quá trình này được minh hoạ ở Hình 11.3.

Xác suất của một mẫu dữ liệu  $x$  thuộc vào phân bố  $p(x)$  là  $D_\phi(x)$ . Ta muốn xác suất này cực đại khi  $x \sim p(x)$  tức là cực đại  $\mathbb{E}_{X \sim p(x)}[\log D_\phi(X)]$ . Ta muốn xác suất này cực tiểu khi  $x = G_\theta(z), z \sim p_z$  tức là cực đại  $\mathbb{E}_{Z \sim p_z}[\log(1 - D_\phi(G_\theta(Z)))]$ .

Ngược lại, ta lại muốn hàm sinh  $G_\theta(z)$  “đánh lừa” được mô hình phân biệt  $D_\phi(x)$ , tức là cực tiểu  $\mathbb{E}_{Z \sim p_z}[\log(1 - D_\phi(G_\theta(Z)))]$ . Kết hợp lại, việc huấn luyện  $G_\theta, D_\phi$  là bài toán cực tiểu hoá một cực

đại sau:

$$\min_G \max_D L(G, D) \triangleq E_{X \sim p(x)} [\log D_\phi(X)] + \mathbb{E}_{Z \sim p_z} [\log(1 - D_\phi(G_\theta(Z)))]$$

---

**Thuật toán 11.5** Thuật toán huấn luyện mô hình GAN
 

---

- 1: **procedure** TRAINGAN( $D, n_s$ )
- 2:    Khởi tạo tham số cho mô hình sinh  $G_\theta$  và mô hình phân biệt  $D_\phi$
- 3:    **while** chưa hội tụ **do**
- 4:     Lấy mẫu  $z_1, \dots, z_{n_s} \sim \mathcal{N}(0, I)$
- 5:     Lấy mẫu  $x_1, \dots, x_{n_s} \sim D$  (dữ liệu thật)
- 6:     Cập nhật tham số  $D_\phi$  theo:

$$\phi \leftarrow \arg \max_{\phi} \frac{1}{n_s} \sum_{i=1}^{n_s} [\log D_\phi(x_i) + \log(1 - D_\phi(G_\theta(z_i)))]$$

- 7:     Cập nhật tham số của  $G_\theta$  để giảm xác suất bị phân biệt:

$$\theta \leftarrow \arg \min_{\theta} \frac{1}{n_s} \sum_{i=1}^{n_s} \log(1 - D_\phi(G_\theta(z_i)))$$

- 8:    **end while**
  - 9:    **return**  $G_\theta, D_\phi$
  - 10: **end procedure**
- 

Gọi  $p_g$  là phân bố của đầu ra của mạng  $G_\theta$ , ta có

$$L(G, D) = \int p(x) \log D_\phi(x) + p_g(x) \log(1 - D_\phi(x)) dx$$

Để tìm  $D_\phi(x)$  cực đại  $L(G, D)$ , xét tổng bên trong tích phân. Đặt  $A = p(x), B = p_g(x), y = D_\phi(x)$ , ta có

$$f(y) = A \log y + B \log(1 - y)$$

suy ra cực đại của hàm này đạt tại  $y = \frac{A}{A+B}$ , tức là  $D_\phi^*(x) = \frac{p(x)}{p(x)+p_g(x)}$ . Khi  $D_\phi^*(x)$  tối ưu như trên thì

$$\begin{aligned} D_{JS}(p||p_g) &= \frac{1}{2} \left[ D_{KL} \left( p \left\| \frac{p+p_g}{2} \right\| \right) + D_{KL} \left( p_g \left\| \frac{p+p_g}{2} \right\| \right) \right] \\ &= \frac{1}{2} \log 2 + \frac{1}{2} \int p(x) \log \frac{p(x)}{p(x)+p_g(x)} \\ &\quad + \frac{1}{2} \log 2 + \frac{1}{2} \int p_g(x) \log \frac{p_g(x)}{p(x)+p_g(x)} \\ &= \frac{1}{2} \log 4 + \frac{1}{2} \mathbb{E}_{X \sim p(x)} [\log D_\phi^*(X)] + \frac{1}{2} \mathbb{E}_{X \sim p_g(x)} [1 - \log D_\phi^*(X)] \\ &= \frac{1}{2} \log 4 + \frac{1}{2} L(G, D^*) \end{aligned}$$

Khi hàm sinh  $G_\theta^*(x)$  cho phân bố  $p_g(x) \approx p(x)$  thì  $D_\phi(x)$  sẽ bị “lừa”, tức là  $D_\phi^*(x) \approx 1/2$ . Khi đó ta có

$$L(G^*, D^*) = \int \left( p(x) \log \frac{1}{2} + p_g(x) \log \frac{1}{2} \right) dx = -2 \log 2$$

Như vậy, hàm mục tiêu  $L(G, D^*)$  của  $G_\theta$  bản chất là khoảng cách Jensen-Shannon giữa hai phân bố  $p$  và  $p_g$ . Khi  $G_\theta^*$  tối ưu thì  $L(G^*, D^*)$  tối ưu và bằng  $-2 \log 2$ .

### Các yếu điểm của mô hình GAN cổ điển

- Dữ liệu nằm trong không gian có số chiều thấp: Các dữ liệu  $x$  về mặt biểu diễn có thể có số chiều rất lớn. Do đó, khi khởi tạo, phân bố  $p_g$  có thể nằm ở vùng không gian khác hoàn toàn so với  $p$ . Do đó rất dễ để mô hình phân biệt  $D_\phi$  học được cách phân biệt hoàn hảo hai phân bố này (phân biệt thật - giả).
- Đạo hàm dùng để huấn luyện bị triệt tiêu: Khi  $D_\phi(x)$  hoàn hảo thì  $D(x) \approx 1$  với  $x \sim p(x)$  và  $D(x) \approx 0$  khi  $x \sim p_g$ . Khi đó  $L(G, D) \approx 0$  và đạo hàm bị triệt tiêu gần bằng 0, khiến cho việc huấn luyện rất khó khăn.

Mô hình Wasserstein GAN (WGAN) tìm cách giải quyết khó khăn của mô hình GAN cổ điển bằng cách đánh giá khoảng cách giữa hai phân bố  $p$  và  $p_g$  “trơn” hơn. Khoảng cách Wasserstein giữa hai phân bố  $p$  và  $p_g$  được định nghĩa như sau

$$W(p, p_g) = \sup_{\|f\| \leq K} \mathbb{E}_{X \sim p(x)}[f(X)] - \mathbb{E}_{X \sim p_g(x)}[f(X)]$$

trong đó  $f$  là hàm số liên tục K-Lipschitz. Tiếp tục nói lỏng điều kiện đối với hàm  $f$  bằng cách biểu diễn hàm  $f$  bằng một mạng nơ-ron với bộ trọng số  $w \in W$ . Ta có hàm mục tiêu

$$L(p, p_g) = W(p, p_g) = \max_{w \in W} \mathbb{E}_{X \sim p(x)}[f_w(X)] - \mathbb{E}_{Z \sim p_z}[f_w(G_\theta(Z))]$$

Tương tự như huấn luyện mô hình GAN, để huấn luyện mô hình Wasserstein GAN, ta huấn luyện  $f_w$  để cực đại  $\mathbb{E}_{X \sim p(x)}[f_w(X)] - \mathbb{E}_{Z \sim p_z}[f_w(G_\theta(Z))]$  nhằm phân biệt  $p$  và  $p_g$ . Còn để huấn luyện  $G_\theta$ , ta cực tiểu đại lượng  $-\mathbb{E}_{Z \sim p_z}[f_w(G_\theta(Z))]$  nhằm đánh lừa “nhà phê bình”  $f_w$  (Thuật toán 11.6).

## 11.6 Tình huống áp dụng: Generative Adversarial Networks (GAN)

Trong hướng dẫn này, chúng ta sẽ triển khai một mô hình GAN đơn giản để sinh ra hình ảnh giống như tập dữ liệu MNIST, một tập dữ liệu phổ biến gồm 60.000 hình ảnh số viết tay kích thước 28x28.

1. Sử dụng các thư viện như `torch`, `torchvision` và `matplotlib` để tải và hiển thị hình ảnh từ tập dữ liệu MNIST.
2. Xây dựng một mạng sinh (Generator) có đầu vào là một véc-tơ ngẫu nhiên (100 chiều) và đầu ra là một hình ảnh 28x28 (Bảng 11.1).

---

**Thuật toán 11.6** Thuật toán huấn luyện mô hình WGAN

---

- 1: **procedure** TRAINWGAN( $D, n_s, n_{\text{critic}}, \alpha, c$ )
- 2:   Khởi tạo tham số mô hình sinh  $\theta^{(0)}$ , mô hình phê bình  $w^{(0)}$
- 3:   **while** chưa hội tụ **do**
- 4:     **for**  $j = 1$  **to**  $n_{\text{critic}}$  **do**
- 5:       Lấy mẫu  $z_1, \dots, z_{n_s} \sim \mathcal{N}(0, I)$
- 6:       Lấy mẫu  $x_1, \dots, x_{n_s} \sim D$
- 7:       Tính đạo hàm:

$$g_w \leftarrow \nabla_w \left[ \frac{1}{n_s} \sum_{i=1}^{n_s} f_w(x_i) - \frac{1}{n_s} \sum_{i=1}^{n_s} f_w(G_\theta(z_i)) \right]$$

- 8:       Cập nhật tham số mô hình phê bình:

$$w \leftarrow w + \alpha \cdot \text{RMSProp}(w, g_w)$$

- 9:       Chặt giá trị:

$$w \leftarrow \text{clip}(w, -c, c)$$

- 10:     **end for**
- 11:     Lấy mẫu  $z_1, \dots, z_{n_s} \sim \mathcal{N}(0, I)$
- 12:     Tính đạo hàm:

$$g_\theta \leftarrow \nabla_\theta \left[ -\frac{1}{n_s} \sum_{i=1}^{n_s} f_w(G_\theta(z_i)) \right]$$

- 13:       Cập nhật tham số mô hình sinh:

$$\theta \leftarrow \theta - \alpha \cdot \text{RMSProp}(\theta, g_\theta)$$

- 14:     **end while**
  - 15:     **return**  $G_\theta, f_w$
  - 16: **end procedure**
-



**Bảng 11.1:** Các lớp trong kiến trúc của Generator trong mô hình GAN

| Lớp      | Đầu vào | Đầu ra | Hàm kích hoạt/Xử lý |
|----------|---------|--------|---------------------|
| Linear 1 | 100     | 128    | LeakyReLU (0.2)     |
| Linear 2 | 128     | 256    | LeakyReLU (0.2)     |
| Linear 3 | 256     | 784    | Tanh                |

3. Xây dựng một mạng phân loại (Discriminator) có đầu vào là một hình ảnh 28x28 và đầu ra là một giá trị xác suất (0-1) cho biết hình ảnh đó có phải là hình ảnh thật hay không (Bảng 11.2).

**Bảng 11.2:** Các lớp trong kiến trúc của Discriminator trong mô hình GAN

| Lớp      | Đầu vào | Đầu ra | Hàm kích hoạt   |
|----------|---------|--------|-----------------|
| Linear 1 | 784     | 512    | LeakyReLU (0.2) |
| Linear 2 | 512     | 256    | LeakyReLU (0.2) |
| Linear 3 | 256     | 1      | Sigmoid         |

4. Để huấn luyện GAN, ta chọn hàm lỗi entropy chéo nhị phân `nn.BCELoss()` để phân biệt ảnh thật, giả và thuật toán tối ưu hóa Adam với các tham số  $lr = 0.0002$  và  $betas = (0.5, 0.999)$  cho cả hai mạng sinh và phân loại.

5. Vòng lặp huấn luyện là phần cốt lõi của mô hình GAN, trong đó huấn luyện mạng sinh và mạng phân biệt xen kẽ nhau.

- *[Chuẩn bị dữ liệu]:* Sử dụng tập dữ liệu MNIST với kích thước lô là 64 (`batch_size`). Dữ liệu được chuẩn hóa về khoảng  $[-1, 1]$  để phù hợp với đầu ra của hàm sinh (hàm kích hoạt Tanh).
- *[Huấn luyện mạng phân biệt]:*

- Với hình ảnh thật từ MNIST, gán nhãn là 1 (thật).
  - Với hình ảnh giả do mạng sinh tạo ra, gán nhãn là 0 (giả).
  - Tính tổng hàm lỗi rồi tối ưu bằng `optimizer_D.step()`.
  - *[Huấn luyện mạng sinh]:*
    - Tạo ra hình ảnh giả (nhãn 0) bằng mạng sinh, sau đó đưa vào Discriminator.
    - Tính hàm lỗi, rồi tối ưu bằng `optimizer_G.step()`.
6. Sau khi huấn luyện, chúng ta có thể sử dụng mạng sinh để tạo ra các hình ảnh mới. Quá trình này bao gồm:
- Tạo lô 64 véc-tơ nhiễu ngẫu nhiên với `latent_dim=100`.
  - Đưa véc-tơ nhiễu qua mạng sinh để sinh ra hình ảnh giả.
  - Đưa kết quả về kích thước 28x28 để hiển thị.

Mã nguồn tham khảo xem tại <https://gist.github.com/tqlong/b1bb2d4678210f413ef1989a532b10aa>

## 11.7 Tổng kết chương

Chương 11 trình bày cách tiếp cận của mô hình sinh trong việc giải quyết các bài toán ước lượng phân phối và tạo dữ liệu mới theo cách thống kê.

Thuật toán kỳ vọng tối ưu (EM) là công cụ hiệu quả để xử lý dữ liệu có biến ẩn, trong khi các mô hình sinh đối nghịch (GAN) mở ra khả năng sáng tạo mạnh mẽ nhờ cơ chế huấn luyện hai thành phần cạnh tranh – sinh và phân biệt.

Mô hình sinh thể hiện bước chuyển từ nhận thức sang sáng tạo trong Học máy, khi hệ thống không chỉ phân loại hay dự đoán mà còn có thể tạo ra dữ liệu mới có độ chân thực cao.

Chương 12 tiếp theo sẽ mở rộng góc nhìn bằng cách kết nối các mô hình Học máy đã học với các mạng đồ thị xác suất, nhằm giới thiệu một hướng tiếp cận tổng quát và có tính thống kê cao hơn cho việc mô hình hóa và suy luận.

## Bài tập

1. Chứng minh rằng độ hợp lý của  $L(\theta) = p_\theta(X) = \sum_Z p_\theta(X, Z)$  không giảm khi thực hiện thuật toán EM.
2. *[Tìm hiểu]* So sánh giữa ước lượng hợp lý cực đại (MLE) và ước lượng hậu nghiệm cực đại (MAP). Đưa ra một ví dụ cụ thể minh họa ưu điểm của MAP so với MLE khi dữ liệu ít hoặc có nhiễu.
3. *[Tìm hiểu]* Giải thích sự khác biệt giữa biểu diễn xác suất bằng hàm năng lượng và hàm sinh. So sánh cách hàm sinh được sử dụng trong mô hình GAN và cách lấy mẫu từ biểu diễn xác suất bằng hàm năng lượng để tạo dữ liệu mới.
4. *[Lập trình]* Triển khai thuật toán EM cho Gaussian Mixture Model (GMM) trên dữ liệu hai chiều. Người đọc có thể chọn 2 chiều trong dữ liệu IRIS.
  - Trực quan hóa quá trình phân cụm và so sánh với K-means.
  - Thử nghiệm với số lượng cụm khác nhau và so sánh kết quả.
5. *[Lập trình]* Nghiên cứu và triển khai biến thể Wasserstein GAN (WGAN). So sánh hiệu suất của WGAN với GAN cơ bản trên tập dữ liệu MNIST bằng cách đánh giá chất lượng ảnh sinh ra và độ ổn định của quá trình huấn luyện.



---

# Tài liệu tham khảo

---

- [1] Dempster, A. P., Laird, N. M., and Rubin, D. B., *Maximum likelihood from incomplete data via the EM algorithm*, Journal of the Royal Statistical Society: Series B, vol. 39, no. 1, pp. 1–22, 1977.
- [2] Silverman, B. W., *Density estimation for statistics and data analysis*, Chapman and Hall, 1986.
- [3] LeCun, Y., Chopra, S., Hadsell, R., Ranzato, M., and Huang, F. J., *A tutorial on energy-based learning*, Proceedings of the IEEE International Joint Conference on Neural Networks, pp. 1–6, 2006.
- [4] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y., *Generative adversarial nets*, Advances in Neural Information Processing Systems, vol. 27, pp. 2672–2680, 2014.
- [5] Kingma, D. P., and Welling, M., *Auto-encoding variational Bayes*, arXiv preprint arXiv:1312.6114, 2013.
- [6] Bishop, C. M., *Mixture density networks*, Technical Report NCRG/94/004, Aston University, 1994.

- [7] Neal, R. M., *Probabilistic inference using Markov chain Monte Carlo methods*, Technical Report CRG-TR-93-1, University of Toronto, 1993.

# Chương 12

---

## Mô hình đồ thị xác suất

---

Chương 12 trình bày về mô hình đồ thị xác suất, một công cụ mạnh mẽ để biểu diễn tri thức và suy luận trong các tình huống bất định hoặc ngẫu nhiên.

Mô hình đồ thị kết hợp lý thuyết xác suất với cấu trúc đồ thị, cho phép mô hình hóa các mối quan hệ phụ thuộc giữa các biến ngẫu nhiên một cách trực quan và hiệu quả. Từ các mô hình đơn giản như mạng Bayes ngây thơ đến mạng Bayes tổng quát và trường ngẫu nhiên Markov, chương này cho thấy khả năng mô tả cấu trúc thống kê tiềm ẩn trong dữ liệu một cách rõ ràng. Các bài toán thực tế như gán nhãn từ loại hay nhận dạng chuỗi hình ảnh vai trò của mô hình đồ thị trong việc khai thác phụ thuộc ngữ cảnh và mối quan hệ giữa các quan sát lân cận.

Nội dung chương giúp hoàn thiện bức tranh tổng thể về các mô hình Học máy, từ phân loại, hồi quy, học sâu đến biểu diễn xác suất, hướng tới những hệ thống thông minh có khả năng suy luận và ra quyết định.

## 12.1 Giới thiệu

Trong học máy và thống kê, *mô hình đồ thị xác suất* là một công cụ mạnh mẽ dùng để biểu diễn và xử lý các mối quan hệ phụ thuộc xác suất giữa các biến ngẫu nhiên. Mô hình này kết hợp lý thuyết đồ thị và lý thuyết xác suất, cho phép biểu diễn cấu trúc phụ thuộc phức tạp giữa các biến trong hệ thống. Trong đó, các đỉnh trong đồ thị biểu diễn các biến ngẫu nhiên và các cạnh biểu diễn quan hệ phụ thuộc (có điều kiện hoặc không có điều kiện) giữa các biến.

Hai loại mô hình đồ thị xác suất chính là *Mạng Bayes* sử dụng đồ thị có hướng và *Trường ngẫu nhiên Markov*. Mạng Bayes thường được sử dụng để mô hình hóa các mối quan hệ nhân quả, trong khi trường ngẫu nhiên Markov thích hợp hơn cho các bài toán mà các biến có quan hệ phụ thuộc lẫn nhau.

Mô hình đồ thị xác suất có nhiều ứng dụng trong các lĩnh vực như xử lý ngôn ngữ tự nhiên, thị giác máy tính, phân tích dữ liệu y khoa và hệ thống khuyến nghị. Chẳng hạn, trong nhận dạng đối tượng, trường ngẫu nhiên Markov có thể được sử dụng để mô hình hóa sự phụ thuộc không gian giữa các điểm ảnh trong hình ảnh. Trong khi đó, mạng Bayes thường được dùng để mô hình hóa mối quan hệ nguyên nhân-kết quả trong các hệ thống chẩn đoán hoặc ra quyết định.

Việc học và suy luận trên mô hình đồ thị xác suất bao gồm hai nhiệm vụ chính: ước lượng tham số - tìm xác suất của các biến và suy luận - tính xác suất hậu nghiệm cho các biến ẩn dựa trên dữ liệu quan sát. Các thuật toán phổ biến trong suy luận trên đồ thị bao gồm thuật toán lan truyền niềm tin, loại bỏ biến (variable elimination) và các phương pháp gần đúng như lấy mẫu ngẫu nhiên Monte Carlo theo chuỗi Markov.

Chương này sẽ đi sâu vào các khái niệm và kỹ thuật liên quan đến mô hình đồ thị xác suất, từ biểu diễn lý thuyết đến các thuật



toán học và suy luận. Đồng thời, chương cũng sẽ giới thiệu một số ứng dụng thực tế để minh họa sức mạnh của mô hình này trong việc giải quyết các bài toán phức tạp.

## 12.2 Mô hình Bayes ngây thơ

Mô hình Bayes ngây thơ (NB - Naive Bayes) là mô hình Học máy tuân theo hướng tiếp cận mô hình sinh. Mô hình NB tìm cách mô tả xác suất  $p(x, y)$  dưới một giả thuyết độc lập xác suất. Ở đây, giả sử dữ liệu vào  $x = (x^1, \dots, x^m)$  gồm  $m$  thuộc tính. Theo định lý Bayes, ta có

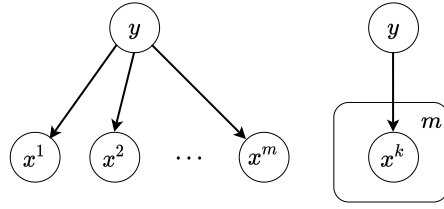
$$p(x, y) = p(x|y)P(y), \forall x, y.$$

Để biểu diễn  $P(y)$ ,  $y = 1, \dots, C$ , ta cần  $C$  tham số (thật ra là  $C - 1$  tham số độc lập do tổng của chúng bằng 1). Để biểu diễn  $p(x|y)$  thì với mỗi giá trị của  $y$ , ta phải mô tả xác suất  $p(x|y)$  với mọi  $x \in \mathcal{X}$ . Nếu không có gì đặc biệt, trong trường hợp đơn giản nhất mỗi thuộc tính  $x^k$  nhận 2 giá trị thì số lượng tham số cần thiết để mô tả  $p(x|y)$  sẽ là  $2^m - 1$  tham số độc lập. Như vậy, nếu sử dụng mô hình xác suất tổng quát, chúng ta không thể lưu trữ, tính toán gì khi số thuộc tính lớn (hàng tỉ tham số khi  $m \geq 32$ ).

Vì vậy, theo hướng tiếp cận mô hình sinh, chắc chắn chúng ta phải giả sử một đặc tính nào đó của xác suất  $p(x|y)$  để có thể làm việc, cài đặt thuật toán trên thực tế. Với mô hình NB, chúng ta sử dụng giả thuyết độc lập xác suất có điều kiện theo công thức (12.1).

$$p(x|y) = p(x^1, \dots, x^m|y) = \prod_{k=1}^m p(x^k|y), \forall x, y. \quad (12.1)$$

Trong đó, ta nói, các thuộc tính  $x^k$  là độc lập có điều kiện với nhau khi biết nhãn  $y$  (nói một cách dễ hiểu, các thuộc tính  $x^k$  chỉ phụ thuộc vào nhãn  $y$ ). Mối quan hệ này được mô tả trong



**Hình 12.1:** Mô hình Bayes ngây thơ.

hình 12.1. Giả thuyết này có thể không đúng trong thực tế, nhưng nó giúp đơn giản hóa rất nhiều các phép toán xác suất. Công thức (12.1) làm giảm số lượng tham số dùng để mô tả xác suất  $p(x|y)$  đáng kể do xác suất này bị tách thành  $m$  phân bố độc lập  $p(x^k|y)$ ,  $k = 1, \dots, m$ . Đặc biệt, các phân bố độc lập  $p(x^k|y)$  là phân bố trên một biến  $x^k$ , dễ dàng thao tác, ước lượng và cài đặt.

### 12.2.1 Pha huấn luyện

Đầu tiên, để huấn luyện mô hình NB, chúng ta cần ước lượng các xác suất  $p(x^k|y)$  và  $P(y)$  với mọi  $k = 1, \dots, m$  và  $y = 1, \dots, C$  từ dữ liệu  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$ .

- *Ước lượng  $P(y)$* : Chúng ta ước lượng  $P(y)$  bằng tần suất xuất hiện của nhãn  $y$  trong dữ liệu  $D$ .

$$\hat{P}(y) = \frac{c_y}{n}, \quad (12.2)$$

với  $c_y$  là số lần nhãn  $y$  xuất hiện trong  $D$ .

- *Ước lượng  $P(x^k|y)$  khi  $x^k \in A^k$  là thuộc tính rời rạc*: Chúng ta ước lượng  $P(x^k|y)$  bằng tần suất xuất hiện  $x^k$  khi nhãn của dữ liệu bằng  $y$  theo ước lượng tần suất ở (12.3).

$$\hat{P}(x^k = a|y) = \frac{c_{ya}^k}{c_y}, \quad (12.3)$$

trong đó  $c_{ya}^k = |\{(x_i, y_i) \in D : x_i^k = a, y_i = y\}|$  với  $a \in A^k$  là số mẫu dữ liệu trong  $D$  thuộc phân lớp  $y$  và có thuộc tính thứ  $k$  là  $a$ .

- Ước lượng  $P(x^k|y)$  khi  $x^k \in A^k \subseteq \mathbb{R}$  là thuộc tính liên tục: Thường chúng ta chọn một phân bố quen thuộc để mô tả phân bố có điều kiện của  $x^k$ . Ví dụ nếu giả sử  $P(x^k|y)$  là phân bố chuẩn (Gauss) có kì vọng  $\mu_y^k$  và phương sai  $(\sigma_y^k)^2$  thì có thể ước lượng các tham số này bằng công thức

$$\hat{\mu}_y^k = \frac{1}{c_y} \sum_{\substack{y_i=y \\ (x_i, y_i) \in D}} x_i^k, (\hat{\sigma}_y^k)^2 = \frac{1}{c_y} \sum_{\substack{y_i=y \\ (x_i, y_i) \in D}} (x_i^k - \hat{\mu}_y^k)^2. \quad (12.4)$$

Đây là các ước lượng hợp lý cực đại của các tham số của phân bố thuộc tính thứ  $k$  cho các mẫu dữ liệu thuộc phân lớp  $y$ .

### 12.2.2 Pha suy luận

Sau khi huấn luyện, có thể dùng mô hình NB để dự đoán phân lớp của một mẫu dữ liệu  $x \in \mathcal{X}$  bất kì. Cụ thể, ta cần tìm nhãn  $y^*$  cực đại hóa xác suất hậu nghiệm  $\hat{P}(y|x)$ .

$$\begin{aligned} y^* &= \arg \max_y \hat{P}(y|x) = \arg \max_y \hat{P}(y|x)p(x) = \arg \max_y \hat{P}(x, y) \\ &= \arg \max_y \hat{P}(y)\hat{P}(x|y) = \arg \max_y \hat{P}(y) \prod_{k=1}^m \hat{P}(x^k|y) \\ &= \arg \max_y \left[ \log \hat{P}(y) + \sum_{k=1}^m \log \hat{P}(x^k|y) \right] \end{aligned}$$

Trong đó, ta đã lợi dụng tính đồng biến của hàm log. Như vậy, để suy luận trên mẫu dữ liệu  $x$ , chỉ cần tìm nhãn  $y$  cực đại hóa tổng của các giá trị của hàm log các xác suất đã được tính trong pha huấn luyện. Việc sử dụng hàm log thay vì giá trị trực tiếp khi làm việc với xác suất, giúp chúng ta tránh được hiện tượng tràn số vì kết quả tính toán quá nhỏ.

---

**Thuật toán 12.1** Thuật toán huấn luyện mô hình Bayes ngây thơ

---

1: **procedure** TRAINNB( $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ )

2:   Đếm số lượng mẫu theo lớp:

$$c_y \leftarrow \#\{i : y_i = y\}, \quad \forall y = 1, \dots, C$$

3:   **for all** thuộc tính  $k$  **do**4:     **if**  $A^k$  là rời rạc **then**5:       **for all**  $a \in A^k, y = 1, \dots, C$  **do**

6:         Đếm số mẫu:

$$c_{ya}^k \leftarrow \#\{i : y_i = y \wedge x_i^k = a\}$$

7:       **end for**8:     **else**9:       **for**  $y = 1$  **to**  $C$  **do**

10:         Tính trung bình và phương sai:

$$\hat{\mu}_y^k \leftarrow \text{mean of } x^k \text{ where } y_i = y$$

$$(\hat{\sigma}_y^k)^2 \leftarrow \text{variance of } x^k \text{ where } y_i = y$$

11:       **end for**12:     **end if**13:   **end for**

14:    Tính các phân bố xác suất:

$$\hat{P}(y) = \frac{c_y}{n}$$

$$\hat{P}(x^k = a|y) = \frac{c_{ya}^k}{c_y} \quad (\text{rời rạc})$$

$$\hat{P}(x^k = a|y) = \mathcal{N}(a|\hat{\mu}_y^k, (\hat{\sigma}_y^k)^2) \quad (\text{liên tục})$$

15:    **return** các ước lượng xác suất  $\hat{P}(y), \hat{P}(x^k = a|y)$ 16: **end procedure**

---

---

**Thuật toán 12.2** Thuật toán suy luận bằng mô hình Bayes ngây thơ
 

---

```

1: procedure TESTNB($\mathbf{x} = (x^1, \dots, x^m)$)
2: for $y = 1$ to C do
3: Tính điểm số:

```

$$s_y \leftarrow \log \hat{P}(y) + \sum_{k=1}^m \log \hat{P}(x^k|y)$$

```

4: end for
5: Dự đoán nhãn:

```

$$y^* \leftarrow \arg \max_y s_y$$

```

6: return y^*
7: end procedure

```

---

**12.2.3 Lưu ý khi ước lượng xác suất**

Nhược điểm của cách ước lượng xác suất trong các công thức (12.2) và (12.3) là chúng đếm số lần xuất hiện của các giá trị  $a \in A^k$  và nhãn  $y$ . Nếu các số đếm này bằng 0 thì chúng ta sẽ nhận được các xác suất bằng 0 trong pha huấn luyện. Đến pha suy luận, xác suất bằng 0 nhân với các xác suất khác trong công thức tính  $P(x, y)$  cũng sẽ cho kết quả bằng 0. Như vậy, các mẫu dữ liệu  $x$  có giá trị thuộc tính không xuất hiện trong tập huấn luyện  $D$  sẽ được ước lượng có xác suất bằng 0 (hiện tượng học quá).

Để khắc phục hiện tượng này, sử dụng phương pháp làm trơn của Laplace, chúng ta có thể thêm các số đếm giả vào cả tử số và mẫu số của các công thức (12.2) và (12.3). Cụ thể, chúng ta thêm một hằng số  $C$  vào số đếm của cả tử số và mẫu số như công thức (12.5)

$$\hat{P}(y) = \frac{c_y + 1}{n + C}, \hat{P}(x^k = a|y) = \frac{c_{ya}^k + 1}{c_y + |A^k|} \quad (12.5)$$

Công thức (12.5) làm tròn các công thức (12.2) và (12.3), đảm bảo ước lượng  $\hat{P}(y)$  và  $\hat{P}(x^k|y)$  luôn dương sau khi huấn luyện.

## 12.3 Mạng Bayes

Như chúng ta đã thấy ở mô hình Bayes ngây thơ, với giả thuyết độc lập xác suất  $p(x|y) = \prod_{k=1}^m p(x^k|y)$  chúng ta đã làm giảm đáng kể khả năng biểu diễn mô hình. Mô hình Bayes ngây thơ là một trường hợp con của *Mạng Bayes* (Bayesian network).

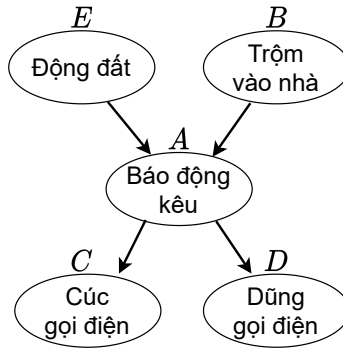
Mạng Bayes là mô hình đồ thị xác suất được biểu diễn bằng một đồ thị có hướng không có chu trình (DAG - directed acyclic graph). Đồ thị DAG thể hiện các mối quan hệ độc lập xác suất cục bộ cũng như toàn cục. Mỗi biến  $x^k$  được biểu diễn bằng một nút (đỉnh) của đồ thị. Gọi  $\text{pa}_k$  là tập các nút cha của  $x^k$  (nút có cạnh tới  $x^k$ ). Theo ngôn ngữ xác suất, ta có định nghĩa mạng Bayes như trong Định nghĩa 12.1.

**Định nghĩa 12.1** (Mạng Bayes). Mạng Bayes được biểu diễn bằng đồ thị DAG  $G = \langle V, E \rangle$  thể hiện phân bố xác suất có khai triển xác suất liên hợp như sau

$$p(x^1, x^2, \dots, x^m) = \prod_{k=1}^m p(x^k | \text{pa}_k)$$

Một số khái niệm thường được sử dụng trong mạng Bayes:

- $\text{pa}_k$ : tập các nút cha của thuộc tính  $x^k$ .
- $\text{an}(X)$ : tập các nút trong  $X$  và tổ tiên của chúng.
- Nếu  $X = \text{an}(X)$  ta nói  $X$  là một tập nút gốc.
- Nút  $x$  không có nút con gọi là nút lá.

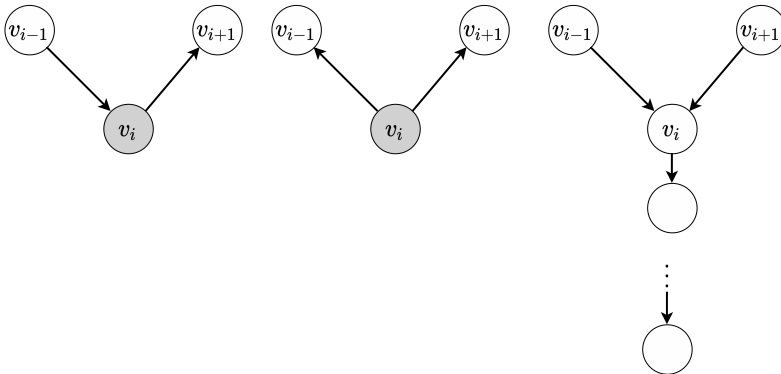


$$P(A, B, C, D, E) = P(E)P(B)P(A|B, E)P(C|A)P(D|A)$$

**Hình 12.2:** Ví dụ minh họa về mạng Bayes mô tả quan hệ giữa các thuộc tính.

### 12.3.1 D-chia cắt

Để nói về quan hệ độc lập xác suất toàn cục, trước tiên ta nói về tính chất D-chia cắt (D-separation) của hai tập thuộc tính trong  $V$ . Hình 12.3 mô tả ba tình huống D-chia cắt: hình trái - đầu nối với đuôi, hình giữa - đuôi nối với đuôi, hình phải - đầu nối với đầu. Các thuộc tính tô màu là thuộc tính trong tập  $Z$ .



**Hình 12.3:** Các tình huống D-chia cắt.

**Định nghĩa 12.2** (D-chia cắt). Tập thuộc tính  $Z$  D-chia cắt hai tập thuộc tính  $X$  và  $Y$  nếu mọi đường đi  $p = v_1, v_2, \dots, v_\ell$  từ  $X$  đến  $Y$  ( $v_1 \in X, v_\ell \in Y$ ) có một trong các tính chất sau (Hình 12.3):

1. Có một đỉnh  $v_i \in Z$  và  $(v_{i-1}, v_i) \in E, (v_i, v_{i+1}) \in E$  (đầu-nối-với-đuôi).
2. Có một đỉnh  $v_i \in Z$  và  $(v_i, v_{i-1}) \in E, (v_i, v_{i+1}) \in E$  (đuôi nối với đầu).
3. Nếu  $(v_{i-1}, v_i) \in E, (v_{i+1}, v_i) \in E$  (đầu-nối-với-đầu) thì  $v_i \notin Z$  và tất cả con cháu của  $v_i$  không thuộc  $Z$ .

**Bổ đề 12.3.** Cho  $y$  là một nút lá trong mạng Bayes  $\mathcal{N}$ , nếu loại bỏ nút  $y$  khỏi  $\mathcal{N}$  được mạng mới  $\mathcal{N}'$  có tập đỉnh là  $X$  thì

$$P_{\mathcal{N}}(X) = P_{\mathcal{N}'}(X).$$

*Chứng minh:*

$$\begin{aligned} P_{\mathcal{N}}(X) &= \sum_y \prod_{x \in X \cup \{y\}} p(x|\text{pa}_x) \\ &= \sum_y \prod_{x \in X} p(x|\text{pa}_x) p(y|\text{pa}_y) \end{aligned}$$

do  $y$  là nút lá nên  $y \notin \text{pa}_x, \forall x$

$$= \underbrace{\prod_{x \in X} p(x|\text{pa}_x)}_{P_{\mathcal{N}'}(X)} \underbrace{\sum_y p(y|\text{pa}_y)}_1$$

□

**Bổ đề 12.4.** Cho  $X$  là một tập nút gốc, nếu loại bỏ tất cả các nút khác khỏi  $\mathcal{N}$  được mạng mới  $\mathcal{N}'$  (cũng có tập đỉnh là  $X$ ) thì

$$P_{\mathcal{N}}(X) = P_{\mathcal{N}'}(X).$$

*Chứng minh:* Sử dụng bổ đề 12.3 lần lượt với từng nút lá không thuộc  $X$  ta được điều phải chứng minh. □



**Bổ đề 12.5.** Nếu  $P(X, Y, Z) = g(X, Z)h(Y, Z)$  thì  $X$  độc lập với  $Y$  khi biết  $Z$ , tức là  $P(X, Y|Z) = P(X|Z)P(Y|Z)$ .

**Bổ đề 12.6.** Nếu  $X, Y, Z$  phân hoạch  $V$  và  $Z$  D-chia cắt  $X$  và  $Y$  thì  $X$  độc lập với  $Y$  khi biết  $Z$ . Khi đó ta viết  $X \perp Y|Z$ .

*Chứng minh:* Vì  $Z$  D-chia cắt  $X$  và  $Y$  nên

- $\forall x \in X, \text{pa}_x \cap Y = \emptyset$  hay  $\text{pa}_x \subset X \cup Z$
- $\forall y \in Y, \text{pa}_y \cap X = \emptyset$  hay  $\text{pa}_y \subset Y \cup Z$

Nếu tách  $Z$  thành hai tập  $Z_1$  và  $Z_2$  sao cho

- $Z_1 = \{z_1 \in Z : \text{pa}_{z_1} \cap X \neq \emptyset\}$
- $Z_2 = Z \setminus Z_1$ .

Do  $Z$  D-chia cắt  $X$  và  $Y$  nên  $\forall z_1 \in Z_1, \text{pa}_{z_1} \cap Y = \emptyset$  (nếu không sẽ tạo thành tình huống đầu-nối-với-đầu). Và theo cách xây dựng  $Z_2$  ta cũng có  $\forall z_2 \in Z_2, \text{pa}_{z_2} \cap X = \emptyset$ .

Vậy ta có

$$\begin{aligned}
 P(V) &= P(X, Z_1, Z_2, Y) \\
 &= \prod_{v \in X \cup Z_1 \cup Z_2 \cup Y} P(v|\text{pa}_v) \\
 &= \underbrace{\prod_{v \in X \cup Z_1} P(v|\text{pa}_v)}_{g(X, Z)} \underbrace{\prod_{v \in Z_2 \cup Y} P(v|\text{pa}_v)}_{h(Y, Z)} \\
 &= g(X, Z)h(Y, Z)
 \end{aligned}$$

Như vậy  $P(X, Y, Z)$  được phân tích thành hai nhân tử  $g(X, Z)$  và  $h(Y, Z)$ , suy ra  $X \perp Y|Z$  theo Bổ đề 12.5.  $\square$

### 12.3.2 Quan hệ độc lập xác suất có điều kiện

**Định lý 12.7** (Độc lập xác suất toàn cục). *Nếu  $Z$  D-chia cắt  $X$  và  $Y$  thì  $X$  và  $Y$  độc lập xác suất khi biết giá trị của  $Z$ .*

$$p(X, Y|Z) = p(X|Z)p(Y|Z) \text{ hay } X \perp Y|Z$$

*Chứng minh:* Theo bổ đề 12.4, ta chỉ cần xét trường hợp

$$\text{an}(X, Y, Z) = V.$$

Gọi  $U$  là tập các đỉnh không bị D-chia cắt với  $X$  bởi  $Z$ . Ta có  $X \subset U$ .

Gọi  $W = V \setminus U \setminus Z$  là tập các đỉnh còn lại ngoài  $U \cup Z$  hay  $W$  là tập các đỉnh bị chia cắt với  $X$  bởi  $Z$ . Ta có  $Y \subset W$ . Đặt  $X' = U \setminus X$  và  $Y' = W \setminus Y$ . Do bổ đề 12.6, ta có  $U \perp W|Z$  hay

$$\begin{aligned} P(U, Z, W) &= g(U, Z)h(Z, W) = g(X, X', Z)h(Z, Y, Y') \\ P(X, Z, Y) &= \sum_{x \in X', y \in Y'} g(X, X', Z)h(Z, Y, Y') \\ &= \sum_{x \in X'} g(X, X', Z) \sum_{y \in Y'} h(Z, Y, Y') \\ &= g'(X, Z)h'(Z, Y). \end{aligned}$$

Suy ra  $X \perp Y|Z$ . □

Từ định lý 12.7, ta có thể xác định tất cả các mối quan hệ độc lập xác suất được biểu diễn bởi mạng Bayes bằng cách kiểm tra tính chất D-chia cắt.

**Mệnh đề 12.8.** *Nếu gọi bao trùm Markov của một đỉnh  $x \in V$  là tập các đỉnh thỏa mãn một trong các điều kiện cha của  $x$ , con của  $x$  hoặc cha của các con của  $x$  thì  $x$  độc lập xác suất với tất cả các đỉnh còn lại nếu biết bao trùm Markov của  $x$ .*

**Mệnh đề 12.9.** *Mỗi thuộc tính  $x^k$  độc lập xác suất với tất cả các thuộc tính không phải con cháu của nó nếu biết các thuộc tính  $\text{pa}_k$  (độc lập xác suất cục bộ).*

Pha huấn luyện của mạng Bayes giống như pha huấn luyện của mô hình Bayes ngây thơ. Theo đó, ta cần xác định tất cả các phân bố  $p(x^k|\text{pa}_k)$  trong Định nghĩa 12.1. Lưu ý rằng phân bố này là phân bố của một thuộc tính  $x^k$  nên rất dễ ước lượng (xem Mục 12.2).

## 12.4 Trường ngẫu nhiên Markov

Trường ngẫu nhiên Markov là một dạng mô hình đồ thị xác suất dựa trên đồ thị vô hướng. Trước khi định nghĩa trường ngẫu nhiên Markov, ta cần một số khái niệm trong đồ thị  $G = \langle V, E \rangle$ .

- Đồ thị con đầy đủ (clique) là một đồ thị con  $C$  của  $G$  có cạnh nối tất cả các cặp đỉnh.
- Đồ thị con đầy đủ lớn nhất là đồ thị con đầy đủ mà nếu thêm bất cứ đỉnh nào sẽ không còn là đồ thị con đầy đủ.
- Tập tất cả các đồ thị con đầy đủ lớn nhất đặt tên là  $\mathcal{C}$ .

**Định nghĩa 12.10** (Trường ngẫu nhiên Markov). Trường ngẫu nhiên Markov được biểu diễn bằng đồ thị vô hướng  $G = \langle V, E \rangle$  thể hiện phân bố xác suất có khai triển xác suất liên hợp thành nhân tử

$$p(x^1, x^2, \dots, x^m) = \frac{1}{Z} \prod_{C \in \mathcal{C}} \psi_C(x_C).$$

Trong đó nhân tử  $\psi_C(x_C) \geq 0$  gọi là hàm thế năng của đồ thị con đầy đủ  $C$  còn  $Z$  là hằng số, còn gọi là hàm phân hoạch, để biểu thức vế phải là một xác suất (có tổng xác suất bằng 1).

Nếu đặt  $\psi_C(x_C) = e^{-E_C(x_C)}$ , ta gọi  $E_C(x_C)$  là hàm năng lượng của  $C$ . Khi đó

$$p(x^1, x^2, \dots, x^m) = \frac{1}{Z} \exp \left\{ - \sum_{C \in \mathcal{C}} E_C(x_C) \right\}.$$

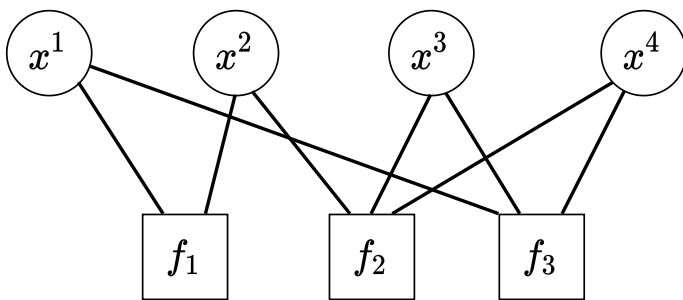
**Định nghĩa 12.11** (Chia cắt). Xét ba tập đỉnh  $X, Y, Z$  rời nhau. Tập đỉnh  $Z$  chia cắt tập đỉnh  $X$  và tập đỉnh  $Y$  nếu mọi đường đi từ  $X$  đến  $Y$  đều phải đi qua một đỉnh trong  $Z$ .

**Định lý 12.12** (Hammersley-Clifford). Nếu các nhân tử  $\psi_C(x_C)$  là các hàm số dương thì  $Z$  chia cắt  $X$  và  $Y$  khi và chỉ khi  $X$  và  $Y$  độc lập xác suất khi biết giá trị của  $Z$ .

### 12.4.1 Đồ thị nhân tử

Cả hai mô hình đồ thị xác suất mạng Bayes và trường ngẫu nhiên Markov đều có thể biểu diễn dưới dạng *đồ thị nhân tử* do xác suất liên hợp  $p(x^1, \dots, x^m)$  đều được phân tích thành nhiều nhân tử, mỗi nhân tử là một hàm trên một tập con các thuộc tính

$$p(x^1, \dots, x^m) = \prod_S f_S(x_S). \quad (12.6)$$



Hình 12.4: Một đồ thị nhân tử có 3 nhân tử.

Trong đó,  $S$  là một tập con các thuộc tính,  $x_S$  là giá trị các thuộc tính trong  $S$ . Khi vẽ đồ thị nhân tử, ta dùng biểu diễn đồ thị hai

phía: một bên là tập thuộc tính, một bên là các nhân tử. Có cạnh nối giữa một nhân tử  $f_S$  và một thuộc tính  $x^k$  nếu thuộc tính đó nằm trong tập đầu vào của nhân tử,  $x^k \in S$  (Hình 12.4) mô tả một đồ thị nhân tử có ba nhân tử  $p(x^1, \dots, x^4) = f_1(x^1, x^2) \times f_2(x^2, x^3, x^4) \times f_3(x^1, x^3, x^4)$ .

## 12.5 Suy luận chính xác trên đồ thị dạng cây

Khi mạng Bayes và trường ngẫu nhiên Markov dựa trên đồ thị là cây (đồ thị không có chu trình), đồ thị nhân tử cũng sẽ là cây. Khi đó tồn tại thuật toán suy luận chính xác rất hiệu quả trên mô hình đồ thị.

Suy luận trên mô hình đồ thị cho phép trả lời các câu hỏi phong phú hơn nhiều các mô hình phân lớp hoặc hồi quy mà chúng ta đã tìm hiểu. Có thể kể đến một số câu hỏi quan trọng của phép suy luận như:

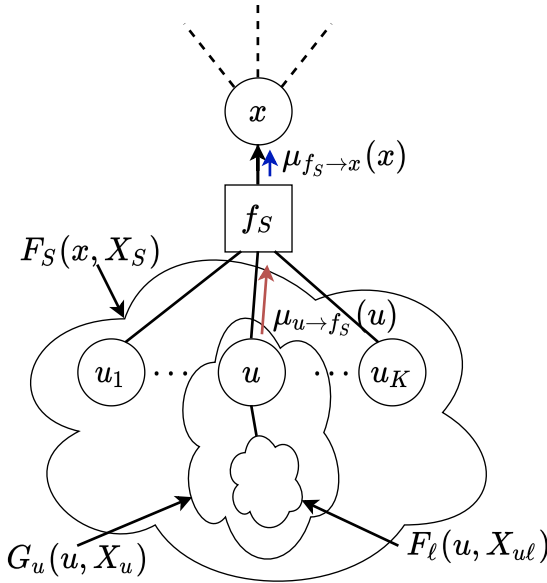
- Tính  $p(y|X)$  trong đó  $y$  là một đỉnh còn  $X$  là tập con các đỉnh mà ta biết giá trị.
- Tính  $y^* = \arg \max_y p(y|X)$ , tìm nhãn  $y$  có xác suất cao nhất khi biết  $X$ . Đây là câu hỏi thường thấy trong các bài toán phân lớp, hồi quy.
- Tính  $p(Y|X)$  trong đó  $X, Y$  là các tập đỉnh rời nhau.
- Tính  $Y^* = \arg \max_Y p(Y|X)$ , tìm tập các nhãn  $Y$  có xác suất cao nhất khi biết  $X$ . Đây là câu hỏi thường thấy trong bài toán tìm kiếm nhãn cho một cấu trúc dữ liệu phức tạp như chuỗi văn bản, âm thanh, hình ảnh.

Thuật toán suy luận trên đồ thị nhân tử dạng cây gồm thuật toán tổng-tích và thuật toán max-tổng.

### 12.5.1 Tính xác suất biên của một đỉnh

Đầu tiên, ta tìm hiểu thuật toán tính xác suất biên của một đỉnh  $p(x)$ . Ta có công thức xác suất liên hợp:

$$p(x) = \sum_{V \setminus \{x\}} p(x_V) = \sum_{V \setminus \{x\}} \prod_S f_S(x_S) \quad (12.7)$$



Hình 12.5: Cây con trong đồ thị nhân tử.

Để tính  $p(x)$ , chúng ta xét các nhân tử  $f_S$  có nối với  $x$  tạo thành một cây con có gốc là  $f_S$  (xem Hình 12.5). Nếu ta nhóm tất cả các nhân tử trong cây con này lại thành một đại lượng  $F_S(x, X_S)$  trong đó  $F_S$  là tích các nhân tử trong cây con,  $X_S$  là tất cả các đỉnh thuộc tính trong cây con đó, Công thức (12.7) trên trở thành

$$p(x) = \sum_{V \setminus \{x\}} \prod_{f_S \in \text{ne}(x)} F_S(x, x_S)$$

Sử dụng tính chất  $X_S$  là các tập rời nhau nên có thể trao đổi tổng và tích, tiếp tục biến đổi ta thu được công thức sau:

$$\begin{aligned} &= \prod_{f_S \in \text{ne}(x)} \left[ \sum_{X_S} F_S(x, x_S) \right] \\ &= \prod_{f_S \in \text{ne}(x)} \mu_{f_S \rightarrow x}(x) \end{aligned} \quad (12.8)$$

với  $\text{ne}(x)$  là tập các nhân tử  $f_S$  có nối với  $x$ . Ta gọi đại lượng

$$\mu_{f_S \rightarrow x}(x) = \sum_{X_S} F_S(x, x_S) \quad (12.9)$$

là thông điệp từ nút nhân tử  $f_S$  tới nút thuộc tính  $x$ .

Để tính được thông điệp trên, nhận thấy rằng, mỗi đỉnh thuộc tính  $u$  nối với  $f_S$  lại tương ứng với một cây con gốc  $u$ . Ta lại nhóm tất cả các nhân tử của từng cây con lại để thu được công thức như sau:

$$F_S(x, X_S) = f_S(x, u_1, \dots, u_k) \prod_{i=1}^k G(u_i, X_{u_i}), \quad (12.10)$$

trong đó,  $x_S = \{x, u_1, \dots, u_k\}$  là các đỉnh thuộc tính của nhân tử  $f_S$ ,  $G(u_i, X_{u_i})$  là tích các nhân tử trong cây con gốc  $u_i$ ,  $X_{u_i}$  là tất cả các đỉnh còn lại của cây con này. Tiếp tục khai triển từ các đỉnh  $u \in \text{ne}(f_S) \setminus \{x\}$ , với  $\text{ne}$  kí hiệu là lân cận của một đỉnh, ta thu được:

$$\begin{aligned} \mu_{f_S \rightarrow x}(x) &= \sum_{u_1, \dots, u_K} f_S(x, u_1, \dots, u_k) \prod_{u \in \text{ne}(f_S) \setminus \{x\}} \left[ \sum_{X_u} G_u(u, X_u) \right] \\ &= \sum_{u_1, \dots, u_K} f_S(x, u_1, \dots, u_k) \prod_{u \in \text{ne}(f_S) \setminus \{x\}} \mu_{u \rightarrow f_S}(u). \end{aligned} \quad (12.11)$$

Ta gọi đại lượng

$$\mu_{u \rightarrow f_S}(u) = \sum_{X_u} G_u(u, X_u)$$

là thông điệp từ nút thuộc tính  $u$  đến nút nhân tử  $f_S$ .

Tiếp tục, khai triển bằng cách sử dụng đẳng thức  $G_u(u, X_u) = \prod_{f_\ell \in \text{ne}(u) \setminus f_S} F_\ell(u, X_{u\ell})$  để thay vào công thức trên ta được

$$\begin{aligned} \mu_{u \rightarrow f_S}(u) &= \prod_{f_\ell \in \text{ne}(u) \setminus f_S} \left[ \sum_{X_{u\ell}} F_\ell(u, X_{u\ell}) \right] \\ &= \prod_{f_\ell \in \text{ne}(u) \setminus f_S} \mu_{f_\ell \rightarrow u}(u) \end{aligned} \quad (12.12)$$

Hai công thức (12.11) và (12.12) được dùng để *lan truyền các thông điệp* trên đồ thị nhân tử giữa các nút thuộc tính và các nút nhân tử của chúng. Riêng các trường hợp ở các nút lá ta xét hai trường hợp sau:

- Nếu  $f_S$  là nút lá thì  $\mu_{f_S \rightarrow x}(x) = f_S(x)$ .
- Nếu  $u$  là nút lá thì  $\mu_{u \rightarrow f_S}(u) = 1$ .

---

**Thuật toán 12.3** Thuật toán tính  $p(x)$  cho mọi thuộc tính  $x$  trên cây

---

- 1: **procedure** TREEINFERENCE( $G$ )
- 2:     Chọn một nút gốc  $r$  và xây dựng cây với  $r$  là gốc
- 3:     **(Truyền đi lên)**: Từ các lá, tính thông điệp  $\mu_{f \rightarrow x}$  truyền dần lên đến  $r$
- 4:     **(Truyền đi xuống)**: Khi  $r$  nhận đủ thông điệp, truyền ngược các thông điệp từ  $r$  về các lá
- 5:     **(Tổng hợp)**: Với mỗi nút  $x$ , tính xác suất:

$$p(x) = \prod_{f_S \in \text{ne}(x)} \mu_{f_S \rightarrow x}(x)$$

- 6:     **return**  $p(x)$  cho mọi  $x$
  - 7: **end procedure**
-



Tính  $p(x_S)$  khi  $x_S$  là các thuộc tính của nhân tử  $f_S$ . Bằng các luận giải như trên ta có

$$p(x_S) = f_S(x_S) \prod_{u \in \text{ne}(f_S)} \mu_{u \rightarrow f_S}(u). \quad (12.13)$$

### 12.5.2 Tính xác suất cực đại

Trong mục này, ta tìm hiểu thuật toán tổng lớn nhất để tính giá trị cực đại của xác suất liên hợp

$$p^{\max} = \max_{x^1, \dots, x^m} p(x^1, \dots, x^m) = \max_{x_V} \prod_S f_S(x_S) \quad (12.14)$$

với  $\max$  là hàm lấy giá trị lớn nhất.

Nếu ta lấy logarit cả hai vế và sử dụng tính chất đồng biến của logarit thì ta thu được phương trình sau

$$\max_{x^1, \dots, x^m} \ln p(x^1, \dots, x^m) = \max_{x_V} \sum_S \ln f_S(x_S). \quad (12.15)$$

Ngoài ra, nhận thấy tính chất phân phối của phép cộng với phép max (giống như tính chất phân phối của phép nhân với phép cộng)

$$\max(a + b, a + c) = a + \max(b, c) \quad (12.16)$$

Nên thuật toán để tính  $\max_{x_V} \sum_S \ln f_S(x_S)$  sẽ giống hệt như thuật toán tính  $\sum_{V \setminus \{x\}} \prod_S f_S(x_S)$ , chỉ cần thay phép nhân bằng phép cộng, phép cộng bằng phép max. Cụ thể ta có biến đổi công thức như sau:

$$\mu_{f_S \rightarrow x}(x) = \max_{u_1, \dots, u_K} \left[ \ln f_S(x, u_1, \dots, u_K) + \sum_{u \in \text{ne}(f_S) \setminus \{x\}} \mu_{u \rightarrow f_S}(u) \right], \quad (12.17)$$

$$\mu_{u \rightarrow f_S}(u) = \sum_{f_\ell \in \text{ne} \setminus f_S} \mu_{f_\ell \rightarrow u}(u) \quad (12.18)$$

nếu  $u$  là nút lá

$$\mu_{u \rightarrow f_S}(u) = 0 \quad (12.19)$$

nếu  $f_S$  là nút lá

$$\mu_{f_S \rightarrow x}(x) = \ln f_S(x) \quad (12.20)$$

xác suất cực đại là

$$p^{\max} = \max_x \left[ \sum_{f_S \in \text{ne}(x)} \mu_{f_S \rightarrow x}(x) \right] \quad (12.21)$$

Các công thức trên được áp dụng để tính  $p^{\max}$  bằng cách lan truyền các thông điệp từ lá đến một nút thuộc tính bất kì được chọn làm gốc. Để lần ngược lại cấu hình các thuộc tính  $x^k$ , ta phải lưu lại bộ giá trị  $u_1, \dots, u_K$  là cực trị ứng với mỗi giá trị của  $x$  trong công thức (12.17).

## 12.6 Mô hình Markov ẩn HMM

Trong các mục trước của chương, chúng ta đã tìm hiểu mô hình đồ thị xác suất tổng quát bao gồm mạng Bayes (đồ thị có hướng) và trường ngẫu nhiên Markov (đồ thị vô hướng). Trong mục này, chúng ta tìm hiểu mô hình Markov ẩn (HMM - Hidden Markov Model) là một mô hình mạng Bayes chuyên dùng để mô hình hoá dữ liệu dạng chuỗi. Mô hình HMM có ứng dụng trong xử lý ngôn ngữ tự nhiên, xử lý tiếng nói, tin sinh học. Đây là những lĩnh vực có dữ liệu dạng chuỗi rất lớn. Mô hình HMM mô tả quá trình sinh ra dữ liệu chuỗi gồm các bước sau:

- Tại thời điểm 0, hệ thống ở một trạng thái  $q_0 \in S$  là tập các trạng thái.

- Hệ thống trải qua các trạng thái theo thời gian, trạng thái  $q_t$  ở thời điểm  $t$  chỉ phụ thuộc vào trạng thái  $q_{t-1}$  ở thời điểm  $t - 1$ .
- Tại mỗi thời điểm, tùy thuộc vào trạng thái hiện tại  $q_t$ , hệ thống sinh ra dữ liệu quan sát được  $o_t \in V$ .

**Ví dụ 12.13** (Mô hình âm thanh).

Mô hình HMM sử dụng một tập rời rạc các trạng thái

$$S = \{s_1, s_2, \dots, s_N\}.$$

Tập  $V$  các dữ liệu có thể quan sát được tùy thuộc vào từng nhiệm vụ.

- $V = \{v_1, v_2, \dots, v_M\}$  rời rạc: dành cho nhiệm vụ xử lý ngôn ngữ (tập từ vựng), tin sinh học (DNA, protein) hoặc khi dữ liệu đã được phân cụm, rời rạc hoá.
- $V = \mathbb{R}^m$  liên tục: dành cho nhiệm vụ có dữ liệu liên tục như mô hình hoá âm thanh (acoustic models).

Để tiện trình bày, chúng ta sẽ bắt đầu với tập  $V$  rời rạc. Mô hình HMM sử dụng các tham số

$\pi_i = \mathbb{P}(q_0 = s_i)$  : xác suất để  $s_i$  là trạng thái đầu tiên

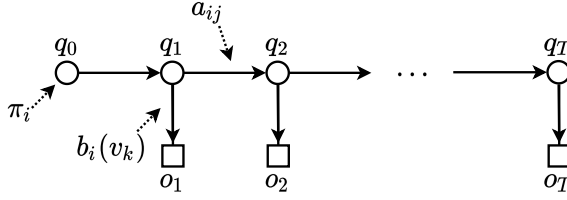
$a_{ij} = \mathbb{P}(q_{t+1} = s_j | q_t = s_i)$  : xác suất chuyển từ  $s_i$  sang  $s_j$

$b_i(v_k) = \mathbb{P}(o_t = v_k | q_t = s_i)$  : xác suất sinh  $v_k$  từ trạng thái  $s_i$

Ta gộp cả ba bộ tham số lại thành một bộ  $\Theta = (A, B, \pi)$ .

Như vậy, quá trình lấy mẫu một chuỗi dữ liệu có  $O = o_1 o_2 \dots o_T$  có độ dài  $T$  gồm các bước sau:

1. Lấy mẫu trạng thái đầu tiên  $q_0$  từ phân bố  $\pi$



**Hình 12.6:** Mạng Bayes cho mô hình HMM.

2. Lần lượt lấy mẫu các trạng thái  $q_1, q_2, \dots, q_T$  từ trạng thái trước đó theo phân bố  $A = [a_{ij}]$
3. Lấy mẫu dữ liệu  $o_t$  từ trạng thái  $q_t$  theo phân bố  $b_i(v_k)$

Biểu diễn mạng Bayes của mô hình HMM sinh dữ liệu dạng chuỗi ở Hình 12.6. Trong đó, các nút tròn thể hiện các trạng thái ẩn (hidden) còn các nút vuông là dữ liệu quan sát được.

Xác suất liên hợp của mô hình HMM gồm nhiều nhân tử tương ứng với các cạnh của đồ thị và trạng thái ban đầu

$$P(Q, O | \Theta) = P(q_0 q_1 \dots q_T, o_1 o_2 \dots o_T) \quad (12.22)$$

$$= \pi_{q_0} a_{q_0 q_1} a_{q_1 q_2} \dots a_{q_{T-1} q_T} b_{q_1}(o_1) b_{q_2}(o_2) \dots b_{q_T}(o_T) \quad (12.23)$$

Với mô hình HMM, có ba bài toán chính:

1. B1 (suy luận): Cho  $\Theta$  và chuỗi dữ liệu  $O$ , hỏi xác suất  $P(O)$  là bao nhiêu?
2. B2 (suy luận): Cho  $\Theta$  và chuỗi dữ liệu  $O$ , hỏi chuỗi trạng thái  $Q = q_1 q_2 \dots q_T$  nào có xác suất cực đại?

$$Q^* = \arg \max_Q P(Q | O).$$

3. B3 (huấn luyện): Cho bộ dữ liệu là tập các chuỗi  $O_1, O_2, \dots$ , hãy tìm bộ tham số  $\Theta$  hợp lý nhất với bộ dữ liệu này.

### 12.6.1 Bài toán suy luận xác suất

Ta sẽ sử dụng thuật toán suy luận trên mạng Bayes lan truyền các thông điệp trên các nút của đồ thị nhân tử tương ứng của mô hình HMM để tính  $P(O|\Theta) = \sum_Q P(Q, O|\Theta)$ .

Nếu chọn cách truyền thông điệp từ trái qua phải, ta có thủ tục lan truyền tới. Sau khi đơn giản hoá các công thức lan truyền thông điệp, chúng ta đặt

$$\alpha_t(i) = \mu_{\rightarrow q_t}(s_i) = \mathbb{P}(q_t = s_i, o_1 o_2 \dots o_t | \Theta)$$

thì ta có công thức truy hồi sau:

$$\begin{aligned} \alpha_0(j) &= \pi_j \\ \alpha_{t+1}(j) &= \left[ \sum_{i=1}^N \alpha_t(i) a_{ij} \right] b_j(o_{t+1}) \end{aligned}$$

Xác suất của chuỗi được tính bởi công thức

$$P(O|\Theta) = \sum_{i=1}^n \alpha_T(i)$$

Nếu chọn cách truyền thông điệp ngược lại từ phải qua trái, ta có thủ tục lan truyền ngược. Tương tự biến đổi ở trên, ta đặt

$$\beta_t(i) = \mu_{q_t \leftarrow}(s_i) = \mathbb{P}(o_{t+1} o_{t+2} \dots o_T | q_t = s_i, \Theta)$$

thì ta thu công thức truy hồi sau

$$\begin{aligned} \beta_T(i) &= 1 \\ \beta_t(i) &= \sum_{j=1}^N \beta_{t+1}(j) a_{ij} b_j(o_{t+1}). \end{aligned}$$

Xác suất của chuỗi được tính bởi công thức

$$P(O|\Theta) = \sum_{i=1}^n \beta_0(i) \pi_i.$$

### 12.6.2 Bài toán suy luận chuỗi trạng thái

Để tìm chuỗi trạng thái có xác suất cực đại khi biết chuỗi dữ liệu  $O$ , ta sử dụng thuật toán max-tổng. Thuật toán này còn có tên là thuật toán Viterbi. Chúng ta đặt

$$\delta_t(i) = \max_{q_0 q_1 \dots q_{t-1}} \ln \mathbb{P}(q_0 q_1 \dots q_{t-1} q_t = s_i, o_1 o_2 \dots o_t | \Theta) \quad (12.24)$$

thì công thức truy hồi như sau

$$\delta_0(i) = \ln \pi_i \quad (12.25)$$

$$\delta_{t+1}(j) = \max_{i=1}^N [\delta_t(i) + \ln a_{ij}] + \ln b_j(o_{t+1}) \quad (12.26)$$

$$\psi_{t+1}(j) = \arg \max_{i=1}^N [\delta_t(i) + \ln a_{ij}] \quad (12.27)$$

$$\ln P^* = \max_{i=1}^N \delta_T(i) \quad (12.28)$$

$$q_T^* = \arg \max_{i=1}^N \delta_T(i) \quad (12.29)$$

$$q_t^* = \psi_{t+1}(q_{t+1}^*) \quad (12.30)$$

Trong đó,  $\delta$  là các thông điệp trong thuật toán max-tổng còn  $\psi$  lưu trữ chỉ số trạng thái làm biểu thức (12.26) đạt cực đại để lần ngược bằng công thức (12.30) khi thuật toán kết thúc.

### 12.6.3 Bài toán huấn luyện

Mô hình HMM được huấn luyện bằng thuật toán Baum-Welch là một thuật toán cực đại hoá kì vọng EM. Thuật toán EM dành cho đồ thị nhân tử dạng cây yêu cầu tính toán xác suất của  $x_S$  cho từng nhân tử  $f_S(x_S)$  (công thức (12.13)). Đối với mô hình HMM, các nhân tử chính là các cạnh của đồ thị và trạng thái bắt đầu. Cụ thể ta cần tính

$$\begin{aligned}
\xi_t(i, j) &= \mathbb{P}(q_t = s_i, q_{t+1} = s_j | O, \Theta) \\
&= \frac{\mathbb{P}(q_t = s_i, q_{t+1} = s_j, O | \Theta)}{\mathbb{P}(O | \Theta)} \\
&= \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{\mathbb{P}(O | \Theta)} \tag{12.31}
\end{aligned}$$

$$\begin{aligned}
\gamma_t(i) &= \mathbb{P}(q_t = s_i | O, \Theta) \\
&= \sum_{j=1}^N \xi_t(i, j) \tag{12.32}
\end{aligned}$$

Đại lượng  $\xi_t(i, j)$  là xác suất để ở hai thời điểm  $t, t + 1$ , hệ thống lần lượt ở các trạng thái  $s_i$  và  $s_j$ . Đại lượng  $\gamma_t(i)$  là xác suất để hệ thống ở trạng thái  $s_i$  tại thời điểm  $t$ .

Việc ước lượng lại bộ tham số được thực hiện như sau

$$\pi_i \leftarrow \gamma_0(i) \tag{12.33}$$

$$a_{ij} \leftarrow \frac{\sum_{t=0}^{T-1} \xi_t(i, j)}{\sum_{t=0}^{T-1} \gamma_t(i)} \tag{12.34}$$

$$b_i(v_k) \leftarrow \frac{\sum_{t=1, o_t=v_k}^T \gamma_t(i)}{\sum_{t=1}^T \gamma_t(i)} \tag{12.35}$$

Thuật toán 12.4 cho phép huấn luyện mô hình HMM từ một chuỗi dữ liệu  $O$ . Nếu tập dữ liệu huấn luyện có nhiều xâu  $O^{(1)}, O^{(2)}, \dots, O^{(n)}$ , ta chỉ cần sửa các công thức ước lượng tham số như sau:

$$\pi_i \leftarrow \frac{1}{n} \sum_{\ell=1}^n \gamma_0^{(\ell)}(i) \tag{12.36}$$

$$a_{ij} \leftarrow \frac{\sum_{\ell=1}^n \sum_{t=0}^{T^{(\ell)}-1} \xi_t^{(\ell)}(i, j)}{\sum_{\ell=1}^n \sum_{t=0}^{T^{(\ell)}-1} \gamma_t^{(\ell)}(i)} \tag{12.37}$$

$$b_i(v_k) \leftarrow \frac{\sum_{\ell=1}^n \sum_{t=1, o_t^{(\ell)}=v_k}^{T^{(\ell)}} \gamma_t^{(\ell)}(i)}{\sum_{\ell=1}^n \sum_{t=1}^{T^{(\ell)}} \gamma_t^{(\ell)}(i)} \tag{12.38}$$

---

**Thuật toán 12.4** Thuật toán Baum-Welch huấn luyện mô hình HMM

---

- 1: **procedure** TRAINHMM( $O$ )  $\triangleright O$ : chuỗi quan sát
- 2:    Khởi tạo ngẫu nhiên các tham số  $\Theta = (\pi, A, B)$
- 3:    **while** chưa hội tụ **do**
- 4:     Tính  $\alpha_t(i)$  và  $\beta_t(i)$  cho mọi  $t, i$
- 5:     Tính  $P(O|\Theta)$  từ  $\alpha$
- 6:     Tính  $\xi_t(i, j)$  và  $\gamma_t(i)$  bằng:

$$\xi_t(i, j) = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{P(O|\Theta)}$$

$$\gamma_t(i) = \sum_j \xi_t(i, j)$$

- 7:     Cập nhật:

$$\begin{aligned}\pi_i &\leftarrow \gamma_1(i) \\ a_{ij} &\leftarrow \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)} \\ b_j(k) &\leftarrow \frac{\sum_{t: o_t=v_k} \gamma_t(j)}{\sum_{t=1}^T \gamma_t(j)}\end{aligned}$$

- 8:    **end while**
  - 9:    **return**  $\Theta = (\pi, A, B)$
  - 10: **end procedure**
-



Trong đó,  $\ell$  là chỉ số của chuỗi dữ liệu  $O^{(\ell)}$  và các đại lượng  $\xi_t^{(\ell)}, \gamma_t^{(\ell)}$  là  $\xi, \gamma$  tính cho chuỗi dữ liệu này.

## 12.7 Trường ngẫu nhiên có điều kiện

Trường ngẫu nhiên có điều kiện (CRF - Conditional Random Field) là một trường ngẫu nhiên Markov dựa trên đồ thị vô hướng. Mô hình CRF mô tả xác suất hậu nghiệm  $p(y|x)$ , trong đó,  $y = y_1 y_2 \dots y_T$  là các nhãn cần dự đoán,  $x$  là tất cả các đặc trưng đầu vào. Trường hợp dự đoán trên một chuỗi, có thể  $x$  được chia thành  $x_1 x_2 \dots x_T$  là đặc trưng của các vị trí trong chuỗi. Lý do mô hình CRF không mô tả xác suất liên hợp  $p(x, y)$  là do  $x$  có thể có tới hàng chục nghìn đặc trưng. Trong khi đó, chỉ cần  $p(y|x)$  là đủ để giải bài toán phân lớp  $y^* = \arg \max_y p(y|x)$ .

**Định nghĩa 12.14** (Trường ngẫu nhiên có điều kiện). Xét một đồ thị nhân tử  $G$  trên cả  $x$  và  $y$ . Nếu với mọi  $x$  ta có  $p(y|x)$  phân tích thành nhân tử dựa trên các nút nhân tử trong  $G$ , ta nói  $x, y$  là một trường ngẫu nhiên có điều kiện.

Gọi  $\{\psi_a\}_{a=1}^A$  là tập các nhân tử trong  $G$ . Ta có công thức phân tích

$$p(y|x) = \frac{1}{Z(x)} \prod_{a=1}^A \psi_a(y_a, x_a), \quad (12.39)$$

trong đó,  $x_a, y_a$  là các thuộc tính đầu vào của nhân tử  $\psi_a$ . Trong công thức trên, ta coi  $x_a$  là các hằng số chứ không phải biến ngẫu nhiên.

Để bài toán huấn luyện và suy luận có thể giải dễ dàng hơn, người ta thường chọn các nhân tử có logarit là một tổ hợp tuyến

tính của các hàm đặc trưng sau:

$$\psi_a(y_a, x_a) = \exp \left\{ \sum_{k=1}^{K_a} \theta_{ak} f_{ak}(y_a, x_a) \right\}. \quad (12.40)$$

Công thức (12.40) mỗi nhân tử  $\psi_a$  được tính nhờ  $K_a$  đặc trưng. Các trọng số  $\theta_{ak}$  thể hiện sự quan trọng của từng đặc trưng. Thường trong đồ thị  $G$  có sự lặp đi lặp lại của các cấu trúc nhân tử thì các cấu trúc này sẽ sử dụng các trọng số và hàm đặc trưng giống nhau. Ví dụ, trong một cấu trúc chuỗi thì nhân tử  $\psi(y_t, y_{t-1}, x_t)$  sẽ giống nhân tử  $\psi(y_{t+1}, y_t, x_{t+1})$ , sử dụng chung trọng số  $\theta$  và tập hàm đặc trưng.

**Ví dụ 12.15** (Mô hình HMM là một CRF).

Kết hợp các công thức trên ta được

$$\begin{aligned} p(y|x) &= \frac{1}{Z(x)} \exp \left\{ \sum_{a=1}^A \sum_{k=1}^{K_a} \theta_{ak} f_{ak}(y_a, x_a) \right\} \\ &= \frac{1}{Z(\Theta)} \exp \{ \Theta^T F(y, x) \} \end{aligned}$$

với  $\Theta = \{\theta_{ak}\}$  là tập tất cả các trọng số và  $F(y, x) = \{f_{ak}(y_a, x_a)\}$  là tập tất cả các hàm đặc trưng.

### 12.7.1 Pha huấn luyện

Giả sử ta có tập huấn luyện  $D = \{(\mathbf{x}_i, y_i)\}, i = 1, 2, \dots, n$ . Theo nguyên lý ước lượng hợp lý cực đại, ta sẽ tìm tham số  $\Theta^*$  cực đại hoá xác suất của dữ liệu tức là ta cần tìm  $\Theta^*$  theo công thức sau:

$$\Theta^* = \arg \max_{\Theta} \ell(\Theta) = \frac{1}{n} \sum_{i=1}^n \ln p(y_i | x_i) \quad (12.41)$$

Cũng giống như thuật toán huấn luyện mô hình hồi quy Logistic, xuất phát từ một nghiệm  $\Theta$ , ta cập nhật nghiệm này theo hướng của đạo hàm

$$\begin{aligned}\frac{\partial \ell}{\partial \Theta} &= \frac{\partial (-\ln Z(\Theta) + \Theta^T [\frac{1}{n} \sum_{i=1}^n F(y_i, x_i)])}{\partial \Theta} \\ &= \frac{1}{n} \sum_{i=1}^n F(y_i, x_i) - \frac{\partial \ln Z(\Theta)}{\partial \Theta}\end{aligned}$$

Ta có khai triển của đạo hàm của hàm phân phối  $Z(\Theta)$  như sau:

$$\begin{aligned}Z(\Theta) &= \frac{1}{Z(\Theta)} \sum_y \exp \{ \Theta^T F(y, x) \} \\ \frac{\partial \ln Z(\Theta)}{\partial \Theta} &= \frac{1}{Z(\Theta)} \sum_y \exp \{ \Theta^T F(y, x) \} F(y, x) \\ &= \sum_y F(y, x) p(y|x) \\ &= \mathbb{E}_y[F(y, x)]\end{aligned}$$

Trong đó kì vọng được tính từ phân bố  $p(y|x)$  dựa trên bộ tham số  $\Theta$ . Như vậy, bài toán huấn luyện được quy về bài toán suy luận, ta phải tính được kỳ vọng của các hàm đặc trưng trong véc-tơ

$$\mathbb{E}_y[F(y, x)] = (\mathbb{E}_{y_a}[f_{ak}(y_a, x_a)]), a = 1 \dots A, k = 1 \dots K_a.$$

Ta có công thức tính

$$\mathbb{E}_{y_a}[f_{ak}(y_a, x_a)] = \sum_{y_a} f_{ak}(y_a, x_a) p(y_a|x). \quad (12.42)$$

Trong đó,  $p(y_a|x)$  được tính từ công thức (12.13) vì  $p(y|x)$  có khai triển theo đồ thị nhân tử như trong công thức (12.39).

### 12.7.2 Pha suy luận

Bước (b) và (c) trong Thuật toán (12.5) chính là một pha suy luận của mô hình CRF. Tổng quát hơn, các suy luận trên mạng CRF bao gồm một số tác vụ cơ bản sau:

---

**Thuật toán 12.5** Thuật toán huấn luyện mô hình CRF cho đồ thị nhân tử dạng cây

---

- 1: **procedure** TRAINCRF( $D, \lambda$ )
- 2:     Khởi tạo ngẫu nhiên các tham số  $\Theta$
- 3:     **while** chưa hội tụ **do**      $\triangleright$  Thực hiện cho mỗi mẫu  $(x, y)$  trong  $D$
- 4:         **for all**  $(x, y) \in D$  **do**
- 5:             Chọn một nút gốc trong đồ thị nhân tử
- 6:             Thực hiện lan truyền thông điệp hai chiều trên cây:

$$\mu_{\psi_a \rightarrow x}(x), \quad \mu_{u \rightarrow \psi_a}(u)$$

- 7:             Tính xác suất biên gần đúng:

$$\hat{p}(y_a|x) = \frac{1}{Z(x)} \exp \left\{ \sum_{k=1}^{K_a} \theta_{ak} f_{ak}(y_a, x_a) \right\} \prod_{u \in \text{ne}(\psi_a)} \mu_{u \rightarrow \psi_a}(u)$$

- 8:             Tính kỳ vọng đặc trưng:

$$\mathbb{E}_{y_a}[f_{ak}(y_a, x_a)] = \frac{\sum_{y_a} f_{ak}(y_a, x_a) \hat{p}(y_a|x)}{\sum_{y_a} \hat{p}(y_a|x)}$$

- 9:             Cập nhật tham số:

$$\theta_{ak} \leftarrow (1 - \lambda)\theta_{ak} + \lambda \mathbb{E}_{y_a}[f_{ak}(y_a, x_a)]$$

- 10:             **end for**
  - 11:             **end while**
  - 12:             **return**  $\Theta$
  - 13: **end procedure**
-

- Tính xác suất  $p(y_a|x)$  cho từng nhân tử.
- Tính kì vọng  $\mathbb{E}_{y_a}[f_a(y_a, x_a)]$  cho từng nhân tử.
- Tìm  $\arg \max_{y_a} p(y_a|x)$  cho từng nhân tử
- Tìm kiếm toàn bộ bộ nhãn  $y^* = \arg \max_y p(y|x)$ .

Hai bài toán đầu đơn giản, chúng ta đã thấy ở bước (b) và (c) của Thuật toán (12.5). Hai bài toán cuối là bài toán tối ưu tổ hợp NP-khó, tức là không thể giải một cách hiệu quả trong thời gian chấp nhận được với mọi trường hợp. Thay vào đó, người ta dùng các thuật toán tìm kiếm xấp xỉ để tìm một nghiệm chấp nhận được. Cụ thể chúng ta cần tìm giá trị  $y^*$  sao cho

$$y^* = \arg \max_y p(y|x) = \arg \max_y \Theta^T F(y, x). \quad (12.43)$$

Riêng đối với một số cấu trúc mạng, ta có thuật toán tìm kiếm hiệu quả

- Đồ thị nhân tử dạng cây: sử dụng thuật toán Viterbi (giống mô hình HMM). Chọn một nút  $y^*$  làm gốc, ta sử dụng các công thức (12.17) và (12.18), áp dụng vào mạng CRF được

$$\mu_{\psi_a \rightarrow y^*}(y^*) = \max_{u \in y_a \setminus \{y^*\}} \left[ \sum_{k=1}^{K_a} \theta_{ak} f_{ak}(y_a, x_a) + \sum_{u \in y_a \setminus \{y^*\}} \mu_{u \rightarrow \psi_a}(u) \right], \quad (12.44)$$

$$\mu_{u \rightarrow \psi_a}(u) = \sum_{\psi_\ell \in \text{ne}(u) \setminus \psi_a} \mu_{\psi_\ell \rightarrow u}(u) \quad (12.45)$$

nếu  $u$  là nút lá

$$\mu_{u \rightarrow \psi_a}(u) = 0 \quad (12.46)$$

nếu  $\psi_a$  là nút lá

$$\mu_{\psi_a \rightarrow y^*}(y^*) = \sum_{k=1}^{K_a} \theta_{ak} f_{ak}(y^*, x^*) \quad (12.47)$$

xác suất cực đại là

$$p^{\max} = \max_{y^*} \left[ \sum_{\psi_a \in \text{ne}(y^*)} \mu_{\psi_a \rightarrow y^*}(y^*) \right] \quad (12.48)$$

Để lần ngược lại kết quả  $y^*$ , ta cần lưu trữ nghiệm của max trong công thức (12.44).

- Đồ thị nhân tử chỉ gồm các nhân tử của cặp thuộc tính

$$\phi_{ij}(y_i, y_j, x_i, x_j)$$

và hàm  $\phi_{ij}$  là hàm sub-modular: sử dụng thuật toán luồng cực đại.

## 12.8 Tình huống áp dụng: Bài toán gán nhãn từ loại

Trong mục này, chúng ta cùng tìm hiểu cách áp dụng Trường ngẫu nhiên có điều kiện (CRF) cho bài toán gán nhãn từ loại (POS Tagging). Chúng ta sẽ sử dụng tập dữ liệu sẵn có từ thư viện NLTK, giúp dễ dàng thực hành mà không cần thu thập dữ liệu phức tạp.

- Chuẩn bị dữ liệu: sử dụng thư viện NLTK để tải tập dữ liệu (treebank) gán nhãn từ loại và chia thành tập huấn luyện (80%) và tập kiểm tra (20%). Bộ dữ liệu **treebank** trong NLTK được trích xuất từ các bài viết của Tạp chí phổ Wall, là một phần của bộ dữ liệu Penn Treebank.

- *Nguồn dữ liệu*: các bài báo theo các chủ đề về kinh tế, tài chính và tin tức chung.
  - *Kích thước*: 3914 câu với tổng cộng hơn 100.000 từ đã được gán nhãn từ loại.
  - *Nhãn từ loại*: gồm 45 nhãn khác nhau (ví dụ: NN cho danh từ số ít, VB cho động từ nguyên thể, JJ cho tính từ, . cho dấu chấm, v.v.).
- *Tiền xử lý dữ liệu*: Tiếp theo, ta trích xuất các đặc trưng từ mỗi từ trong câu gồm từ hiện tại, từ trước đó, từ sau đó và các thuộc tính như từ có viết hoa hay không. Các đặc trưng này sẽ là đầu vào cho mô hình CRF.
    - *Trích xuất đặc trưng*: Mỗi từ được gán các đặc trưng như dạng chữ thường (`word.lower`), viết hoa (`isupper`), viết hoa chữ cái đầu (`istitle`) và là số (`isdigit`).
    - *Ngữ cảnh*: Bao gồm từ trước (`-1:word.lower`) và từ sau (`+1:word.lower`) để mô hình hóa mối quan hệ chuỗi.
    - *Đánh dấu ranh giới*: Thêm đặc trưng BOS (bắt đầu câu) và EOS (kết thúc câu) để xác định vị trí trong câu.
    - *Tách nhãn*: Tách nhãn từ loại (ví dụ: NN, VB) từ mỗi từ để tạo đầu ra cho mô hình.
    - *Định dạng đầu vào*: chuyển thành cặp đặc trưng ( $X$ ) và nhãn ( $y$ ) cho tập huấn luyện và kiểm tra.
  - *Xây dựng mô hình CRF*: Mô hình CRF được xây dựng bằng thư viện `sklearn-crfsuite`. Thư viện cho phép thiết lập các tham số như thuật toán huấn luyện (`lbfgs`), hệ số điều chuẩn (`c1`, `c2`), số lần lặp tối đa và bật tính năng tạo tất cả các chuyển tiếp có thể để mô hình hóa tốt hơn.
  - *Huấn luyện mô hình*: sử dụng hàm `fit` để huấn luyện. trên tập dữ liệu huấn luyện đã tiền xử lý.

- Đánh giá mô hình: sử dụng thư viện `sklearn-crfsuite` để đánh giá mô hình bằng các độ đo như độ chính xác (accuracy) và F1-score.

Mã nguồn ví dụ có thể xem tại <https://gist.github.com/tqlong/5847a91992c59eae9e7ec4df1f66e1d8>

## 12.9 Tổng kết chương

Mô hình đồ thị xác suất là bước cuối cùng giúp hoàn thiện bức tranh toàn diện, từ các mô hình tuyến tính, phi tuyến, mạng nơ-ron sâu cho đến hệ thống thống kê phức tạp có cấu trúc rõ ràng.

Thông qua các mô hình như mạng Bayes và trường ngẫu nhiên Markov, chương đã minh họa cách kết hợp giữa xác suất và cấu trúc để xây dựng các hệ thống suy luận, có khả năng xử lý thông tin không chắc chắn trong môi trường thực tế.

Hy vọng rằng, các nền tảng lý thuyết cùng với các ví dụ thực tiễn sẽ truyền cảm hứng và hỗ trợ bạn đọc tiếp tục nghiên cứu, triển khai và sáng tạo các giải pháp Học máy trong nhiều lĩnh vực khác nhau.

## Bài tập

1. Cho tập dữ liệu với 2 lớp  $y \in \{0, 1\}$  và 3 thuộc tính nhị phân  $x^1, x^2, x^3 \in \{0, 1\}$ . Dữ liệu gồm:
  - Lớp 0: 5 mẫu, trong đó 2 mẫu có  $x^1 = 1$ , 3 mẫu có  $x^2 = 1$ , 4 mẫu có  $x^3 = 1$ .
  - Lớp 1: 5 mẫu, trong đó 4 mẫu có  $x^1 = 1$ , 3 mẫu có  $x^2 = 1$ , 1 mẫu có  $x^3 = 1$ .

Hãy



- Tính  $\hat{P}(y)$  và  $\hat{P}(x^k | y)$  cho mỗi  $y$  và  $k$ , sử dụng làm trơn Laplace với hằng số 1.
  - Sử dụng các xác suất này để ra quyết định xem mẫu  $x^1 = 1, x^2 = 0, x^3 = 1$  thuộc lớp nào.
2. Chứng minh rằng trong mô hình Bayes ngây thơ, việc chọn lớp  $y^* = \arg \max_y P(y | \mathbf{x})$  tương đương với tối ưu hóa hàm log-likelihood
- $$y^* = \arg \max_y \log P(y) + \sum_k \log P(x^k | y).$$
3. Cho mạng Bayes với đồ thị  $A \rightarrow B \rightarrow C$  và  $A \rightarrow D$ . Chứng minh rằng  $B$  D-chia cắt  $A$  và  $C$  và  $A$  độc lập với  $C$  khi biết  $B$ . Hơn nữa, hãy kiểm tra xem  $C$  có độc lập với  $D$  khi biết  $A$  hay không.
4. [Tìm hiểu] Hãy tìm hiểu chứng minh định lý Hammersley-Clifford cho trường ngẫu nhiên Markov.
5. Cho đồ thị nhân tử dạng cây với các nhân tử trong bảng 12.1.

**Bảng 12.1:** Giá trị của các nhân tử trong đồ thị nhân tử dạng cây

| Nhân tử               | Biến   | $x_1$ | $x_2$ | Giá trị |
|-----------------------|--------|-------|-------|---------|
| $\psi_{12}(x_1, x_2)$ | (0, 0) | 0     | 0     | 0.6     |
|                       | (0, 1) | 0     | 1     | 0.4     |
|                       | (1, 0) | 1     | 0     | 0.3     |
|                       | (1, 1) | 1     | 1     | 0.7     |
| $\psi_{23}(x_2, x_3)$ | (0, 0) | 0     | 0     | 0.5     |
|                       | (0, 1) | 0     | 1     | 0.5     |
|                       | (1, 0) | 1     | 0     | 0.8     |
|                       | (1, 1) | 1     | 1     | 0.2     |

Tính  $P(x_2)$  bằng thuật toán tổng-tích và xác định

$$\arg \max_{x_2} P(x_2 | x_1 = 1, x_3 = 0)$$

bằng thuật toán max-tổng (Viterbi).

6. [*Lập trình*] Triển khai mô hình CRF đơn giản bằng Python (sử dụng thư viện `sklearn-crfsuite`) cho bài toán gán nhãn từ loại. Huấn luyện và đánh giá trên tập dữ liệu giả lập.

---

# Tài liệu tham khảo

---

- [1] McCallum, A., and Nigam, K., *A comparison of event models for Naive Bayes text classification*, Proceedings of the AAAI Workshop on Learning for Text Categorization, pp. 41–48, 1998.
- [2] Pearl, J., *Reverend Bayes on inference engines: A distributed hierarchical approach*, Proceedings of the National Conference on Artificial Intelligence, pp. 133–136, 1982.
- [3] Lauritzen, S. L., and Spiegelhalter, D. J., *Local computations with probabilities on graphical structures and their application to expert systems*, Journal of the Royal Statistical Society: Series B, vol. 50, no. 2, pp. 157–224, 1988.
- [4] Geman, S., and Geman, D., *Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images*, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 6, no. 6, pp. 721–741, 1984.
- [5] Rabiner, L. R., *A tutorial on hidden Markov models and selected applications in speech recognition*, Proceedings of the IEEE, vol. 77, no. 2, pp. 257–286, 1989.

- [6] Lafferty, J. D., McCallum, A., and Pereira, F. C. N., *Conditional random fields: Probabilistic models for segmenting and labeling sequence data*, Proceedings of the 18th International Conference on Machine Learning, pp. 282–289, 2001.

## Chương 13

---

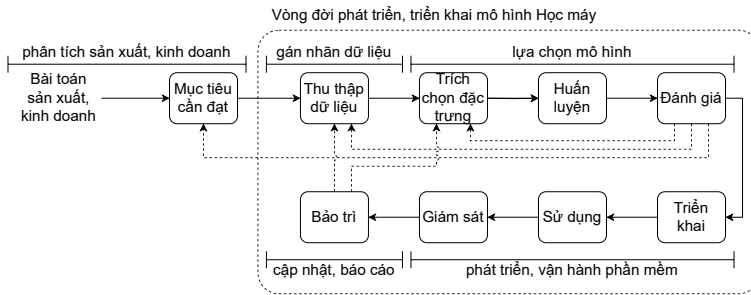
# Triển khai mô hình học máy

---

Xây dựng mô hình chỉ là bước khởi đầu trong chuỗi giá trị của Học máy. Chương 13 trình bày cách đưa mô hình học máy vào vận hành trong môi trường thực tế, giúp người học nắm bắt toàn bộ vòng đời triển khai mô hình.

Nội dung chương bao gồm các bước quan trọng như tiền xử lý dữ liệu, đóng gói mô hình, triển khai dịch vụ, giám sát hiệu suất và tối ưu hoá sau triển khai. Chương cũng giới thiệu các công cụ và nền tảng phổ biến trong triển khai hệ thống Học máy hiện đại, như Docker để tạo môi trường độc lập, FastAPI để xây dựng dịch vụ RESTful và CI/CD để tự động hóa quy trình triển khai liên tục.

Thông qua các ví dụ thực tiễn, chương cung cấp kiến thức thiết yếu để chuyển đổi một mô hình từ môi trường nghiên cứu sang hệ thống hoạt động ổn định và có thể mở rộng trong thực tế.



Hình 13.1: Quá trình phát triển, triển khai mô hình Học máy.

### 13.1 Vòng đời phát triển mô hình học máy

Quá trình phát triển và triển khai mô hình học máy bắt đầu từ nhu cầu sản xuất, kinh doanh của tổ chức. Nhà lãnh đạo đặt ra các *mục tiêu cần đạt* đối với phần mềm. Nhóm phát triển sẽ phân tích xem bài toán có phù hợp với cách tiếp cận học máy hay không, tức là có xác định được nhiệm vụ, độ đo hiệu năng và có sẵn dữ liệu kinh nghiệm hay không. Nếu câu trả lời là *có*, nhóm phát triển sẽ khởi động vòng đời phát triển mô hình học máy (Hình 13.1).

1. *Thu thập và chuẩn bị*: Dữ liệu được thu thập từ lịch sử hoạt động, sau đó được chuẩn hoá, làm sạch và chuyển thành định dạng mô hình có thể xử lý. Dữ liệu được chia thành *tập huấn luyện*, *tập kiểm thử* (dành cho phát triển) và *tập kiểm tra* (dành cho đánh giá độc lập, không dùng để tinh chỉnh mô hình).
2. *Trích chọn đặc trưng dữ liệu*: Người phát triển phân tích dữ liệu để chọn đặc trưng phù hợp, loại bỏ đặc trưng nhiễu hoặc dư thừa. Có thể tạo thêm đặc trưng mới từ các đặc trưng có sẵn. Việc này giúp mô hình học hiệu quả hơn và phản ánh đúng nhiệm vụ.
3. *Huấn luyện mô hình*: Chọn và huấn luyện mô hình học máy phù

hợp bằng dữ liệu huấn luyện. Mô hình được tinh chỉnh dần để hiệu suất  $P$  cải thiện theo thời gian và số lượng mẫu huấn luyện.

4. *Đánh giá mô hình*: Đánh giá mô hình bằng dữ liệu kiểm thử để đảm bảo mô hình đạt yêu cầu và hoạt động tốt trên dữ liệu chưa từng thấy. Không dùng dữ liệu huấn luyện để đánh giá để tránh thiên lệch.
5. *Triển khai mô hình*: Sau khi đánh giá, mô hình được đóng gói và chuyển sang định dạng thích hợp (ví dụ: tối ưu bằng TensorRT, ONNX). Sau đó được tích hợp vào hệ thống thực (máy chủ, ứng dụng web, v.v.).
6. *Sử dụng mô hình*: Mô hình được kết nối qua API để hệ thống dễ dàng gửi dữ liệu đầu vào và nhận kết quả. Mô hình thường được nạp sẵn vào bộ nhớ để giảm độ trễ và chia sẻ tài nguyên hiệu quả.
7. *Giám sát mô hình*: Khi mô hình vận hành, ta giám sát các chỉ số như thời gian xử lý, độ chính xác, lỗi và mức độ hài lòng. Có thể triển khai song song mô hình cũ–mới để so sánh (A/B testing).
8. *Bảo trì mô hình*: Dựa trên giám sát và yêu cầu mới, mô hình có thể cần cập nhật để cải thiện hiệu suất, mở rộng API, điều chỉnh đầu vào–đầu ra hoặc tái huấn luyện với dữ liệu mới thu thập.

## 13.2 Thu thập và tiền xử lý dữ liệu

Dữ liệu đóng vai trò cốt lõi trong học máy, vì mô hình không thể hoạt động nếu không có dữ liệu đầu vào. Dữ liệu cung cấp thông tin để thuật toán học, nhận diện mẫu, dự đoán và tự động hóa quyết định. Tuy nhiên, không phải mọi dữ liệu đều có giá trị như nhau.

Chất lượng dữ liệu ảnh hưởng trực tiếp đến hiệu suất mô hình. Dữ liệu thiếu, nhiễu, sai lệch hoặc không đầy đủ có thể khiến mô hình hoạt động kém hoặc đưa ra dự đoán sai. Do đó, việc thu thập, làm sạch và chuẩn bị dữ liệu kỹ lưỡng là điều kiện tiên quyết để xây dựng mô hình mạnh mẽ và đáng tin cậy.

Quy trình xử lý dữ liệu trước học máy thường gồm ba bước chính:

- **Thu thập dữ liệu:** Lấy dữ liệu từ các nguồn như cơ sở dữ liệu công khai, API, cảm biến hoặc hệ thống thực.
- **Làm sạch và chuyển đổi dữ liệu:** Loại bỏ giá trị thiếu, xử lý ngoại lai, chuẩn hoá định dạng để phù hợp với yêu cầu đầu vào của mô hình.
- **Trích xuất đặc trưng:** Lựa chọn hoặc tạo đặc trưng giúp giảm độ phức tạp và làm nổi bật yếu tố quan trọng nhất trong dữ liệu.

Một quy trình xử lý dữ liệu khoa học giúp mô hình học hiệu quả hơn, đạt hiệu suất cao và đưa ra kết quả đáng tin cậy trong thực tế.

### 13.2.1 Thu thập dữ liệu

**Nguồn dữ liệu.** Dữ liệu là nền tảng thiết yếu trong học máy và có thể được thu thập từ nhiều nguồn khác nhau. Hai loại chính gồm:

- *Dữ liệu có sẵn:* Bao gồm các bộ dữ liệu công khai (Kaggle, UCI, Google Dataset Search) và dữ liệu nội bộ từ doanh nghiệp. Dữ liệu công khai tiện lợi cho thử nghiệm; dữ liệu nội bộ thường mang tính đặc thù cao, cần bảo mật nhưng có giá trị thực tiễn lớn.



- *Dữ liệu thu thập theo thời gian thực*: Từ API, cảm biến, thiết bị IoT hoặc web scraping. Các phương pháp này cho phép cập nhật liên tục, mở rộng linh hoạt.

Bất kể nguồn dữ liệu nào, quá trình thu thập cần đảm bảo tính đầy đủ, toàn vẹn và phù hợp với mục tiêu phân tích.

**Định dạng dữ liệu.** Dữ liệu học máy có thể tồn tại dưới ba định dạng chính:

- *Cấu trúc*: Dữ liệu bảng (CSV, SQL), dễ phân tích bằng các công cụ truyền thống, thích hợp với mô hình tuyến tính hoặc cây quyết định.
- *Phi cấu trúc*: Văn bản tự nhiên, hình ảnh, âm thanh, video. Yêu cầu kỹ thuật đặc thù như NLP, xử lý ảnh, nhận dạng giọng nói.
- *Bán cấu trúc*: Dữ liệu có tổ chức nhưng không cứng nhắc, ví dụ JSON, XML. Phù hợp cho các ứng dụng web, dữ liệu phân cấp hoặc linh hoạt.

Việc hiểu rõ định dạng giúp lựa chọn đúng kỹ thuật xử lý và mô hình phù hợp.

**Lưu trữ và quản lý dữ liệu.** Hệ thống học máy hiện đại yêu cầu lưu trữ hiệu quả và dễ dàng mở rộng:

- *Cơ sở dữ liệu quan hệ (RDBMS)*: Phù hợp dữ liệu có cấu trúc; sử dụng SQL để truy vấn; ví dụ: MySQL, PostgreSQL.
- *Cơ sở dữ liệu phi quan hệ (NoSQL)*: Dữ liệu bán cấu trúc hoặc phi cấu trúc; linh hoạt hơn RDBMS; ví dụ: MongoDB, Cassandra, DynamoDB.

- *Lưu trữ đám mây*: AWS S3, Google Cloud Storage, Azure hỗ trợ lưu trữ lớn, bảo mật, sao lưu, tích hợp công cụ như Spark, Hadoop.

Việc lựa chọn công nghệ lưu trữ phụ thuộc vào quy mô, loại dữ liệu và yêu cầu mở rộng của dự án.

### 13.2.2 Tiền xử lý dữ liệu

**Khám phá dữ liệu.** Kiểm tra dữ liệu là bước đầu trong tiền xử lý, giúp hiểu rõ đặc điểm và chất lượng dữ liệu trước khi huấn luyện mô hình. Một kỹ thuật quan trọng là phân tích dữ liệu khám phá (Exploratory Data Analysis – EDA), kết hợp trực quan hóa và thống kê mô tả.

EDA dùng các công cụ như biểu đồ phân phối, biểu đồ hộp, biểu đồ tương quan để phát hiện dữ liệu thiếu, ngoại lệ, hoặc sự không đồng nhất giữa các thuộc tính. Nó cũng hỗ trợ định hướng cách xử lý dữ liệu và chọn mô hình phù hợp.

Phân phối thuộc tính cho biết mức độ tuân theo phân phối chuẩn, giúp chọn phương pháp chuẩn hóa phù hợp. Nếu không chuẩn, có thể áp dụng biến đổi log, căn bậc hai hoặc Box-Cox.

Các thống kê như giá trị lớn nhất, nhỏ nhất, trung bình và độ lệch chuẩn cho biết phạm vi và mức độ biến thiên của dữ liệu, hỗ trợ phát hiện bất thường hoặc dữ liệu cần xử lý tiếp.

**Làm sạch dữ liệu.** Làm sạch đảm bảo đầu vào cho mô hình là đầy đủ, nhất quán và không gây nhiễu. Các vấn đề cần xử lý gồm: giá trị thiếu, dữ liệu ngoại lai, lỗi chính tả và định dạng.

*Xử lý giá trị thiếu:*

- **Loại bỏ:** Loại các hàng/cột nếu tỷ lệ thiếu nhỏ và không ảnh hưởng đến tổng thể.

- **Điền giá trị:** Sử dụng trung bình, trung vị, mode hoặc mô hình dự đoán (như hồi quy).

*Xử lý dữ liệu ngoại lai:*

- **IQR:** Loại các giá trị nằm ngoài  $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$ .
- **Z-score:** Loại giá trị có điểm Z vượt quá ngưỡng (thường là  $|Z| > 3$ ).

*Xử lý lỗi định dạng và chính tả:*

- **Lỗi chính tả:** Phát hiện và sửa bằng so khớp gần đúng, ví dụ với thư viện `fuzzywuzzy`.
- **Định dạng không đồng nhất:** Chuẩn hóa ngày tháng, chữ viết hoa/thường, hoặc đơn vị đo lường.

Làm sạch dữ liệu đúng cách giúp cải thiện chất lượng dữ liệu và hiệu năng mô hình.

### 13.2.3 Trích chọn đặc trưng

Chúng ta đã đề cập đến một số kỹ thuật trích chọn đặc trưng trong Chương 10.

**Chuyển đổi dữ liệu.** Đây là bước quan trọng trong tiền xử lý, giúp biến đổi dữ liệu thô thành dạng phù hợp với mô hình học máy. Chuyển đổi đúng cách giúp tăng độ chính xác, giảm độ lệch phân phối, đẩy nhanh tốc độ huấn luyện và cải thiện khả năng tổng quát hóa. Có một số kỹ thuật phổ biến như sau:

- **Chuẩn hoá Min-Max:** Đưa dữ liệu về một khoảng cố định, thường là  $[0, 1]$ . Phù hợp với dữ liệu có đơn vị đo khác nhau,

nhưng nhạy cảm với ngoại lai:

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}.$$

- **Chuẩn hoá Z-score:** Biến dữ liệu về phân phối chuẩn với trung bình 0 và độ lệch chuẩn 1. Giúp xử lý dữ liệu có thang đo khác nhau và ổn định hơn với ngoại lai:

$$X_{\text{norm}} = \frac{X - \mu}{\sigma}.$$

- **Mã hóa one-hot:** Chuyển các thuộc tính phân loại thành dạng nhị phân. Phù hợp khi số danh mục không quá lớn. Nếu số lượng danh mục nhiều, có thể gây tăng chiều dữ liệu:

$$\{\text{"đỏ"}, \text{"xanh dương"}, \text{"xanh lá"}\} \rightarrow [1,0,0], [0,1,0], [0,0,1].$$

- **Biến đổi phi tuyến:** Dùng log, căn bậc hai, hoặc hàm mũ để xử lý phân phối lệch, giảm độ biến thiên, và tăng tuyến tính hóa. Hữu ích khi dữ liệu lệch hoặc có giá trị cực lớn.
- **Mã hoá nhãn:** Với các mô hình cây, có thể dùng cách gán số nguyên cho từng danh mục. Tuy nhiên, cần tránh với mô hình tuyến tính vì gây hiểu nhầm về thứ tự.

Chuyển đổi dữ liệu đúng cách là nền tảng cho việc trích chọn đặc trưng hiệu quả, giúp mô hình học tốt và ổn định hơn trên nhiều loại dữ liệu.

**Trích chọn đặc trưng.** Đây là quá trình lựa chọn tập con các đặc trưng đầu vào có ảnh hưởng nhiều nhất đến đầu ra, giúp giảm chiều dữ liệu, tăng hiệu năng và tránh quá khớp.

- **Dựa trên thống kê:** Dùng phương pháp lọc như kiểm định  $\chi^2$ , ANOVA, hay tương quan Pearson để loại bỏ đặc trưng yếu.

- **Dựa trên mô hình:** Dùng trọng số của mô hình tuyến tính (như Lasso), cây quyết định hoặc Random Forest để đánh giá tầm quan trọng của đặc trưng.
- **Rút trích tuần tự:** Phương pháp gói thử nghiệm từng tổ hợp đặc trưng với mô hình thực tế để chọn tổ hợp tốt nhất.

Việc chọn đúng đặc trưng giúp giảm độ nhiễu, rút ngắn thời gian huấn luyện và cải thiện khả năng tổng quát.

**Tạo đặc trưng.** Là quá trình xây dựng đặc trưng mới từ dữ liệu gốc để mô hình học tốt hơn. Đây là công việc quan trọng nhưng thường mang tính sáng tạo và phụ thuộc vào kiến thức lĩnh vực.

Một số ví dụ điển hình:

- **Tổ hợp đặc trưng:** Kết hợp nhiều thuộc tính để tạo đặc trưng mới. Ví dụ: tỷ lệ giá/số phòng trong bài toán bất động sản.
- **Phân loại thời gian:** Tách ngày thành các phần như ngày trong tuần, tháng, mùa, v.v. để phát hiện quy luật theo chu kỳ.
- **Làm rời:** Phân nhóm giá trị liên tục thành các khoảng rời rạc.
- **Đếm và tần suất:** Tạo đặc trưng thống kê như số lần xuất hiện của từ trong văn bản hoặc số giao dịch của khách hàng.
- **Embedding:** Với dữ liệu dạng văn bản, danh mục hoặc đồ thị, có thể dùng kỹ thuật học sâu để tạo ra véc-tơ đặc trưng có tính ngữ nghĩa cao.

Việc tạo và chọn đặc trưng tốt có thể ảnh hưởng mạnh mẽ đến hiệu suất mô hình, trong nhiều trường hợp, có tác động lớn hơn cả việc lựa chọn thuật toán, mô hình Học máy.

## 13.3 Triển khai mô hình học máy

### 13.3.1 Triển khai mô hình

Việc triển khai mô hình học máy không chỉ dừng ở xây dựng mô hình mà còn bao gồm tích hợp mô hình vào ứng dụng thực tế để tạo ra giá trị cụ thể. Một mô hình được triển khai hiệu quả sẽ trở thành thành phần của hệ thống, hỗ trợ tự động hóa hoặc ra quyết định trong các lĩnh vực như tài chính, y tế hay thương mại điện tử.

Mục tiêu là đảm bảo mô hình hoạt động liên mạch trong hệ thống hiện có (web, di động, IoT), đồng thời tối ưu về thời gian xử lý, hiệu suất hệ thống và khả năng mở rộng khi truy cập tăng cao.

Triển khai mô hình là một quá trình phức tạp, thường gặp nhiều thách thức về hiệu năng, bảo mật và độ ổn định. Nếu không được giải quyết tốt, các vấn đề này có thể làm giảm chất lượng hệ thống hoặc gây rủi ro an toàn dữ liệu.

Các yếu tố cần xem xét trong triển khai mô hình bao gồm:

- **Hiệu suất:** Mô hình phải đáp ứng yêu cầu về tốc độ xử lý, đặc biệt trong hệ thống thời gian thực và môi trường có tải cao.
- **Tính khả dụng:** Mô hình cần duy trì hoạt động ổn định, không bị gián đoạn khi gặp lỗi hệ thống hoặc lượng truy cập tăng đột biến.
- **Bảo mật:** Cần bảo vệ mô hình khỏi các tấn công như đảo ngược mô hình, xâm nhập dữ liệu và bảo đảm an toàn đầu vào/đầu ra, tránh rò rỉ thông tin nhạy cảm.

Để triển khai thành công, cần kết hợp kỹ thuật phần mềm, hạ tầng đám mây và các công cụ như Docker, REST API, hoặc hệ thống CI/CD để đảm bảo mô hình hoạt động tin cậy và dễ dàng bảo trì.

### 13.3.2 Chuẩn bị mô hình trước khi triển khai

#### Lưu trữ và quản lý mô hình

Lưu trữ mô hình là bước quan trọng nhằm đảm bảo mô hình có thể tái sử dụng, theo dõi và cải thiện. Việc sử dụng định dạng lưu trữ phù hợp giúp tích hợp dễ dàng vào hệ thống triển khai. Một số công cụ phổ biến:

- **Pickle/Joblib**: Dùng trong Python để lưu mô hình cơ bản. Joblib tối ưu hơn với mô hình có kích thước lớn.
- **TensorFlow SavedModel**: Lưu trữ cấu trúc, trọng số và siêu tham số của mô hình TensorFlow, thuận tiện triển khai trên nhiều nền tảng.
- **ONNX**: Định dạng mở hỗ trợ chuyển đổi mô hình giữa các framework như PyTorch và TensorFlow, phù hợp cho môi trường triển khai không đồng nhất.

Để quản lý vòng đời mô hình, có thể dùng **MLflow**, công cụ mã nguồn mở hỗ trợ lưu trữ, so sánh, theo dõi hiệu suất và phiên bản mô hình. Việc theo dõi mô hình giúp đảm bảo mô hình triển khai là bản tốt nhất và có thể truy vết khi có sự cố.

#### Kiểm thử và tối ưu mô hình

Trước khi triển khai, cần đánh giá mô hình về độ chính xác, tốc độ phản hồi và mức sử dụng tài nguyên:

- **Độ chính xác**: Kiểm tra chất lượng dự đoán trên tập kiểm tra để xác nhận mô hình đạt tiêu chuẩn chất lượng.
- **Thời gian phản hồi**: Đo thời gian xử lý một yêu cầu, đặc biệt quan trọng với hệ thống thời gian thực.

- **Tài nguyên sử dụng:** Ghi nhận mức dùng CPU, GPU và RAM để đánh giá khả năng mở rộng và tính hiệu quả của mô hình.

*Tối ưu hoá mô hình* là kỹ thuật giúp giảm kích thước, tăng tốc độ và tiết kiệm tài nguyên hệ thống:

- **Lượng tử hóa:** Giảm độ chính xác của trọng số từ 32-bit xuống 16 hoặc 8-bit, giúp giảm dung lượng và tăng tốc suy luận.
- **Cắt tỉa:** Loại bỏ trọng số không quan trọng để giảm kích thước mô hình và giảm chi phí tính toán.
- **Chất lọc thông tin:** Huấn luyện mô hình nhỏ học từ mô hình lớn để giảm độ phức tạp mà vẫn giữ hiệu năng.
- **Tối ưu hoá mã nguồn:** Sử dụng thư viện chuyên dụng như TensorFlow Lite, ONNX Runtime để tối ưu tốc độ và tính tương thích trên các thiết bị triển khai.

Việc kiểm thử và tối ưu đảm bảo mô hình hoạt động chính xác, nhanh, và ổn định khi tích hợp vào hệ thống thực tế. Đây là bước then chốt để đảm bảo chất lượng dịch vụ và khả năng mở rộng của ứng dụng học máy.

### 13.3.3 Đóng gói mô hình

Đóng gói mô hình là bước quan trọng nhằm đảm bảo mô hình hoạt động ổn định và dễ tích hợp vào hệ thống. Quá trình này thường bao gồm tạo môi trường độc lập và chọn định dạng API phù hợp để triển khai. Công cụ đóng gói phổ biến bao gồm:

- **Docker:** Tạo container chứa toàn bộ mã nguồn, thư viện và phụ thuộc, giúp mô hình chạy ổn định trên nhiều môi trường khác nhau.



- **Kubernetes:** Quản lý và điều phối nhiều container ở quy mô lớn, hỗ trợ tự động mở rộng, cân bằng tải và đảm bảo tính khả dụng cao.

Một số định dạng API phổ biến là:

- **REST API:** Phổ biến và dễ triển khai; truyền dữ liệu qua HTTP với định dạng JSON/XML; phù hợp với ứng dụng web và di động.
- **gRPC:** Hiệu suất cao hơn, truyền dữ liệu nhị phân, phù hợp với hệ thống phân tán hoặc thời gian thực yêu cầu độ trễ thấp.

Việc đóng gói hiệu quả giúp mô hình dễ triển khai, bảo trì và tích hợp vào các dịch vụ hoặc hạ tầng hiện tại.

#### 13.3.4 Các phương pháp triển khai mô hình

Tùy vào yêu cầu hệ thống, có thể triển khai mô hình theo ba hướng chính:

- **Triển khai tại chỗ:** Chạy mô hình trên máy chủ nội bộ của tổ chức. Ưu điểm: bảo mật cao, kiểm soát hoàn toàn. Nhược điểm: chi phí hạ tầng lớn, khó mở rộng.
- **Triển khai trên đám mây:** Sử dụng nền tảng như AWS SageMaker, Google AI Platform. Ưu điểm: mở rộng linh hoạt, tích hợp tốt. Nhược điểm: chi phí phụ thuộc vào tài nguyên sử dụng.
- **Triển khai trên thiết bị biên:** Mô hình chạy trực tiếp trên thiết bị IoT, điện thoại hoặc hệ nhúng. Ưu điểm: phản hồi nhanh, giảm tải hệ thống trung tâm. Nhược điểm: hạn chế về tài nguyên.

### 13.3.5    Đánh giá và giám sát mô hình sau triển khai

Sau triển khai, cần giám sát và đánh giá để đảm bảo mô hình duy trì hiệu suất trong môi trường thực tế.

#### 1. Giám sát hiệu suất hệ thống:

- **Độ trễ:** Thời gian xử lý một yêu cầu dự đoán.
- **Tải lượng:** Số lượng yêu cầu mô hình xử lý trong một khoảng thời gian.
- **Tài nguyên sử dụng:** CPU, GPU, RAM. Phát hiện tắc nghẽn giúp tối ưu hệ thống.

Các công cụ như **Prometheus** và **Grafana** hỗ trợ thu thập và trực quan hoá các chỉ số trên.

#### 2. Giám sát chất lượng dự đoán:

- Phát hiện *drift dữ liệu* khi phân phối đầu vào thay đổi so với lúc huấn luyện.
- Theo dõi sự suy giảm hiệu suất do thay đổi trong môi trường ứng dụng.

#### 3. Cập nhật và tái triển khai mô hình:

- Sử dụng quy trình CI/CD để tự động hóa kiểm thử, đánh giá và triển khai lại khi có mô hình mới.
- CI/CD giúp đảm bảo mô hình luôn được cập nhật kịp thời, giữ hiệu suất ổn định trong thời gian dài.

Tóm lại, đánh giá và giám sát mô hình sau triển khai là quy trình liên tục giúp duy trì chất lượng và khả năng thích ứng. Kết

hợp công cụ giám sát và CI/CD giúp hệ thống luôn đạt hiệu suất tối ưu.

## 13.4 Các nền tảng hỗ trợ triển khai mô hình

Triển khai mô hình học máy là quá trình phức tạp gồm đóng gói, quản lý, giám sát và tối ưu. Việc sử dụng các công cụ hỗ trợ giúp đơn giản hoá quy trình và đảm bảo mô hình hoạt động hiệu quả trong môi trường sản xuất.

### 13.4.1 Công cụ đóng gói và triển khai

**Docker:** Công cụ phổ biến để đóng gói mô hình cùng môi trường vào container độc lập, đảm bảo tính di động và nhất quán khi triển khai trên các hệ thống khác nhau.

**Kubernetes:** Nền tảng mã nguồn mở quản lý và điều phối nhiều container. Hỗ trợ cân bằng tải, mở rộng tự động và đảm bảo tính sẵn sàng cao cho hệ thống.

**TensorFlow Serving:** Giải pháp triển khai mô hình TensorFlow nhanh chóng, hỗ trợ phục vụ nhiều phiên bản mô hình, tối ưu cho ứng dụng web và API.

**TorchServe:** Dịch vụ triển khai mô hình PyTorch, hỗ trợ cấu hình API REST, logging, batch inference và quản lý tài nguyên.

### 13.4.2 Công cụ quản lý và giám sát mô hình

**MLflow:** Nền tảng mã nguồn mở hỗ trợ quản lý vòng đời mô hình (tracking, packaging, deployment). Giúp theo dõi, lưu trữ và tái sử dụng mô hình trong nhiều môi trường.

**Prometheus + Grafana:** Prometheus thu thập chỉ số hệ thống và mô hình (latency, throughput, CPU/GPU); Grafana hiển thị trực quan bằng biểu đồ, hỗ trợ phát hiện và cảnh báo sự cố.

**Kubeflow:** Nền tảng MLOps toàn diện xây dựng trên Kubernetes, hỗ trợ huấn luyện, phục vụ và theo dõi mô hình. Phù hợp với quy trình tự động hoá và triển khai ở quy mô lớn.

### 13.4.3 Công cụ tối ưu hóa và tăng tốc suy luận

**TensorRT:** Công cụ tối ưu mô hình từ NVIDIA, tăng tốc suy luận trên GPU bằng kỹ thuật như lượng tử hoá, hợp nhất tầng và tái sắp xếp mạng.

**ONNX Runtime:** Runtime đa nền tảng hỗ trợ mô hình từ nhiều framework (PyTorch, TensorFlow, MXNet), tối ưu trên cả CPU và GPU, phù hợp cho triển khai phân tán.

**OpenVINO:** Bộ công cụ tối ưu mô hình từ Intel, hỗ trợ triển khai trên CPU, GPU, FPGA và thiết bị IoT. Thích hợp cho suy luận tại thiết bị biên (edge deployment).

### 13.4.4 Công cụ CI/CD cho học máy

**GitHub Actions và GitLab CI/CD:** Hỗ trợ tự động hoá quy trình huấn luyện, kiểm thử và triển khai mô hình. Giúp đảm bảo tính liên tục và phản ứng nhanh với thay đổi dữ liệu.

**Jenkins:** Hệ thống CI/CD mạnh mẽ, cho phép tích hợp linh hoạt nhiều công cụ, hỗ trợ toàn bộ quy trình phát triển và triển khai mô hình một cách tự động.

**DVC (Data Version Control):** Quản lý phiên bản dữ liệu, mã và mô hình. Đảm bảo tính tái lập và minh bạch cho các thí nghiệm học máy. Hữu ích trong dự án cộng tác và kiểm soát mô hình phức tạp.

Việc lựa chọn nền tảng phụ thuộc vào yêu cầu dự án, độ phức tạp triển khai và hạ tầng hiện có. Kết hợp nhiều công cụ sẽ giúp

xây dựng hệ thống triển khai học máy hiệu quả, linh hoạt và có thể mở rộng.

## 13.5 Tình huống áp dụng: Triển khai mô hình qua giao thức lập trình ứng dụng REST

Trong thực tế, việc triển khai mô hình học máy thông qua giao thức lập trình ứng dụng REST giúp tích hợp vào các hệ thống dễ dàng và linh hoạt. Hướng dẫn này sử dụng thư viện *FastAPI* để triển khai một mô hình học máy đơn giản cho bài toán phân loại hoa IRIS. Mô hình này sẽ nhận dữ liệu đầu vào từ người dùng thông qua API và trả về kết quả dự đoán. Chúng ta sẽ dựa trên mô hình Random Forest đã được huấn luyện trên tập dữ liệu IRIS, sử dụng thư viện *scikit-learn* để thực hiện các bước huấn luyện và lưu trữ mô hình. Các bước triển khai chính bao gồm:

- Chuẩn bị thư viện: Trong bước này, chúng ta sẽ cài đặt các thư viện cần thiết để triển khai mô hình học máy, bao gồm *FastAPI* cho việc xây dựng API, *scikit-learn* cho việc huấn luyện mô hình và *joblib* để lưu trữ mô hình và các thư viện liên quan. Các thư viện được lưu trữ thông qua tệp `requirements.txt` để dễ dàng quản lý và cài đặt.
- Huấn luyện mô hình: Trong bước này, chúng ta sẽ sử dụng tập dữ liệu IRIS để huấn luyện mô hình Random Forest và lưu trữ mô hình đã huấn luyện. Các bước triển khai chính người học có thể tham khảo các Chương 2 và 9 để hiểu rõ hơn về cách sử dụng thư viện *sklearn* để huấn luyện mô hình Random Forest trên tập IRIS.
- Lưu trữ mô hình: Mô hình sau khi huấn luyện sẽ được lưu trữ dưới dạng tệp để có thể sử dụng lại trong các lần triển khai sau. Chú ý rằng việc lưu trữ này bao gồm toàn bộ tham số và quá

trình tính toán kèm theo. Trong bước này, chúng ta sẽ sử dụng thư viện *joblib* để lưu trữ mô hình đã huấn luyện.

- Triển khai mô hình: Chúng ta sẽ sử dụng thư viện *FastAPI* để xây dựng một giao diện lập trình ứng dụng REST đơn giản. Giao diện này sẽ nhận dữ liệu đầu vào từ người dùng và trả về kết quả dự đoán từ mô hình đã huấn luyện. Sau khi xây dựng xong, chúng ta sẽ chạy ứng dụng FastAPI để phục vụ các yêu cầu từ người dùng. Giả sử tệp triển khai là `main.py`, chúng ta sẽ sử dụng lệnh *uvicorn* để chạy ứng dụng FastAPI.

#### Đoạn mã 13.1: Chạy FastAPI Server

```
uvicorn main:app --reload
```

Server sẽ khởi động tại địa chỉ `http://127.0.0.1:8000`, bao gồm các giao thức:

1. **GET /**: Trả về thông báo chào mừng.
  2. **POST /predict**: Nhận dữ liệu đầu vào và trả về kết quả dự đoán.
- Kiểm thử: Chúng ta có thể sử dụng các công cụ như Postman hoặc curl để gửi các yêu cầu POST với dữ liệu đầu vào và nhận kết quả dự đoán từ mô hình. Ví dụ, gửi dữ liệu JSON đến `/predict` theo định dạng sau:

#### Đoạn mã 13.2: Yêu cầu gửi đến giao thức dự đoán

```
{
 "sepal_length": 5.1,
 "sepal_width": 3.5,
 "petal_length": 1.4,
 "petal_width": 0.2
}
```

Kết quả trả về sẽ có dạng:

**Đoạn mã 13.3: Kết quả dự đoán nhận được**

```
{
 "prediction": 0
}
```

Người đọc có thể tham khảo mã nguồn tại: <https://gist.github.com/cuongtv312/cd68e6989b11cf48e8d8b5ed0d306e5c>

## 13.6 Tổng kết chương

Chương 13 nhấn mạnh tầm quan trọng của việc triển khai và quản lý mô hình Học máy một cách chuyên nghiệp và toàn diện.

Từ khâu thu thập và làm sạch dữ liệu, đóng gói mô hình, xây dựng API đến giám sát hiệu suất sau triển khai, mỗi bước trong vòng đời mô hình đều đòi hỏi quy trình bài bản và sự phối hợp hiệu quả giữa các nhóm phát triển, vận hành và hạ tầng.

Các công cụ như Docker, FastAPI và hệ thống CI/CD không chỉ hỗ trợ tự động hóa mà còn giúp đảm bảo tính ổn định, khả năng mở rộng và duy trì chất lượng của hệ thống Học máy trong môi trường thực tế.

Nội dung chương trang bị cho người học kiến thức và kỹ năng thiết yếu để đưa mô hình từ nghiên cứu vào ứng dụng, đóng vai trò quan trọng trong việc xây dựng các hệ thống Học máy vận hành thực tế.

## Bài tập

Dưới đây là các bài tập giúp sinh viên tìm hiểu và thực hành các công cụ phổ biến trong việc triển khai mô hình học máy. Sinh viên cần nghiên cứu, cài đặt và thực hành để nắm vững cách sử dụng từng công cụ.

1. *[Tìm hiểu]* Sử dụng **MLflow** hoặc **WandB** để theo dõi quá trình huấn luyện một mô hình phân loại trên tập dữ liệu **IRIS**. Ghi lại các siêu tham số và độ chính xác qua các lần huấn luyện.
2. *[Tìm hiểu]* Tìm hiểu kỹ thuật lượng tử hóa mô hình và chứng minh rằng nó có thể giảm kích thước mô hình, tăng tốc độ suy luận mà không làm giảm đáng kể độ chính xác.
3. *[Lập trình]* Sử dụng **TensorRT** hoặc nền tảng **ONNX** để tối ưu hóa một mô hình học sâu. Người đọc có thể lựa chọn kiến trúc **ResNet** hoặc các kiến trúc khác được đề cập trong Chương 5.
4. *[Tìm hiểu]* Thiết lập một luồng **CI/CD** đơn giản bằng công cụ **GitHub Actions** để tự động huấn luyện và triển khai một mô hình học máy khi mã nguồn thay đổi.
5. *[Lập trình]* Sử dụng **FastAPI** để triển khai một mô hình phân loại **MNIST** qua giao thức lập trình ứng dụng **REST**. Viết mã nguồn để huấn luyện mô hình, lưu trữ, tạo giao thức nhận dữ liệu đầu vào và trả về dự đoán.



---

# Tài liệu tham khảo

---

- [1] Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D., *The ML test score: A rubric for ML production readiness and technical debt reduction*, Proceedings of the IEEE International Conference on Big Data, pp. 1123–1132, 2017.
- [2] Polyzotis, N., Roy, S., Whang, S. E., and Zinkevich, M., *Data management challenges in production machine learning*, Proceedings of the 2018 ACM International Conference on Management of Data, pp. 1723–1726, 2018.
- [3] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., et al., *Hidden technical debt in machine learning systems*, Advances in Neural Information Processing Systems, vol. 28, pp. 2503–2511, 2015.
- [4] Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., et al., *TensorFlow: Large-scale machine learning on heterogeneous systems*, arXiv preprint arXiv:1603.04467, 2015.
- [5] Bergstra, J., and Bengio, Y., *Random search for hyperparameter optimization*, Journal of Machine Learning Research, vol. 13, pp. 281–305, 2012.

- [6] Mäkinen, S., Skogström, E., Laaksonen, J., and Järvinen, H., *Who needs MLOps: What data scientists seek to accomplish and how can MLOps help?*, arXiv preprint arXiv:2010.13335, 2020.
- [7] Hazelwood, K., Bird, S., Brooks, D., Chintala, S., Diril, U., Dzhulgakov, D., Fawzy, M., et al., *Applied machine learning at Facebook: A datacenter infrastructure perspective*, Proceedings of the IEEE International Symposium on High-Performance Computer Architecture, pp. 620–629, 2018.

---

# Chỉ mục

---

Adam, 74

ANOVA, 285

Bayes ngây thơ, 335

bài toán hồi quy, 173

bài toán phân lớp, 26, 29

bản đồ đặc trưng, 118

bộ lọc, 118

bộ nhớ ngắn hạn dài, 158

Bộ nhớ đệm, 230

bộ tự mã hoá biến phân, 281

bộ tự mã hóa sâu, 279

C4.5, 48

CART, 194

chiến lược huấn luyện, 137

chuỗi thời gian, 151

chính sách tham lam, 224

CIFAR10, 140

CNN, 113, 123

CRf, 359

Cây quyết định, 44

cây quyết định, 44, 193

cường độ, 114

cận dưới bằng chứng, 307

DQN, 229

dữ liệu, 28

dữ liệu dạng chuỗi, 150

dữ liệu ảnh, 114

entropy chéo, 60

GAN, 322

GD, 75

giá trị riêng, 183

HMM, 352

hàm hợp lý, 301

hàm kích hoạt, 58

hàm Lagrange, 91, 275

hàm lỗi, 174

hàm lỗi entropy chéo, 60

Hàm mục tiêu, 230

hàm nhân hoá, 101

hàm phân hoạch, 345

hàm phân lớp tối ưu, 35  
hàm sigmoid, 36  
hàm tuyến tính, 36  
hệ số chiết khấu, 206  
học bán giám sát, 15  
học chuyển đổi, 17  
học chủ động, 16  
học có giám sát, 11  
học hàm Q, 219  
học không giám sát, 13  
học kết hợp, 18, 244  
học máy, 1, 4  
học trực tuyến, 17  
học tăng cường, 14  
học tự giám sát, 15  
học đa nhiệm vụ, 16  
hồi quy Logistic, 36  
hồi quy tuyến tính, 178  
hội tụ, 68  
  
ID3, 47  
ImageNet, 136  
Inception, 128  
  
khoảng cách KL, 298, 321  
kinh nghiệm, 3  
KNN, 42  
  
lan truyền ngược, 64  
LASSO, 287  
likelihood, 37  
LSTM, 158  
lấy mẫu, 296

lập theo chính sách, 216  
lập theo giá trị, 209  
lớp chuẩn hoá loạt, 126  
lớp gộp, 123  
lớp nơ-ron, 59  
lớp triệt tiêu ngẫu nhiên, 125  
lớp tích chập, 117  
  
MAP, 301  
MDP, 204  
MLE, 38, 318  
MLP, 61, 125  
MNIST, 135  
MobileNet, 132  
mô hình sinh, 296  
mô hình đồ thị xác suất, 334  
Mạng Bayes, 334, 340  
mạng nơ-ron nhiều lớp, 58, 61  
mạng xương sống, 127  
  
nhiệm vụ, 8  
  
PCA, 271  
perceptron, 86  
phân lớp tuyến tính, 86  
phân tích thành phần chính,  
271  
phép toán tự chú ý, 161  
phương pháp bagging, 245  
phương pháp nhân hoá, 101,  
103  
phương sai, 176  
phương trình Bellman, 209

phương trình chuẩn tắc, 182  
Pytorch, 77

quy hoạch động, 209  
quỹ đạo, 205

relu, 59  
ResNet, 130

RNN, 153  
RSS, 181

rủi ro kì vọng, 30  
rủi ro thực nghiệm, 30  
rừng ngẫu nhiên, 248

SGD, 72

SMO, 100

softmax, 60

Spearman, 284

suy luận, 347

SVM, 85, 90

SVM lề cứng, 91

SVM lề mềm, 95

sự tráo đổi giữa độ lệch và phương  
sai, 177

thu thập dữ liệu, 373

thuật toán học máy, 3

thuật toán lan truyền ngược,  
64

thông tin tương hỗ, 285

Transformer, 161

triển khai, 380

triển khai mô hình, 18

triệt tiêu đạo hàm, 131

trung bình bình phương sai số,  
13, 175

trích chọn đặc trưng, 283

trường ngẫu nhiên Markov, 334,  
345

tính chất Markov, 203

Tăng cường bằng đạo hàm, 256

tỉ lệ lỗi, 30

tỉ lệ lỗi trung bình, 28

Tối thiểu hóa rủi ro thực nghiệm,  
31

tự giám sát, 280

VGG, 127

vòng đời mô hình, 18

vòng đời phát triển, 372

vùng tiếp nhận, 118

XGBoost, 261

xác suất lỗi, 29

điểm số, 36

đánh giá mô hình, 183

đặc trưng ẩn, 63

định lý PAC, 31

độ hợp lý, 37, 299

độ lệch, 176

ước lượng hợp lý cực đại, 38,  
299

ước lượng mật độ, 297

